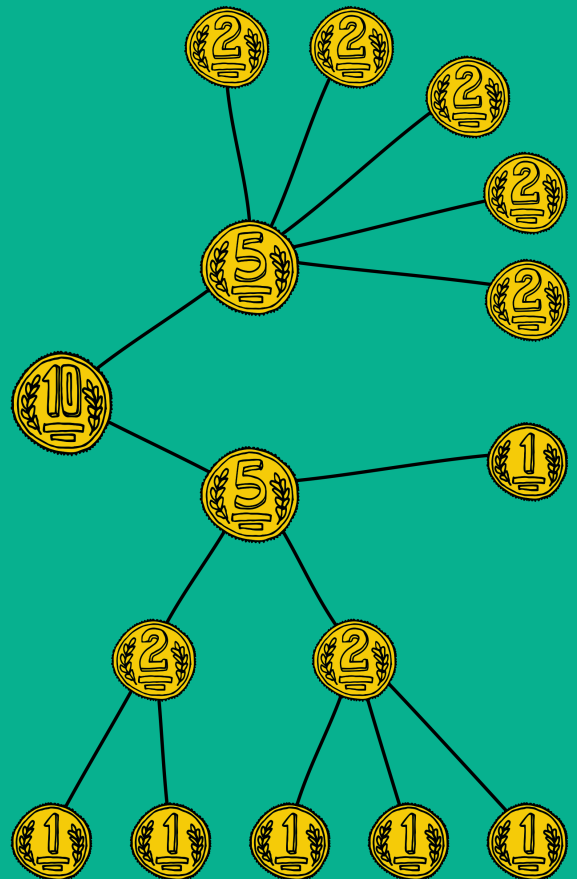


Massively parallel algorithms for sparse graphs

Rustam Latypov



Aalto University publication series
Doctoral Theses 82/2026

Massively parallel algorithms for sparse graphs

Rustam Latypov

A doctoral thesis completed for the degree of Doctor of Science (Technology) to be defended, with the permission of the Aalto University School of Science, at a public examination held at the lecture hall M232 M1 of the school on the 10th of April 2026 at 12:00.

Aalto University
School of Science
Department of Computer Science

Supervising professor

Assistant Professor Jara Uitto, Aalto University, Finland

Preliminary examiners

Professor Artur Czumaj, University of Warwick, United Kingdom

Associate Professor Sepehr Assadi, University of Waterloo, Canada

Opponent

Professor Artur Czumaj, University of Warwick, United Kingdom

Aalto University publication series

Doctoral Theses 82/2026

© Rustam Latypov

Image on the cover: Cover illustration by Rustam Latypov, based on licensed artwork by Nikolai Shitov / Adobe Stock.

ISBN 978-952-64-3082-9 (soft cover)

ISBN 978-952-64-3081-2 (PDF)

ISSN 1799-4934 (printed)

ISSN 1799-4942 (PDF)

<https://urn.fi/URN:ISBN:978-952-64-3081-2>

Unigrafia Oy, Helsinki 2026

PunaMusta Oy, Joensuu, 2026

Author Rustam Latypov

Name of the doctoral thesis Massively parallel algorithms for sparse graphs

Article-based thesis

Number of pages 167

Keywords Massively parallel computation model, uniformly sparse graphs, connectivity, graph coloring, maximal matching, maximal independent set

Most real-life graphs are large and sparse, consisting of billions of nodes connected by billions of edges. Single-machine computation for such graphs is often slow and inefficient. The modern approach is to deploy a fleet of machines that work on a given graph problem in parallel. The Massively Parallel Computation (MPC) model captures the strengths and limitations of such parallel systems and is a widely accepted theoretical framework for analyzing modern parallel algorithms. This thesis considers fundamental graph problems for uniformly sparse graphs in the MPC model, focusing on memory-optimal solutions. Computation is performed on arbitrarily many machines simultaneously while using roughly the same amount of memory that is needed to store the input itself.

We resolve open problems and advance the state of the art by developing novel deterministic algorithms for graph problems such as connectivity, vertex coloring, maximal matching, and maximal independent set. Our focus is on uniformly sparse graphs, technically known as low-arboricity graphs. These graph families form the setting for most known lower bounds in MPC, and designing memory-optimal algorithms for them is notoriously difficult.

Algorithm complexity is measured in communication rounds and is typically expressed as a function of the total number of nodes n or the diameter of the graph D . We establish the first $O(\log D)$ -round algorithm for connectivity, as well as the first $O(\log \log n)$ - and $O(\log D)$ -round algorithms for 3-coloring, maximal matching, and maximal independent set on forest graphs. We also show that strengthening the MPC model, by allowing adaptive queries to a distributed data store, enables a constant-round $O(\alpha)$ -coloring algorithm for graphs with bounded arboricity α .

One major technical contribution of this thesis is a novel graph exploration technique called balanced exponentiation, on which many of our results rely. This technique has been developed into a parameterized, standalone tool that allows nodes to explore surrounding sparse regions of the graph without prior knowledge of their location, while incurring minimal memory overhead.

Tekijä Rustam Latypov

Väitöskirjan nimi Massively parallel algorithms for sparse graphs

Artikkeliväitöskirja

Sivumäärä 167

Avainsanat Massiivisen rinnakkaislaskennan malli, tasaisen harvat verkot, yhtenäisyys, verkon väritys, maksimaalinen pariutus, maksimaalinen riippumaton joukko

Useimmat tosielämän verkot ovat suuria ja harvoja, koostuen miljardeista solmuista, joita yhdistävät miljardit kaaret. Yksittäisen tietokoneen suorittama laskenta tällaisille verkoille on usein hidasta ja tehontonta. Nykyaikainen lähestymistapa on käyttää useita tietokoneita ratkaisemaan annettua verkko-ongelmaa rinnakkain. Massiivisen rinnakkaislaskennan (MPC) -malli on vakiintunut teoreettinen viitekehys modernien rinnakkaisalgoritmien analysointiin, kuvaten rinnakkaisjärjestelmien vahvuuksia ja rajoituksia. Tämä väitöskirja tarkastelee perustavanlaatuisia ongelmia MPC-mallissa, keskittyen muistioptimaalisiin ratkaisuihin tasaisen harvoissa verkoissa. Laskenta suoritetaan samanaikaisesti mielivaltaisella määrällä koneita, käyttäen suunnilleen yhtä paljon muistia kuin itse syötteen tallentamiseen tarvitaan.

Ratkaisemme avoimia ongelmia ja parannamme alan aiempia huipputuloksia uusilla deterministisillä algoritmeilla yhtenäisyydelle, väritykselle, maksimaaliselle pariutukselle sekä maksimaaliselle riippumattomalle joukolle. Keskitymme tasaisen harvoin verkkoihin, jotka tunnetaan teknisesti matalan puumaisuuden verkkoina. Suurin osa tunnetuista alarajoista MPC-mallissa on osoitettu juuri näissä verkoissa ja muistioptimaalisten algoritmien suunnittelu niille on tunnetusti haastavaa.

Algoritmien kompleksisuus mitataan kommunikaatiokierrosten kokonaismääränä ja se ilmaistaan tyypillisesti solmujen määrän n tai verkon halkaisijan D funktiona. Kehitämme ensimmäisen $O(\log D)$ -kierroksisen algoritmin yhtenäisyydelle sekä ensimmäiset $O(\log \log n)$ - ja $O(\log D)$ -kierroksiset algoritmit 3-väritykselle, maksimaaliselle pariutukselle ja maksimaaliselle riippumattomalle joukolle metsäverkoissa. Vahvistamalla MPC-mallia siten, että adaptiiviset kyselyt hajautettuun tietokantaan ovat sallittuja, kehitämme vakiokierroksisen $O(\alpha)$ -väritysalgoritmin verkoille, joiden puumaisuus α on vakio.

Yksi tämän väitöskirjan merkittävimmistä teknisistä kontribuutioista on uusi verkon kartoitusmenetelmä, nimeltään tasapainotettu eksponointi, johon monet väitöskirjan tulokset perustuvat. Menetelmä on jalostettu erilliseksi parametrisoiduksi työkaluksi, jonka avulla solmut voivat kartoittaa niitä ympäröiviä verkon harvoja alueita ilman ennakkotietoa niiden sijainnista, aiheuttamatta merkittävää muistikuormaa.

Preface

What originally drew me to the theoretical study of graph problems was their striking duality — they are often simple to state but difficult to solve. One does not need to be a math graduate to understand, for example, the graph coloring problem, which I can easily explain to my friends, my parents, or even my daughter.

First and foremost, I want to thank Jara for being an exceptional supervisor. You have always been patient and generous with your time. Our frequent whiteboard marathons and debugging sessions were often the most exciting part of any project. Your calming presence in the face of broken proofs and impending deadlines made me feel that everything would work out. In addition to genuinely caring for my development as a researcher, you understood my need for a healthy work-life balance. I could not have asked for a better mentor.

I've been fortunate to have collaborated extensively with Yannic, whose green comments tend to appear all over the manuscript just minutes after resolving the previous ones. The color green will haunt me long after my graduation. Your rigorous attention to detail and precision has shaped the way I approach research.

I'm deeply grateful to Bernhard for hosting me at ETH. Your endless enthusiasm for research and your sweet personality made our stay in Zürich truly memorable. During my visit, I had the pleasure of spending time with Christoph, Václav, Nick, Antti R., and Goran. Thank you for the epic research sessions and late nights over Hanabi and Wizard.

Thank you to all my co-authors: Alkida Balliu, Sebastian Brandt, Karl Bringmann, Manuela Fischer, Nick Fischer, Christoph Grunau, Chetan Gupta, Bernhard Haeupler, Jakub Łącki, Yannic Maus, Dennis Olivetti, Shreyas Pai, Antti Röyskö, Simo Särkkä, Jan Studený, Aurelio Sulser, Jukka Suomela, Jara Uitto, and Hossein Vahidi. I want to thank the Academy of Finland, the Nokia Foundation, and the Finnish Foundation for Technology Promotion for funding my research. I'm very grateful to Artur Czumaj and Sepehr Assadi for serving as the pre-examiners for my thesis, and to Artur for also serving as my opponent.

I've been lucky to have amazing companions throughout this journey. Thank you to the wonderful people I've met at various summer and winter schools and conferences over the years: Giovanna, Marc, Maxime, Zahra, Cem, Zurab, and many others. Every event was made better by the jokes and puzzles we shared. To my longtime friends Saara, Olli, Alisa, Visa, Veera, Sampo, Kalle, Juho, Otto, and Jani, thank you for the years of friendship that had nothing to do with research. To my officemate Mélanie, thank you for the peer support and a mellow workspace.

Climbing is one of my greatest passions, one I've been lucky enough to pursue alongside research. I'm grateful for all the wild vertical adventures I've shared with Joshua, Antti S., Arttu, Paavo, Simon, Tim, Martin, Gavin, and Olga. I could not have asked for a better crew to be routinely scared to death with.

My deepest gratitude goes to my family. My parents' courage to immigrate to Finland from post-Soviet Russia is the only reason I had access to quality education and was free to pursue international academia. Finally, to my beloved partner Olga and our daughter Liana. You were always certain of me, even when I was not, and your "permission" to keep working was the only reason deadlines were met and papers were published. Becoming a father mid-studies was far from easy, but I would not have had it otherwise.

Helsinki, March 8, 2026,

Rustam Latypov

Contents

Preface	1
Contents	3
List of Publications	5
Author’s Contribution	7
List of Figures	9
1. Introduction	11
1.1 The Graph Coloring Problem	11
1.2 Local Problems	13
1.3 Global Problems	15
1.4 How to Solve Graph Problems?	16
1.4.1 Distributed Computing Models	17
1.4.2 Modern Parallel Computing Models	17
1.5 Research Output and Roadmap	18
2. Preliminaries	21
2.1 Graph Notations	21
2.2 Nash-Williams Decomposition	23
2.3 Locally Checkable Labeling (LCL) Problems	23
2.4 The LOCAL Model	24
2.5 The Massively Parallel Computation (MPC) Model	26
2.5.1 Node-Centric Approach	27
2.5.2 Graph Exponentiation	27
2.5.3 Lower Bounds and the 1 vs. 2 Cycles Conjecture	27
2.5.4 Component Stability	28
2.6 The Adaptive MPC Model	28
3. Contributions and Related Work	31
3.1 Exponential Speedup Over Locality	32

3.2	Forest Connectivity	33
3.3	Coloring, MIS and Maximal Matching	35
3.4	Coloring Sparse Graphs in AMPC	36
4.	Exponential Speedup Over Locality	39
4.1	Tiny-Regime $f(n) = \Theta(1)$ and $f(n) = \Theta(\log^* n)$	40
4.2	High-Regime $f(n) = \Theta(n^{1/k})$, for all $k \in \mathbb{N}$	41
4.3	Mid-Regime $f(n) = \Theta(\log n)$	41
4.4	Low-Regime $g(n) = \Theta(\log \log n)$	42
5.	The Balanced Exponentiation Technique	43
5.1	Forest Connectivity	43
5.2	Coloring, MIS and Maximal Matching	45
6.	Coin Dropping Game via Adaptive Queries	49
6.1	Sublinear LCA	49
6.1.1	Where Should One Query?	50
6.1.2	Our Solution	52
6.2	Using Our LCA Recursively	53
	References	55
	Publications	63

List of Publications

This thesis consists of an overview and of the following publications which are referred to in the text by their Roman numerals.

- I** Alkida Balliu, Sebastian Brandt, Manuela Fischer, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Exponential Speedup Over Locality in MPC with Optimal Memory. In *International Symposium on Distributed Computing (DISC) 2022 (also in Journal of Distributed Computing, February 2025)*, Volume 246, 9:1-9:21, doi.org/10.4230/LIPIcs.DISC.2022.9, Augusta, USA, October 2022.
- II** Alkida Balliu, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Optimal Deterministic Massively Parallel Connectivity on Forests. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA) 2023*, pages 2589-2631, doi.org/10.1137/1.9781611977554.ch99, Florence, Italy, January 2023.
- III** Christoph Grunau, Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. Conditionally Optimal Parallel Coloring of Forests. In *International Symposium on Distributed Computing (DISC) 2023*, Volume 281, 23:1-23:20, doi.org/10.4230/LIPIcs.DISC.2023.23, L'Aquila, Italy, October 2023.
- IV** Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. Adaptive Massively Parallel Coloring in Sparse Graphs. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC) 2024*, pages 508-518, doi.org/10.1145/3662158.3662821, Nantes, France, June 2024.

Author's Contribution

Publication I: “Exponential Speedup Over Locality in MPC with Optimal Memory”

The problem of exponential speedup over locality was suggested by Jara Uitto. The author of this thesis served as the main author of the paper, writing approximately 50% of it. The mid-regime result was developed and written by the thesis author. The high-, low-, and tiny-regime results were a collaborative effort by all authors, with most proof verification carried out by the thesis author. Writing the high-regime result was primarily the work of Sebastian Brandt and Jara Uitto. The tiny-regime result was mostly written by Alkida Balliu and Dennis Olivetti.

Publication II: “Optimal Deterministic Massively Parallel Connectivity on Forests”

The author of this thesis served as the main author of the paper, writing approximately 80% of it. The thesis author developed the core technique of balanced exponentiation. Formulating the result as a connectivity algorithm without a runtime dependence on the number of nodes was a collaborative effort by all authors. The thesis author derived all technical proofs, with guidance from Yannic Maus. Alkida Balliu and Dennis Olivetti were responsible for reformulating the LCL solver, originally written by the thesis author in an unpublished manuscript.

Publication III: “Conditionally Optimal Parallel Coloring of Forests”

The idea of utilizing balanced exponentiation to solve the coloring problem was conceived jointly by Christoph Grunau and the author of this thesis.

The solution required generalizing balanced exponentiation, which was done by the thesis author, who wrote approximately 60% of the paper. The techniques for constructing and merging the necessary graph decompositions were developed collectively by all authors, with most of the writing carried out by Christoph Grunau. The introduction and motivation sections were written by Jara Uitto and Yannic Maus.

Publication IV: “Adaptive Massively Parallel Coloring in Sparse Graphs”

The author of this thesis served as the main author of the paper, writing approximately 80% of it. The thesis author developed most of the algorithmic ideas, including the key technique of coin-dropping along with its critical forwarding rules. Formulating the technique as an LCA was a joint effort by Jara Uitto and Yannic Maus. The section on various trade-offs between the number of colors and runtime was written by Shreyas Pai.

List of Figures

1.1	A colored map of South African provinces and its graph representation.	12
1.2	Graph G_1 is colored incorrectly and graph G_2 is colored correctly.	13
1.3	The immediate neighborhoods of every node in G_1 . The upper left-hand corner corresponds to the neighborhood of node 1, the upper right-hand corner to node 2, and so on. The neighborhoods with conflicting colors are highlighted.	14
1.4	The immediate neighborhoods of every node in G_2 . The upper left-hand corner corresponds to the neighborhood of node 1, the upper right-hand corner to node 2, and so on. .	15
1.5	Two different maximal matchings for the same graph, where dashed edges represent matched neighbors. The matching on the left is maximum, while that on the right is not. . . .	16
5.1	(Publication II, Figure 1) Light subtrees highlighted in green.	44
6.1	(Publication IV, Figure 1) An illustration of a β -partition where most nodes are assigned a layer.	50
6.2	(Publication IV, Figures 2 & 3) Examples of different dependency graphs.	50

1. Introduction

A *graph* is a mathematical structure consisting of *nodes* (or *vertices*) connected by *edges*, used to represent relationships between objects. If two nodes share an edge, they are called *neighbors*. The *topology* of a graph is the arrangement of its elements, and can be derived from structures such as social networks, transportation systems, molecular structures, or maps.

1.1 The Graph Coloring Problem

The *graph coloring problem* asks to color all nodes with a distinct color (e.g., green, blue, or yellow) such that all neighbors have different colors, while using as few colors as possible. When deriving a graph topology from a map, every country is represented by a node, and if two countries share a border, the corresponding nodes are joined by an edge. If a graph represents a map, its coloring corresponds to coloring every country on the map such that countries sharing a border have different colors. As an example, we color a map of South African provinces and show that it is equivalent to coloring its graph representation in Figure 1.1.

A trivial solution to the coloring problem would be to use a unique color for every node, in which case it is obvious that all neighboring nodes will have different colors. While this solution would be correct, it would use an excessive number of colors — the same number as there are nodes in the graph. Hence, the real challenge lies in using as few colors as possible, regardless of the number of nodes in the graph.

Coloring is often encountered in practice, without even thinking about it in terms of nodes and edges. The last time you saw a colored world map, neighboring countries were most likely colored with different colors for clarity. But how many colors were used? An even more intriguing question is, what is the smallest number of colors one has to use? An old graph theoretical result from 1890 by [Hea90] shows that 5 colors suffice for any imaginable map. This result was later improved to just 4 colors by [AH76], being the first major result with a computer-aided proof.

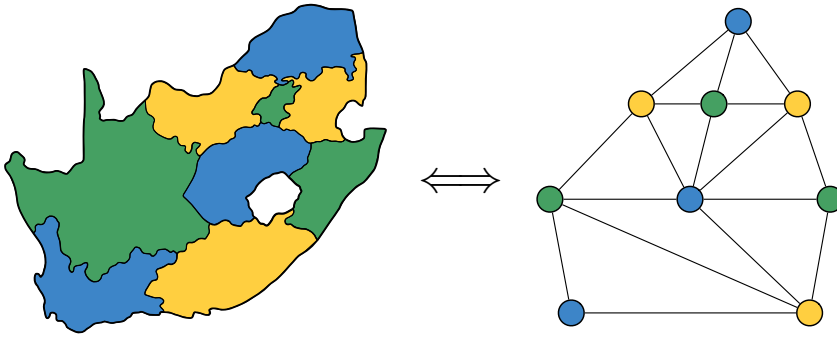


Figure 1.1. A colored map of South African provinces and its graph representation.

While coloring maps is great fun, it is not particularly useful in the grand scheme of things. Luckily, the coloring problem makes perfect sense for all kinds of graph structures, not only for those derived from maps. This makes graph coloring an important subroutine for solving advanced problems across computer science and other disciplines such as statistical mechanics, bioinformatics, and resource scheduling.

From a theoretical point of view, graph coloring is a fascinating problem in itself. The focus is often on minimizing the number of colors used. Let Δ denote the maximum number of neighbors any node has. A trivial greedy approach shows that every graph can be colored with $(\Delta + 1)$ colors: fix a color palette $1, \dots, \Delta + 1$, then visit every node, one by one, and color it with some color from the palette that is not used by any of its neighbors. This approach relies heavily on having exactly $\Delta + 1$ colors, and it fails when only Δ colors are available. This restriction is addressed by Brooks' Theorem, which gives a clean characterization of graphs that admit a Δ -coloring [Bro41; Lov75]. In general, coloring a graph with the minimum number of colors, whatever that number may be, is shown to be an NP-hard (essentially 'extremely difficult') problem by [Kar72].

In the context of distributed or parallel computing (defined formally in Chapter 2), finding a Δ -coloring is significantly more difficult than finding a $(\Delta + 1)$ -coloring; the latter can be solved by local greedy choices, whereas the former requires navigating global constraints.

Besides restricting the number of colors, one could also focus on certain types of graphs. How difficult is it to color a *path*, a *cycle*, or a *tree* graph? We will define these different graphs in Chapter 2.

Graph coloring is a *local problem*, in the sense that the correctness of a solution can be checked by inspecting only the immediate neighborhoods of every node in the graph. Problems that follow this general line of thought are called *locally checkable labeling problems* (LCLs), formally defined in Section 2.3. In addition to coloring, this umbrella term encompasses problems such as *maximal matching* and *maximal independent set*, which

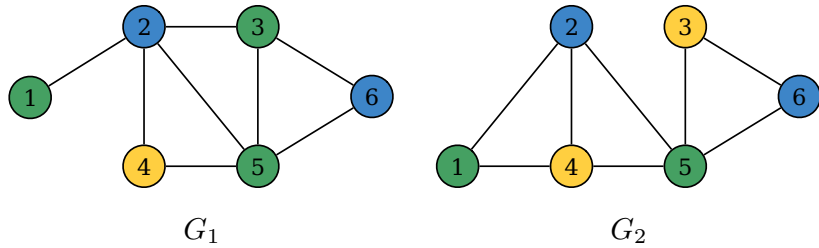


Figure 1.2. Graph G_1 is colored incorrectly and graph G_2 is colored correctly.

we will define in Section 1.2. What these problems have in common is that their solutions consist of labeling all nodes and/or edges with some label, and that the validity of any proposed solution can be verified by inspecting the local neighborhoods of every node.

On the other hand, if a problem has to satisfy some condition that depends on the whole topology of the graph, inspecting only the local neighborhoods of every node is not sufficient to validate a solution. Such problems are called *global*. In Section 1.3, we will define problems such as *maximum matching*, *maximum independent set*, and *connectivity*, which are global and whose candidate solutions may look valid in the local neighborhoods of every node, but could nevertheless turn out to be invalid when considering the whole graph.

This thesis is dedicated to solving problems from both ends of the locality spectrum by developing novel techniques in a modern parallel computing framework called the *Massively Parallel Computation (MPC) model*, defined formally in Section 2.5.

1.2 Local Problems

The protagonist of this thesis is graph coloring, which is the canonical example of an LCL problem. A coloring is correct, or is proper, if the color of every node differs from the color of its neighbors. Otherwise, the coloring is incorrect. Figure 1.2 presents a concrete example: a graph G_1 that is colored incorrectly and a graph G_2 that is colored correctly.

A candidate solution can be verified by only checking the immediate neighborhoods surrounding every node in the graph. If the solution is “locally” correct, meaning that the solution looks correct around every node in the graph, then the solution is also “globally” correct, and vice versa. Conversely, if there exists at least one node around which the solution looks invalid, then the whole solution is invalid. More precisely, we can validate a solution by inspecting and approving, one by one, the immediate neighborhood of every node in the graph. We illustrate this explicitly for graph G_1 in Figure 1.3 and for graph G_2 in Figure 1.4. We can do

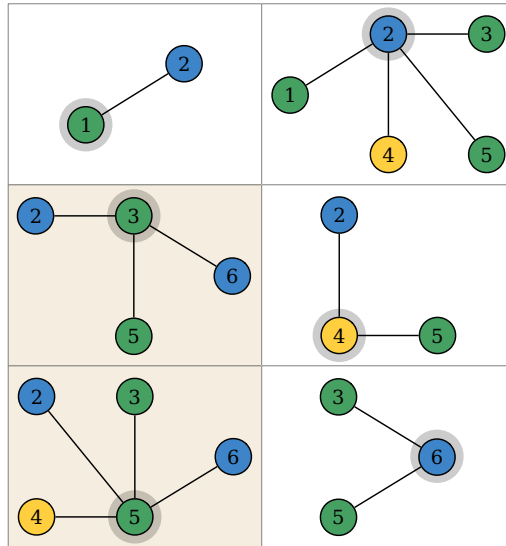


Figure 1.3. The immediate neighborhoods of every node in G_1 . The upper left-hand corner corresponds to the neighborhood of node 1, the upper right-hand corner to node 2, and so on. The neighborhoods with conflicting colors are highlighted.

this because the correctness requirement of coloring is local: if a node is colored blue, we only care that its neighbors are not colored blue, and we don't care about how the rest of the graph is colored (for the sake of this particular node). Figure 1.3 shows that the coloring of G_1 is incorrect, since the neighborhoods of nodes 3 and 5 reveal a conflicting coloring. Using the same verification method, Figure 1.4 shows that the coloring of G_2 is correct.

Another example of an LCL problem is the *maximal independent set* problem. Here, the goal is to select a set of nodes such that there are no neighboring nodes within this set, and no nodes can be added to the set without violating the aforementioned condition. In other words, every node in the graph either belongs to the maximal independent set or one of its neighbors belongs to it.

Much like the maximal independent set problem, but defined over edges rather than nodes, is the *maximal matching* problem. In this setting, the goal is to match neighboring nodes together, such that every node takes part in only one matching, and no additional matches can be made. In other words, neighboring nodes are either matched together or at least one of them is matched with some other node. See Figure 1.5 for an example.

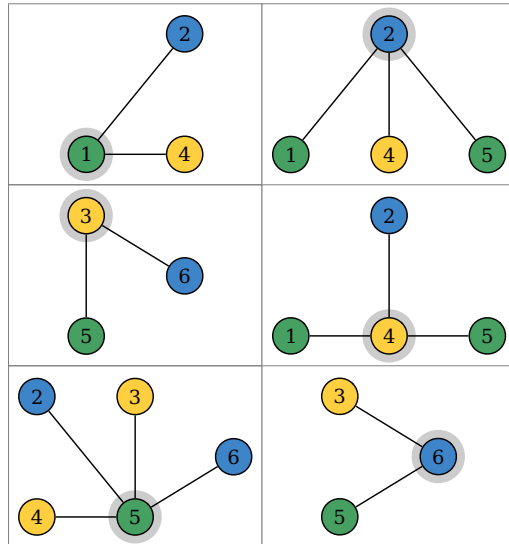


Figure 1.4. The immediate neighborhoods of every node in G_2 . The upper left-hand corner corresponds to the neighborhood of node 1, the upper right-hand corner to node 2, and so on.

1.3 Global Problems

Global problems have some inherently global conditions baked in that they are meant to satisfy, making it impossible to verify a candidate solution solely based on the local neighborhoods of the nodes.

Let us dissect a popular global problem — *maximum matching*. Note that there is a huge difference between maximal matching, which is an LCL problem, and maximum matching, which is a global problem. A maximal matching is maximum if the number of matches in the graph is the most you can have for a given graph. In Figure 1.5 we see two maximal matchings for the same graph. The left solution has 3 matches, while the right side has 2. One can prove that for this graph, 3 matches is the most you can have. Hence, the matching on the left side is maximum, while the one on the right is simply maximal. It is straightforward to realize that a candidate solution to the maximum matching problem can only be verified by looking at the whole graph, and not by inspecting the local neighborhoods.

Similarly to maximum matching, the *maximum independent set* problem is a global problem, whereas the maximal independent set problem is an LCL problem. A maximum independent set is a maximal independent set that is the largest you can have for a given graph.

Another example of a global problem is *connectivity*, where the goal is to assign a unique identifier to each *connected component* of the graph, which is a part of the graph where every node is connected via some series of edges. Specifically, two nodes must receive the same identifier if and only if

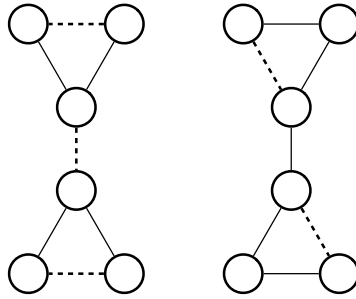


Figure 1.5. Two different maximal matchings for the same graph, where dashed edges represent matched neighbors. The matching on the left is maximum, while that on the right is not.

they belong to the same connected component. Connectivity is not an LCL problem: since a local neighborhood only ever contains nodes from the same connected component, it is impossible to verify (using local neighborhoods) whether nodes in different components have received different identifiers.

1.4 How to Solve Graph Problems?

So far, we have introduced graph problems and their solutions, without really diving into the algorithmic means to obtain them. What even is an *algorithm*? An algorithm is a precise sequence of instructions that tells a computer how to solve a problem. It entails a computational model, which is a set of ground rules to be followed when working on the solution, and some metric based on which the quality of an algorithm is evaluated. This metric is referred to as *computational complexity* or simply *complexity*, and it measures the amount of resources required to execute an algorithm.

The resource that is most commonly considered is *time*, which in itself is quite ambiguous. The earthly units of time, such as seconds and minutes, are not used to measure time complexity, since they depend on the choice of a specific computer and the evolution of hardware. For instance, a specific algorithm will run much faster on a modern computer than on one from the 90s. This, however, is not a virtue of the algorithm itself, but rather a consequence of technological advances in computer architecture and software. What time complexity aims to capture is the intrinsic time requirements of algorithms — the basic, immutable time units an algorithm would require on any computer. This is measured by tracking how many elementary operations, like summation, multiplication, and comparison, are executed until the algorithm terminates. Other complexities of interest besides time include *memory complexity* (computer memory that is required) and *communication complexity* (the necessary amount of communication between computational entities when there are multiple).

The model of computing that people unconsciously resort to when faced with a graph problem is *centralized*, which is indeed very intuitive. Here, the solver has a God’s eye view of the whole graph, and the complexity measure is time complexity: the number of elementary operations it takes to reach a solution. However, this is by far not the only computational model that makes sense to study.

1.4.1 Distributed Computing Models

In *distributed computing* models like *LOCAL* and *CONGEST*, which we define formally in Section 2.4, the approach to algorithm design is fundamentally different from the centralized setting. Here, every node is thought of as a processor with a unique identification number, and every edge is a communication link over which the endpoints can communicate. The goal is to design a communication protocol (algorithm) that is distributed to every node (processor) to follow and eventually reach a solution. One round of communication allows all nodes to communicate with all of their neighbors once. The complexity measure is communication: the number of communication rounds required to reach a solution.

For example, when solving a graph coloring problem, nodes communicate with their neighbors to exchange information, and eventually every node concludes that for the graph to be properly colored, they have to be labeled with a certain color. In fast *LOCAL* algorithms, nodes can only communicate with nodes that reside in close vicinity, highlighting the central theme of this framework — *locality*. What kind of problems can be solved based only on the information from nearby nodes, and how fast?

There are parallels to be drawn between distributed algorithms and nature. Ant colonies and schools of fish seemingly consist of identical independent entities that communicate only locally, while still achieving complex goals efficiently.

1.4.2 Modern Parallel Computing Models

Modern *parallel computing* models like *Massively Parallel Computation (MPC)* [KSV10; BKS17] and *Adaptive MPC* [Beh+19c] share some similarities with distributed models, but differ fundamentally in their communication capabilities. We define them formally in Sections 2.5 and 2.6. These parallel systems consist of multiple machines, each with a restricted amount of *local space* (or *local memory*), which refers to the amount of memory they have to store data. The amount of memory used by the whole system is called *global space* (or *total space*). Graph problems are solved in such a way that the input is first divided among the machines, so that each machine knows parts of the graph, but no machine knows the topology of the whole graph. One round of communication allows every machine to

communicate with all other machines, as long as the message sizes are small and machines do not exceed their local space. In this setting, there are two complexity measures of interest: how many rounds of communication are required to reach a solution (communication complexity) and how much local and global space is required throughout the algorithm execution (memory complexity).

Note the difference in communication capabilities between these parallel and distributed models. In parallel models, machines can communicate with all other machines, whereas distributed models allow communication only across the edges of a given graph. The main challenge of modern parallel models is hence related to the communication complexity: how to efficiently utilize this all-to-all communication? Another major challenge is related to the memory complexity: how to solve problems space-efficiently, without needing to store too much auxiliary data?

By allowing adaptive queries to a distributed data store on top of the MPC model, we arrive at the Adaptive MPC model, AMPC for short. This model is a practically motivated extension of the MPC model, which takes into account the more advanced capabilities of real-world data centers, while still serving as a clean theoretical framework for analysis.

The MPC model and its variants are inspired by real-life data centers, where communication is the most time-consuming part of the operation, and where memory is a limited resource. They are used as abstractions of modern frameworks of parallel computing, such as Hadoop [Whi09], Spark [Zah+10], MapReduce [DG08], and Dryad [Isa+07].

1.5 Research Output and Roadmap

In this thesis, we consider fundamental graph problems for uniformly sparse graphs in the MPC model, focusing on memory-optimal solutions. We resolve open problems and advance the state of the art by developing novel deterministic algorithms for graph problems such as connectivity, vertex coloring, maximal independent set, and maximal matching.

Most of our algorithms are memory-optimal in two ways. First, the local space of individual machines is strongly sublinear in the size of the input. Second, the total space used by the whole system remains linear in the size of the input — the best one can hope for.

By focusing on uniformly sparse graphs, we are forced to develop novel algorithmic techniques, as current approaches fundamentally do not extend to memory-optimal algorithms for these graphs. Even when not striving for memory-optimal algorithms, these graph families are hard instances to solve, since they form the setting for most known lower bounds in MPC.

Chapter 2 is dedicated to basic graph theoretical notations and the formal definitions of the LOCAL model, the MPC model, and the AMPC model. In

Chapter 3, we give an overview of the contributions of this thesis and the related work. In Chapters 4 to 6 we dive deeper into the individual results of this thesis.

In Chapter 4, we present the results of Publication I, which include exponentially faster algorithms for LCL problems on forests in the MPC model compared to the LOCAL model. In Chapter 5, we present the results of Publication II and Publication III, which give the first deterministic $O(\log D)$ -round algorithm for connectivity, as well as the first $O(\log \log n)$ - and $O(\log D)$ -round algorithms for 3-coloring, maximal independent set, and maximal matching on forests. Here, n denotes the total number of nodes in the graph, and D denotes the maximum diameter of the graph. Publication II and Publication III are grouped because both rely on our novel graph exploration technique called *balanced exponentiation*, which is one of the major technical contributions of this thesis. This technique has been developed into a parameterized, standalone tool that allows nodes to explore surrounding sparse regions of the graph without prior knowledge of their location, while incurring minimal memory overhead. Lastly, in Chapter 6, we present the results of Publication IV, which include AMPC algorithms for *arboricity-dependent* colorings. Arboricity is a general measure of the density of a graph, and is defined formally in Chapter 2. We present several algorithms with varying trade-offs between the number of colors and runtime. Most notably, we achieve a constant-round $O(\alpha)$ -coloring algorithm for graphs with bounded arboricity α .

2. Preliminaries

This chapter is not meant to serve as an exhaustive survey of distributed and parallel computing, but rather as prerequisite information for understanding the contributions of this thesis.

The theory of computation focuses on the asymptotic complexity of functions, ignoring constant factors and lower-order functions. Big- O , Little- o , Big- Ω , Little- ω , and Θ are the basic formal notations for stating the growth of a function. Formally,

- $T(n)$ is $O(f(n))$, iff¹ for some constants c and n_0 , $T(n) \leq c \cdot f(n)$, $\forall n \geq n_0$.
- $T(n)$ is $\Omega(f(n))$, iff for some constants c and n_0 , $T(n) \geq c \cdot f(n)$, $\forall n \geq n_0$.
- $T(n)$ is $\Theta(f(n))$, iff $T(n)$ is $O(f(n))$ and $T(n)$ is $\Omega(f(n))$.
- $T(n)$ is $o(f(n))$, iff $T(n)$ is $O(f(n))$ and $T(n)$ is not $\Theta(f(n))$.
- $T(n)$ is $\omega(f(n))$, iff $T(n)$ is $\Omega(f(n))$ and $T(n)$ is not $\Theta(f(n))$.

Informally, Big- O describes the upper bound, Big- Ω describes the lower bound, Θ describes the exact bound, and Little- o (Little- ω) describes the upper (lower) bound that can never be reached.

2.1 Graph Notations

A graph $G = (V, E)$ is a pair of sets V and E , such that $E \subseteq [V]^2$. The elements of V are called nodes or vertices, and the elements of E are called edges. The number of nodes in the graph is denoted by $n = |V|$ and the number of edges in the graph is denoted by $m = |E|$. A common way to picture a graph is by drawing a circle for each node and joining two of these

¹iff stands for “if and only if”

circles with a line if the corresponding two nodes form an edge. For this thesis, we assume that graphs are undirected, finite, and simple.

The set of neighbors of a node $v \in V$ in a graph G is defined as $N_G(v) := \{u \mid \{u, v\} \in E\}$. The degree of a node v refers to the number of neighbors v has in G and is defined as $\deg_G(v) := |N_G(v)|$. The maximum degree of any node in G is denoted by Δ_G . The distance $\text{dist}_G(u, v)$ between two vertices $v, u \in V$ is the length of a shortest path between nodes v and u in G . If no such path exists, we set $\text{dist}_G(v, u) = \infty$. Note that in undirected graphs, it holds that $\text{dist}_G(v, u) = \text{dist}_G(u, v)$. The largest distance between any two vertices in G is the diameter of G , denoted by $\text{diam}(G)$. The r -hop neighborhood of a node v , or in other words an r -radius ball around node v , is the subgraph $B_G(v, r) = (V(v, r), E(v, r))$, where

$$V(v, r) = \{u \in V \mid \text{dist}(u, v) \leq r\}, \text{ and}$$

$$E(v, r) = \{\{u, w\} \in E \mid \text{dist}(v, u) \leq r \text{ and } \text{dist}(v, w) \leq r\}.$$

If the underlying graph G is clear from the context, we may omit the subscript G from all of the notation above.

Consider graphs $G = (V, E)$ and $H = (V', E')$. If $V' \subseteq V$ and $E' \subseteq E$, we say that H is a subgraph of G . If $V' \subseteq V$ and $E' = \{\{u, v\} \in E \mid u, v \in V'\}$, we say that H is a subgraph induced by the node set V' , and it can be written as $G[V']$.

A graph $G = (V, E)$ is called *connected* if $\text{dist}(v, u) \neq \infty, \forall v, u \in V$. If the graph is not connected, it is *disconnected* and consists of multiple *connected components*. A connected graph where all nodes have degree 2 is a *cycle*. A connected graph where two of its nodes have degree 1, and all other nodes have degree 2, is a *path*. A connected graph without cycles as subgraphs is called a *tree*. A disconnected graph consisting of multiple paths/cycles is called a collection of paths/cycles. A disconnected graph consisting of multiple trees is called a *forest*.

Arboricity is a general measure of density. The arboricity α of a graph is defined as the smallest number of forests into which its edge set can be partitioned. More precisely, for any subgraph S of a graph G , let n_S and m_S denote the number of vertices and edges in S . Then the arboricity of G equals

$$\max_S \lceil m_S / (n_S - 1) \rceil, \quad n_S \geq 2.$$

Low-arboricity graphs are uniformly sparse, in the sense that they cannot contain dense subgraphs. Conversely, any graph with high arboricity is guaranteed to contain a dense subgraph.

2.2 Nash-Williams Decomposition

Due to Nash-Williams [Nas64], it is well known that the edge set E of any graph $G = (V, E)$ with arboricity α can be decomposed into α edge-disjoint forests. This fundamental theorem has many applications in graph theory and combinatorics.

The first efficient distributed algorithm for computing such decompositions was given by Barenboim and Elkin [BE08], who coined the term *H-partitions* for the resulting constructions. An *H-partition* entails partitioning the vertex set of the graph into $l = \lfloor 2/\varepsilon \cdot \log n \rfloor$ disjoint subsets $H_1, H_2, \dots, H_l$ that satisfy that every vertex $v \in H_i, i \in \{1, 2, \dots, l\}$, has at most $(2 + \varepsilon)\alpha$ neighbors in the vertex set $\bigcup_{j=i}^l H_j$. It is worth noting that Miller and Reif introduced a related concept in the parallel setting for trees [MR85], where a logarithmic number of so-called *rake* and *compress* operations remove all nodes from a tree.

Many algorithms of this thesis will involve computing *H-partitions*, or suitable variants thereof, for low-arboricity graphs efficiently and in parallel.

2.3 Locally Checkable Labeling (LCL) Problems

In their seminal work [NS93], Naor and Stockmeyer introduced the notion of *locally checkable labeling problems* (LCLs, for short). Their definition focuses on problems where nodes are labeled (e.g., vertex coloring), though they remark that an analogous definition applies to problems where edges are labeled (e.g., edge coloring). A modern formulation that unifies both perspectives and their combinations labels *half-edges*, i.e., pairs (v, e) where e is an edge incident to vertex v . Furthermore, it is known that on trees every LCL problem can be expressed in a special form called *node-edge-checkable LCL problems* [Bal+21; GRB22]. Next, we define half-edge labelings formally and then present this modern definition of LCL problems.

Definition 2.3.1 (Publication I, Definition 3, Half-edge labeling). A half-edge in a graph $G = (V, E)$ is a pair (v, e) , where $v \in V$ is a vertex, and $e \in E$ is an edge incident to v . A half-edge (v, e) is incident to some vertex w if $v = w$. We denote the set of half-edges of G by $H = H(G)$. A half-edge labeling of G with labels from a set Σ is a function $g: H(G) \rightarrow \Sigma$.

We distinguish between two types of half-edge labelings: *input labelings*, which are part of the problem instance, and *output labelings*, which are produced by an algorithm executed on the instance. For our purposes, we assume that any input graph G is equipped with an input labeling $g_{\text{in}}: H(G) \rightarrow \Sigma_{\text{in}}$, where Σ_{in} denotes the *set of input labels*. If an LCL problem has no input labels, we simply take $\Sigma_{\text{in}} = \perp$ and assume that each node is labeled with $\perp$.

Although the formal definition of a node-edge-checkable LCL (see below) may seem complicated, the underlying intuition is simple: we specify (i) a set of allowed output label combinations around nodes, (ii) a set of allowed output label combinations on edges, and (iii) a set of allowed input-output label combinations. A correct solution to the LCL must satisfy all of these constraints.

Definition 2.3.2 (Publication I, Definition 4, Node-edge-checkable LCL).

Let Δ be some non-negative integer constant. A node-edge-checkable LCL is a quintuple $\Pi = (\Sigma_{\text{in}}, \Sigma_{\text{out}}, \mathcal{N}, \mathcal{E}, g)$ where Σ_{in} and Σ_{out} are finite sets, $\mathcal{N} = \{\mathcal{N}_1, \dots, \mathcal{N}_\Delta\}$ consists of sets $\mathcal{N}_i$ of cardinality- i multisets with elements from Σ_{out} , $\mathcal{E}$ is a set of cardinality-2 multisets with elements from Σ_{out} , and $g: \Sigma_{\text{in}} \rightarrow 2^{\Sigma_{\text{out}}}$ is a function mapping input labels to sets of output labels. We call $\mathcal{N}_1 \cup \dots \cup \mathcal{N}_\Delta$ and $\mathcal{E}$ the node constraint and edge constraint of Π , respectively. Furthermore, we call each element of $\mathcal{N}$ a node configuration, and each element of $\mathcal{E}$ an edge configuration. For a node v , denote the half-edges of the form (v, e) for some edge e by $h_1^v, \dots, h_{\deg(v)}^v$ (in arbitrary order). For an edge e , denote the half-edges of the form (v, e) for some node v by h_1^e, h_2^e (in arbitrary order).

A correct solution for Π on a graph G is a half-edge labeling $g_{\text{out}}: H(G) \rightarrow \Sigma_{\text{out}}$ such that

1. for each node v , the multiset of outputs assigned by g_{out} to $h_1^v, \dots, h_{\deg(v)}^v$ is an element of $\mathcal{N}_{\deg(v)}$,
2. for each edge e , the cardinality-2 multiset of outputs assigned by g_{out} to h_1^e, h_2^e is an element of $\mathcal{E}$, and
3. for each half-edge $h \in H(G)$, we have $g_{\text{out}}(h) \in g(\iota)$, where $\iota = g_{\text{in}}(h)$ is the input label assigned to h .

An algorithm $\mathcal{A}$ is said to solve an LCL problem Π on a graph class $\mathcal{G}$ if it produces a correct solution for Π on every $G \in \mathcal{G}$. Note that the definitions of LCL problems above implicitly require that class $\mathcal{G}$ has constant degree.

2.4 The LOCAL Model

Distributed computing [Pel00] studies the capabilities and limitations of a network, where every node is occupied by a computing entity (computer, processor) and messages can be exchanged only over the links of this network. The primary focus of this research is *locality*: what problems can be solved when each processor makes decisions based only on information from nearby nodes.

The LOCAL model [Lin92] has been the standard model for studying locality in distributed computing for over 30 years, see surveys by [Suo13; Roz24]. In this model, a given network is modeled as a connected simple graph $G = (V, E)$, with nodes being processors that can communicate with each other only over the edges of the graph G . Every node has a unique identification number (ID) of size $O(\log n)$ bits, where n is the number of nodes in G . Initially, every node is only aware of its immediate neighbors. The model is synchronous, so the computation proceeds in sequential rounds with all nodes starting at the same time. In each round, every node sends messages to its neighbors, receives messages from its neighbors, and performs some unlimited computation. The message sizes are unbounded, and nodes are assumed to be infinitely powerful, as in any computation finishes instantly. Every node aims to output its part of the solution, e.g., in the case of coloring, nodes only need to output their own color. The communication complexity, or round complexity, of an algorithm is the number of rounds until all nodes have finished. The system is fault-free, meaning that nodes never crash and messages never get corrupted.

There is an important alternative way to view LOCAL algorithms as pointed out by [Lin92]. Any t -round algorithm $\mathcal{A}$ can be simulated by another t -round algorithm $\mathcal{B}$ which proceeds in two distinct phases. First, every node gathers all the information about the network at a distance t around it. Second, every node simulates the behavior of the original algorithm $\mathcal{A}$ without communicating with other nodes. In other words, any t -round LOCAL algorithm is simply a function from the ball $B_G(v, t)$ of radius t around every node $v \in V$, to an output set.

There are two randomized variants of the LOCAL model. One allows private randomness, as in every node is given access to an infinite string of random bits that is guaranteed to be independent of the strings of other nodes in the graph. Another allows shared randomness, where nodes are given simultaneous access to the same infinite string. Both variants have been proven to be stronger models than the deterministic LOCAL model for many problems [Bal+20b; Bal+25].

The LOCAL model is a particularly clean and streamlined model that has been the medium for a very active line of research for over 30 years. However, despite all of its merits, the model has been subject to criticisms as far as practical applications are concerned. Being fault-free, synchronous, and having unbounded message sizes and computational power is not something one can assume of a practical computing system. However, it's worth emphasizing that this apparent weakness is simultaneously a strength: this ensures that any LOCAL lower bounds also hold in strictly weaker models of computing. If some problem is impossible to solve faster than, say, $O(\log n)$ time using unbounded message sizes and computational power in LOCAL, it must also be impossible in models that limit message sizes and computing power.

2.5 The Massively Parallel Computation (MPC) Model

The Massively Parallel Computation (MPC) model [KSV10; BKS17] is a mathematical abstraction of modern frameworks of parallel computing such as Hadoop [Whi09], Spark [Zah+10], MapReduce [DG08], and Dryad [Isa+07]. In the MPC model, there are M machines, each with local space S . Local space refers to the amount of memory each machine has to store data. Machines can communicate with each other in an all-to-all fashion, in synchronous rounds. Within a round, every machine sends messages to any other machine, receives messages from other machines, and then performs some unbounded computation on local data. Initially, an input graph $G = (V, E)$ consisting of n nodes and m edges is arbitrarily and equally distributed among the machines. Every node has a unique identification number (ID) of size $O(\log n)$ bits. The time complexity, or round complexity, of an algorithm is the number of rounds it takes until every machine knows the output for each node and/or edge it holds in memory, e.g., whether or not an edge belongs to the maximal matching.

The MPC model is typically divided into three regimes according to the size of the local space S . The memory unit used is a *word* of size $O(\log n)$ bits, which is enough to store a node or a machine identifier from a polynomial (in n) domain. The *superlinear* regime allows for $S = n^{1+\Omega(1)}$ words of space, and the *linear* regime allows for $S = \tilde{O}(n)$ words². The local space of a machine restricts the amount of data a machine can store during the execution of an algorithm, and hence also restricts the amount of data a machine is allowed to send and receive within one round. Both linear and superlinear regimes allow for very fast algorithms because machines can get a “global view” of the graph in the sense that they can store some information about every node in the graph. Due to the size of ever-growing real-world graphs, it is impractical to assume that a single machine can get such a global view. Hence, the research in recent years has focused on the most challenging *low-space* (*scalable*, *sublinear*) regime with $S = O(n^\delta)$, for some constant $\delta \in (0, 1)$. In this regime, it is possible that a machine cannot even store its immediate neighborhood in its local space. As each machine can only gather a small amount of local graph information around a node, the low-space regime is closely connected to the study of locality and the LOCAL model of distributed computing.

The total space T that the collection of M machines uses has historically been an overlooked metric of the model. However, it has gained traction in recent years, partly due to the contributions of this thesis. The aim is to use as little total space as possible, with linear $T = \Theta(n + m)$ being the gold standard, as $\Omega(n + m)$ words are *required* to even store the input graph.

²The $\tilde{O}$ notation hides polylogarithmic factors, so $\tilde{O}(f(n))$ equals $O(f(n) \cdot \log(f(n)))$.

2.5.1 Node-Centric Approach

To make graph algorithm design simpler in the MPC model, one often adopts a node-centric framework, which has some conceptual connections to the LOCAL model. This approach is common in literature, but is rarely written down explicitly. In this approach, every machine is assumed to simulate several virtual machines, such that every node in the graph is assigned to a unique virtual machine. Now, an algorithm can be designed from a node's perspective, like in the LOCAL model, except that it can communicate with any other node in the graph directly. This approach comes with one major caveat with regard to total space.

We would like to assume that during an algorithm, every virtual machine hosting a node has S local space available. However, if we were to reserve this amount of space for every virtual machine, it would defeat the purpose of having virtual machines, since every node would require its own physical machine, and the total space would always be $\Theta(n \cdot S)$. The workaround for obtaining better total space bounds is as follows. During every round of an algorithm, we bound the total space by $O(\sum_{v \in V} S_v)$, where $S_v = O(S)$ is how much local space the virtual machine hosting a node v is actively using. We assume that the underlying architecture does load balancing for us and re-shuffles the virtual machines between physical machines so that the local space S of every physical machine is not violated.

2.5.2 Graph Exponentiation

A technique called *graph exponentiation* [LW10] is so integral to the MPC model that it is worth defining as a preliminary.

Lemma 2.5.1 (Graph exponentiation). *Let $G = (V, E)$ be a graph. In the low-space MPC model, every node $v \in V$ can collect its 2^r -hop neighborhood $B(v, 2^r)$ in $O(r)$ rounds as long as for every node $v \in V$ it holds that $|B(v, 2^r)| \leq n^\delta$.*

Proof. Initially, every node $v \in V$ knows its 1-hop neighborhood $B(v, 1)$. For the induction step, consider a node u in some round $t < r$ that knows $B(u, 2^t)$. Node u can communicate with every node $v \in B(u, 2^t)$ and acquire their neighborhoods $B(v, 2^t)$, effectively collecting $B(u, 2^{t+1})$. By induction, node u collect $B(u, 2^r)$ after $O(r)$ rounds. $\square$

2.5.3 Lower Bounds and the 1 vs. 2 Cycles Conjecture

All current lower bounds in low-space MPC are conditional, meaning that they hold if some other commonly assumed statement is true. The conjecture, on which essentially all lower bounds in this model condition, is the widely believed 1 vs. 2 cycles conjecture. It states that $\Omega(\log n)$ rounds

are required to distinguish between an n -node cycle and two $n/2$ -node cycles [GKU19].

It is worth noting that establishing unconditional lower bounds appears unattainable. Any unconditional non-constant lower bound in the low-space MPC model for any problem in P would imply a breakthrough result in circuit complexity, namely a separation between NC^1 and P [RVW16], which seems far beyond the reach of current techniques [RR97; AB09].

2.5.4 Component Stability

The concept of component-stable low-space MPC algorithms was introduced by [GKU19] as the first broad class of MPC algorithms for which meaningful conditional lower bounds can be derived. This is achieved by lifting lower bounds in the LOCAL model to the MPC model via a sophisticated lifting framework. In essence, algorithms in this class require that the outputs of nodes belonging to different connected components are independent.

After its introduction, the notion of component stability has been refined by [CDP21a], encompassing many existing randomized MPC algorithms, and allowing for more robust conditional lower bounds. These refinements extend the original framework to also cover deterministic algorithms as well as algorithms whose runtime depends on the maximum degree Δ .

Definition 2.5.2 (Component-stability, [CDP21a]). *A randomized MPC algorithm A_{MPC} is component-stable if its output at any node v is entirely, deterministically, dependent on the topology and IDs (but independent of names) of v 's connected component (which we will denote $CC(v)$), v itself, the exact number of nodes n and maximum degree Δ in the entire input graph, and the input random seed S . That is, the output of A_{MPC} at v can be expressed as a deterministic function $A_{\text{MPC}}(CC(v), v, n, \Delta, S)$. A deterministic MPC algorithm A_{MPC} is component-stable under the same definition, but omitting dependency on the random seed S .*

This thesis adopts the revised version of component stability by [CDP21a].

2.6 The Adaptive MPC Model

The Adaptive MPC (AMPC) model [Beh+19c] is designed to reflect the challenges faced by modern large-scale data processing platforms. It extends the standard MPC model in a practically motivated way: communication takes place through distributed key-value storage rather than all-to-all message passing. In this model, there are P machines, each equipped with $S = O(n^\delta)$ local space where $\delta \in (0, 1)$ and n denotes the number of vertices in the input graph $G = (V, E)$. The total space across all machines is $T = S \cdot P$. The basic unit of measure is a word consisting of $O(\log n)$ bits.

Communication among machines is represented by a sequence of distributed data stores (DDS), denoted by $D_0, D_1, \dots, D_f$. Each DDS supports key-value semantics: it maintains collections of key-value pairs, where querying with a key retrieves its associated value. Both keys and values are of constant size. The initial input resides in D_0 , which is indexed by a set of keys known to all machines, such as consecutive integers. The final solution to the problem must be written into the last data store D_f . If a key x is associated with $k > 1$ values in some DDS, then these values can be accessed via extended keys $(x, 1), \dots, (x, k)$, where the indices from 1 to k are assigned arbitrarily. A query to a key not present in the DDS returns an empty result.

The computation proceeds in synchronous rounds. In round i , each machine reads from D_{i-1} and writes to D_i . During a round, a machine may issue $O(S)$ queries (reads) and perform $O(S)$ writes, in addition to carrying out unrestricted computation on local data. A query with key x corresponds to reading the value linked with x in D_{i-1} , and each write represents storing a single key-value pair in D_i .

The defining feature of the model is *adaptivity*, which distinguishes it from the standard MPC model. Specifically, queries within the same round may depend on one another. For example, suppose g is a function mapping a set X to itself, and for every $x \in X$, the store D_{i-1} contains a pair $(x, g(x))$. Then, in round i , a machine can compute $g^k(y)$ for any $y \in X$, provided $k = O(S)$. A more detailed discussion of the practical realism of the AMPC model can be found in [Beh+19c].

3. Contributions and Related Work

There is a successful line of research on MPC algorithms that are exponentially faster than the best LOCAL algorithms. Most such results are achieved with *superlinear global space*, that is, $\omega(m + n)$ words [Beh+19a; GGJ20; CDP21b; CDP21a], where n is the number of nodes and m the number of edges in the input graph. One of the main themes of this thesis is that we deviate from the usual approach of using superlinear global space, and instead strive for strictly *linear global space*, forcing us to develop novel algorithmic techniques. We restrict ourselves to optimal MPC parameters across the board: we only allow $O(n^\delta)$ words of local space and $O(m + n)$ words of global space. A simple example illustrates why minimizing global space matters: for an input of one million data points, would you prefer storing 10^6 values (linear global space) or 10^{12} (quadratic global space)?

Most of our algorithms are for forests and other low-arboricity graphs. At first glance, it may seem that forests are an easy graph class for which to develop algorithms because they are sparse. We would like to point out that, when striving for linear global space, the converse is actually true. When the input graph is dense, it requires a huge amount of global space to store it, which can be leveraged by an algorithm via *sparsification*. Indeed, a common algorithm design pattern for dense graphs is to first sparsify the input graph, i.e., make the graph less dense, and then solve the problem in this sparser instance [And+18; GU19; CDP21a; CDP20] using superlinear global space with respect to the current graph size, while remaining linear with respect to the original graph size. An additional step is then needed to show how a solution for the original input can be derived from a solution for the sparsified graph. Such techniques are fundamentally incompatible with low-arboricity graphs such as forests, as they are already maximally sparse by definition.

Another compelling reason to focus on trees and forests is their central role in understanding problem complexity in the LOCAL model: many problems of interest are challenging even on trees, and often even in regular balanced trees of small degree. In fact, most known lower bounds in the LOCAL model are established in this setting. Through lifting techniques, these lower

bounds naturally extend to forests in the MPC model. Therefore, decreasing the significance of trees and forests would require a fundamental shift in techniques: either the development of entirely new lower bound frameworks for the LOCAL model, along with new lifting theorems, or novel lower bound techniques tailored directly for the MPC model.

3.1 Exponential Speedup Over Locality

The central result of Publication I is that for any LCL problem (see Section 2.3) on trees with deterministic (respectively, randomized) complexity T in the LOCAL model, we can systematically derive an exponentially faster MPC algorithm with deterministic (respectively, randomized) complexity $O(\log T)$ on forests. While there have been prior works focusing on individual LCL problems in the MPC model, this kind of automated procedure that solves all LCL problems is the first of its kind. In particular, we establish the following.

Theorem 3.1.1. *[Publication I, Theorem 1] Consider an LCL problem on trees with deterministic time complexity $f(n)$ and randomized time complexity $g(n)$ in the LOCAL model. This problem has deterministic time complexity $O(\log f(n))$ and randomized time complexity $O(\log g(n))$ in the low-space MPC model on forests using optimal $O(m + n)$ words of global memory. The provided algorithms are component-stable.*

A rich line of work [NS93; Cha+19b; CP17; Bal+19; Bal+18; Bal+20a; Cha20; GRB22] recently came to an end, showing that a problem in the LOCAL model can only have deterministic complexity

$$f(n) \in \{\Theta(1), \Theta(\log^* n), \Theta(\log n)\} \cup \{\Theta(n^{1/k}) : k \in \mathbb{N}\}.$$

We show that knowing the asymptotic value of $f(n)$ is enough to obtain a deterministic MPC algorithm with complexity

$$O(\log(f(n))) \in \{O(1), O(\log \log^* n), O(\log \log n), O(\log n)\}.$$

Furthermore, it is known that for all complexities $f(n) \notin \Theta(\log n)$, the randomized and deterministic LOCAL complexities coincide. However, when $f(n) \in \Theta(\log n)$, the randomized LOCAL complexity $g(n)$ can be either $\Theta(\log n)$ or $\Theta(\log \log n)$. In the latter case, we present an MPC algorithm with randomized complexity $O(\log \log \log n)$. If the component-stability (see Section 2.5.4) condition is dropped, the same $O(\log \log \log n)$ complexity can be achieved deterministically in the MPC model.

Theorem 3.1.2. *[Publication I, Theorem 2] Consider an LCL problem on trees with randomized time complexity $g(n) = \Theta(\log \log n)$ in the LOCAL model. This problem has deterministic time complexity $O(\log \log \log n)$ in*

the low-space MPC model on forests using optimal $O(m+n)$ words of global memory. This algorithm is component-unstable.

The work of [GKU19] (Theorem I.4) and [CDP21a] (Theorem 14) show that Theorem 3.1.1 is optimal for LCL problems with complexity $O(\log n)$ in the LOCAL model: if a problem requires T rounds in the LOCAL model, then any component-stable algorithm in the low-space MPC model requires $\Omega(\log T)$ rounds — assuming the widely believed 1 vs. 2 cycles conjecture holds (see Section 2.5.3).

Consider an LCL problem $\mathcal{P}$ with deterministic complexity $\Theta(\log n)$ and randomized complexity $\Theta(\log \log n)$ in the LOCAL model. By [CDP21a], problem $\mathcal{P}$ has a low-space MPC lower bound of $\Omega(\log \log n)$ for deterministic component-stable algorithms. In Theorem 3.1.2, we show that a component-unstable algorithm can solve problem $\mathcal{P}$ deterministically in $O(\log \log \log n)$ rounds. In other words, we show that the lower bounds obtained by the lifting framework of [GKU19] can be bypassed for all LCL problems in the aforementioned complexity class with component-unstable algorithms. Obtaining the same improvement for even a single problem while preserving component stability would constitute a breakthrough, as it would refute the 1 vs. 2 cycles conjecture mentioned in Section 2.5.3.

As a key subroutine for speeding up problems to $O(\log n)$ complexity in the MPC model, we provide a component-stable MPC algorithm for rooting a forest in $O(\log n)$ rounds. This algorithm supports arbitrary degrees and may be of independent interest.

3.2 Forest Connectivity

The primary contribution of Publication II is a deterministic algorithm that detects the connected components of a forest in time logarithmic in the maximum diameter of any component. Crucially, without any dependence on the total number of nodes n .

The problem of connectivity has attracted considerable attention in recent years. Most notably by [And+18] who developed a randomized $O(\log D \cdot \log \log n)$ algorithm for general graphs and by [Beh+19b] who achieved a randomized $O(\log D + \log \log n)$ algorithm for general graphs. Later, [CC22] derandomized the work of [Beh+19b], achieving the same runtime. Another notable result is by [ASW19] who designed an $O(\log \log n + \log(1/\lambda))$ -round algorithm that finds all connected components with spectral gap at least λ .

Our result contrasts these prior approaches, where the runtime seems to inherently depend on n . Our algorithm is deterministic and has a runtime of $O(\log D)$ for forests. Moreover, we prove that our algorithm is asymptotically optimal under the 1 vs. 2 cycles conjecture. Formally, we prove the following statement.

Theorem 3.2.1. *[Publication II, Theorem 1.1] Consider the family of forests. There is a deterministic low-space MPC algorithm to detect the connected components on this family of graphs. In particular, each node learns the maximum ID of its component. The algorithm runs in $O(\log D)$ rounds, where D is the maximum diameter of any component. The algorithm requires $O(n + m)$ words of global memory, it is component-stable, and it does not need to know D . Under the 1 vs. 2 cycles conjecture, the runtime is asymptotically optimal.*

We can extend the techniques from Theorem 3.2.1 to also obtain a rooting algorithm with the same time and space complexity guarantees.

Theorem 3.2.2. *[Publication II, Theorem 1.2] Consider the family of forests with component-wise maximum diameter D . There is a deterministic low-space MPC algorithm that roots the forest in $O(\log D)$ rounds using $O(n + m)$ words of global memory, and it is component-stable.*

While rooting a forest is not particularly groundbreaking, it does provide a convenient handle for algorithm design and symmetry breaking in low-space MPC. As a concrete example, consider the 2-coloring problem. Without a rooting in place, this is a daunting task. However, if you can invoke a $O(\log D)$ time rooting algorithm as a black-box, the task of 2-coloring in $O(\log D)$ time becomes almost trivial. Without going into technical details, this can be achieved using a rather simple and efficient algorithm: each node is colored based on the parity of its distance to the root, which can be computed via *pointer jumping*, which is a similar technique to graph exponentiation of Lemma 2.5.1.

Observe that the rooting algorithm in Theorem 3.2.2 supports *arbitrary* degrees, making it the only known $O(\log D)$ time forest rooting algorithm. Prior connectivity algorithms for general graphs by [Beh+19b; CC22] yield slightly slower rooting algorithms, achieving a runtime of $O(\log D \cdot \log \log n)$.

Our final contribution is a component-stable algorithm for solving any LCL problem (which includes 2-coloring) deterministically in $O(\log D)$ time. Our result employs the rooting algorithm of Theorem 3.2.2 as a subroutine. Furthermore, we show that if $D \in \Omega(\log n)$ and $D \in n^{o(1)}$, there cannot exist (under the 1 vs. 2 cycles conjecture) an MPC algorithm that solves all LCL problems in time $o(\log D)$ on forests with component-wise maximum diameter D . This claim holds even for component-unstable algorithms and even when allowing $\text{poly}(n)$ global space. This implies that our $O(\log D)$ time LCL solver is conditionally optimal.

Theorem 3.2.3 (Publication II, Theorem 1.3). *Consider an LCL problem Π on forests and let D be the component-wise maximum diameter. There is a deterministic low-space MPC algorithm that solves Π in $O(\log D)$ rounds using $O(n + m)$ words of global memory. Under the 1 vs. 2 cycles conjecture, the runtime is asymptotically optimal.*

3.3 Coloring, MIS and Maximal Matching

Since LCL problems are limited to graphs with only constant degree by definition, a very natural question to ask is how fast can we solve classical symmetry-breaking problems such as coloring, maximal independent set (MIS), and maximal matching on forests when degrees are arbitrary?

A long line of research has studied this question [BFU19; Beh+19a; LU21; GGJ20; FGG23]. In Publication III, we consolidate and unify these works by presenting a conceptually simple deterministic algorithm that simultaneously solves 3-coloring, MIS, and maximal matching. Our approach is based on computing a variant of the classical H -partition (see Section 2.2), which provides a unified framework for addressing these problems. While H -partitions are a natural tool for trees and forests, efficiently computing them in the MPC model had remained beyond the reach of prior techniques.

Theorem 3.3.1 (Publication III, Theorem 1). *There are deterministic $O(\log \log n)$ -round low-space MPC algorithms for 3-coloring, maximal matching, and maximal independent set (MIS) on forests. These algorithms use $O(n)$ global space.*

Our algorithms are conditionally optimal under the 1 vs. 2 cycles conjecture, at least when restricted to the class of component-stable algorithms.

Algorithms achieving similar guarantees for maximal matching and maximal independent set on forests were recently obtained by [FGG23]. However, their methods are highly technical, and they rely on sophisticated derandomization machinery. Moreover, their techniques inherently cannot extend to coloring with a small number of colors. Our main result, which is 3-coloring, is widely considered to be harder than MIS and maximal matching. For example, once a graph is 3-colored, an MIS can be trivially computed in constant time. Furthermore, prior MPC algorithms for MIS and maximal matching crucially exploit the property that any partial solution can be extended to a full solution for the entire graph — a property that does not hold for 3-coloring.

The best previously known deterministic algorithm for 3-coloring trees, due to the author of this thesis in an unpublished manuscript [LU21], achieves a runtime of $O(\log^2 \log n)$. Allowing one extra color and randomization makes the problem substantially easier: the best such algorithm [GGJ20] uses 4 colors and runs in $O(\log \log n)$ rounds using a divide-and-conquer strategy. Each node joins one of two partitions uniformly at random, creating induced connected components of logarithmic diameter. Due to their small diameter, each component can then be 2-colored independently in $O(\log \log n)$ rounds in a straightforward manner.

How is this different to LCLs? It is worth emphasizing that the problems addressed in Theorem 3.3.1 are not LCL problems because the degrees of the graphs can be arbitrary, whereas LCL problems are only defined

for constant-degree graphs. This seemingly minor change in the initial conditions leads to a fundamentally harder algorithmic setting.

One might ask whether the techniques developed in Publication I can be ported to this setting? Unfortunately, the answer is no. The underlying techniques in Publication I are either too slow or heavily exploit the fact that nodes have constant degree.

Another avenue of approach could be to employ the LCL solver from Publication II (Theorem 3.2.3), since under the hood, this solver can handle arbitrary degrees. However, this approach is also too slow. The solver runs in $O(\log D)$ rounds, where D is the component-wise maximum diameter, which in the worst case, can result in an $O(\log n)$ runtime.

3.4 Coloring Sparse Graphs in AMPC

The problem of finding arboricity-dependent colorings has received significant attention across various settings, including LOCAL [BE08; BE10; Bar15; FHK16; Mau21; MT20], MPC [GS19; FGG23; BCG20; GG25], dynamic algorithms [HNW20; CNR23; CR22; Bha+24], and streaming algorithms [BCG20]. Obtaining an $O(\alpha)$ -coloring on graphs with arboricity α efficiently remains a challenging open problem in low-space MPC. A step towards solving this open problem was taken by [GG25], who gave a $\text{poly}(\log \log n)$ algorithm for $O(\alpha \log \log n)$ -coloring. The solution relies on computing an edge orientation such that the maximum out-degree of any node is $O(\alpha \log \log n)$.

By shifting from the MPC to the AMPC model, we gain a new avenue of attack. While a randomized constant-time¹ $(\Delta + 1)$ -coloring algorithm in the AMPC model is already known [Cha+19a], this number of colors is often far from optimal for sparse graphs, where Δ can be significantly larger than α . In Publication IV, we present deterministic AMPC algorithms that in constant, or near-constant, time give $\text{poly } \alpha$ - and $O(\alpha)$ -colorings for sparse graphs.

Theorem 3.4.1 (Publication IV, Theorem 1.3). *For any constant $\varepsilon > 0$ on graphs with possibly non-constant arboricity α , there are deterministic low-space AMPC algorithms to compute:*

1. An $O(\alpha^{2+\varepsilon})$ -coloring in $O(1/\varepsilon)$ rounds.
2. An $O(\alpha^2)$ -coloring in $O(\log \alpha)$ rounds.
3. A $((2 + \varepsilon)\alpha + 1)$ -coloring in $\tilde{O}(\alpha/\varepsilon)$ rounds.

¹Under the assumption that Δ is a small enough polynomial in n .

Although Theorem 3.4.1 offers several trade-offs between the number of colors and runtime, its main strength is captured in the following corollary.

Corollary 3.4.2 (Publication IV, Corollary 1.4). *For any constant $\varepsilon > 0$, there is a constant time AMPC algorithm to compute a $((2+\varepsilon)\alpha+1)$ -coloring on any graph with bounded arboricity $\alpha = O(1)$.*

A standard and powerful approach for computing arboricity-dependent colorings is based on H -partitions (see Section 2.2), which define an acyclic edge orientation where each node has bounded out-degree. Our main technical contribution is designing efficient deterministic algorithms to construct such orientations, and showing how to leverage them to obtain colorings in AMPC.

Why using less than $\Delta + 1$ colors is hard? The standard approaches for $(\Delta + 1)$ -coloring are essentially oblivious to the graph’s topology. They rely on the fact that *any* partial coloring can always be extended to a valid coloring of the entire graph. However, this property no longer holds when aiming for colorings with only $O(\alpha)$ (or similarly small in terms of α) colors, which is typically far fewer than $\Delta + 1$. In such cases, extending partial colorings may be impossible, giving rise to intricate dependencies that must be carefully resolved.

4. Exponential Speedup Over Locality

In this chapter, we give an overview of the main techniques used to address the challenges encountered in Publication I. Earlier work has established that in the LOCAL model, the deterministic complexity of a problem on trees can only take the following forms:

$$f(n) \in \{\Theta(1), \Theta(\log^* n), \Theta(\log n)\} \cup \{\Theta(n^{1/k}) \mid k \in \mathbb{N}\}.$$

We show that knowing the asymptotic behavior of $f(n)$ is sufficient to design a deterministic MPC algorithm with runtime

$$O(\log(f(n))) \in \{O(1), O(\log \log^* n), O(\log \log n), O(\log n)\}.$$

It is known that for all $f(n) \notin \Theta(\log n)$, the randomized and deterministic LOCAL complexities coincide. In contrast, when $f(n) \in \Theta(\log n)$, the randomized complexity $g(n)$ in the LOCAL model can be either $\Theta(\log n)$ or $\Theta(\log \log n)$. In the latter case, we design an MPC algorithm with randomized complexity $O(\log \log \log n)$.

Our central result can be summarized as follows.

Theorem 3.1.1. *[Publication I, Theorem 1] Consider an LCL problem on trees with deterministic time complexity $f(n)$ and randomized time complexity $g(n)$ in the LOCAL model. This problem has deterministic time complexity $O(\log f(n))$ and randomized time complexity $O(\log g(n))$ in the low-space MPC model on forests using optimal $O(m + n)$ words of global memory. The provided algorithms are component-stable.*

We group these runtimes into four distinct regimes:

- the tiny-regime: $f(n) \in \{\Theta(1), \Theta(\log^* n)\}$,
- the low-regime: $f(n) = \Theta(\log n)$,
- the mid-regime: $g(n) = \Theta(\log \log n)$,
- the high-regime: $f(n) = \Theta(n^{1/k}), \forall k \in \mathbb{N}$.

The tiny-regime is the simplest case, which still illustrates the difficulties in staying within the optimal global space bound. The most technically demanding case is the high-regime, where we establish that every LCL problem admits a deterministic $O(\log n)$ -round MPC algorithm.

Graph exponentiation. A central obstacle across all regimes is the restriction to linear global space, which in practice means that each node is limited to using only a constant amount of space on average. This is particularly problematic because nearly all recent advances in MPC algorithms, especially those achieving exponential speedups, depend heavily on the memory-intensive technique of graph exponentiation (see Lemma 2.5.1). Informally, this method allows a node to collect its 2^k -hop neighborhood within k communication rounds. However, performing this in parallel for all nodes incurs a global space overhead of Δ^{2^k} . Almost all effective algorithms need $k = \omega(1)$, which results in a super-constant multiplicative increase in global space. In order to use this technique effectively while using only linear global space, we develop new techniques outlined in the following sections.

4.1 Tiny-Regime $f(n) = \Theta(1)$ and $f(n) = \Theta(\log^* n)$

For $f(n) = \Theta(1)$, the approach is straightforward: any LOCAL algorithm for LCL problems can be directly simulated in the MPC model. For $f(n) = \Theta(\log^* n)$, prior work has shown that such problems in the LOCAL model can be solved by reducing them to computing a distance- k coloring, where k is a constant depending on the specific problem. In a distance- k c -coloring, each vertex is assigned a color from $\{1, \dots, c\}$ such that no two vertices within distance k share the same color. This coloring can be obtained in $O(\log^* n)$ rounds in the LOCAL model. Using graph exponentiation, the same can be achieved in the MPC model in $O(\log \log^* n)$ rounds, but this requires an additional factor of $O(\log^* n)$ in global space.

We show that this space overhead is not necessary. It is known that, in the LOCAL model, coloring general graphs can be reduced to coloring directed pseudoforests, i.e., graphs in which every edge is oriented and each node has at most one outgoing edge. We give a memory-efficient MPC algorithm for coloring such structures using a variant of graph exponentiation that only requires keeping track of a constant number of IDs. As a result, each node uses constant memory, and the overall space requirement remains linear.

4.2 High-Regime $f(n) = \Theta(n^{1/k})$, for all $k \in \mathbb{N}$

For every solvable LCL problem, we design a novel algorithm with runtime $O(\log n)$. The algorithm processes each tree in the input forest independently, so it suffices to consider the case of trees. On a high level, we first root the tree in $O(\log n)$ rounds, and then solve the problem in two phases:

1. Working bottom-up from the leaves, we compute, for a substantial number of nodes v , the set of output labels that could be assigned to v such that any assignment would extend to a valid solution within the subtree rooted at v .
2. Using the information from Phase 1, we construct a valid solution top-down, starting from the root.

While this outline appears simple, there are several technical difficulties. For example, the input tree may have depth $\omega(\log n)$, preventing a purely sequential execution of the above idea. Furthermore, storing the required compatibility information using standard graph exponentiation causes an exponential blowup in memory requirements, violating the linear global space constraint. The key technique for resolving these challenges is a method for interleaving graph exponentiation with graph compression, while also storing compatibility information in a reversible way (so it can be reused in Phase 2). The most difficult aspect of this approach is that exponentiation processes on different subtrees must be merged — sometimes simultaneously, sometimes at different times — into a single coherent process. In order to analyze the resulting complex and highly non-sequential algorithm, we develop a finely tuned potential function.

4.3 Mid-Regime $f(n) = \Theta(\log n)$

Ideally, one would hope to use the algorithm of Chang and Pettie [CP17] as a black box. On a high level, their LOCAL algorithm constructs a rake-and-compress decomposition of depth $O(\log n)$, resembling the classical decomposition introduced by Miller and Reif [MR85]. Compatibility information is then propagated upward layer by layer, labels are fixed at the top, and the assignments are propagated downward.

However, directly applying this algorithm in the MPC model does not work out of the box. First, the compatibility information grows exponentially, which causes congestion in MPC communication. Second, a straightforward application of graph exponentiation would exceed the linear global space limit. We overcome these barriers as follows. We show that the compatibility information can be reduced to a constant size at each iteration, preventing

congestion. We interleave graph exponentiation with memory-releasing steps in a balanced fashion, ensuring that the overall space usage remains linear.

4.4 Low-Regime $g(n) = \Theta(\log \log n)$

For this regime, prior work by [Bal+21] shows a constant-time reduction to instances of size $N = \log n$, and gives a LOCAL algorithm with runtime $\text{poly}(\log N) = \text{poly}(\log \log n)$. With a naïve application of graph exponentiation, this reduction translates into an MPC algorithm with runtime $O(\log \log \log n)$ and an $O(\log n)$ multiplicative factor in global space.

Without using additional global space, however, solving these reduced instances within triple-logarithmic time is more challenging. Our solution is to apply the algorithm developed for the mid-regime to the reduced instances, yielding a memory-efficient algorithm with runtime $O(\log \log \log n)$. To the best of our knowledge, no prior work is capable of efficiently handling such small instances under strict global space constraints.

5. The Balanced Exponentiation Technique

This chapter introduces the *balanced exponentiation* technique, explaining its origins, motivation, and applications. The method was first developed for solving connectivity in Publication II, was later adapted for coloring forests with arbitrary degree in Publication III, and was the inspiration for the coin-dropping game used in arboricity-based colorings (see Chapter 6) in Publication IV.

5.1 Forest Connectivity

Our goal is to design a deterministic algorithm for connectivity on forests with runtime $O(\log D)$, where D denotes the maximum diameter of any connected component. Prior algorithms for general graphs achieve runtimes with an unavoidable dependence on n , the number of nodes, which we are determined to eliminate.

A natural approach for solving connectivity on forests is to root each tree and then identify it by the ID of its root. Rooting, however, already illustrates the inherent difficulty of the problem. A standard rooting method is to iteratively perform rake operations: at each step, every leaf chooses its unique neighbor as their parent. This procedure roots a tree in $O(D)$ parallel rounds, and in a forest, each tree can be rooted independently. If we disregard space constraints, this process could be accelerated to $O(\log D)$ rounds using graph exponentiation: in $O(\log D)$ rounds, every node could collect its D -hop neighborhood, effectively the entire graph. However, in the low-space MPC model, where local and global space are restricted to $O(n^\delta)$ and $O(n + m)$ respectively, this is infeasible. Graph exponentiation requires every node to simultaneously expand its neighborhood, which, under the strict space limit, means that each node can only afford to gather its constant-radius neighborhood. On graphs with arbitrary degrees, even this is not possible, since the degree of a single node could be $\omega(n^\delta)$. As a result, only a constant number of rake steps can be simulated in an MPC round, preventing the desired speedup.

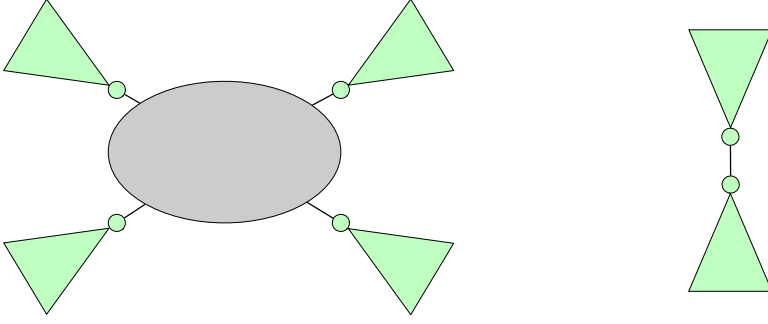


Figure 5.1. (Publication II, Figure 1) Light subtrees highlighted in green.

One might hope to exploit the fact that the ratio of the “original” total space to the number of active nodes increases as we rake the graph, an observation made in earlier work [And+18; Beh+19b; CDP21a]. If each round shrinks the graph by a constant factor, then the average available space per node also increases, potentially enabling nodes to gather progressively larger neighborhoods. Yet, even with such a guarantee, the runtime would remain dependent on n , since progress would still be measured using the size of the graph. This suggests that a fundamentally different approach is needed, rather than one that gradually sparsifies the graph.

The algorithm. The core idea of our algorithm in Publication II is to iteratively compress parts of the graph while preserving connectivity, ensuring that the maximum identifier within each compressed part is retained in the reduced graph. This process is repeated until only a single node remains, which holds the maximum identifier (ID) of the whole connected component. We then reverse the process by iteratively decompressing the graph and propagating ID back to all nodes. Eventually, the original graph is restored, with every node aware of ID.

The two types of structures we compress are light subtrees and paths. The definition of a light subtree (highlighted in green in Figure 5.1) is a bit convoluted, because we are in an unrooted setting. Intuitively, a subtree T_v at a node v is the graph connected to v if we ignore exactly one edge adjacent to v . This subtree is light if $|T_v| \leq n^{\delta/8}$. We refer to v as the root of T_v , and we refer to the nodes in T_v as the descendants of v , even though we do not have any global rooting in place. To make the definition more formal, one needs to include the ignored edge in the notation, which is precisely what we do in the technical parts of the paper.

The algorithm proceeds in $\ell = O(1)$ phases, followed by the same number of reversal phases. In each phase, all light subtrees are compressed into single nodes, and paths are compressed into single edges, yielding a sequence of graphs $G_0, G_1, \dots, G_\ell$. During the reversal phases, the compression steps are undone in reverse order, spreading ID across the graph.

Balanced exponentiation. The main technical challenge lies in compressing light subtrees, or in other words, ensuring that every root of a light subtree can gather the subtree. Tackling this challenge led to the development of the balanced exponentiation technique, which ensures that every root of a light subtree gathers a part of its neighborhood that is balanced in the following sense. If a node u is the root of a light subtree T_u of size $\gamma \leq n^{\delta/8}$, we ensure that u gathers T_u , as well as only $O(\gamma)$ nodes in the direction of the edge that was ignored when defining T_u . Thus, the memory requirement at u is $O(\gamma)$.

A crucial step in our analysis is showing that even if every node gathered its γ descendants and some other $O(\gamma)$ additional nodes, the total space increases by only a multiplicative D factor. The following lemma is central to this argument.

Lemma 5.1.1 (Publication II, Lemma 4.5). *Consider an n -node tree T with diameter D that is rooted at node r . Let $T(v, r)$ denote the subtree rooted at v (including v) when T is rooted at r . It holds that $\sum_{v \in V} |T(v, r)| \leq (D+1) \cdot n$.*

Proof. Consider the unique path P_{rv} from the root r to a node v . Observe that v appears only in the subtrees of the nodes in P_{rv} . Since $|P_{rv}| \leq D+1$, node v is overcounted at most D times, and $\sum_{v \in V} |T(v, r)| \leq (D+1) \cdot n$. $\square$

By the lemma above, if every node u would gather $\min\{n^\delta, O(|T_u|)\}$ information, where T_u is a light subtree rooted at u , the global space would be bounded by $O(m \cdot D)$. With some simple preprocessing, we can eliminate the extra factor of D , ensuring that the algorithm operates within linear global space.

The major crux of this approach is the fact that the input graph is not rooted, so nodes do not know in advance who their descendants are and whether or not they are roots of light subtrees. Nevertheless, we show that, without asymptotic slowdown, each node can deterministically identify its “worst-case” parent choice and gather information accordingly. Using this strategy, nodes with light subtrees can identify them in $O(\log D)$ time, independent of n . Since the number of phases is constant, the overall runtime is $O(\log D)$.

Another issue is that executing balanced exponentiation requires knowing the diameter D , which is not given as input. This issue is resolved by running $O(\log \log n)$ parallel instances of the algorithm with exponentially increasing diameter guesses, one of which will be sufficiently close to the correct value.

5.2 Coloring, MIS and Maximal Matching

In Publication III, we present a unified algorithmic framework that solves 3-coloring, maximal independent set (MIS), and maximal matching in

$O(\log \log n)$ rounds. The core technical ingredient enabling this unification is an efficient procedure for computing H -partitions (see Section 2.2).

Theorem 5.2.1 (Publication III, Theorem 2). *There is a deterministic $O(\log \log n)$ -round low-space MPC algorithm that computes a strict H -decomposition with $O(\log n)$ layers on forests in $O(n)$ global space.*

On forests, an H -partition entails partitioning the vertex set into layers such that each node has at most two neighbors in the same or higher layers. It is known that every forest admits an H -partition with $O(\log n)$ layers. In the LOCAL model, such a decomposition immediately yields an $O(\log n)$ -round algorithm for 3-coloring. Essentially, one can process the layers in reverse order sequentially, coloring all nodes in a layer while avoiding conflicts with already colored neighbors in higher layers.

The novelty of our work does not lie in the use of H -partitions to obtain a 3-coloring — this approach is already well known and even taught in graduate courses — but rather in how we compute the decomposition efficiently. We extend the balanced exponentiation technique, originally developed for our connectivity algorithm in Publication II, and adapt it to this setting. Our method steers each node to explore portions of the graph (largely in an uncoordinated manner) so that it can locally construct a *partial* H -partition. These partial partitions are then combined in a non-trivial way into a consistent global decomposition. To the best of our knowledge, this is the first MPC algorithm for H -partitions that follows such an approach.

Furthermore, we formulate balanced exponentiation as a standalone technique, making it available as a general tool that can be applied to a broader range of problems.

Balanced exponentiation. Our balanced exponentiation procedure enables nodes to explore meaningful, by some metric, parts of the graph, while keeping space usage minimal. To describe this method, we first need to introduce the notion of a *subtree*: a connected subgraph of a tree whose removal does not disconnect the graph. A node is considered *important* if it lies within a subtree of size at most $n^{\delta/8}$. Informally, balanced exponentiation can be summarized as follows (the precise formulation is given in Lemma 5.2.2).

Publication III, page 23:5: *Let $0 < k \leq n^{\delta/8}$ be a parameter. There is a deterministic low-space MPC algorithm that, given an n -node forest F , uses $O(\log k)$ rounds in which every important node $v \in F$ discovers its k -hop neighborhood in every direction of the graph, except for at most one.*

For a node $v \in F$, each neighbor $x \in N(v)$ defines a direction relative to v . The portion of the forest that v can explore in direction x is simply the subgraph of F that is connected to v via x . This subgraph is unique because F is a forest.

The above technique may be of independent interest and could find applications in other graph problems. We obtain it by generalizing the exponentiation technique introduced in Publication II. In that earlier work, the parameter k was fixed to be the maximum diameter of the graph, which enabled an $O(\log D)$ -round algorithm for solving connectivity on forests. The same technique has since been adapted for certain dynamic programming tasks on tree-structured data by [Gup+23] (a publication by the author not included in this thesis), also achieving runtimes logarithmic in the diameter of the graph.

The key advantage of our generalized result is its flexibility: parameter k can be tuned so that both runtime and space remain small whenever nodes require only a limited “view” of the graph. This property plays a central role in our algorithm for computing H -partitions. For completeness, we include the formal statement of this generalized balanced exponentiation procedure below.

Lemma 5.2.2 (Full version of Publication III [Gru+23], Lemma 8.4). *Let $0 < k \leq n^{\delta/8}$ be a parameter that may or may not be constant. Given an n -node forest F , there is a deterministic $O(\log k)$ time low-space MPC algorithm after which the following holds. For every important node $v \in V(F)$, there is a node $z \in N(v)$ and a machine that for all $x \in N(v) \setminus z$ holds $N^k(v) \cap F_{v \rightarrow x}$ in memory. The algorithm requires $O(n \cdot \text{poly}(k))$ total space.*

Our algorithm in a nutshell. As outlined previously, the central technical component of our approach is computing an H -partition of the input forest F . Concretely, we seek a partition

$$V(F) = V_1 \sqcup V_2 \sqcup \dots \sqcup V_L$$

of $L = O(\log n)$ layers such that each node in V_i has at most two neighbors in $\bigcup_{j \geq i} V_j$. A straightforward way to obtain such a decomposition is via an iterative peeling process: in each step, remove all nodes of degree at most 2 and define V_i as the set of nodes removed in the i -th round. A simple counting argument shows that at least half of the remaining nodes are removed in every iteration, which yields a decomposition of $O(\log n)$ layers. Moreover, one can determine the layer of a node solely based on its $O(\log n)$ -hop neighborhood. Hence, if we could store the $O(\log n)$ -neighborhood of each node, the layer assignment could be decided locally without further communication.

However, the space constraints of the low-space MPC model prevent us from gathering the full $O(\log n)$ -neighborhood for every node. To overcome this, we employ the balanced graph exponentiation technique for forests described earlier. When setting parameter k to $O(\log n)$, balanced exponentiation provides a slightly weaker guarantee than collecting the complete $O(\log n)$ neighborhood of every node:

1. It only applies to nodes within small subtrees, specifically of size $\leq n^{\delta/8}$.
2. For such nodes, it gathers the $O(\log n)$ -neighborhood in all directions except one.

We first address the limitation of handling only small subtrees. Suppose we repeatedly remove, from F , all nodes that either belong to a subtree of size at most x or have degree 2 outside said subtrees. Within $O(\log_x n)$ iterations, the entire forest is removed using a simple argument presented in Publication II. Consequently, if we assign nodes in subtrees of size at most $n^{\delta/8}$ together with nodes of degree 2 to one of $O(\log n)$ layers in each round, then after $O(1/\delta)$ such rounds every node has been assigned to some layer. This results in a decomposition of $O((1/\delta) \log n)$ layers. Hence, it suffices to restrict our attention to nodes contained in subtrees of size at most $n^{\delta/8}$ (nodes of degree 2 are trivial to handle).

Addressing the second challenge is more involved. In general, no machine stores the full $O(\log n)$ neighborhood of any node, making it impossible to directly replicate the peeling process. Instead, each node v runs a *conservative* peeling algorithm based only on the portion of the graph stored locally. In this conservative peeling algorithm, not all nodes are assigned a layer — some remain unlayered and must be handled in subsequent iterations. Consequently, each iteration yields only a *partial* H -partition, consisting of both layered and unlayered nodes. An additional complication is that these partial decompositions may not be globally consistent because a node might get assigned to different layers by different nodes. This discrepancy arises because balanced exponentiation can give different nodes very different perspectives on the graph. Fortunately, this inconsistency does not pose a real obstacle. We prove a key structural property of (partial) H -partitions: given multiple (partial) decompositions, one can construct another valid (partial) decomposition by assigning each node to the smallest layer it is assigned to across all decompositions. Using this property, we can merge the various locally computed partial H -partitions into a single globally consistent partial H -partition, ensuring that every node in a sufficiently small subtree is assigned to one of the $O(\log n)$ layers.

6. Coin Dropping Game via Adaptive Queries

Balanced exponentiation makes an appearance in the AMPC model in the form of the so-called *coin-dropping*. This technique is used to develop a deterministic *local computation algorithm* (LCA) [Rub+11; Alo+12] that enables us to compute $\text{poly}(\alpha)$ - and $O(\alpha)$ -colorings in constant, or near-constant, time on graphs with arboricity α in the AMPC model. On a high level, the approach is very similar to the tree-coloring algorithm in the MPC model from Publication III, in that the core task reduces to computing an H -partition of the graph. The key difference, however, is that the input graphs are no longer trees. This alone renders the generalized balanced exponentiation technique from Publication III useless because it fundamentally relies on a property unique to trees: the subgraph connected to a node v via its neighbor $u \in N(v)$ is disconnected from the subgraph connected to a node v via some other neighbor $w \in N(v), w \neq u$. However, we show that we can bypass this limitation in the AMPC mode by designing an LCA that computes (partial) H -partitions efficiently, even on graphs with cycles.

6.1 Sublinear LCA

We develop a deterministic LCA that produces an acyclic orientation with small out-degree $\beta = \Omega(\alpha)$ for the vast majority of nodes on a graph with arboricity α , using a sublinear number of queries. In particular, we construct a β -partition, i.e., a layering of the vertex set where each node has at most β neighbors in its own or higher layers, while minimizing the total number of layers. Such a partition directly induces the desired acyclic orientation: inter-layer edges are oriented from lower to higher layers, and intra-layer edges are oriented from lower to higher ID (or, if a layer is colored, from lower to higher color). Previous work suggests that achieving this for all nodes with sublinear query complexity is highly unlikely. Instead, our algorithm focuses on computing layers for *most* of the vertices, thereby achieving an LCA with sublinear-query complexity that produces a near-complete β -partition (see Figure 6.1).

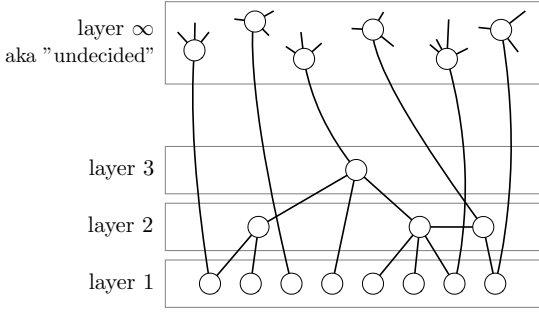
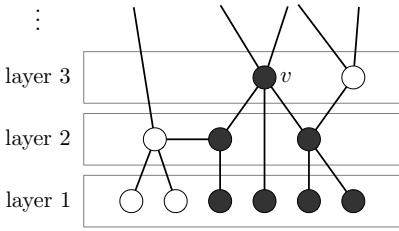
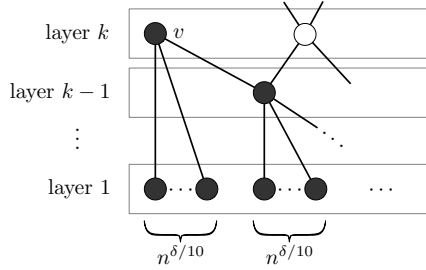


Figure 6.1. (Publication IV, Figure 1) An illustration of a β -partition where most nodes are assigned a layer.



(a) The first three layers of a β -partition. Black nodes form the dependency graph $D(v)$ of node v .



(b) A skewed dependency graph, which is a counter-example to naïve volume-based querying.

Figure 6.2. (Publication IV, Figures 2 & 3) Examples of different dependency graphs.

The informal guarantee we achieve is summarized below.

Lemma 6.1.1 (Publication IV, Lemma 1.1). *For any constant $\delta > 0$ there is deterministic LCA that uses at most $O(n^\delta)$ queries per node on a graph G with arboricity α and assigns each node a layer from $\mathbb{N} \cup \{\infty\}$ such that the following holds:*

▷ *There exists a subset $S \subseteq V$ containing at least a $1 - 1/n^{O(\delta)}$ fraction of vertices such that the layering of S forms a β -partition of $G[S]$ with $O(\log_\beta n)$ layers.* ◁

6.1.1 Where Should One Query?

A fundamental question to answer is which part of the graph a node needs to query to determine its layer? The answer is straightforward to answer if we assume that the target β -partition ℓ is already known, because this partition naturally defines the *dependency graph* of each node v . The dependency graph $D(v)$ is the set of all nodes u that can be reached from v along a path of strictly decreasing layers (see Figure 6.2a). Every node in $D(v)$ has at most β neighbors outside of $D(v)$. The dependency graph certifies the layer

of v , in the sense that the layer of v in ℓ can be determined solely based on the nodes in $D(v)$ and their degrees (see Figure 6.2a for an example). Once a node has discovered its dependency graph through queries, it can correctly compute its layer. While this imagined partition ℓ is not available to guide the algorithm, simply because it is not known beforehand, it is precisely what we aim to compute, and it provides valuable intuition and a useful framework for analysis.

Observe that if a node's dependency graph is too large, then learning it with a sublinear number of queries is impossible. Thus, our strategy relies on the fact that many nodes have *small* dependency graphs (of size $\leq n^\delta$). For such nodes, we hope to explore their dependency graphs and determine their layers.

Everything boils down to solving what appears to be an impossible task: guiding the exploration of a node v so that most queries are concentrated within its dependency graph, despite having no prior knowledge of where this region lies. Deterministic exploration is particularly difficult, as the local neighborhood of v can look identical in every direction. However, only a fraction of these directions actually lead to $D(v)$, and there is no obvious way to identify them in advance. Let us briefly explain why the classic search paradigms fail in this setting.

- *Depth-First Search (DFS)*: A node v may have up to β neighbors outside its dependency graph. If the algorithm initiates DFS from one of these, all queries may end up being wasted on irrelevant parts of the graph.
- *Breadth-First Search (BFS)*: To avoid the pitfalls of DFS, one might try BFS in hopes of balancing exploration. At first glance, this seems promising: after the first round of queries, at least a $1/(\beta+1)$ fraction of queried nodes belong to $D(v)$. However, the gains of this approach quickly deteriorate. Suppose v has a neighbor u outside $D(v)$ with very high degree, say $\deg(u) = n^\delta$. Then, most second-level queries are spent on exploring u 's neighbors, which most likely lie outside $D(v)$ and provide no useful information.

Clearly, a more refined exploration strategy is needed. Our solution is to explore the graph in a *volume-balanced* manner, rather than in a *distance-balanced* way like BFS or DFS. This method draws direct inspiration from the balanced exponentiation technique of Publication II and Publication III.

Volume-based querying. To guide the queries, consider the following coin-dropping *game*. Suppose node v is given n^δ coins. A naïve strategy (which, as we will see, still fails) is for v to distribute its coins evenly among its neighbors, so that each neighbor receives the same share. Each neighbor then repeats this process recursively, forwarding the coins it received to its own neighbors, and so on, until the coins can no longer be divided.

Conceptually, this procedure acts like a “volume-based” version of BFS. The act of passing a coin corresponds to querying a node, and any node that receives at least one coin is thereby “discovered” by v . Ideally, this process would succeed in forwarding a coin to every node in $D(v)$. Unfortunately, this approach breaks down in the presence of skewed dependency graphs, an example of which is depicted in Figure 6.2b: a long “descending” path where each node has $n^{\delta/10}$ neighbors in the lowest layer. In this case, the initial n^δ coins given out by node v are exhausted already after ten rounds of distribution, discovering only around $10 \cdot n^{\delta/10}$ nodes of the dependency graph — far fewer than desired. The exploration stalls because most of the coins get “stuck” at the lowest layer very fast and cannot be utilized where they are truly needed. Repeating this coin-dropping game yields no improvement, since the coins will always get stuck in the same way.

6.1.2 Our Solution

Our approach builds on the coin-dropping game described earlier, but instead of naïvely distributing coins, we design more refined *forwarding rules* that dictate how coins are passed along. These rules are changed dynamically as more of the graph is revealed by the queries of node v , allowing us to steer exploration more effectively and fully exploit the adaptivity of the queries.

Let us think about how to guarantee that a coin-dropping game will always reveal new parts of the dependency graph. The previous approach failed because nodes distributed coins to all of their neighbors blindly. What if nodes only forwarded coins to a carefully chosen subset of their neighbors? Recall that every node in $D(v)$ has at most β neighbors outside of $D(v)$. Consequently, if every node $u \in D(v)$ forwards coins to $\beta + 1$ of its neighbors (chosen by some rule), then at least one of those neighbors must belong to $D(v)$. Hence, at least a $1/(\beta + 1)$ fraction of u ’s coins is guaranteed to stay inside the dependency graph. After k forwarding rounds, at least $x/(\beta + 1)^k$ of the original x coins distributed by v remain within $D(v)$, which sounds promising. Does this approach work? Is it possible to distribute coins long enough to learn new parts of the dependency graph? Crucially, does performing multiple of these coin-dropping games lead to v learning the whole dependency graph?

Everything boils down to how we choose, for every node $u \in D(v)$, the $\beta + 1$ neighbors that u forwards coins to. It seems that any fixed choice is insufficient: after some number of coin-dropping games, coins will begin accumulating within already discovered regions of $D(v)$. In such cases, the process does not lead to v to querying new, undiscovered parts of the dependency graph. Hence, the rules for selecting *which* neighbors receive coins must be defined adaptively.

Our coin forwarding rules. We aim to define forwarding rules such that a sufficient fraction of coins flows into unexplored regions of $D(v)$ while minimizing leakage outside of the dependency graph or stagnation in areas already known. To achieve this, node v maintains an approximation of the β -partition (called the *induced* β -partition) of the subgraph it has discovered so far (denoted S_v). In this approximation, every node in S_v is assigned a tentative layer (or ∞ if no layer is yet assigned). Every time a coin-dropping game is initiated by node v , a new approximation is computed and every node $u \in D(v)$ forwards coins to its $\beta + 1$ neighbors with the highest tentative layer, with ties broken arbitrarily. These adaptive rules guarantee that enough coins flow toward undiscovered parts of $D(v)$ in every coin-dropping game.

Interestingly, the process cannot simply stop once node v is assigned a tentative layer different from ∞ . Even if v has already uncovered most of $D(v)$, its tentative layer might still be an overestimate: for example, $n^{\delta/10}$, while its true layer (had the entire dependency graph been explored) could be much smaller. Such inaccuracies are unacceptable, since we aim for $O(\log n)$ layers overall. The success of our method, therefore, hinges on how many times we reinitiate the coin-dropping game and how long individual games are allowed to run.

6.2 Using Our LCA Recursively

Our LCA computes the β -partition layers for a large majority of the nodes, but not for all of them. To obtain a complete β -partition, one would hope to simply recurse on the subgraph that remains after removing all nodes assigned a finite layer. Unfortunately, this step turns out to require more care. From Lemma 6.1.1, we only know that there exists a subset $S \subseteq V$ of size at least $(1 - 1/n^{O(\delta)}) \cdot |V|$ such that the layering of S forms a valid β -partition of $G[S]$. Utilizing this property is difficult because we do not know which nodes belong to this “good” set S , since there may be nodes outside of S that also compute a finite layer. Including such nodes in the partition risks creating a *globally inconsistent* layering. For example, consider nodes $w, u \notin S$. It can happen (because the nodes are not in S) that w assigns itself to layer 10 because it has computed u to be in layer 9, while u independently assigns itself to layer 11. Which result to accept, and could this create an unwanted domino effect? By choosing poorly, nodes could end up with more than β neighbors in higher layers, violating the β -partition property. To overcome this issue, we strengthen our LCA beyond the informal guarantee of Lemma 6.1.1. In addition to outputting its own layer, each node $u \in V$ also provides a *proof* in the form of a β -partition¹ ℓ_u . Intuitively, this proof

¹We actually require this partition to be *partial*, analogous to partial H -partitions introduced in Chapter 5.

acts as a local certificate that justifies why u can be safely assigned to layer $\ell_u(u)$: it describes how to layer parts of the graph so that u 's assignment is valid. With access to these proofs, we no longer need to decide between different layer assignments or identify which finite-layer nodes belong to S . Instead, we can merge all discovered β -partitions into a single partition by defining

$$\ell(v) = \min_{u \in V} \ell_u(v)$$

to be the layer of a node v . This construction yields a valid and globally consistent β -partition of the subgraph induced by all finite-layer nodes. This consistency guarantee is sufficient for applying our LCA recursively in the AMPC model.

References

- [AB09] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009. doi: 10.1017/CB09780511804090 (cit. on p. 28).
- [AH76] Kenneth Appel and Wolfgang Haken. “Every planar map is four colorable”. In *Bulletin of the American Mathematical Society* 82 (1976), pp. 711–712. doi: 10.1090/S0002-9904-1976-14122-5 (cit. on p. 11).
- [Alo+12] Noga Alon, Ronitt Rubinfeld, Shai Vardi, and Ning Xie. “Space-Efficient Local Computation Algorithms”. In *the Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2012, pp. 1132–1139. doi: 10.1137/1.9781611973099.89 (cit. on p. 49).
- [And+18] Alexandr Andoni, Zhao Song, Clifford Stein, Zhengyu Wang, and Peilin Zhong. “Parallel Graph Connectivity in Log Diameter Rounds”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 2018, pp. 674–685. doi: 10.1109/FOCS.2018.00070 (cit. on pp. 31, 33, 44).
- [ASW19] Sepehr Assadi, Xiaorui Sun, and Omri Weinstein. “Massively Parallel Algorithms for Finding Well-Connected Components in Sparse Graphs”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2019, pp. 461–470. doi: 10.1145/3293611.3331596 (cit. on p. 33).
- [Bal+18] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. “Almost Global Problems in the LOCAL Model”. In *the International Symposium on Distributed Computing (DISC)*. 2018, 9:1–9:16. doi: 10.4230/LIPIcs.DISC.2018.9 (cit. on p. 32).
- [Bal+19] Alkida Balliu, Juho Hirvonen, Dennis Olivetti, and Jukka Suomela. “Hardness of Minimal Symmetry Breaking in Distributed Computing”. In *the Proceedings of the ACM Symposium on Prin-*

- ciples of Distributed Computing (PODC)*. 2019, pp. 369–378. doi: 10.1145/3293611.3331605 (cit. on p. 32).
- [Bal+20a] Alkida Balliu, Sebastian Brandt, Yuval Efron, Juho Hirvonen, Yannic Maus, Dennis Olivetti, and Jukka Suomela. “Classification of Distributed Binary Labeling Problems”. In *the International Symposium on Distributed Computing (DISC)*. 2020, 17:1–17:17. doi: 10.1145/3382734.3405703 (cit. on p. 32).
- [Bal+20b] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. “How much does randomness help with locally checkable problems?” In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2020, pp. 299–308. doi: 10.1145/3382734.3405715 (cit. on p. 25).
- [Bal+21] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. “Locally Checkable Labelings with Small Messages”. In *the International Symposium on Distributed Computing (DISC)*. 2021, 8:1–8:18. doi: 10.4230/LIPIcs.DISC.2021.8 (cit. on pp. 23, 42).
- [Bal+25] Alkida Balliu, Mohsen Ghaffari, Fabian Kuhn, Augusto Modanese, Dennis Olivetti, Mikaël Rabie, Jukka Suomela, and Jara Uitto. “Shared randomness helps with local distributed problems”. In *the Proceedings of the International Colloquium on Automata, Languages, and Programming (ICALP)*. 2025, 16:1–16:18. doi: 10.4230/LIPIcs.ICALP.2025.16 (cit. on p. 25).
- [Bar15] Leonid Barenboim. “Deterministic $(\Delta + 1)$ -Coloring in Sub-linear (in Δ) Time in Static, Dynamic and Faulty Networks”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2015, pp. 345–354. doi: 10.1145/2767386.2767410 (cit. on p. 36).
- [BCG20] Suman K. Bera, Amit Chakrabarti, and Prantar Ghosh. “Graph Coloring via Degeneracy in Streaming and Other Space-Conscious Models”. In *the Proceedings of the International Colloquium on Automata, Languages, and Programming (ICALP)*. 2020, 11:1–11:21. doi: 10.4230/LIPIcs.ICALP.2020.11 (cit. on p. 36).
- [BE08] Leonid Barenboim and Michael Elkin. “Sublogarithmic distributed MIS algorithm for sparse graphs using Nash-Williams decomposition”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2008, pp. 25–34. doi: 10.1145/1400751.1400757 (cit. on pp. 23, 36).
- [BE10] Leonid Barenboim and Michael Elkin. “Deterministic Distributed Vertex Coloring in Polylogarithmic Time”. In *the Proceedings of the ACM Symposium on Principles of Distributed Comput-*

- ing (PODC). 2010, pp. 410–419. doi: 10.1145/1835698.1835797 (cit. on p. 36).
- [Beh+19a] Soheil Behnezhad, Sebastian Brandt, Mahsa Derakhshan, Manuela Fischer, MohammadTaghi Hajiaghayi, Richard M. Karp, and Jara Uitto. “Massively Parallel Computation of Matching and MIS in Sparse Graphs”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2019, pp. 481–490. doi: 10.1145/3293611.3331609 (cit. on pp. 31, 35).
- [Beh+19b] Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Łącki, and Vahab Mirrokni. “Near-Optimal Massively Parallel Graph Connectivity”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 2019, pp. 1615–1636. doi: 10.1109/FOCS.2019.00095 (cit. on pp. 33, 34, 44).
- [Beh+19c] Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Łącki, Vahab Mirrokni, and Warren Schudy. “Massively Parallel Computation via Remote Memory Access”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2019, pp. 59–68. doi: 10.1145/3323165.3323208 (cit. on pp. 17, 28, 29).
- [BFU19] Sebastian Brandt, Manuela Fischer, and Jara Uitto. “Breaking the Linear-Memory Barrier in MPC: Fast MIS on Trees with Strongly Sublinear Memory”. In *the Proceedings of the Colloquium on Structural Information and Communication Complexity (SIROCCO)*. 2019, pp. 124–138. doi: 10.1007/978-3-030-24922-9_9 (cit. on p. 35).
- [Bha+24] Sayan Bhattacharya, Martín Costa, Nadav Panski, and Shay Solomon. “Arboricity-Dependent Algorithms for Edge Coloring”. In *the Proceedings of the Scandinavian Symposium and Workshops on Algorithm Theory (SWAT)*. 2024, 12:1–12:15. doi: 10.4230/LIPIcs.SWAT.2024.12 (cit. on p. 36).
- [BKS17] Paul Beame, Paraschos Koutris, and Dan Suciu. “Communication Steps for Parallel Query Processing”. In *the Journal of the ACM* 64.6 (2017). doi: 10.1145/3125644 (cit. on pp. 17, 26).
- [Bro41] Rowland L. Brooks. “On colouring the nodes of a network”. In *the Mathematical Proceedings of the Cambridge Philosophical Society* 37 (1941), pp. 194–197. doi: 10.1017/S030500410002168X (cit. on p. 12).
- [CC22] Sam Coy and Artur Czumaj. “Deterministic Massively Parallel Connectivity”. In *the Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*. 2022, pp. 162–175. doi: 10.1145/3519935.3520055 (cit. on pp. 33, 34).

- [CDP20] Artur Czumaj, Peter Davies, and Merav Parter. “Graph Sparsification for Derandomizing Massively Parallel Computation with Low Space”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2020, pp. 175–185. doi: 10.1145/3350755.3400282 (cit. on p. 31).
- [CDP21a] Artur Czumaj, Peter Davies, and Merav Parter. “Component Stability in Low-Space Massively Parallel Computation”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2021, pp. 481–491. doi: 10.1145/3465084.3467903 (cit. on pp. 28, 31, 33, 44).
- [CDP21b] Artur Czumaj, Peter Davies, and Merav Parter. “Improved Deterministic $(\Delta + 1)$ Coloring in Low-Space MPC”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2021, pp. 469–479. doi: 10.1145/3465084.3467937 (cit. on p. 31).
- [Cha+19a] Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. “The Complexity of $(\Delta + 1)$ -Coloring in Congested Clique, Massively Parallel Computation, and Centralized Local Computation”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2019, pp. 471–480. doi: 10.1145/3293611.3331607 (cit. on p. 36).
- [Cha+19b] Yi-Jun Chang, Qizheng He, Wenzheng Li, Seth Pettie, and Jara Uitto. “Distributed Edge Coloring and a Special Case of the Constructive Lovász Local Lemma”. In *ACM Trans. Algorithms* (2019), pp. 1–51. doi: 10.1145/3365004 (cit. on p. 32).
- [Cha20] Yi-Jun Chang. “The Complexity Landscape of Distributed Locally Checkable Problems on Trees”. In *the International Symposium on Distributed Computing (DISC)*. 2020, 18:1–18:17. doi: 10.4230/LIPIcs.DISC.2020.18 (cit. on p. 32).
- [CNR23] Aleksander B. G. Christiansen, Krzysztof Nowicki, and Eva Rotenberg. “Improved Dynamic Colouring of Sparse Graphs”. In *the Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*. 2023, pp. 1201–1214. doi: 10.1145/3564246.3585111 (cit. on p. 36).
- [CP17] Yi-Jun Chang and Seth Pettie. “A Time Hierarchy Theorem for the LOCAL Model”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 2017, pp. 156–167. doi: 10.1109/FOCS.2017.23 (cit. on pp. 32, 41).
- [CR22] Aleksander B. G. Christiansen and Eva Rotenberg. “Fully-Dynamic $\alpha + 2$ Arboricity Decompositions and Implicit Colouring”. In *the Proceedings of the International Colloquium on*

- Automata, Languages, and Programming (ICALP)*. 2022, 42:1–42:20. doi: 10.4230/LIPIcs.ICALP.2022.42 (cit. on p. 36).
- [DG08] Jeffrey Dean and Sanjay Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters”. In *Communications of the ACM* (2008), pp. 107–113. doi: 10.1145/1327452.1327492 (cit. on pp. 18, 26).
- [FGG23] Manuela Fischer, Jeff Giliberti, and Christoph Grunau. “Deterministic Massively Parallel Symmetry Breaking for Sparse Graphs”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2023, pp. 89–100. doi: 10.1145/3558481.3591081 (cit. on pp. 35, 36).
- [FHK16] Pierre Fraigniaud, Marc Heinrich, and Adrian Kosowski. “Local Conflict Coloring”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 2016, pp. 625–634. doi: 10.1109/FOCS.2016.73 (cit. on p. 36).
- [GG25] Mohsen Ghaffari and Christoph Grunau. “Density-Dependent Graph Orientation and Coloring in Scalable MPC”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2025, pp. 349–359. doi: 10.1145/3732772.3733501 (cit. on p. 36).
- [GGJ20] Mohsen Ghaffari, Christoph Grunau, and Ce Jin. “Improved MPC Algorithms for MIS, Matching, and Coloring on Trees and Beyond”. In *the International Symposium on Distributed Computing (DISC)*. 2020, 34:1–34:18. doi: 10.4230/LIPIcs.DISC.2020.34 (cit. on pp. 31, 35).
- [GKU19] Mohsen Ghaffari, Fabian Kuhn, and Jara Uitto. “Conditional Hardness Results for Massively Parallel Computation from Distributed Lower Bounds”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 2019, pp. 1650–1663. doi: 10.1109/FOCS.2019.00097 (cit. on pp. 28, 33).
- [GRB22] Christoph Grunau, Václav Rozhoň, and Sebastian Brandt. “The Landscape of Distributed Complexities on Trees and Beyond”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2022, pp. 37–47. doi: 10.1145/3519270.3538452 (cit. on pp. 23, 32).
- [Gru+23] Christoph Grunau, Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. “Conditionally Optimal Parallel Coloring of Forests”. In *arXiv*. 2023. doi: 10.48550/arXiv.2308.00355 (cit. on p. 47).

- [GS19] Mohsen Ghaffari and Ali Sayyadi. “Distributed Arboricity-Dependent Graph Coloring via All-to-All Communication”. In *the Proceedings of the International Colloquium on Automata, Languages, and Programming (ICALP)*. 2019, 142:1–142:14. doi: 10.4230/LIPIcs.ICALP.2019.142 (cit. on p. 36).
- [GU19] Mohsen Ghaffari and Jara Uitto. “Sparsifying Distributed Algorithms with Ramifications in Massively Parallel Computation and Centralized Local Computation”. In *the Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2019, pp. 1636–1653. doi: 10.1137/1.9781611975482.99 (cit. on p. 31).
- [Gup+23] Chetan Gupta, Rustam Latypov, Yannic Maus, Shreyas Pai, Simo Särkkä, Jan Studený, Jukka Suomela, Jara Uitto, and Hossein Vahidi. “Fast Dynamic Programming in Trees in the MPC Model”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2023, pp. 443–453. doi: 10.1145/3558481.3591098 (cit. on p. 47).
- [Hea90] Percy J. Heawood. “Map-Colour Theorem”. In *Quarterly Journal of Pure and Applied Mathematics* 24 (1890), pp. 332–338. url: <https://babel.hathitrust.org/cgi/pt?id=inu.30000050138159> (cit. on p. 11).
- [HNW20] Monika Henzinger, Stefan Neumann, and Andreas Wiese. “Explicit and Implicit Dynamic Coloring of Graphs with Bounded Arboricity”. In *arXiv*. 2020. doi: 10.48550/arXiv.2002.10142 (cit. on p. 36).
- [Isa+07] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. “Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks”. In *ACM SIGOPS Operating Systems Review* (2007), pp. 59–72. doi: 10.1145/1272996.1273005 (cit. on pp. 18, 26).
- [Kar72] Richard M. Karp. “Reducibility among Combinatorial Problems”. In *the Proceedings of the Symposium on the Complexity of Computer Computations*. 1972, pp. 85–103. doi: 10.1007/978-1-4684-2001-2_9 (cit. on p. 12).
- [KSV10] Howard J. Karloff, Siddharth Suri, and Sergei Vassilvitskii. “A Model of Computation for MapReduce”. In *the Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2010, pp. 938–948. doi: 10.1137/1.9781611973075.76 (cit. on pp. 17, 26).
- [Lin92] Nathan Linial. “Locality in Distributed Graph Algorithms”. In *SIAM Journal on Computing* (1992), pp. 193–201. doi: 10.1137/0221015 (cit. on p. 25).

- [Lov75] László Lovász. “Three short proofs in graph theory”. In *the Journal of Combinatorial Theory, Series B* 19.3 (1975), pp. 269–271. doi: 10.1016/0095-8956(75)90089-1 (cit. on p. 12).
- [LU21] Rustam Latypov and Jara Uitto. “Coloring Trees in Massively Parallel Computation”. In *arXiv*. 2021. doi: 10.48550/arXiv.2105.13980 (cit. on p. 35).
- [LW10] Christoph Lenzen and Roger Wattenhofer. “Brief Announcement: Exponential Speed-up of Local Algorithms Using Non-Local Communication”. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. 2010, pp. 295–296. doi: 10.1145/1835698.1835772 (cit. on p. 27).
- [Mau21] Yannic Maus. “Distributed Graph Coloring Made Easy”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2021, pp. 362–372. doi: 10.1145/3409964.3461804 (cit. on p. 36).
- [MR85] Gary L. Miller and John H. Reif. “Parallel tree contraction and its application”. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*. 1985, pp. 478–489. doi: 10.1109/SFCS.1985.43 (cit. on pp. 23, 41).
- [MT20] Yannic Maus and Tigran Tonoyan. “Local Conflict Coloring Revisited: Linial for Lists”. In *the International Symposium on Distributed Computing (DISC)*. 2020, 16:1–16:18. doi: 10.4230/LIPICS.DISC.2020.16 (cit. on p. 36).
- [Nas64] Crispin Nash-Williams. “Decomposition of Finite Graphs Into Forests”. In *Journal of the London Mathematical Society* s1–39 (1964), p. 12. doi: 10.1112/jlms/s1-39.1.12 (cit. on p. 23).
- [NS93] Moni Naor and Larry J. Stockmeyer. “What can be computed locally?” In *the Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*. 1993, pp. 184–193. doi: 10.1145/167088.167149 (cit. on pp. 23, 32).
- [Pel00] David Peleg. *Distributed Computing: A Locality-Sensitive Approach*. Society for Industrial and Applied Mathematics, 2000. doi: 10.1137/1.9780898719772 (cit. on p. 24).
- [Roz24] Václav Rozhoň. “Invitation to Local Algorithms”. In *arXiv*. 2024. doi: 10.48550/arXiv.2406.19430 (cit. on p. 25).
- [RR97] Alexander A. Razborov and Steven Rudich. “Natural Proofs”. In *Journal of Computer and System Sciences* 55 (1997), pp. 24–35. doi: 10.1006/jcss.1997.1494 (cit. on p. 28).

- [Rub+11] Ronitt Rubinfeld, Gil Tamir, Shai Vardi, and Ning Xie. “Fast Local Computation Algorithms”. In *the Proceedings of Innovations in Computer Science (ICS)*. 2011, pp. 223–238. doi: 10.48550/arXiv.1104.1377 (cit. on p. 49).
- [RVW16] Tim Roughgarden, Sergei Vassilvitskii, and Joshua R. Wang. “Shuffles and Circuits: (On Lower Bounds for Modern Parallel Computation)”. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 2016, pp. 1–12. doi: 10.1145/2935764.2935799 (cit. on p. 28).
- [Suo13] Jukka Suomela. “Survey of local algorithms”. In *ACM Computing Surveys* (2013), 24:1–24:40. doi: 10.1145/2431211.2431223 (cit. on p. 25).
- [Whi09] Tom White. *Hadoop: The Definitive Guide*. O’Reilly Media, Inc., 2009. isbn: 0596521979 (cit. on pp. 18, 26).
- [Zah+10] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. “Spark: Cluster Computing with Working Sets”. In *the Proceedings of the USENIX Conference on Hot Topics in Cloud Computing (HotCloud)*. 2010, p. 10. doi: 10.5555/1863103.1863113 (cit. on pp. 18, 26).

Publication I

Alkida Balliu, Sebastian Brandt, Manuela Fischer, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Exponential Speedup Over Locality in MPC with Optimal Memory. In *International Symposium on Distributed Computing (DISC) 2022 (also in Journal of Distributed Computing, February 2025)*, Volume 246, 9:1-9:21, doi.org/10.4230/LIPIcs.DISC.2022.9, Augusta, USA, October 2022.

© 2022 Schloss Dagstuhl – Leibniz-Zentrum für Informatik
Reprinted with permission.

Exponential Speedup over Locality in MPC with Optimal Memory

Alkida Balliu 

Gran Sasso Science Institute, L'Aquila, Italy

Manuela Fischer 

ETH Zürich, Switzerland

Yannic Maus 

TU Graz, Austria

Jara Uitto 

Aalto University, Espoo, Finland

Sebastian Brandt 

CISPA Helmholtz Center for Information Security,
Saarbrücken, Germany

Rustam Latypov 

Aalto University, Espoo, Finland

Dennis Olivetti 

Gran Sasso Science Institute, L'Aquila, Italy

Abstract

Locally Checkable Labeling (LCL) problems are graph problems in which a solution is correct if it satisfies some given constraints in the local neighborhood of each node. Example problems in this class include maximal matching, maximal independent set, and coloring problems. A successful line of research has been studying the complexities of LCL problems on paths/cycles, trees, and general graphs, providing many interesting results for the LOCAL model of distributed computing. In this work, we initiate the study of LCL problems in the low-space Massively Parallel Computation (MPC) model. In particular, on forests, we provide a method that, given the complexity of an LCL problem in the LOCAL model, automatically provides an exponentially faster algorithm for the low-space MPC setting that uses optimal global memory, that is, truly linear.

While restricting to forests may seem to weaken the result, we emphasize that all known (conditional) lower bounds for the MPC setting are obtained by lifting lower bounds obtained in the distributed setting *in tree-like networks* (either forests or high girth graphs), and hence the problems that we study are challenging already on forests. Moreover, the most important technical feature of our algorithms is that they use optimal global memory, that is, memory linear in the number of edges of the graph. In contrast, most of the state-of-the-art algorithms use more than linear global memory. Further, they typically start with a dense graph, sparsify it, and then solve the problem on the residual graph, exploiting the relative increase in global memory. On forests, this is not possible, because the given graph is already as sparse as it can be, and using optimal memory requires new solutions.

2012 ACM Subject Classification Theory of computation → Distributed algorithms

Keywords and phrases Distributed computing, Locally checkable labeling problems, Trees, Massively Parallel Computation, Sublinear memory

Digital Object Identifier 10.4230/LIPIcs.DISC.2022.9

Related Version *Full Version*: <https://arxiv.org/abs/2208.09453>

Funding *Rustam Latypov*: Supported in part by the Academy of Finland, Grant 334238.

1 Introduction

The Massively Parallel Computation (MPC) model, introduced in [42] and later refined by [2, 12, 39], is a mathematical abstraction of modern data processing platforms such as MapReduce [28], Hadoop [55], Spark [56], and Dryad [41]. Recently, tremendous progress has been made on fundamental *graph problems* in this model, such as maximal independent set (MIS), maximal matching (MM) [37, 25], and coloring problems [19, 27]. All these



© Alkida Balliu, Sebastian Brandt, Manuela Fischer, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto;

licensed under Creative Commons License CC-BY 4.0

36th International Symposium on Distributed Computing (DISC 2022).

Editor: Christian Scheideler; Article No. 9; pp. 9:1–9:21



Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

problems, and many others, fall under the umbrella of *Locally Checkable* problems, in which the feasibility of a solution can be checked by inspecting local neighborhoods. They also serve as abstractions for fundamental primitives in large-scale graph processing and have recently gained a lot of attention [9, 7, 27, 29, 3, 18, 32]. *Locally checkable labelings (LCLs)* are locally checkable problems restricted to constant degree graphs. A more formal definition of LCLs is deferred to Section 2.

LCLs have been a rich source of research in various models of computation, because they can be seen as a starting point to understand locally checkable problems in general, and this holds independently of the model. For example, in the distributed setting, techniques developed to understand LCLs [16] have then been used to prove lower bounds in the unbounded degree setting, which the LCL setting does not include, e.g., for the maximal independent set problem, or the Δ -coloring problem [4, 6, 5]. In the distributed LOCAL model of computing, a lot is known about LCLs: for example, if the graph on which we want to solve the problem is a tree, then there is a discrete set of possible complexities, and in some cases, given an LCL, we can even automatically decide its distributed time complexity. Our goal is to bring to the parallel setting, and in particular to the MPC model, the knowledge that researchers developed about LCLs in the distributed setting, while also developing new techniques that can be used in the parallel setting. We show that, on forests, the mere knowledge of what is the distributed complexity of a problem is enough to obtain blazingly fast algorithms in the MPC setting. In particular, we obtain MPC algorithms that are *exponentially faster* than the best distributed ones. We summarize our main result.

The complexity of any LCL problem on forests in the MPC model is exponentially lower than its distributed complexity, even when using optimal memory bounds.

More in detail, in our work, we solve LCL problems in forests in the most restrictive *low-space* MPC model with *linear* total memory, which is the most *scalable* variant of the MPC model. Our results provide an automatic method that, for all LCL problems, yields an algorithm that solves the given problem *exponentially* faster than its optimal distributed counterpart. The resulting algorithms are *component-stable* [35, 26], which implies that the solutions in individual connected components are independent of the other components. Our results are in some sense optimal: for problems that in the LOCAL model can be solved in $n^{o(1)}$, finding more-than-exponentially faster component-stable algorithms would violate the widely-believed 1 vs. 2 cycle conjecture in the MPC setting.

Why do we care about trees and forests? All known conditional lower bounds¹ for problems in the MPC setting are derived by lifting lower bounds that hold in the LOCAL model of distributed computing [35, 26]. Most of the lower bounds known in the LOCAL model are actually proved either on trees or on high-girth graphs (where the neighborhood of each node corresponds to a tree): see, e.g., [44, 5, 4, 6, 16]. It follows that essentially all the conditional lower bounds known in the MPC setting already hold on forests². Despite this fact, with a few exceptions, there is no work on upper bounds on forests in the MPC model – a gap we aim to fill.

¹ Proving unconditional lower bounds for the MPC model would imply a major breakthrough in circuit complexity and seems out of reach [53].

² As lifting lower bounds from the LOCAL model to the MPC model requires hereditary graph classes one cannot immediately lift a lower bound in the LOCAL model that holds on trees. Instead, a lower bound in the LOCAL model on trees implies the same lower bound in the LOCAL model for forests which can then be lifted to a lower bound for MPC algorithms on forests.

Moreover, understanding the complexity of problems on trees has been already shown to be essential in the LOCAL model: it is typically the case that interesting problems are already challenging on trees, and often even in regular balanced trees of small degree. In fact, most lower bounds known in the LOCAL model hold exactly in this setting. Due to the lifting, the same statement adapted to forests is true for all recent MPC lower bounds. Hence, to decrease the relevance of trees and forests, we either need completely new lower bound techniques in the LOCAL model coupled with completely new lifting theorems, or completely new lower bound techniques for the MPC model.

At first glance it may seem that our results are easy to achieve, because we restrict to forests. Conversely, we would like to emphasize that many state-of-the-art algorithms for problems like MIS and coloring work as follows [37, 25]: start with a dense graph which requires a lot of memory to store, sparsify it, and then use the freed global memory to solve the problem faster on the sparsified part. On forests, this is not possible, because the given graph is already as sparse as it can be.

The MPC Model. In the MPC model, we have M machines who communicate in an all-to-all fashion. We focus on problems where the input is modeled as a graph with n vertices, m edges and maximum degree Δ ; we call this graph the *input graph*. Each node has a unique ID of size $b = O(\log n)$ bits from a domain $\{1, 2, \dots, N\}$, where $N = \text{poly}(n)$. Each node and its incident edges are hosted on a machine(s) with $S = O(n^\delta)$ *local memory*, where $\delta \in (0, 1)$ and the units of memory are *words* of $O(\log n)$ bits. When the local memory is bounded by $O(n^\delta)$, the model is called *low-space* (or *sublinear*). The number of machines is chosen such that $M = m/S = \Theta(m/n^\delta)$. For trees, where $m = \Theta(n)$, this results in $\Theta(n^{1-\delta})$ machines, that is, a *total memory* (or *global memory*) of $M \cdot S = \Theta(n)$. For simplicity³, we assume that each machine i simulates one virtual machine for each node and its incident edges that i hosts, such that the local memory restriction becomes that no virtual machine can use more than $O(n^\delta)$ memory.

During the execution of an MPC algorithm, computation is performed in synchronous, fault-tolerant rounds. In each round, every machine performs some (unbounded) computation on the locally stored data, then sends/receives messages to/from any other machine in the network. Each message is sent to exactly one other machine specified by the sending machine. All messages sent and received by each machine in each round, as well as the output, have to fit into local memory. The time complexity is the number of rounds it takes to solve a problem. Upon termination, each node (resp. its hosting machine) must know its own part of the solution. For example in the case of node-coloring, the machine hosting node u must decide on the color of u upon termination of the algorithm.

Unlike in most other works, our algorithms employ $O(m + n)$ words of total memory, which is the strictest possible as it is only enough to store a constant number of copies of the input graph. Note that if we were to allow superlinear $O(m^{1+\delta})$ global memory in our constant-degree setting, many LOCAL algorithms with complexity $O(\log n)$ could be trivially sped up exponentially in the low-space MPC model by applying the well-known graph exponentiation technique by Lenzen and Wattenhofer [45]. A crucial challenge that comes with the linear global memory restriction is that only a small fraction of $n^{1-\delta}$ of the (virtual) machines can simultaneously utilize all of their available local memory. Thus, with

³ In practice, it is assumed that the virtual machines can be shuffled between physical machines, such that the sum of the memory of the virtual machines hosted on any single physical machine is $O(n^\delta)$.

strictly linear global memory we are forced to develop new techniques which must avoid gathering local neighborhoods, i.e., fundamentally divert from direct simulations of message passing algorithms.

1.1 The Distributed Complexity Landscapes

In the last decade, there has been tremendous progress in understanding the complexities of LCLs in various models of distributed and parallel computing. A prime example is the LOCAL model [46], where the input graph corresponds to a message passing system, and the nodes must output their part of the solution according only to local information about the graph. Another example is the CONGEST model, which is a LOCAL model variant where the message size is restricted to $O(\log n)$ bits [52]. A curious fact about LCLs in the distributed setting is the existence of *complexity gaps*, that is, some complexities are not possible at all. For example, it is known that there are no LCLs with a distributed time complexity in the LOCAL and CONGEST model that lies between $\omega(\log^* n)$ and $o(\log n)$. In these two models, the *whole* complexity landscape of LCL problems is now understood for some important graph families. For instance, a rich line of work [50, 20, 22, 11, 8, 3, 18] recently came to an end when a complexity gap between $\omega(1)$ and $o(\log^* n)$ was proved [40], completing the randomized/deterministic complexity landscape of LCL problems in the LOCAL model for trees. In the CONGEST model, the authors of [9] showed that, on trees, the complexity of an LCL problem is asymptotically equal to its complexity in the LOCAL model, whereas the same does not hold in general graphs. In the randomized/deterministic LOCAL and CONGEST models, recent work showed that the complexity landscapes of LCL problems for rooted regular trees are fully understood [7], while the complexity landscapes of LCL problems in the LOCAL model for rings and tori have already been known for some while [17]. Even for general (constant-degree) graphs, the LOCAL complexity landscape of LCL problems is almost fully understood [50, 16, 21, 22, 36, 30, 34, 10, 8, 54, 33], only missing a small part of the picture related to the randomized complexity of Lovász Local Lemma (LLL).

In the case of trees, for deterministic algorithms in the LOCAL model, it is known that there is a discrete set of possible complexities, that we divide into four categories:

- **Tiny regime:** contains the complexities $O(1)$ and $\Theta(\log^* n)$.
 - Example problems: maximal independent set, maximal matching, $(\Delta + 1)$ -vertex coloring⁴, $(2\Delta - 1)$ -edge coloring, and trivial problems (e.g., all nodes must output 0).
- **Mid regime:** contains the complexity $\Theta(\log n)$.
 - Example problems: sinkless orientation [16], 3-coloring, and Δ -coloring.
- **High regime:** contains the complexities $\Theta(n^{1/k})$, for all $k \in \mathbb{N}$.
 - Example problems: 2-coloring and $2^{\frac{1}{2}}$ -coloring [22].

Moreover, it is known that randomness can help only in the mid regime, and in particular that *some* problems requiring $\Theta(\log n)$ for deterministic algorithms have randomized complexity $\Theta(\log \log n)$, which constitutes our fourth category – **Low regime**. Problems residing in the low regime include sinkless orientation and Δ -coloring.

On forests, the complexity landscape in the LOCAL model is the same as on trees. While this is intuitively evident, it can also be shown formally using an analogous approach to the one used in the proof of [40, Lemma 3.3].

⁴ We denote the maximum degree of the graph by Δ .

1.2 Our Contributions

Our main contribution is showing that, given any LCL problem (see Definition 4) on trees that has deterministic (resp. randomized) complexity T in the LOCAL model, we can automatically obtain an MPC algorithm with deterministic (resp. randomized) complexity $O(\log T)$ on forests. In particular, we prove the following.

► **Theorem 1.** *Consider an LCL problem on trees with deterministic time complexity $f(n)$ and randomized time complexity $g(n)$ in the LOCAL model. This problem has deterministic time complexity $O(\log f(n))$ and randomized time complexity $O(\log g(n))$ in the low-space MPC model on forests using optimal $O(m + n)$ words of global memory. The provided algorithms are component-stable.*

Put differently, a problem in the LOCAL model can only have a deterministic complexity $f(n) \in \{\Theta(1), \Theta(\log^* n), \Theta(\log n)\} \cup \{\Theta(n^{1/k}) \mid k \in \mathbb{N}\}$, and we show that it is enough to know the asymptotic value of $f(n)$ in order to obtain a deterministic MPC algorithm with complexity $O(\log(f(n))) \in \{O(1), O(\log \log^* n), O(\log \log n), O(\log n)\}$.

Moreover, it is known that for all $f(n) \notin \Theta(\log n)$, the LOCAL randomized complexity of the problem is the same as the deterministic one. Instead, for $f(n) \in \Theta(\log n)$, the LOCAL randomized complexity $g(n)$ can be either $\Theta(\log n)$ or $\Theta(\log \log n)$. If it is $\Theta(\log \log n)$, then we provide an MPC algorithm with randomized complexity $O(\log \log \log n)$. If we dismiss the component-stability requirement, we can obtain the same $O(\log \log \log n)$ runtime with a deterministic MPC algorithm.

► **Theorem 2.** *Consider an LCL problem on trees with randomized time complexity $g(n) = \Theta(\log \log n)$ in the LOCAL model. This problem has deterministic time complexity $O(\log \log \log n)$ in the low-space MPC model on forests using optimal $O(m + n)$ words of global memory. This algorithm is component-unstable.*

By [35, 26], we know that Theorem 1 is in some sense optimal: if a problem requires T deterministic rounds in the LOCAL model, then it requires $\Omega(\min\{\log T, \log \log n\})$ rounds in the low-space MPC setting for component-stable algorithms, assuming that the infamous 1 vs. 2 cycle conjecture holds [12, 35, 53]. In contrast, Theorem 2 shows that one can break the conditional lower bound of $\Omega(\log \log n)$ for deterministic MPC algorithms for all LCL problems in the aforementioned class by diverting to component-unstable algorithms. Achieving the same result even for a single problem without dismissing the component-stability requirement would be a major breakthrough, as it would falsify the conjecture.

As a subroutine for solving all problems that belong to the high regime in $O(\log n)$ MPC rounds, we also develop an $O(\log n)$ round MPC algorithm for rooting a forest. This rooting algorithm is component-stable, and may be of independent interest, since it is also compatible with arbitrary degrees.

Additional observations. There is a long line of research that provided algorithms for MPC that are exponentially faster than the best algorithms for the LOCAL model. Most existing results achieved these speedup results by using additional global memory, that is, $\omega(m)$ words [13, 32, 27, 26]. We emphasize that, deviating from the usual approach, all of our results use *optimal* MPC parameters, in the sense that we work in the low-space setting with $O(n^\delta)$ words of local memory and $O(m + n)$ words of global memory.

Hence, our contribution is twofold, on the one hand we prove that we can indeed achieve this exponential speedup for all LCLs, while on the other hand we show that this exponential speedup can be achieved without requiring any additional memory. Furthermore, graph

problems in trees and forests are widely unexplored, despite their central role that we have already elaborated on. It is known that a 4-coloring, MIS, and maximal matching can be found in $O(\log \log n)$ rounds [32]. However, the coloring result heavily relies on randomness and the MIS and matching results require a (small) overhead in the total memory. To compare, our results deterministically yield a 3-coloring in $O(\log \log n)$ rounds with linear total memory. It is not clear whether randomness can even help in the case of 3-coloring, which is a significant difference to the case of 4-coloring. Furthermore, it is not clear whether the previous approaches to MIS and matching can be extended to work deterministically with the same runtime and with linear total memory. While the previous work is designed for arbitrary degree graphs, it is not clear whether the algorithms could be tuned to work faster with constant degrees.

Open Questions. In the tiny regime, our results extend to general graphs (see Theorem 5). In the low regime, our results extend to general graphs if we allow slightly more global memory (see Theorem 6.5 in the full version). Once we reach the mid regime, i.e., logarithmic distributed complexities, we do not know the behaviour in general graphs. This leads to an interesting open question. As mentioned, the asymptotic complexity of any problem on trees is identical in the LOCAL and CONGEST model, and the same is true (modulo the exact complexity of the LLL in both models) on general graphs as long as the complexity is sublogarithmic [9]. However, there is an exponential separation between the models for complexities that are at least logarithmic [9]. Does such a separation between the complexity of an LCL in the LOCAL model and the MPC model also hold for large complexities? Here, of course, we would want to have a doubly exponential separation.

Interestingly, current conditional lower bounds for the MPC model cannot prove MPC lower bounds that are $\omega(\log \log n)$. So, while our results in the high regime show that any problem on forests can be solved in $O(\log n)$ rounds in the MPC model, it remains unclear whether we cannot improve on this bound, even without falsifying the 1 vs. 2 cycle conjecture.

Component-stability. The term of a *component-stable* MPC algorithm has been introduced in [35] in the context of lifting distributed lower bounds to the MPC setting. By their definition, informally, an algorithm is component-stable if the output of a node does not change if other connected components in the graph are altered (see Definition 11).

While initially believed that it might be an artifact of their lifting techniques, Czumaj, Davies and Parter [26] showed the contrary, i.e., they showed that *component-unstable* algorithms can beat the conditional lower bounds of [35]. Their results hold assuming their revised definition of component-stability, which is argued to be more robust (see Definition 13). Under their definition, it is not strictly easier nor harder to design algorithms to be component-stable, as compared to the definition of [35]. The main difference is that they allow the output of component-stable algorithms to depend on the total number of nodes in the graph and the maximum degree. In our work, we adopt the revised definition of component-stability [26]. See Appendix A and the discussion therein for further details.

1.3 Challenges & Key Techniques

We now provide an overview of the challenges that we had to tackle in order to prove our results, and a very high level explanation of the key techniques that we used to solve them.

The tiny regime serves as a good warm-up to see why using an optimal amount of global memory is difficult. The most technically involved part is the high regime, where we obtain an $O(\log n)$ -time MPC algorithm for any LCL problem.

Graph Exponentiation. A reoccurring challenge for all regimes lies in respecting the linear global memory, which roughly means that on average, every node can use only a constant amount of memory. This is particularly unfortunate because almost all recent MPC results – and in particular all that achieve exponential speedups – rely on the memory-intense *graph exponentiation* technique [45]. Informally, this technique enables a node to gather its 2^k -hop neighborhood in k communication rounds. Doing this in parallel for every node in the graph results in a Δ^{2^k} overhead in global memory. For this technique to be useful, k has to be $\omega(1)$, yielding a non-constant multiplicative increase in the global memory requirement. In order to use this technique but not violate linear global memory, we develop new solutions that are discussed in the following paragraphs.

Tiny regime $f(n) = \Theta(1)$ and $f(n) = \Theta(\log^* n)$. Handling the $\Theta(1)$ complexity is trivial, since any LOCAL algorithm for LCLs can be simulated in the MPC setting. For the $\Theta(\log^* n)$ class, it is known from prior work that all problems can be solved in the LOCAL model in a very specific way: reduce to the problem of computing a distance- k coloring with a small enough number of colors, where k is a constant that depends on the problem. In a distance- k c -coloring, each node is assigned a color in $\{1, \dots, c\}$ such that nodes at distance at most k have different colors. Such a coloring can be computed in $O(\log^* n)$ rounds in the LOCAL model, and it could be computed easily in the MPC setting in $O(\log \log^* n)$ rounds, by exploiting the graph exponentiation technique, if we allow an additional $O(\log^* n)$ factor overhead in the amount of global memory.

We show that this overhead is not required, by developing a novel MPC algorithm for coloring. The algorithm that we provide reduces the problem of coloring a general graph to coloring directed pseudoforests, that is, graphs where all edges are oriented and every node has at most one outgoing edge. Then, we show that in directed pseudoforests, it is possible to solve the coloring problem through a variant of graph exponentiation that only requires keeping track of a constant number of IDs. This way, the memory use of each node is constant, and the global memory is linear.

High regime $f(n) = \Theta(n^{1/k})$, for all $k \in \mathbb{N}$. We explicitly provide, for any solvable LCL, a novel algorithm that has a runtime of $O(\log n)$. Essentially, we solve each tree in the forest separately, hence we will consider trees in the following argumentation. On a high level, our algorithm first roots the tree using our $O(\log n)$ -time tree rooting algorithm, and then proceeds in two phases. In the first phase, roughly speaking, the goal is to compute, for a substantial number of nodes v , the set of possible output labels that can be output at v such that the label choice can be extended to a (locally) correct solution in the subtree hanging from v . This is done in an iterative manner, proceeding from the leaves towards the root. The second phase consists of using the computed information to solve the given LCL from the root downwards.

While this outline sounds simple, there are a number of intricate challenges that require the development of novel techniques, both in the design of the algorithm and its analysis. For instance, the depth of the input tree may be $\omega(\log n)$ (which prevents us from performing the above ideas in a sequential manner), and the storage of the required completability information grows exponentially when using graph exponentiation, exceeding the available global memory. Our key technical contributions are the following.

- The design of a process that allows for interleaving graph exponentiation steps and compressing the graph (and compatibility information) such that the process is also reversible (second phase of the algorithm). The main challenge here is that multiple graph exponentiation processes executed on individual parts of the tree have to be merged, simultaneously or at different times, into one process during the execution.

- The design of a fine-tuned potential function for the analysis of the complex algorithm resulting from addressing the aforementioned issues and the highly non-sequential behavior arising from interleaving graph exponentiation steps.

Mid regime $f(n) = \Theta(\log n)$. We would wish to use the algorithm of Chang and Pettie [22] as a black box. On a very high level idea, their LOCAL algorithm uses $O(\log n)$ rounds to compute a rake-and-compress decomposition of size $O(\log n)$, which is essentially the classic H -partition by Miller and Reif [49]. Then, compatibility information of the given LCL problem is propagated layer by layer to the top, and then labels are fixed at the top and propagated down.

Applying known MPC techniques like graph exponentiation to speed up this process does not work out of the box for several reasons. First, the compatibility information they propagate grows exponentially, which creates congestion in the MPC model. Secondly, since the input graph is as sparse as it could possibly be, the direct application of graph exponentiation would violate the optimal global memory bounds we are striving for. We resolve the first issue by first observing that the compatibility information can be reduced to constant size in every iteration. The second issue is remedied by interleaving exponentiation steps with memory freeing steps in a balanced way.

Low regime $g(n) = \Theta(\log \log n)$. With an additional $O(\log n)$ factor of global memory, this result is easy to obtain. Previous work [9] has a constant time reduction to instances of size $N = \log n$, resulting in a LOCAL algorithm with runtime $\text{poly}(\log N) = \text{poly}(\log \log n)$. A straightforward application of graph exponentiation would yield an MPC algorithm with runtime $O(\log \log \log n)$. Exploiting additional global memory in this manner has been used in a similar setting in [26]. However, without the additional memory it is harder to solve the small instances in triple logarithmic time. The work around for this memory issue is to use our mid regime algorithm on the small instances, yielding a memory efficient algorithm with runtime $O(\log \log \log n)$. To the best of our knowledge there is no other paper that can efficiently deal with such occurring small instances – small instances occur also in many other problems like MIS and graph coloring – with optimal global memory.

1.4 Further Related Work

For many of the classic graph problems, simple $O(\log n)$ -time MPC algorithms follow from classic literature in the LOCAL model and PRAM [1, 46, 48]. In particular in the case of bounded degree graphs, it is often straightforward to simulate algorithms from other models. However, it is usually desirable to get algorithms that run *much faster* than their LOCAL counterparts. If the MPC algorithms are given *linear* $\Theta(n)$ or even *superlinear* $\Theta(n^{1+\delta})$ local memory, fast algorithms are known for many classic graph problems.

In the sublinear (or low-space) model, [19] provided a randomized algorithm for the $(\Delta+1)$ -coloring problem that, combined with the new network decomposition results [54, 33], yields an $O(\log \log \log n)$ MPC algorithm, that is exponentially faster than its LOCAL counterpart. A recent result by Czumaj, Davies, and Parter [27] provides a deterministic $O(\log \log \log n)$ -time algorithm for the same problem using derandomization techniques. For many other problems, the current state of the art in the sublinear model is still far from the aforementioned exponential improvements over the LOCAL counterparts, at least in the case of general graphs. For example, the best known MIS, maximal matching, $(1 + \epsilon)$ -approximation of maximum matching, and 2-approximation of minimum vertex cover algorithms run in $\tilde{O}(\sqrt{\log \Delta} + \sqrt{\log \log n})$ time [37], whereas the best known LOCAL algorithm has a

logarithmic dependency on Δ [31]. For restricted graph classes, such as trees and graphs with small arboricity⁵ α , better algorithms are known [15, 13]. Through a recent work by Ghaffari, Grunau and Jin, the current state of the art for MIS and maximal matching are $O(\sqrt{\log \alpha} \cdot \log \log \alpha + \log \log n)$ -time algorithms using $\tilde{O}(n + m)$ words of global memory [32].

As for lower bounds, [35] gave conditional lower bounds of $\Omega(\log \log n)$ for component-stable sublinear MPC algorithms for constant approximation of maximum matching and minimum vertex cover, and MIS. In addition, the authors provided a lower bound of $\Omega(\log \log \log n)$ for LLL. Their hardness results are conditioned on a widely believed conjecture in MPC about the complexity of the connectivity problem, which asks to detect the connected components of a graph. It is argued that disproving this conjecture would imply rather strong and surprising implications in circuit complexity [53]. When assuming component-stability, they also argue that all known algorithms in the literature are component-stable or can easily be made component-stable with no asymptotic increase in the round complexity. However, recent work [26] gave a separation between stable and unstable algorithms, and that some particular problems (e.g., computing an independent set of size $\Omega(n/\Delta)$) can be solved faster with unstable algorithms than with stable ones.

It is also worth discussing the complexity of rooting a tree, as it is an important subroutine in our high regime. On the randomized side, [15] gave an $O(\log d \cdot \log \log n)$ time algorithm, where d is the diameter of the graph. On the deterministic side, Coy and Czumaj [24] gave an $O(\log n)$ time algorithm using (component-unstable) derandomization methods, which is the current state of the art. In the full version we provide a totally different rooting algorithm that is also deterministic and takes $O(\log n)$ time, but is component-stable. We note that [43] uses similar techniques in a more general setting, but in $\omega(\log n)$ time.

1.5 Outline

After the formal introduction of LCL problems and other notations in Section 2, we start proving the exponential speedup for the different regimes in Theorem 1 in separate sections. In Section 3, we warm up with the tiny regime. In Section 4, we present the high level ideas for our most involved result, the exponential speedup for the high regime. We provide full proofs and handle the remaining regimes in the full version of the paper.

Some of our speedup results use a description of a distributed algorithm with the claimed runtime to obtain the speedup. In the full version of the paper, we show that such a description can be inferred merely by knowing the distributed complexity class in which the problem resides.

2 Definitions and Notation

We work with undirected, finite, simple graphs $G = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges such that $E \subseteq [V]^2$ and $V \cap E = \emptyset$. Let $\deg_G(v)$ denote the degree of a node v in G and let Δ denote the maximum degree of G . The distance $d_G(v, u)$ between two vertices v, u in G is the length of a shortest $v - u$ path in G ; if no such path exists, we set $d_G(v, u) := \infty$. The greatest distance between any two vertices in G is the diameter of G , denoted by $\text{diam}(G)$. For a subset $S \subseteq V$, we use $G[S]$ to denote the subgraph of G induced by nodes in S . Let G^k , where $k \in \mathbb{N}$, denote the k :th power of a graph G , which is another graph on the same

⁵ The arboricity of a graph is the minimum number of disjoint forests into which the edges of the graph can be partitioned.

vertex set, but in which two vertices are adjacent if their distance in G is at most k . In the context of MPC, G^k is the resulting virtual graph after performing $\log k$ steps of graph exponentiation [45].

For each node v and for every radius $k \in \mathbb{N}$, we denote the k -hop (or k -radius) neighborhood of v as $N^k(v) = \{u \in V : d(v, u) \leq k\}$. The topology of a neighborhood $N^k(v)$ of v is simply $G[N^k(v)]$. However, with slight abuse of notation, we sometimes refer to $N^k(v)$ both as the node set and the subgraph induced by node set $N^k(v)$. Neighborhood topology knowledge is often referred to as vision, e.g., node v sees $N^k(v)$. In trees and forests, the number n of nodes and the number m of edges are asymptotically equal, and we may use them interchangeably throughout the paper when reasoning about global memory.

2.1 LCL Definitions

In their seminal work [50], Naor and Stockmeyer introduced the notion of a locally checkable labeling problem (LCL problem or just LCL for short). The definition they provide restricts attention to problems where nodes are labeled (such as vertex coloring problems), but they remark that a similar definition can be given for problems where edges are labeled (such as edge coloring problems). A modern way to define LCL problems that captures both of the above types of problems (and combinations thereof) labels *half-edges* instead, i.e., pairs (v, e) where e is an edge incident to vertex v . Moreover, on trees it is known that all LCL problems can be rephrased in a special form, called *node-edge-checkable LCL problems* [9, 40]. Let us first define a half-edge labeling formally, and then provide this modern LCL problem definition.

► **Definition 3 (Half-edge labeling).** *A half-edge in a graph $G = (V, E)$ is a pair (v, e) , where $v \in V$ is a vertex, and $e \in E$ is an edge incident to v . A half-edge (v, e) is incident to some vertex w if $v = w$. We denote the set of half-edges of G by $H = H(G)$. A half-edge labeling of G with labels from a set Σ is a function $g: H(G) \rightarrow \Sigma$.*

We distinguish between two kinds of half-edge labelings: *input labelings* that are part of the input and *output labelings* that are provided by an algorithm executed on input-labeled instances. Throughout the paper, we will assume that any considered input graph G comes with an input labeling $g_{\text{in}}: H(G) \rightarrow \Sigma_{\text{in}}$ and will refer to Σ_{in} as the *set of input labels*; if the considered LCL problem does not have input labels, we can simply assume that $\Sigma_{\text{in}} = \{\perp\}$ and that each node is labeled with $\perp$.

While the formal definition of a node-edge-checkable LCL (see below) appears complicated, the intuition behind it is simple: essentially, we have a list of allowed output label combinations around nodes, a list of allowed output label combinations on edges, and a list of allowed input-output label combinations, all of which a correct solution for the LCL has to satisfy.

► **Definition 4 (Node-edge-checkable LCL).** *Let Δ be some non-negative integer constant. A node-edge-checkable LCL is a quintuple $\Pi = (\Sigma_{\text{in}}, \Sigma_{\text{out}}, \mathcal{N}, \mathcal{E}, g)$ where Σ_{in} and Σ_{out} are finite sets, $\mathcal{N} = \{\mathcal{N}_1, \dots, \mathcal{N}_\Delta\}$ consists of sets $\mathcal{N}_i$ of cardinality- i multisets with elements from Σ_{out} , $\mathcal{E}$ is a set of cardinality-2 multisets with elements from Σ_{out} , and $g: \Sigma_{\text{in}} \rightarrow 2^{\Sigma_{\text{out}}}$ is a function mapping input labels to sets of output labels. We call $\mathcal{N}_1 \cup \dots \cup \mathcal{N}_\Delta$ and $\mathcal{E}$ the node constraint and edge constraint of Π , respectively. Furthermore, we call each element of $\mathcal{N}$ a node configuration, and each element of $\mathcal{E}$ an edge configuration. For a node v , denote the half-edges of the form (v, e) for some edge e by $h_1^v, \dots, h_{\deg(v)}^v$ (in arbitrary order). For an edge e , denote the half-edges of the form (v, e) for some node v by h_1^e, h_2^e (in arbitrary order).*

A correct solution for Π on a graph G is a half-edge labeling $g_{\text{out}}: H(G) \rightarrow \Sigma_{\text{out}}$ such that

1. for each node v , the multiset of outputs assigned by g_{out} to $h_1^v, \dots, h_{\deg(v)}^v$ is an element of $\mathcal{N}_{\deg(v)}$,
2. for each edge e , the cardinality-2 multiset of outputs assigned by g_{out} to h_1^e, h_2^e is an element of $\mathcal{E}$, and
3. for each half-edge $h \in H(G)$, we have $g_{\text{out}}(h) \in g(\iota)$, where $\iota = g_{\text{in}}(h)$ is the input label assigned to h .

We say that an algorithm $\mathcal{A}$ solves an LCL problem Π on a graph class $\mathcal{G}$ if it provides a correct solution for Π for every $G \in \mathcal{G}$. Note that the LCL definitions above implicitly require that graph class $\mathcal{G}$ has constant degree.

3 The Tiny Regime

In this section, we show that any LCL problem on general graphs that can be solved in the LOCAL model in $O(\log^* n)$ rounds, can be solved in the MPC model in $O(\log \log^* n)$ rounds. By combining this result with known gaps in the landscape of possible complexities in the LOCAL model [21], we obtain the following result.

► **Theorem 5.** *Let Π be an LCL problem on general graphs. Assume that there is a deterministic algorithm for the LOCAL model that solves Π in $o(\log n)$ rounds, or a randomized algorithm that solves it in $o(\log \log n)$ rounds. Then, the problem Π can be solved deterministically in $O(\log \log^* N)$ rounds in the low-space MPC model using $O(m + n)$ words of global memory, where $N = \text{poly}(n)$ is the size of the ID space. The algorithm works even if the graph consists of disconnected components, and it is components-stable.*

The rest of this section is devoted to proving Theorem 5.

A Universal Algorithm. In the LOCAL model, it is known that, if an LCL can be solved with an algorithm A in $o(\log n)$ deterministic rounds, or in $o(\log \log n)$ randomized rounds, then it can also be solved with a deterministic algorithm A' that requires just $O(\log^* n)$ rounds [21]. In order to prove this result, [21] shows how to convert any such algorithm A into an algorithm A' that works as follows (for some constant k that depends on the problem Π and the algorithm A):

1. Compute a distance- k $O(\Delta^{2k})$ -coloring of the graph;
 2. Run a k -round algorithm B that uses the computed coloring to produce the final output.
- In [21] is shown that the constant k , and the k -round algorithm B , can be mechanically determined from the original algorithm A . The runtime of algorithm A' is $O(\log^* n)$ rounds since this is the runtime for the first step, while the second step only requires constant time.

Why it Works. The high-level purpose of computing the coloring in Item 1 is to provide new identifiers at the nodes that are unique up to distance k and come from a much smaller space than the original identifiers (that are part of the setting in the LOCAL model). Roughly speaking, this ensures that the k -hop view of any node that interprets the computed colors as identifiers is consistent with the node living in a constant-sized graph (with a constant-sized identifier space).

In [21], it is argued why this approach works, and on a high level, the reason can be summarized as follows. For some sufficiently large constant k , algorithm A can be executed on all graphs of a suitable constant size with a runtime of just k rounds. Since each node of the original graph executing this k -round algorithm cannot distinguish between living in

the original graph with the generated new identifiers and living in (a suitable) one of these constant-sized graphs (on all of which the algorithm is correct), the k -round algorithm must also be correct on the (much larger) original graph. This is just a high-level sketch of the proof presented in [21]; there are a number of intricate details that have to be taken care of and are explained in [21].

How We Proceed. For our purpose, we do not actually need to know the details of [21] on how A' is constructed as a function of A , and we just use the following statement that comes from [21]: if the problem Π can be solved in $o(\log n)$ deterministic rounds or $o(\log \log n)$ randomized rounds, then it can also be solved in $O(\log^* n)$ deterministic rounds using an algorithm that first applies Item 1 and then applies Item 2. In fact, in our case, we are not even given the algorithm A as input: we just know that the problem can be solved in $o(\log n)$ deterministic or $o(\log \log n)$ randomized rounds, but we are not given an algorithm A with such a complexity. Hence, we cannot apply the construction of [21] directly.

In Section 7 of the full version, we show that this is not an issue, in the sense that, if an algorithm exists, then it can be found by brute force. To show that, we use the following two important ingredients presented in [50]:

- Any constant time algorithm that solves an LCL in the LOCAL model can be transformed into an algorithm that does not require nodes to have IDs.
- For every k , it is decidable whether there exists a k -round algorithm that solves a given problem in a setting where we do not have IDs and we are given a (suitable) distance- k coloring. The reason is that, in this setting, there are only a finite number of possible algorithm candidates (and they can be enumerated), and given a candidate, it is possible to check if it constitutes a correct algorithm by using a centralized offline procedure.

We use the above ingredients as follows. If we just know that Π can be solved in $o(\log n)$ deterministic rounds or $o(\log \log n)$ randomized rounds, even if no algorithm is given, we can use [21] to claim that there exists a k for which there is a k -round algorithm B that solves Π given a distance- k coloring, and then use the first ingredient to claim that this algorithm does not need the presence of IDs. Finally, we use the second ingredient to say that if we try increasing values of k , we are going to find the algorithm B that we need.

From the above discussion, in order to prove Theorem 5, we only need to show how to compute a distance- k $O(\Delta^{2k})$ -coloring in $O(\log \log^* n)$ deterministic MPC rounds.

3.1 LOCAL Algorithm

We start by presenting an algorithm for computing such a coloring in the LOCAL model. While computing such a coloring in the LOCAL model is easy, we present an algorithm amenable to be converted into a faster MPC algorithm. This algorithm is not new: it has been already presented in [38, 51], and we report it here, with minor modifications, for completeness.

► **Lemma 6.** *For any constant k , the distance- k $O(\Delta^{2k})$ -coloring problem on general graphs can be solved in the LOCAL model with a deterministic algorithm running in $O(\log^* n)$ rounds.*

Proof. We present an algorithm that is able to compute an $O(\Delta^2)$ coloring of a given graph G , where Δ is the maximum degree of G , in $O(\log^* n)$ rounds. By simulating such an algorithm on G^k , the k -th power of G , which has maximum degree Δ^k , we obtain the claimed result. Note that the running time is also asymptotically the same, since k is a constant.

The algorithm works as follows. At the beginning, each edge is oriented arbitrarily. Then, each node marks its incident outgoing edges with different numbers from $\{1, \dots, \Delta\}$. In this way, we decomposed our graph G into Δ edge-disjoint directed subgraphs $G_1, \dots, G_\Delta$,

where each G_i is the graph induced by edges marked i . Also, notice that by construction, for each i , each node in G_i has at most a single outgoing edge, and hence each G_i is a directed pseudoforest.

Assume we can color each directed pseudoforest with 3 colors in $O(\log^* n)$ rounds. Then, we can obtain a proper coloring for the nodes of G with 3^Δ colors, by letting each node construct the tuple $c(v) = (c_1(v), \dots, c_\Delta(v))$, where $c_i(v)$ is the color of v in G_i . In fact, consider two neighboring nodes u and v connected through an edge e . Assume that e is oriented from u to v , and that u marked e with value i . Then, in G_i , u and v are neighbors, and hence they obtained different colors $c_i(u)$ and $c_i(v)$, implying that $c(u) \neq c(v)$. Once a 3^Δ -coloring is obtained, we can then spend $O(3^\Delta)$ rounds to reduce the number of colors to $O(\Delta^2)$, by using a simple greedy algorithm.

We now show that each pseudoforest can be 3-colored efficiently. Let P be an arbitrary pseudotree. At first, we can use the IDs of the nodes to produce a $\text{poly}(n)$ -coloring of P . Then we apply 1 round of Linial's coloring algorithm [47] in order to obtain an $O(\log n)$ -coloring of P . While this step of coloring is not necessary for the LOCAL algorithm, it allows us to reduce the amount of information that we will later need to transmit in the MPC algorithm. Nodes can then spend $T = O(\log^* n)$ rounds to gather the color of their successors in P at distance at most T , and it is known that, with this information, nodes can compute a proper coloring of P , by simulating $O(\log^* n)$ steps of a color reduction algorithm for directed paths [38, 23]. ◀

3.2 MPC Implementation

We now show how to convert the LOCAL algorithm into an exponentially faster low-space MPC algorithm. The LOCAL algorithm consists of two main steps: The distance- k $O(\Delta^{2k})$ -coloring and the k -round algorithm. Since k and Δ are constant, the latter step is trivial, and the former step can be computed efficiently using graph exponentiation, where nodes keep track of the IDs of the two outermost nodes, and the colors of all nodes in between. Lemma 9 of the following paragraph proves the former step, completing the proof for Theorem 5. Component-stability and compatibility with disconnected components follows directly from the fact that all arguments are local, i.e., nodes in separate components never communicate, and that the runtime depends only on N .

Distance- k Coloring. We show that the initial distance- k coloring can be computed in $O(\log \log^* n)$ low-space MPC rounds, while respecting linear global memory. First, we observe that using the standard graph exponentiation technique, we can compute the k th power of a graph; for constant k , the memory overhead is only a constant. Then, we will apply techniques similar to the ones used in the LOCAL model in Lemma 6.

► **Observation 7.** *For an input graph G with n nodes, m edges, and maximum degree Δ , the power graph G^k can be computed deterministically in $O(\log k)$ low-space MPC rounds with $O(\Delta^k)$ words of local and $O(m + n \cdot \Delta^k)$ words of global memory, as long as $\Delta^k < n^\delta$.*

► **Observation 8.** *Every k -round LOCAL algorithm can be simulated in $O(\log k)$ low-space MPC rounds with $O(\Delta^k)$ words of local and $O(m + n \cdot \Delta^k)$ words of global memory, as long as $\Delta^k < n^\delta$. If the LOCAL algorithm is deterministic, then the MPC algorithm is deterministic as well.*

Proof. Using Observation 7, we can collect the k -hop neighborhood of each node and hence, simulate a k -round LOCAL algorithm in an additional $O(1)$ low-space MPC rounds. Observe that this also holds for general graphs. ◀

► **Lemma 9.** *The distance- k $O(\Delta^{2k})$ -coloring problem on general graphs can be solved in the low-space MPC model with a $O(\log \log^* n + \log k)$ -time deterministic algorithm, as long as $\Delta^k < n^\delta$. The algorithm requires $O(\Delta^k)$ words of local and $O(m + n \cdot \Delta^k)$ words of global memory. If k and Δ are constants, the runtime reduces to $O(\log \log^* n)$ and we require $O(1)$ words of local and $O(m + n)$ words of global memory.*

Proof. Using Observation 7, we can first compute G^k in $O(\log k)$ rounds, and operate on G^k instead of the input graph G henceforth. The application of Observation 7 requires $O(\Delta^k)$ words of local memory and $O(m + n \cdot \Delta^k)$ words of global memory. Then, similarly to Lemma 6, we can reduce the coloring problem to $O(1)$ -coloring of directed pseudoforests that are initially colored with $O(\log n)$ colors.

Next, our goal is to use the graph exponentiation technique such that each node can collect the topology and the colors of its $O(\log^* n)$ successors in its pseudoforest in $O(\log \log^* n)$ time. Here, we have to take care of the subtle detail that the *color* of a successor is not enough to determine the machine on which this successor lies. Suppose that each node is initially labeled with its $O(\log \log n)$ -bit color and its $O(\log n)$ -bit identifier that encodes both the identity (color) of the node and the machine containing the node. Then, in round 1, each node knows the identifier and the color of its successor. For an inductive argument, suppose that each node u knows the identifier the successor v_i in distance i and the vector of colors of all nodes in between u and v_i , on the directed path from u to v_i . Then, in $O(1)$ MPC rounds, u can learn the identifier of the $2i$:th successor v_{2i} and the colors of all nodes between u and v_{2i} . After learning the identifier of v_{2i} , node u can forget about the identifier of v_i and hence, u only keeps track of one identifier. By induction, node u learns the colors of its $O(\log^* n)$ successors in $O(\log \log^* n)$ MPC rounds.

Using the vector of colors of the successors, in $O(1)$ MPC rounds, each node can simulate the $O(\log^* n)$ -time LOCAL algorithm to obtain an $O(\Delta^{2k})$ -coloring. This requires $O(\log^* n \cdot \log \log n + \log n) = O(\log n)$ bits of memory per node per pseudoforest that the node belongs to, counting the colors of the successors and the identifier of the furthest successor. Altogether, this results in a global memory requirement of $O(n \log n \cdot \Delta^k)$ bits which fits $O(n \cdot \Delta^k)$ words. ◀

4 The High Regime

In this section, we will prove that all solvable LCL problems on forests, i.e., *all LCL problems that have a correct solution on every forest*, can be solved deterministically in $O(\log n)$ time in the low-space MPC model using $O(m + n)$ words of global memory. Our proof is constructive: we explicitly provide, for any solvable LCL, an algorithm that has a runtime of $O(\log n)$. In fact, our construction can be used to find an $O(\log n)$ -time algorithm *even for unsolvable LCLs*, with the guarantee that on any instance that admits a correct solution the given output will be correct (while the algorithm detects it if no solution exists). We show the following theorem.

► **Theorem 10.** *For any solvable LCL problem Π on a forest, there is an $O(\log n)$ -time deterministic low-space MPC algorithm that is component-stable and uses $O(m + n)$ words of global memory.*

In the full version, we provide a method to solve any LCL on forests if we can solve it on trees. Hence, we can restrict attention to trees.

4.1 High-level Overview of the Algorithm and Its Analysis

Consider an arbitrary solvable LCL problem Π on trees. In the following, we will give a slightly simplified view of the algorithm we will use to solve Π in $O(\log n)$ time. First, we root the input tree by using the $O(\log n)$ -round rooting algorithm described in the full version. Then, on a high level, the rest of the algorithm proceeds in 2 phases.

In the first phase, which we will refer to as the *leaves-to-root phase*, roughly speaking, the goal is to compute, for a substantial number of edges $e = (u, v)$, the set of output labels that can be output at half-edge (v, e) such that the label choice can be extended to a (locally) correct solution in the subtree hanging from v via e . This is done in an iterative manner, proceeding from the leaves towards the root. When, at last, the root has computed this set of output labels for each incident half-edge, it can, on each such half-edge, select an output label from the computed set such that the obtained node configuration is contained in the node constraint of Π and the input-output constraints of Π (given by the function g in the definition of Π) are satisfied. Such a selection must exist due to the fact that Π has a correct solution on the considered instance. We refer to these sets as the *completeness information*.

The second phase, which we will refer to as the *root-to-leaves phase*, consists of completing the solution from the root downwards, by iteratively propagating the selected solution further towards the leaves. With the same argumentation as at the root, certain nodes v can select an output label at the half-edge leading to its parent and output labels from the sets computed on its incident half-edges leading to its children such that the obtained node configuration is contained in the node constraint of Π , the obtained edge configuration on the edge from v to its parent is contained in the edge constraint of Π , and the input-output constraints of Π are satisfied. The fact that the selected labels come from the sets computed in the first phase ensures that after each choice the current partial solution is part of a correct global solution. While this outline sounds simple, there are a number of intricate challenges to make the mentioned ideas work in $O(\log n)$ rounds while staying within the memory bound of $O(m + n)$.

Unfortunately, if the depth of the input tree is $\omega(\log n)$ the outlined approach has $\omega(\log n)$ steps and running them sequentially is insufficient for an $O(\log n)$ -time algorithm. In order to mitigate this issue, we will not only process the leaves of the remaining unprocessed tree in each iteration, but also the nodes of degree 2, inspired by the rake-and-compress decomposition by Miller and Reif [49] which guarantees that after $O(\log n)$ iterations of removing all degree-1 and degree-2 nodes all nodes have been removed. The advantage of degree-2 nodes over higher-degree nodes w.r.t. storing completeness information (as in the above outline) is that they form paths, which by definition only have two endpoints; the idea, when processing such a path, is to simply store in the two endpoints the information for which pairs of labels at the two half-edges at the ends of the path there exists a correct completion of the solution inside the path. This allows to naturally add processing degree-2 nodes to the leaves-to-root phase, while for the root-to-leaves phase, the information stored at the endpoints s, t of a path essentially allows us to start extending the current partial solution on the path itself (and thereafter on the subtrees *hanging* from nodes on the path) one step after the output labels at s and t are selected. Note that the degrees of nodes change throughout the process due to the removal of nodes and hence new nodes might become degree-2 nodes after every step of the algorithm.

Unfortunately, there are further challenges in obtaining an $O(\log n)$ runtime. In the leaves-to-root phase, even when using graph exponentiation, processing a path of degree-2 nodes of length L involves coordination between its endpoints and takes $\Omega(\log L)$ time, whereas the $O(\log n)$ time guarantee of the rake-and-compress technique crucially relies on

the fact that each iteration (optimally, an iteration would remove all leaves and all degree-2 nodes) can be performed in constant time. Hence, essentially, we will only perform one step of graph exponentiation on paths in each iteration. Here, a new obstacle arises: before the graph exponentiation is finished, new nodes (that just became degree-2 nodes due to all except one of their remaining children being conclusively processed in the most recent iteration) might join the path. Nevertheless, we will show that this process still terminates in logarithmic time by designing a fine-tuned potential function that is inspired by the idea of counting how many nodes from certain groups of degree-2 nodes are contained in any fixed “pointer chain” from some leaf to the root.

Another issue is that we have to be able to store the completability information that we compute in the leaves-to-root phase until we use it (again) in the root-to-leaves phase. Recall that the graph exponentiation technique adds new edges/pointers. Even on paths their number can be up to logarithmic in n per node (even on average), yielding a logarithmic overhead in global memory.

In order to remedy this problem, we perform preprocessing before the leaves-to-root phase, and, as a result thereof, postprocessing after the root-to-leaves phase. The preprocessing can be thought of as a more memory-efficient (hence relatively slower) version of (a few iterations in) the leaves-to-root phase. It differs by processing the degree-2 nodes, i.e., paths, in a way that guarantees that the number of new edges introduced by the graph exponentiation (which we should rather call pointer forwarding at this point) on each path in each iteration is only a constant fraction of the length of the respective path. This is achieved by finding, in each iteration, a maximal independent set (MIS) on each path, letting only MIS nodes forward pointers, and removing the MIS nodes afterwards. The preprocessing runs for $\Theta(\log \log n)$ iterations, and computing an MIS on paths in each of them takes $O(\log^* N)$ time, where N is the size of the ID space. Note that due to the removal of vertices and the way we treat paths, new paths can appear in each iteration and we need to pay the $O(\log^* N)$ runtime in each iteration, yielding a runtime of $O(\log \log n \cdot \log^* N)$ for the preprocessing, which is much less than the target runtime of $O(\log n)$ rounds.

We will show that the number of remaining nodes is $O(n/\log n)$ after the preprocessing. This property ensures that the memory overhead of $O(\log n)$ edges per node introduced in the leaves-to-root phase does not exceed the desired global memory of $O(m + n)$ words. The postprocessing runs for $\Theta(\log \log n)$ iterations and is conceptually very similar to the preprocessing. We simply iteratively extend the partial solution (computed so far) on the edges that were processed during preprocessing, analogous to the approach in the root-to-leaves phase. Lastly, we also have to ensure that the local memory restrictions of low-space MPC are not exceeded.

References

- 1 Noga Alon, László Babai, and Alon Itai. A Fast and Simple Randomized Parallel Algorithm for the Maximal Independent Set Problem. *Journal of Algorithms*, pages 567–583, 1986. doi:10.1016/0196-6774(86)90019-2.
- 2 Alexandr Andoni, Aleksandar Nikolov, Krzysztof Onak, and Grigory Yaroslavtsev. Parallel algorithms for geometric graph problems. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 574–583, 2014. doi:10.1145/2591796.2591805.
- 3 Alkida Balliu, Sebastian Brandt, Yuval Efron, Juho Hirvonen, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Classification of Distributed Binary Labeling Problems. In *DISC*, pages 17:1–17:17, 2020. doi:10.1145/3382734.3405703.

- 4 Alkida Balliu, Sebastian Brandt, Juho Hirvonen, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. Lower Bounds for Maximal Matchings and Maximal Independent Sets. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 481–497, 2019. doi:10.1109/FOCS.2019.00037.
- 5 Alkida Balliu, Sebastian Brandt, Fabian Kuhn, and Dennis Olivetti. Distributed Δ -Coloring Plays Hide-and-Seek. In *Proceedings of the Symposium on Theory of Computing (STOC)*, 2022. arXiv:2110.00643.
- 6 Alkida Balliu, Sebastian Brandt, and Dennis Olivetti. Distributed Lower Bounds for Ruling Sets. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 365–376, 2020. doi:10.1109/FOCS46700.2020.00042.
- 7 Alkida Balliu, Sebastian Brandt, Dennis Olivetti, Jan Studeny, Jukka Suomela, and Aleksandr Tereshchenko. Locally Checkable Problems in Rooted Trees. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 263–272, 2021. doi:10.1145/3465084.3467934.
- 8 Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and J. Suomela. Almost Global Problems in the LOCAL Model. In *Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 9:1–9:16, 2018. doi:10.4230/LIPIcs.DISC.2018.9.
- 9 Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally Checkable Labelings with Small Messages. In *Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 8:1–8:18, 2021. doi:10.4230/LIPIcs.DISC.2021.8.
- 10 Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. New Classes of Distributed Time Complexity. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 1307–1318, 2018. doi:10.1145/3188745.3188860.
- 11 Alkida Balliu, Juho Hirvonen, Dennis Olivetti, and Jukka Suomela. Hardness of Minimal Symmetry Breaking in Distributed Computing. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 369–378, 2019. doi:10.1145/3293611.3331605.
- 12 Paul Beame, Paraschos Koutris, and Dan Suciu. Communication steps for parallel query processing. *Journal of the ACM (JACM)*, 64(6):40, 2017. doi:10.1145/3125644.
- 13 Soheil Behnezhad, Sebastian Brandt, Mahsa Derakhshan, Manuela Fischer, MohammadTaghi Hajiaghayi, Richard M. Karp, and Jara Uitto. Massively Parallel Computation of Matching and MIS in Sparse Graphs. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 481–490, 2019. doi:10.1145/3293611.3331609.
- 14 Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Łącki, and Vahab Mirrokni. Near-Optimal Massively Parallel Graph Connectivity. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1615–1636, 2020. doi:10.1109/FOCS.2019.00095.
- 15 Sebastian Brandt, Manuela Fischer, and Jara Uitto. Breaking the Linear-Memory Barrier in MPC: Fast MIS on Trees with Strongly Sublinear Memory. In *the Proceedings of the International Colloquium on Structural Information and Communication Complexity*, pages 124–138, 2019. doi:10.1007/978-3-030-24922-9_9.
- 16 Sebastian Brandt, Orr Fischer, Juho Hirvonen, Barbara Keller, Tuomo Lempiäinen, Joel Rybicki, Jukka Suomela, and Jara Uitto. A Lower Bound for the Distributed Lovász Local Lemma. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 479–488. ACM Press, 2016. doi:10.1145/2897518.2897570.
- 17 Sebastian Brandt, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Patric R.J. Östergård, Christopher Purcell, Joel Rybicki, Jukka Suomela, and Przemysław Uznański. LCL Problems on Grids. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 101–110, 2017. doi:10.1145/3087801.3087833.
- 18 Yi-Jun Chang. The Complexity Landscape of Distributed Locally Checkable Problems on Trees. In *Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 18:1–18:17, 2020. doi:10.4230/LIPIcs.DISC.2020.18.

- 19 Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. The Complexity of $(\Delta + 1)$ Coloring in Congested Clique, Massively Parallel Computation, and Centralized Local Computation. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 471–480, 2019.
- 20 Yi-Jun Chang, Qizheng He, Wenzheng Li, Seth Pettie, and Jara Uitto. Distributed Edge Coloring and a Special Case of the Constructive Lovász Local Lemma. *ACM Transactions on Algorithms (TALG)*, pages 1–51, 2019. doi:10.1145/3365004.
- 21 Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. An Exponential Separation between Randomized and Deterministic Complexity in the LOCAL Model. *SIAM Journal on Computing*, 48(1):122–143, 2019. doi:10.1137/17M1117537.
- 22 Yi-Jun Chang and Seth Pettie. A Time Hierarchy Theorem for the LOCAL Model. *SIAM Journal of Computing*, 48(1):33–69, 2019. doi:10.1137/17M1157957.
- 23 Richard Cole and Uzi Vishkin. Deterministic Coin Tossing with Applications to Optimal Parallel List Ranking. *Inf. Control.*, 70(1):32–53, 1986. doi:10.1016/S0019-9958(86)80023-7.
- 24 Sam Coy and Artur Czumaj. Deterministic massively parallel connectivity. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, 2022. doi:10.1145/3519935.3520055.
- 25 Artur Czumaj, Peter Davies, and Merav Parter. Graph Sparsification for Derandomizing Massively Parallel Computation with Low Space. In *Proceedings of the Symposium on Parallel Algorithms and Architectures (SPAA)*, pages 175–185, 2020. doi:10.1145/3350755.3400282.
- 26 Artur Czumaj, Peter Davies, and Merav Parter. Component Stability in Low-Space Massively Parallel Computation. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 481–491, 2021. doi:10.1145/3465084.3467903.
- 27 Artur Czumaj, Peter Davies, and Merav Parter. Improved Deterministic $(\Delta + 1)$ Coloring in Low-Space MPC. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 469–479, 2021. doi:10.1145/3465084.3467937.
- 28 Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. In *Communications of the ACM*, pages 107–113, 2008.
- 29 Michal Dory, Orr Fischer, Seri Khoury, and Dean Leitersdorf. Constant-Round Spanners and Shortest Paths in Congested Clique and MPC. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 223–233, 2021. doi:10.1145/3465084.3467928.
- 30 Manuela Fischer and Mohsen Ghaffari. Sublogarithmic Distributed Algorithms for Lovász Local Lemma, and the Complexity Hierarchy. In *Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 18:1–18:16, 2017. doi:10.4230/LIPIcs.DISC.2017.18.
- 31 Mohsen Ghaffari. An Improved Distributed Algorithm for Maximal Independent Set. In *Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 270–277, 2016. doi:10.1137/1.9781611974331.ch20.
- 32 Mohsen Ghaffari, Christoph Grunau, and Ce Jin. Improved MPC Algorithms for MIS, Matching, and Coloring on Trees and Beyond. In *Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 34:1–34:18, 2020. doi:10.4230/LIPIcs.DISC.2020.34.
- 33 Mohsen Ghaffari, Christoph Grunau, and Václav Rozhon. Improved Deterministic Network Decomposition. In *Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2904–2923, 2021. doi:10.1137/1.9781611976465.173.
- 34 Mohsen Ghaffari, David G. Harris, and Fabian Kuhn. On Derandomizing Local Distributed Algorithms. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 662–673, 2018. doi:10.1109/FOCS.2018.00069.
- 35 Mohsen Ghaffari, Fabian Kuhn, and Jara Uitto. Conditional Hardness Results for Massively Parallel Computation from Distributed Lower Bounds. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1650–1663, 2019. doi:10.1109/FOCS.2019.00097.

- 36 Mohsen Ghaffari and Hsin-Hao Su. Distributed Degree Splitting, Edge Coloring, and Orientations. In *Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2505–2523, 2017. doi:10.1137/1.9781611974782.166.
- 37 Mohsen Ghaffari and Jara Uitto. Sparsifying Distributed Algorithms with Ramifications in Massively Parallel Computation and Centralized Local Computation. In *Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1636–1653, 2019. doi:10.1137/1.9781611975482.99.
- 38 Andrew V. Goldberg, Serge A. Plotkin, and Gregory E. Shannon. Parallel symmetry-breaking in sparse graphs. *SIAM J. Discret. Math.*, 1(4):434–446, 1988. doi:10.1137/0401044.
- 39 Michael T. Goodrich, Nodari Sitchinava, and Qin Zhang. Sorting, Searching, and Simulation in the Mapreduce Framework. In *International Symposium on Algorithms and Computation (ISAAC)*, pages 374–383, 2011. doi:10.1007/978-3-642-25591-5_39.
- 40 Christoph Grunau, Václav Rozhon, and Sebastian Brandt. The landscape of distributed complexities on trees and beyond. In Alessia Milani and Philipp Woelfel, editors, *PODC '22: ACM Symposium on Principles of Distributed Computing, Salerno, Italy, July 25 - 29, 2022*, pages 37–47. ACM, 2022. doi:10.1145/3519270.3538452.
- 41 Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. In *SIGOPS Operating Systems Review*, pages 59–72, 2007. doi:10.1145/1272996.1273005.
- 42 Howard J. Karloff, Siddharth Suri, and Sergei Vassilvitskii. A Model of Computation for MapReduce. In *Proceedings of ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 938–948, 2010. doi:10.1137/1.9781611973075.76.
- 43 Raimondas Kiveris, Silvio Lattanzi, Vahab Mirrokni, Vibhor Rastogi, and Sergei Vassilvitskii. Connected Components in MapReduce and Beyond. In *ACM Symposium on Cloud Computing*, pages 18:1–18:13, 2014. doi:10.1145/2670979.2670997.
- 44 Fabian Kuhn, Thomas Moscibroda, and Roger Wattenhofer. Local computation: Lower and upper bounds. *Journal of the ACM (JACM)*, 63:17:1–17:44, 2016. doi:10.1145/2742012.
- 45 Christoph Lenzen and Roger Wattenhofer. Brief Announcement: Exponential Speed-Up of Local Algorithms Using Non-Local Communication. In *the Proceedings of the International Symposium on Principles of Distributed Computing (PODC)*, pages 295–296, 2010. doi:10.1145/1835698.1835772.
- 46 Nathan Linial. Distributive Graph Algorithms - Global Solutions from Local Data. In *the Proceedings of the IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 331–335, 1987. doi:10.1109/SFCS.1987.20.
- 47 Nathan Linial. Locality in Distributed Graph Algorithms. *SIAM Journal on Computing*, 21(1):193–201, 1992. doi:10.1137/0221015.
- 48 Michael Luby. A Simple Parallel Algorithm for the Maximal Independent Set Problem. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 1–10, 1985. doi:10.1145/22145.22146.
- 49 Gary L. Miller and John H. Reif. Parallel Tree Contraction Part 1: Fundamentals. *Adv. Comput. Res.*, 5:47–72, 1989.
- 50 Moni Naor and Larry Stockmeyer. What Can Be Computed Locally? *SIAM Journal on Computing*, 24(6):1259–1277, 1995. doi:10.1137/S0097539793254571.
- 51 Alessandro Panconesi and Romeo Rizzi. Some Simple Distributed Algorithms for Sparse Networks. *Distributed Computing*, 14(2):97–100, 2001. doi:10.1007/PL00008932.
- 52 David Peleg. *Distributed Computing: A Locality-Sensitive Approach*. Society for Industrial and Applied Mathematics, 2000. doi:10.1137/1.9780898719772.
- 53 Tim Roughgarden, Sergei Vassilvitskii, and Joshua R. Wang. Shuffles and Circuits (On Lower Bounds for Modern Parallel Computation). *J. ACM*, pages 1–12, 2018. doi:10.1145/3232536.
- 54 Václav Rozhon and Mohsen Ghaffari. Polylogarithmic-Time Deterministic Network Decomposition and Distributed Derandomization. In *Proceedings of the Symposium on Theory of Computing (STOC)*, pages 350–363, 2020. doi:10.1145/3357713.3384298.

- 55 Tom White. *Hadoop: The Definitive Guide*. O'Reilly Media, Inc., 2012. doi:10.5555/1717298.
- 56 Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010. doi:10.5555/1863103.1863113.

A Component-stability

The notion of a *component-stable* MPC algorithm has been introduced in [35] in the context of lifting distributed lower bounds to the MPC setting. It was later revised by Czumaj, Davies and Parter [26] and argued to be made more robust.

► **Definition 11** (Component-stability, [35]). *An MPC algorithm is component-stable if the outputs of nodes in different connected components are independent. Formally, assume that for a graph G , $\mathcal{D}_G$ denotes the initial distribution of the edges of G among the M machines and the assignment of unique IDs to the nodes of G . For a subgraph H of G let $\mathcal{D}_H$ be defined as $\mathcal{D}_G$ restricted to the nodes and edges of H . Let H_v be the connected component of node v . An MPC algorithm $\mathcal{A}$ is called component-stable if for each node $v \in V$, the output of v depends (deterministically) on the node v itself, the initial distribution and ID assignment $\mathcal{D}_{H_v}$ of the connected component H_v of v , and on the shared randomness S_M .*

In their revised definition, [26] assume the setting where all input graphs are legal.

► **Definition 12** (Legal graph). *A graph G is called legal if it is equipped with functions $ID, name: V(G) \rightarrow [\text{poly}(n)]$ providing nodes with IDs and names, such that all names are fully unique and all IDs are unique in every connected component.*

► **Definition 13** (Component-stability (revised), [26]). *A randomized MPC algorithm A_{MPC} is component-stable if its output at any node v is entirely, deterministically, dependent on the topology and IDs (but independent of names) of v 's connected component (which we will denote $CC(v)$), v itself, the exact number of nodes n and maximum degree Δ in the entire input graph, and the input random seed S . That is, the output of A_{MPC} at v can be expressed as a deterministic function $A_{MPC}(CC(v), v, n, \Delta, S)$. A deterministic MPC algorithm A_{MPC} is component-stable under the same definition, but omitting dependency on the random seed S .*

As opposed to [35], [26] allow the output of component-stable algorithms to depend on the total number of nodes in the graph and the maximum degree of the graph. Additionally, they assume the following setting: all input graphs are *legal* (see Definition 12), i.e., all nodes have an ID that is unique in every connected component, and a *name* that is unique across the whole input graph. Assuming the above setting, the output of a component-stable algorithm is allowed to depend on the IDs of all nodes in the same components, but not the names.

In our work, we adopt the revised definition of component-stability [26]. In all of our algorithms, nodes from different components only communicate in order to maintain a certain global synchrony. This synchrony influences *when* certain steps are executed and hence the *execution* of our algorithms. However, the output at each node is not influenced by the global communication.

Theorem 2 shows that the lower bounds for component-stable algorithms can be beaten for a large class of problems on trees and forests even with optimal memory. The long term effect of the term component-stable in this setting is unclear, but it provides room for many interesting open questions. One interesting aspect would be to see under which circumstances one can obtain algorithms with stronger component dependent guarantees, e.g., one may

want to develop algorithms for which not just the output of a node, but also the time until it has computed its output can only depend on the size of its component. Our algorithms do not meet this stronger definition. Besides an ID space dependence our algorithms have the following runtime behaviour. In the low and mid regime the time until we know the output of a node depends on the number of nodes in the largest connected component. In the high regime this time depends on the number of nodes in the whole graph. Going from trees to forests in the high regime relies on the recent beautiful (deterministic) connected components algorithm by Czumaj and Coy [24, 14].

Publication II

Alkida Balliu, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Optimal Deterministic Massively Parallel Connectivity on Forests. In *Proceedings of the ACM-SIAM Symposium on Discrete Algorithms (SODA) 2023*, pages 2589-2631, doi.org/10.1137/1.9781611977554.ch99, Florence, Italy, January 2023.

© 2023 ACM-SIAM

Reprinted with permission.

Optimal Deterministic Massively Parallel Connectivity on Forests*

Alkida Balliu
Gran Sasso Science Institute

Rustam Latypov[†]
Aalto University

Yannic Maus
TU Graz

Dennis Olivetti
Gran Sasso Science Institute

Jara Uitto
Aalto University

Abstract

We show fast deterministic algorithms for fundamental problems on forests in the challenging low-space regime of the well-known Massive Parallel Computation (MPC) model. A recent breakthrough result by Coy and Czumaj [STOC'22] shows that, in this setting, it is possible to deterministically identify connected components on graphs in $O(\log D + \log \log n)$ rounds, where D is the diameter of the graph and n the number of nodes. The authors left open a major question: is it possible to get rid of the additive $\log \log n$ factor and deterministically identify connected components in a runtime that is completely independent of n ?

We answer the above question in the affirmative in the case of forests. We give an algorithm that identifies connected components in $O(\log D)$ deterministic rounds. The total memory required is $O(n + m)$ words, where m is the number of edges in the input graph, which is optimal as it is only enough to store the input graph. We complement our upper bound results by showing that $\Omega(\log D)$ time is necessary even for component-unstable algorithms, conditioned on the widely believed 1 vs. 2 cycles conjecture. Our techniques also yield a deterministic forest-rooting algorithm with the same runtime and memory bounds.

Furthermore, we consider Locally Checkable Labeling problems (LCLs), whose solution can be verified by checking the $O(1)$ -radius neighborhood of each node. We show that any LCL problem on forests can be solved in $O(\log D)$ rounds with a canonical deterministic algorithm, improving over the $O(\log n)$ runtime of Brandt, Latypov and Uitto [DISC'21]. We also show that there is no algorithm that solves all LCL problems on trees asymptotically faster.

1 Introduction

Graphs offer a versatile abstraction to relational data and there is a growing demand for processing graphs at scale. One of the most central graph problems in massive graph processing is the detection of connected components of the input graph. This problem both captures challenges in the study of the fundamentals of parallel computing and has a variety of practical applications. In this work, we introduce new parallel techniques for finding connected components of a graph. Furthermore, we show that our techniques can be applied to solve a broad family of other central graph problems.

The Massively Parallel Computation (MPC) model [29] is a mathematical abstraction of modern frameworks of parallel computing such as Hadoop [38], Spark [39], MapReduce [25], and Dryad [28]. In the MPC model, we have M machines that communicate in synchronous rounds. In each round, every machine receives the messages sent in the previous round, performs (arbitrary) local computations, and is allowed to send messages to any other machine. Initially, an input graph of n nodes and m edges is distributed among the machines. At the end of the computation, each machine needs to know the output of each node it holds, e.g., the identifier of its connected component. We work in the low-space regime, where the *local memory* S of each machines is limited to n^δ words of $O(\log n)$ bits, where $0 < \delta < 1$. A word is enough to store a node or a machine identifier from a polynomial (in n) domain. The local memory restricts the amount of data a machine initially holds and is allowed to send and receive per round. Furthermore, we focus on the most restricted case of *linear total memory*, i.e., $S \cdot M = \Theta(n + m)$. Notice that $\Omega(n + m)$ words are required to store the input graph.

In recent years, identifying connected components of a graph has gained a lot of attention. As a baseline, the widely believed 1 vs. 2 cycles conjecture states that it takes $\Omega(\log n)$ rounds to tell whether the input graph is a

*The full version of the paper can be accessed at <https://arxiv.org/abs/2211.03530>

[†]Supported by the Academy of Finland, Grant 334238

cycle of n nodes or two cycles with $n/2$ nodes [36, 26, 14]. We note that proving any unconditional lower bounds seems out of reach as any non-constant lower bound in the low-space MPC model for any problem in P would imply a separation between NC^1 and P [36]. It has been shown that this conjecture also implies conditional hardness of detecting connected components in time $o(\log D)$ on the family graphs with diameter at most D [14, 22].

This bound has been almost matched in a sequence of works. First, a randomized $O(\log D \cdot \log \log_{m/n} n)$ time algorithm was designed in [2]. This was further improved to $O(\log D + \log \log_{m/n} n)$ in [14] and derandomized with the same asymptotic runtime in [22]. All of the aforementioned algorithms require only $O(n + m)$ words of global memory. A fundamental question is whether the runtime *necessarily* depends on n for some range of m ; we give evidence towards a negative answer. We show that in the case of forests, we can identify the connected components of a graph in $O(\log D)$ time, which we show to be *optimal* under the 1 vs. 2 cycles conjecture.

Connected Components on Forests. Consider the family of forests with component-wise maximum diameter D . There is a deterministic low-space MPC algorithm to find the connected components in time $O(\log D)$. The algorithm uses $O(n + m)$ global memory. Under the 1 vs. 2 cycles conjecture, this is optimal.

Sparsification and Dependence on n . In previous works on connected components, the algorithms have an inherent dependency on the total number of nodes n in the input graph. There is a technical reason for this dependency, also in the context of problems beyond connected components. A common algorithm design pattern is to first *sparsify* the input graph, i.e., the graph is made much smaller and the problem is solved in the sparser instance [2, 27, 24, 23]. Then, it is shown that a solution to the original input can be recovered from a solution on the sparsified graph. As an example, a method to sparsify graphs for connectivity is to perform node/edge contractions, that make the graph smaller and preserve connectivity.

In this pattern, the denser the input graph is, the more global memory the algorithm has on the sparsified graph, relatively speaking. In the aforementioned previous works, the base m/n of the logarithm can be replaced by F/n , where F is the global memory. Hence, if $F = n^{1+\Omega(1)}$, the dependency on n disappears. This suggests that the hardest instances are *sparse* graphs, as the n dependency in the runtime of $O(\log D + \log \log_{F/n} n)$ becomes better the larger the global memory F is. A limitation to solving connected components through independent node/edge contractions comes from the global memory bound. If the graph is already sparse, then the sparsification cannot make the graph any sparser, and hence we do not have an advantage in terms of global memory on the sparsified graph. In the case that $m = O(n)$ and the global memory is linear in n , the best we could hope for in the first round of contractions is to drop a constant fraction of the nodes. The low-level details for the reasons behind this can be extracted from the analysis of [2, 27, 24, 23]. Through the relative increase in global memory, the (remainder) graph size can be bounded by $n \cdot 2^{-2^i}$ in the i th round of contractions, which leads to an $\Omega(\log \log n)$ runtime.

In previous works, there is even more evidence towards sparse graphs being the hardest instances. Recently, it was shown that lower bound results from the LOCAL model of distributed message passing can be lifted to MPC under certain conditions [26, 24]. In the LOCAL model, almost all hardness results are obtained on trees or in high-girth graphs, implying lower bounds on forests with potentially many connected components [30, 5, 6, 8, 7, 11]. It was shown that a *component-stable* algorithm cannot solve a problem π faster than in $O(\log T(n, \Delta))$, where $T(n, \Delta)$ is the complexity of π in the LOCAL model¹ on a graph with n nodes and maximum degree Δ . Roughly speaking, an MPC algorithm is component-stable if the output on each node u only depends on the size of the graph and the connected component of u (see Definition 5.4 for more details [24]). While these methods do not yield unconditional hardness in the MPC model, we face similar difficulties in sparse graphs in the MPC model as in the message passing models.

Rooted Forests and Applications to Locally Checkable Problems. We believe that our technique to obtain connected components is of interest beyond solving the connectivity problem. For example, through minor adjustments to our technique, we obtain an algorithm that roots an (unrooted) input forest. Furthermore, we show that in a rooted tree, all *Locally Checkable Labeling* (LCL) problems can be solved very efficiently through a canonical algorithm. This generalizes to forests and gives an algorithm that can be executed on each connected component independently of the other components.

¹The LOCAL algorithm is allowed to access shared randomness.

Locally Checkable Labelings on Forests. On the family of forests with component-wise maximum diameter D , all LCL problems can be solved deterministically in $O(\log D)$ time in the low-space MPC model with $O(n + m)$ global memory. Under the 1 vs. 2 cycles conjecture, this is optimal.

A range of central graph problems, in particular in the area of parallel and distributed computing, are locally checkable, where the correctness of the whole solution can be verified by checking the partial solution around the local neighborhood of each node. In particular, the class of LCL problems consists of problems with a finite set of outputs per node/edge and a finite set of locally feasible solutions (see Definition 6.2), and includes fundamental problems such as MIS, node/edge-coloring and the algorithmic Lovász Local Lemma (LLL). Our work shows that any LCL problem can be solved in $O(\log D)$ time and that the same runtime can be obtained for many problems that are not restricted to finite descriptions. We complement our results by showing that for LCL problems, this bound is tight under the 1 vs. 2 cycles conjecture.

In recent works, the complexity of LCLs in MPC was compared against *locality* [17, 4], where locality refers to the round complexity of solving an LCL in the LOCAL model, as a function of n . It was shown that all LCLs on trees can be solved exponentially faster in MPC as compared to LOCAL. As a consequence, all LCLs on trees can be solved in $O(\log n)$ rounds in the low-regime MPC model. We note that it is often the case that the diameter of a graph is small, potentially much smaller than the locality of a certain graph problem (which is *independent* of the diameter). Hence, our novel technique significantly improves on the state-of-the-art runtimes for various graph problems in a broad family of graphs.

1.1 Our Contributions Our main contribution is an algorithm that deterministically detects the connected components of a forest in time logarithmic on the maximum component-wise diameter; crucially, independent of the size n of the input graph, whose dependence is inherently present in the techniques used in previous works. We also show that our approach is asymptotically optimal under the 1 vs. 2 cycles conjecture. Next, we present our results more formally.

THEOREM 1.1. (CONNECTED COMPONENTS) *Consider the family of forests. There is a deterministic low-space MPC algorithm to detect the connected components on this family of graphs. In particular, each node learns the maximum ID of its component. The algorithm runs in $O(\log D)$ rounds, where D is the maximum diameter of any component. The algorithm requires $O(n + m)$ words of global memory, it is component-stable, and it does not need to know D . Under the 1 vs. 2 cycles conjecture, the runtime is asymptotically optimal.*

The techniques for Theorem 1.1 can be extended to also obtain a rooted forest, where each node also knows the ID of the corresponding root.

THEOREM 1.2. (ROOTING) *Consider the family of forests with component-wise maximum diameter D . There is a deterministic low-space MPC algorithm that roots the forest in $O(\log D)$ rounds using $O(n + m)$ words of global memory, and it is component-stable.*

The rooting of the input forest gives us a handle for easier algorithm design and memory allocation in low-space MPC. As a concrete example, our results yield an $O(\log D)$ algorithm for deterministically 2-coloring forests. Without going into technical details, this can be achieved through a rather simple algorithm, where each node decides its color based on the parity of its distance to the root, and only needs to keep one pointer in memory for the parity counting. In a sense, we outsource the tedious implementation details to the rooting algorithm in Theorem 1.2 and obtain a convenient tool for algorithm design.

More broadly, we show how to solve any LCL problem in rooted forests in $O(\log D)$ deterministic rounds. LCLs have gotten ample attention in various distributed models of computation, e.g., [17, 10, 12, 20, 21]. Roughly speaking, the family of LCLs is a subset of the problems for which we can check if a given solution is correct by inspecting the constant radius neighborhood of each node. (see Definition 6.2 for a formal definition of LCLs). Furthermore, we show that for any fixed $D \in \Omega(\log n)$ and $D \in n^{o(1)}$, there cannot exist an algorithm that solves all LCL problems in time $o(\log D)$ in the family of unrooted forests of diameter at most D . This holds even if $\text{poly}(n)$ global memory is allowed.

THEOREM 1.3. (LCLS ON TREES) *Consider an LCL problem Π on forests and let D be the component-wise maximum diameter. There is a deterministic low-space MPC algorithm that solves Π in $O(\log D)$ rounds using $O(n + m)$ words of global memory. Under the 1 vs. 2 cycles conjecture, the runtime is asymptotically optimal.*

1.2 Challenges and Techniques A canonical approach to solve connected components on forests is to root each tree and identify each tree with the ID of the root. Also, examining the challenges in rooting demonstrates the challenges we face when identifying connected components. A natural approach to root a tree is to iteratively perform *rake* operations, i.e., pick all the leaves of the tree and each leaf picks the unique neighbor as its parent. This approach clearly roots a tree in $O(D)$ parallel rounds and furthermore, in the case of a forest, each tree performs its rooting process independently. If we ignore the memory considerations in the low-space MPC model, this process could be implemented in $O(\log D)$ rounds using the graph exponentiation technique, where, in $O(\log D)$ rounds, *every* node gathers their D -hop neighborhoods, i.e., the whole graph, to simulate the process fast. However, when we limit the global memory to $O(n + m)$, we get into trouble. A simulation through graph exponentiation requires that *all* nodes iteratively gather larger and larger neighborhoods *simultaneously*. With the strict memory bound, this implies that a node can only gather a *constant* radius neighborhood (and in non-constant degree graphs that we deal with even that is not possible!), which allows only for simulating a constant number of rake-iterations in one MPC round.

A hope towards a more efficient approach is to show that the amount of total memory *relative* to the nodes remaining in the graph increases as we rake the graph (similarly to previous work [2, 14, 24]). If one can reduce the size of the graph by a constant factor in each MPC round, then the available total memory increases by a constant factor per remaining node. Then, we can gather a slightly larger neighborhood per node in the next step of the simulation. However, even if we had this guarantee, the best we could hope for is a runtime that depends on n , since this approach relies on a progress measure that depends on shrinking the graph. Informally speaking, this observation says that we need to have a fundamentally different approach than gradually sparsifying the graph.

Balanced Exponentiation. One of our main technical contributions is to introduce a new method to gather a *part* of the neighborhood of each node that is balanced in the following sense. Suppose, for the sake of argument, that we have a rooted tree. Then, if a node u has, say, γ descendants, we ensure that u will only gather $O(\gamma)$ nodes in the direction of the root, i.e., the direction opposing its descendants. Furthermore, it will also gather its γ descendants, resulting in a memory demand of $O(\gamma)$ (for u). A crucial step in our analysis is to show that even if each node gathered their γ descendants and $O(\gamma)$ nodes in the direction of the root, we do not create too much redundancy and we respect the linear total memory bound. A key technical challenge here is that there is no way for a node to know who are its descendants (because the input graph is unrooted). We show that, without an asymptotic loss in the runtime, we can deterministically determine which neighbor of u is the *worst case* for a choice of a parent and gather the respective nodes slower.

Progress Measure. As mentioned above, to obtain a runtime independent of n , we need to avoid arguments that are based on the size of the graph getting smaller during the execution of our algorithm. The topology gathering through exponentiation can be seen as creating a virtual graph, where a virtual edge $\{u, v\}$ corresponds to the fact that u knows how to reach v and vice versa. The base of our progress measure is to aim to show that in this virtual graph, the diameter is reduced by a constant factor in each iteration. Unfortunately, having this type of guarantee seems too good to be true. Already on a path, it requires too much memory to create a virtual graph where the distances between all pairs of nodes are reduced. Our contribution is to show that this example is degenerate in the sense that either we can guarantee that the balanced exponentiation reduces the diameter or we can reduce it through a node-contraction type of operation.

1.3 Further Related Work In relation to our work, previous works have studied finding rooted spanning forests. In [15, 2], $O(\log D \cdot \log \log n)$ algorithms for rooting were given and the runtime was improved to $O(\log D + \log \log n)$ by [22].

Locally checkable problems have been intensively studied in the MPC model. Many classic algorithms from PRAM imply MPC algorithms with the same runtime, e.g., the MIS, maximal matching and coloring [34, 1]. The runtime of such simulations are typically polylogarithmic and, in MPC, the aim is to obtain something significantly faster. For MIS and maximal matching, there are $\tilde{O}(\sqrt{\log \Delta} + \log \log \log n)$ time algorithms [27] and $(\Delta + 1)$ -node-coloring can be solved in $O(\log \log \log n)$ rounds, even deterministically [19, 24].

Many of the current state-of-the-art algorithms for locally checkable problems are (at least to some degree) based on distributed message-passing algorithms. The common design pattern is to design a message-passing algorithm, for example in the LOCAL model of distributed computing [33] where the output of each node is decided according to their t -hop neighborhood in t -rounds. These algorithms are then implemented faster in the MPC model through the graph exponentiation technique [31] that, in the ideal case, collects the t -hop ball around each node in $O(\log t)$ -rounds. This framework was used to obtain an exponential speedup for many locally checkable problems in general, and in particular, it was used recently to show that all LCL problems on trees with t -round complexity in LOCAL can be solved in $O(\log t)$ MPC rounds [17, 4]. Our work broadens our understanding on the complexities of LCLs as a function of the diameter, which is somewhat orthogonal to previous works.

On a technical level, a related work gave a clever approach to encode the feasible outputs around each node into a constant sized *type* of the node [21]. Given a rooted tree, the type of a node u (or its subtree) is determined through the set of possible outputs of its descendants. This encoding gives rise to an efficient convergecast protocol, where the root learns its type and effectively broadcasts a valid global solution to the rest of the tree. In a recent work, related techniques were used to implement a message passing algorithm for LCLs on trees using small messages [10]. In our work, we employ similar ideas to aggregate and broadcast information efficiently through the input tree.

Lower Bounds. While simulating LOCAL message passing algorithms in MPC has been fruitful in algorithm design, there is an inherent limitation to this approach. A naïve implementation results in a *component-stable* algorithm, where we can show that the simulation cannot be more than exponentially faster than the message passing algorithm [26, 24]. An algorithm is said to be *component-stable* if the output on node v depends (deterministically) only on the topology, the input of the nodes, and the IDs of the nodes in the connected component of v . Furthermore, the output is allowed to depend on the number of nodes n , the maximum degree Δ of the input graph, and in case of randomized algorithms, the output can depend on shared randomness. It was shown that for component-stable algorithms and under the 1 vs. 2 cycles conjecture, $\Omega(\log t)$ rounds cannot be beaten if t is a lower bound on the complexity of the given problem in the LOCAL model. We emphasize that our lower bounds also work for component-unstable algorithms (still relying on the 1 vs. 2 cycles conjecture).

2 Overview, Roadmap and Notation

Our formal results are presented in Sections 4 to 7. In Section 3, we present the core techniques of our algorithm. The formal version of this algorithm appears in Section 4.

Section 4: This section contains the most involved part of our work, i.e., an algorithm that lets every node of an input tree output the maximum identifier of the tree. On a tree G , our algorithm runs in $O(\log \hat{D})$ rounds and uses global memory $O((n + m) \cdot \hat{D}^3)$ where the parameter $\hat{D} \in [\text{diam}(G), n^{1/8}]$ needs to be known to the algorithm. This sounds like a foolish approach, as this problem can be trivially solved in $O(1)$ rounds if we were really given a single tree as input. We still chose to present our result in this way, as our seemingly naïve algorithm is the core of our connected components algorithm that we present in Section 5.

Section 5: In fact, we show that our algorithm can be correctly extended to forests. Note that if every node knows the maximum identifier of its tree, we automatically solve the connected components problem. In this section, we also show how to remove the requirement of knowing $\hat{D}$ via doubly exponentially increasing guesses for $\hat{D}$. We also show that we can reduce the overall memory requirement to $O(n + m)$ by preprocessing the graph, that is, we spend additional $O(\log \hat{D})$ rounds to reduce the size of the graph by a factor $\hat{D}^3$. Lastly, we show that the runtime reduces to $O(\log \max_i \{D_i\})$ where D_i is the diameter of the i -th component of the input graph. In this section, we also present the full proof of our connected components algorithm (Theorem 1.1) and our rooting algorithm (Theorem 1.2).

Section 6: In this section, we show a nice application of our rooting algorithm from Theorem 1.2. In particular, we show that any LCL problem can be solved in just $O(\log D)$ rounds, once each tree of the forest is rooted (Theorem 1.3).

Our approach has a dynamic programming flavor and we explain it for a single tree of the forest. We iteratively reduce the size of the tree, by compressing small subtrees into single nodes, and paths into single edges. While performing these compressions, we set additional constraints on the solution allowed on the nodes into which we compress subtrees, and on the edges that represent compressed paths. We maintain the invariant that, if we obtain a solution in the smaller tree, then it can be extended to the original one. We show that, by performing a constant number of compression steps, we obtain a tree that is comprised of a single node, where it

is straightforward to compute a solution. We then perform the same operations in the reverse order, in order to extend the solution to the whole tree. All of this is preceded by using Theorem 1.2 to compute a rooting of the tree/forest. The rooting helps, as with a given rooting it is significantly easier to identify the suitable subtrees to compress without breaking memory bounds.

Section 7: In this section, we show that the runtimes of our algorithms are tight, conditioned on the widely believed 1 vs. 2 cycles conjecture. Our aim is to use a reduction from the 1 vs. 2 cycles problem to solving connectivity on paths. In previous work [26], a reduction to connectivity on paths was introduced, but for technical reasons, it is not sufficient for our purposes. We require a guarantee that each path is of bounded diameter, which is not directly guaranteed by the previous work. Hence, we start by defining a problem on forests, called D -diameter s - t path-connectivity, for which we can prove conditional hardness. By a reduction, we obtain a conditional lower bound of $\Omega(\log D)$ for the connected components problem.

We then define an LCL problem such that, given an algorithm for it, we can use it to solve the D -diameter s - t path-connectivity problem. Hence, we obtain a lower bound of $\Omega(\log D)$ for the problem, implying that our generic LCL solver is also conditionally tight.

2.1 Definitions and Notation Given a graph $G = (V, E)$, we denote with Δ the maximum degree of G , with $n = |V|$ the number of nodes in G , and with $m = |E|$ the number of edges in G . We denote with $N_G(v)$ the neighbors of v , that is, the set $\{u \mid \{u, v\} \in E\}$. We denote with $\deg_G(v)$ the degree of a node v , that is, the number of neighbors of v in G . If G is clear from the context, we may omit G and simply write $N(v)$ and $\deg(v)$. If G is a directed graph, we denote with $\deg(v)$ the degree of v in the undirected version of G , and with $\deg_{\text{in}}(v)$ and $\deg_{\text{out}}(v)$ its indegree and its outdegree, respectively. We define $\text{dist}_G(u, v)$ as the hop-distance between u and v in G . Again, we may omit G if it is clear from the context. The *radius- r neighborhood* of a node v is the subgraph $G_r(v) = (V_r(v), E_r(v))$, where $V_r(v) = \{u \in V : \text{dist}(u, v) \leq r\}$, and $E_r(v) = \{(u, w) \in E : \text{dist}(v, u) \leq r \text{ and } \text{dist}(v, w) \leq r\}$. Also, we denote with G^k the k -th power of G , that is, a graph containing the same nodes of G , where we connect two nodes u and v ($u \neq v$) if and only if they satisfy $\text{dist}_G(u, v) \leq k$. The *eccentricity* of a node v in a graph G is the maximum of $\{\text{dist}(u, v) \mid u \in V\}$.

3 The MAX-ID Problem: Overview and Techniques

In this section, we present the core techniques of our specialized algorithm for solving the MAX-ID problem, that is the core ingredient for solving connected components on forests (upper bound of Theorem 1.1). In the MAX-ID problem, one is given a connected tree with a unique identifier for each node, and all nodes must output the maximum identifier in the tree. We note that it is trivial to solve the problem in $O(1)$ MPC rounds using a broadcast tree; however, this approach does not extend to forests, and hence a more sophisticated solution is required. The purpose of this section is to present the high level ideas of an algorithm that solves MAX-ID and can also be extended to forests. Some lemma statements have been adapted to fit this (informal) version.

Lemma 4.1 (Solving MAX-ID on trees). *Consider the family of trees. There is a deterministic low-space MPC algorithm that solves MAX-ID on any graph G of that graph family when given $\hat{D} \in [\text{diam}(G), n^{1/8}]$. The algorithm runs in $O(\log \hat{D})$ rounds, is component-stable², and requires $O(m \cdot \hat{D}^3)$ words of global memory.*

We begin with definitions that are essential not only to define our algorithm but also for proving its memory bounds. Let v be a vertex of a tree G . For all nodes $u \in N(v)$, define

$$G_{v \rightarrow u} = \{w \in V(G) \mid u \text{ is contained in the shortest path from } v \text{ to } w\}$$

to be all nodes in the tree that are reachable from v via u , including u . Also, let $G_{v \nrightarrow u} := V(G) \setminus G_{v \rightarrow u}$. For every $w \in G$, let $r_v(w)$ be the node $u \in N(v)$ satisfying that $w \in G_{v \rightarrow u}$, i.e., $r_v(w)$ is the neighbor of v which is on the unique path from v to w .

DEFINITION (LIGHT AND HEAVY NODES) *Let $0 < \delta < 1$ be a constant. A node v is light against a neighbor $u \in N(v)$ if $|G_{v \nrightarrow u}| \leq n^{1/8}$. A node is light if it is light against at least one of its neighbors. Nodes that are not light are heavy.*

²By the formulation of Definition 4.1, any algorithm solving MAX-ID is component-stable by definition. This is discussed in detail in Section 5.4

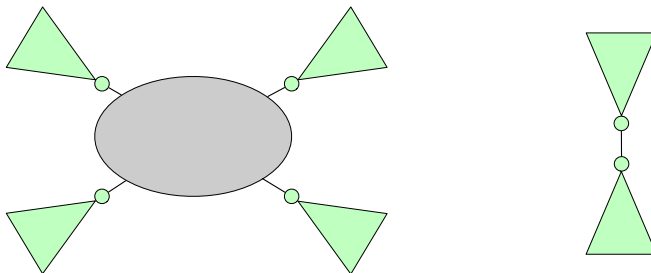


Figure 1: Light nodes are green and heavy nodes are gray.

If there are no heavy nodes, the graph is small and fits into the local memory of one machine.

LEMMA (SEE LEMMA 4.4) *Any tree with no heavy nodes contains at most $2n^{1/2}$ vertices.*

We prove that as soon as there is at least one heavy node, the graph has to look like the one depicted on the left hand side of Figure 1, that is, light subtrees that are attached to a connected component of heavy nodes. We exploit this structure in our algorithm.

3.1 MAX-ID: The Algorithm The high-level idea is to iteratively compress parts of the graph (without disconnecting it) such that the knowledge of the maximum identifier of the compressed parts is always kept within the resulting graph. We repeat this process until there remains only one node, that knows ID , the maximum identifier in the graph. Then, we backtrack the process by iteratively decompressing and broadcasting the knowledge about ID . Eventually, we are left with the original graph where all nodes know ID .

More in detail, our algorithm consists of $\ell = O(1)$ phases and the same number of reversal phases. During the phases, we first compress all light subtrees into single nodes (a procedure that we refer to as **CompressLightSubTrees**) and then replace all paths by a single edge (**CompressPaths**). We denote the resulting graphs by $G_0, G_1, \dots, G_\ell$. The phases are followed by reversal phases, in which we undo all compression steps of the regular phases in reverse order to spread ID to the whole graph.

Bounding the number of Phases. Consider some phase i and graph G_i with heavy nodes that looks as illustrated in Figure 1 (left). If we remove all light subtrees from graph G_i , for the resulting graph G_{i+1} , it holds that every leaf (aka a formerly heavy node) corresponds to a distinct removed subtree of size at least $n^{1/8}$ (if the subtree was smaller the leaf would not be heavy). If we then contract all paths in G_{i+1} into single edges, leaving no degree-2 nodes in G_{i+1} , it holds that at least half of the nodes in G_{i+1} corresponds to a removed subtree. As each of these subtrees has $\geq n^{1/8}$ distinct nodes, we have removed a polynomial-in- n fraction of nodes from G_i to obtain G_{i+1} . Hence, we can only repeat the process a constant time until the graph becomes small.

3.2 MAX-ID: Compressing Light Subtrees For the sake of this high level overview, we focus on our most involved part, the procedure that compresses (maximal) light subtrees into the adjacent heavy node (**CompressLightSubTrees**). The difficulty is that nodes do not know whether they are light or heavy, and already one single exponentiation step in the “wrong” direction of the graph can ruin local and global memory bounds. However, there seems to be no way to obtain a runtime that is logarithmic in the diameter without exponentiation. Thus, we perform careful exponentiations that always ensure the memory bounds but at the same time make enough progress.

Consider a graph G with n nodes—starting from the second phase we will actually use this algorithm on graphs with fewer than n nodes. At all times, every node v has some set of nodes S_v in its memory, which we initialize to $N(v)$. Set S_v can be thought of as the node’s view or knowledge. During the execution, S_v grows, and if $|S_v| \geq 2n^{1/4}$, v becomes full. Similarly to definitions $G_{v \rightarrow u}$ and $G_{v \nrightarrow u}$, let us define the following. For a node v and a node $u \in N(v)$, let $S_{v \rightarrow u} = S_v \cap G_{v \rightarrow u}$. Also, let $S_{v \nrightarrow u} := S_v \setminus S_{v \rightarrow u}$.

All nodes in the graph have the property that they are either light or heavy. Initially, nodes themselves do not know whether they are light or heavy, since these properties depend on the topology of the graph. During

the algorithm each node is in one of the four states: **active**, **happy**, **full**, or **sad**. Initially, all nodes are **active**. A node v becomes **happy**, if at some point during the execution, there exists $u \in N(v)$ such that $G_{v \nrightarrow u} \subseteq S_v$ and $|G_{v \nrightarrow u}| \leq n^{1/8}$. If a node, that is not full, realizes that it can never become **happy** (for example by having $|S_{v \rightarrow u}| > n^{1/8}$ for two different neighbors u), it becomes **sad**. Upon becoming **happy**, **sad** or **full**, nodes do not partake in the algorithm except for answering queries from **active** nodes. We call nodes **unhappy** if they are in some other state than **happy** (including state **active**). The goal is that all light nodes become **happy**. We will prove that the algorithm that we will provide satisfies the following lemma.

LEMMA 4.16. *After $O(\log \hat{D})$ iterations, all light nodes become **happy**, while heavy nodes always remain **unhappy**.*

Intuition for its correctness requires further details and is deferred to the end of this section.

When comparing the definitions of **happy** and **light**, it is evident that when a node becomes **happy**, it knows that it is **light**. Similarly, a node becoming **full** or **sad** knows that it is **heavy**. At the end of the algorithm, **happy** nodes with a **full** or **sad** neighbor compress their whole subtree in that neighbor. A crucial challenge here is to ensure that these compressions are not conflicting as all such nodes execute these in parallel and without a global view.

Exponentiation. Recall the definition of $r_v(w)$ at the beginning of the section. For a node v and any $X \subseteq N(v)$, define an exponentiation operation as

$$\text{Exp}(X) : S_v \leftarrow \bigcup_{u \in X} \bigcup_{w \in S_{v \rightarrow u}} S_{w \nrightarrow r_w(v)}.$$

We say that node v *exponentiates in the direction of* $u \in N(v)$ if v performs $\text{Exp}(X)$ with $u \in X$.

The algorithm consists of $O(\log \hat{D})$ iterations, in each of which nodes perform a carefully designed graph exponentiation procedure. The aim is for light nodes v to become **happy** by learning their subtrees, after which, (certain) light nodes compress into their **unhappy** neighbor.

Failed Exponentiation Approaches. If there were no memory constraints and every node could do a proper (uniform) exponentiation step in every iteration of the algorithm, i.e., execute $\text{Exp}(N(v))$, after $O(\log \hat{D})$ iterations *all* nodes would learn the whole graph—a proper exponentiation step executed on all nodes halves the diameter—and the highest ID node could compress the whole graph into itself. However, uniform exponentiation would result in all nodes exceeding their local memory $O(n)$, and also significantly breaking the global memory requirement. Even if we were to steer the exponentiation procedure such that light nodes would learn a D_v radius ball around them, where D_v is the diameter of their light subtree, this would still break global memory. In fact, we cannot even do a single exponentiation step for all nodes in the graph without breaking memory bounds!

Our Solution (Careful Exponentiation & Probing). Hence, we let every node exponentiate in all but one direction, sparing the direction which currently looks most likely to be towards the heavy parts of the graph. Note that the knowledge of a node about the tree changes over time and in different iterations it may spare different directions. This step is further complicated as nodes neither know whether they are heavy or light nor do they know the size of their subtree, nor in which direction the heavy parts of the graph lie. Thus, in our algorithm, nodes perform a careful probing for the number of nodes into all directions to determine in which directions they can safely exponentiate without using too much memory. More formally, a node v computes $B_{v \rightarrow u} = \sum_{w \in S_{v \rightarrow u}} |S_{w \nrightarrow r_w(v)}|$ for every neighbor $u \in N(v)$ as an estimate for the number of nodes it may learn when exponentiating towards u . This estimate may be inaccurate and may contain a lot of doublecounting. The precise guarantees that this probing provides are technical and presented in Section 5.

We now reason (in a nutshell) why this algorithm meets the memory requirements and why we still make enough progress in order to make all light nodes **happy** in $O(\log \hat{D})$ iterations.

Local and Global Memory Bounds. If there were no memory limitation, we would already know that after $\ell = O(1)$ phases G_ℓ would consist of a single node r . For the sake of analysis, we assume a rooting of G at r . We emphasize that fixing a rooting is only for analysis sake, and we do not assume that the tree is actually rooted beforehand.

Given the rooting at r , we define $T(v, r)$ as the subtree rooted at v (including v itself). Then, the following lemma is crucial to bound the memory. The lemma is standalone as it does not use any properties of r .

LEMMA 4.5. *Consider an n -node tree T with diameter D that is rooted at node r . Let $T(v, r)$ denote the subtree rooted at v (including v) when T is rooted at r . It holds that $\sum_{v \in V} |T(v, r)| \leq (D + 1) \cdot n$.*

Proof. Consider the unique path P_{rv} from the root r to a node v . Observe that node v is only in the subtrees of the nodes in P_{rv} . Since $|P_{rv}| \leq D + 1$, node v is overcounted at most D times, and $\sum_{v \in V} |T(v, r)| \leq (D + 1) \cdot n$. $\square$

The probing ensures that a node, if it exponentiates into a direction, essentially never learns more nodes than there are contained in its “rooted subtree” $T(v, r)$.

LEMMA (SEE LEMMA 4.17) *Let v be any node with a parent u (according to the hypothetical rooting at r). If in some iteration, node v exponentiates in the direction of u , i.e., it performs $\text{Exp}(X)$ with $u \in X$, the size of the resulting set $S_{v \rightarrow u}$ is bounded by $|T(v, r)| \cdot \hat{D}$.*

This is sufficient to sketch the global memory bound.

LEMMA (SEE LEMMA 4.18) *In `CompressLightSubTrees`, the global memory never exceeds $O(n \cdot \hat{D}^3)$.*

Proof. [Proof sketch] Assume that there is at least one heavy node and consider an arbitrary iteration j of the algorithm. For node v define the set $C_v \subseteq S_v$ as the set of nodes that v has added to S_v as a result of performing Exp in all iterations up to iteration j . Let u be the parent of v (according to the hypothetical rooting at r). For that u , let $C_{v \rightarrow u} := C_v \cap G_{v \rightarrow u}$. We obtain

$$|C_v| \leq |C_{v \rightarrow u}| + \sum_{w \in N(v) \setminus u} |S_{v \rightarrow w}| \leq |T(v, r)| \cdot \hat{D} + |T(v, r)| = (1 + \hat{D})|T(v, r)|.$$

The bound on $|C_{v \rightarrow u}|$ is obtained by applying the previous lemma for the last iteration where v has exponentiated in the direction of u , and the bound on the sum is by the definition of $T(v, r)$.

We need to introduce the notation C_v , as in our actual algorithm, exponentiations are not symmetric. In order to ensure a *symmetric enough* view, nodes v that add some vertex u to their set S_v also add themselves to the set S_u . Thus $C_v \neq S_v$. However, this results in at most a factor 2 increase in global memory. The total memory is then bounded by

$$\sum_{v \in V} |S_v| = \sum_{v \in V} 2|C_v| \leq \sum_{v \in V} 2(1 + \hat{D})|T(v, r)| = O(n \cdot \hat{D}^2).$$

Here, the bound on $\sum_{v \in V} |T(v, r)|$ is due to Lemma 4.5. The additional $\hat{D}$ factor in the lemma statement is due to the fact that a node may learn about the same node $\hat{D}$ times in a single exponentiation step resulting in a local peak in global memory; details are given in the full proof. $\square$

The bounds on local memory use that the probing ensures that we do not exponentiate into a direction if it would provide us with too many new nodes.

Measure of Progress. In order to show that all light nodes become happy, we prove that the distance between a light node and a leaf in its subtree decreases by a constant fraction in a constant number of rounds. Distance, in this case, can be measured via a virtual graph where there is an edge between two nodes u and v if $v \in S_u$ or $u \in S_v$. Our algorithm design ensures that light nodes always exponentiate in all but one direction. This is sufficient to show that each segment $x_1, \dots, x_5$ of length 5 of a shortest path in the virtual graph H , shortens by at least one edge in each iteration. Intuitively, one can simply use that x_3 in such a segment either exponentiates in the direction of x_1 or x_5 and will hence add the respective node to its memory. The actual proof needs a more careful reasoning, e.g., as we cannot rely on x_1 being part of the memory of x_2 , due to non homogeneous exponentiations in previous iterations.

4 The MAX-ID Problem

In this section, we give a specialized algorithm for the MAX-ID problem on trees, which will be the core ingredient for solving connected components on forests (upper bound of Theorem 1.1). Once having the algorithm for solving MAX-ID, one can extend it to work as a connected components algorithm. We defer this extension and its proofs to Section 5. We define the problem as follows.

DEFINITION 4.1. (THE MAX-ID PROBLEM) *Given a connected graph with a unique identifier for each node, all nodes output its maximum identifier.*

LEMMA 4.1. (SOLVING MAX-ID ON TREES) *Consider the family of trees. There is a deterministic low-space MPC algorithm that solves MAX-ID on any graph G of that graph family when given $\hat{D} \in [\text{diam}(G), n^{1/8}]$. The algorithm runs in $O(\log \hat{D})$ rounds, is component-stable³, and requires $O(m \cdot \hat{D}^3)$ words of global memory.*

4.1 Definition and Structural Results We begin with structural properties of trees that are essential for proving our memory bounds. Also, the introduced notation plays a central role in each step of our algorithms. Let v be a vertex of a tree G . For all nodes $u \in N(v)$, define

$$G_{v \rightarrow u} = \{w \in V(G) \mid u \text{ is contained in the shortest path from } v \text{ to } w\}$$

to be all nodes in the tree that are reachable from v via u , including u . Also, let $G_{v \nrightarrow u} := V(G) \setminus G_{v \rightarrow u}$. For every $w \in G$ let $r_v(w)$ be $u \in N(v)$ such that $w \in G_{v \rightarrow u}$, i.e., $r_v(w)$ is the neighbor of v which is on the unique path from v to w .

DEFINITION 4.2. (LIGHT AND HEAVY NODES) *Let $0 < \delta < 1$ be a constant. A node v is light against a neighbor $u \in N(v)$ if $|G_{v \nrightarrow u}| \leq n^{1/8}$. A node is light if it is light against at least one of its neighbors. When v is light against u , let $T_{v,u}$ denote $G_{v \nrightarrow u}$. Nodes that are not light are heavy.*

Observe that a light node v can be light against multiple neighbors u and hence, we need to use a subscript in the notation $T_{v,u}$. We emphasize that $v \in T_{v,u}$. Throughout most of our proofs we need to consider the cases that a (virtual) tree contains heavy nodes and the case that it only consists of light nodes separately. Both situations are depicted in Figure 1. We continue with proving structural properties for both cases. Any tree has light nodes as its leaves are light.

LEMMA 4.2. *Consider a tree G that contains a heavy node and let $v \in G$ be a light node against neighbor u . Then all nodes $x \in T_{v,u}$ are light. Moreover, any $x \in T_{v,y}$, $x \neq v$ is light against $r_x(v)$.*

Proof. The first part of the claim must hold since $|G_{x \nrightarrow r_x(v)}| < |G_{v \nrightarrow u}| \leq n^{1/8}$. The second part must hold, since otherwise, all nodes are light, contradicting the assumption that there exists a heavy node. $\square$

LEMMA 4.3. *For every tree G with at least one heavy node, it holds that (i) heavy nodes induce a connected component, and that (ii) every light node is light against exactly one neighbor.*

Proof. For both parts, assume the opposite. Then there is a heavy node in $T_{v,u}$ for some light node v , which contradicts Lemma 4.2. $\square$

Due to Lemma 4.3, we write T_v instead of $T_{v,u}$ for a light node in a tree with (a) heavy node(s) and call T_v the node's *subtree*.

OBSERVATION 4.1. *For any tree G and any two adjacent nodes $u, v \in G$, we have $G = G_{v \nrightarrow u} \cup G_{u \nrightarrow v}$.*

Proof. Since $G_{v \rightarrow u} = G_{u \nrightarrow v}$, we obtain $G_{u \nrightarrow v} \cup G_{v \nrightarrow u} = G_{v \rightarrow u} \cup G_{v \nrightarrow u} = G$. $\square$

LEMMA 4.4. *Any tree with no heavy nodes contains at most $2n^{1/2}$ vertices.*

Proof. For the sake of analysis, let each node v put one token on each incident edge $\{v, u\}$ where v is light against u , i.e., $|G_{v \nrightarrow u}| \leq n^{1/8}$. As all nodes are light the total number of tokens is at least as large as the number of nodes. Since the graph is a tree, at least one edge receives two tokens. Let $\{u, v\}$ be such an edge and observe that $G = G_{v \nrightarrow u} \cup G_{u \nrightarrow v}$ holds due to Observation 4.1. It holds that $|G| = |G_{v \nrightarrow u} \cup G_{u \nrightarrow v}| \leq 2n^{1/8}$, because v is light against u and u is light against v . $\square$

³By the formulation of Definition 4.1, any algorithm solving MAX-ID is component-stable by definition. This is discussed in detail in Section 5.4

The following lemma will be central to bounding the global memory of our algorithm. It considers a rooted tree, which we will only use for analysis; we do not assume a rooting is given as input.

LEMMA 4.5. *Consider an n -node tree T with diameter D that is rooted at node r . Let $T(v, r)$ denote the subtree rooted at v (including v) when T is rooted at r . It holds that $\sum_{v \in V} |T(v, r)| \leq (D + 1) \cdot n$.*

Proof. Consider the unique path P_{rv} from the root r to a node v . Observe that node v is only in the subtrees of the nodes in P_{rv} . Since $|P_{rv}| \leq D + 1$, node v is overcounted at most D times, and $\sum_{v \in V} |T(v, r)| \leq (D + 1) \cdot n$. $\square$

4.2 MAX-ID: The Algorithm In this section, we present a MAX-ID algorithm for trees, which we refer to as MAX-ID-Solver. In our algorithm, every node of an input tree G outputs the maximum identifier of the tree, which we denote by $\underline{\text{ID}}$. We assume we are given $\hat{D} \in [\text{diam}(G), n^{1/8}]$. The runtime of our algorithm is $O(\log \hat{D})$ and it requires $O(n \cdot \hat{D}^3)$ words of global memory.

The high-level idea is to iteratively compress parts of the graph (without disconnecting it) such that the knowledge of the maximum identifier of the compressed parts is always kept within the resulting graph. We repeat this process until there remains only one node, that knows $\underline{\text{ID}}$. Then, we backtrack the process by iteratively decompressing and broadcasting the knowledge about $\underline{\text{ID}}$. Eventually, we are left with the original graph where all nodes know $\underline{\text{ID}}$.

As reasoned in Section 1 it is far from clear how to implement this simple outline with neither breaking the runtime nor the global memory bounds. From a high level point of view our algorithm consists of $O(1)$ **phases** and $O(1)$ **reversal phases**. During the phases, we first compress all light subtrees into single nodes (a procedure that we refer to as **CompressLightSubTrees**) and then replace all paths by a single edge (**CompressPaths**). In this section, we blackbox the properties of both procedures and prove that $O(1)$ phases are sufficient to reduce the graph to a single node (Lemma 4.11). By far the most technically involved part of our algorithm is the procedure **CompressLightSubTrees**, which we explain in detail in Section 4.3. The phases are followed by reversal phases, in which we undo all compression steps of the regular phases in reverse order to spread $\underline{\text{ID}}$ to the whole graph.

Let us be more formal and define the compression/decompression steps. Throughout the algorithm, every node v keeps track of a variable id_v , which is initially set to be the identifier of v . The intuition behind variable id_v is that it represents the largest identified v has “seen” so far. Let us define compressing and decompressing operations for node v and any node set X . Note that decompressing X from v is only defined for X, v such that X was at some point compressed into v .

- **Compress X into v :** set $\text{id}_v \leftarrow \max_u \{\text{id}_u \mid u \in X\}$ remove X (and its incident edges) from the graph. For any edge $\{x, y\}$ with $x \in X$ and $v \neq y \notin X$ we introduce a new edge $\{v, y\}$.
- **Decompress X from v :** set $\text{id}_u \leftarrow \text{id}_v$, $\forall u \in X$ and add X (and its incident edges) back to the graph. Remove any edge $\{v, y\}$ that was added during the compression step of X into v .

Phases. We initialize G_0 as the input graph. From G_0 , we derive a sequence $G_1, G_2, \dots, G_\ell$ of smaller trees until eventually, for some $\ell = O(1)$, it holds that $G_\ell = \{v\}$ for which $\text{id}_v = \underline{\text{ID}}$. The tree G_{i+1} ($0 < i \leq \ell$) is obtained from G_i as follows: first compressing all light subtrees via **CompressLightSubTrees**($G_i, \hat{D}$) and call the resulting tree G'_i , then G_{i+1} is the result of compressing all paths of G'_i into single edges via **CompressPaths**($G'_i, \hat{D}$).

Throughout the sequence, we maintain the properties that compressions do not overlap, every G_i is connected and non-empty, and that $\text{id}_w = \underline{\text{ID}}$ for *some* node $w \in G_i$.

Reversal Phases. From $G_\ell = \{v\}$, we derive a reversal sequence $G_{\ell-1}, G_{\ell-2}, \dots, G_0$ such that any G_i ($\ell > i \geq 0$) has the same node and edge sets as G_i during the regular phases, and $\text{id}_w = \underline{\text{ID}}$ for *every* node $w \in G_i$. The tree G_{i-1} is obtained from G_i as follows: first decompressing all paths via **DecompressPaths**(G_i) and call the resulting tree G'_{i-1} , then G_{i-1} is the result of decompressing all light subtrees via **DecompressLightSubTrees**(G'_{i-1}). Note that in reversal phase i we only decompress paths and subtrees that were compressed during the regular phase i .

MAX-ID-Solver($G, \hat{D}$)

Initialize $G_0 \leftarrow G$

1. For $i = 0, \dots, \ell - 1$ phases:

(a) $G'_i = \text{CompressLightSubTrees}(G_i, \hat{D})$

// If there are heavy nodes, all light nodes are compressed into the closest heavy node. Otherwise, all nodes are light and are compressed into a single node.

(b) $G_{i+1} = \text{CompressPaths}(G'_i, \hat{D})$

// All paths are compressed into single edges.

2. For $i = \ell - 1, \dots, 0$ reversal phases:

(a) $G'_i = \text{DecompressPaths}(G_{i+1})$

// All paths that were compressed during Step 1(b) are decompressed.

(b) $G_i = \text{DecompressLightSubTrees}(G'_i)$

// All light nodes that were compressed during Step 1(a) are decompressed from v .

The correctness of MAX-ID-Solver is contained in the following lemma.

LEMMA 4.6. *There exists some ℓ such that*

1. *after ℓ phases, graph G_ℓ consists of exactly one node v for which $\text{id}_v = \underline{\text{ID}}$.*
2. *after ℓ reversal phases, graph G_0 is the input graph and all nodes know $\underline{\text{ID}}$.*

Proof. The proof is straightforward, given the three essential lemmas (Lemmas 4.7 to 4.9) on the subroutines that we prove in the sections hereafter. Let us prove the two claims separately.

1. Consider graph G_i at the start of any phase i . We first claim that G_i never becomes empty during phase i , for which there are two cases: either G_i contains heavy nodes, or all nodes in G_i are light. In the case of the former: in Step 1(a), by Lemma 4.7, if there are heavy nodes in the graph, they are never compressed. In Step 1(b), by Lemma 4.8, all degree-2 nodes are compressed into single edges, leaving the graph non-empty. In the case of the latter, by Lemma 4.7, we are left with a single node. Observe that since any tree always contains light nodes (leaves are always light), the number of nodes decreases in every phase, and the first part of the claim 1 holds for some ℓ . Since $G_\ell = \{v\}$ is a result of consecutive compression steps applied to the input graph G_0 without disconnecting it, by the definition of compression, it holds that $\text{id}_v = \underline{\text{ID}}$.
2. **Observation.** *Graph G_i during reversal phases i has the same node and edge sets as graph G_i during phase i .*

Proof. We prove the claim by induction. The base case holds since G_ℓ from Step 1 is given directly to Step 2 as input. Assume that the claim holds for reversal phase $i + 1$. By Lemma 4.9, all nodes that were compressed in phase i during Step 1(a) (resp. (b)) can decompress themselves in reversal phase i during Step 2(b) (resp. (a)), proving the claim. $\square$

Consider graph G_ℓ that consists of a single node v for which $\text{id}_v = \underline{\text{ID}}$ by Lemma 4.6. Since graph G_0 after ℓ reversal phases (which is the input graph by the observation above) is a result of consecutive decompression steps applied to G_ℓ , by the definition of decompression, it holds that $\text{id}_u = \underline{\text{ID}}$ for all $u \in G_0$.

$\square$

LEMMA 4.7. (**CompressLightSubTrees**) *Let G be a tree and $\hat{D} \in [\text{diam}(G), n^{1/8}]$. If G contains a heavy node, then $\text{CompressLightSubTrees}(G, \hat{D})$ returns a tree in which all light nodes of G are compressed into the closest heavy node. If G does not contain any heavy nodes, all nodes are compressed into a single node. The algorithm runs in $O(\log D)$ low-space MPC rounds using $O(n \cdot \hat{D}^3)$ words of global memory.*

LEMMA 4.8. (**CompressPaths**) For any tree G and $\hat{D} \in [\text{diam}(G), n^{1/8}]$, $\text{CompressPaths}(G, \hat{D})$ returns the graph that is obtained from G by replacing all paths of G with a single edge. The algorithm runs in $O(\log \hat{D})$ low-space MPC rounds using $O(n \cdot \hat{D}^2)$ words of global memory.

LEMMA 4.9. (**DecompressPaths, DecompressLightSubTrees**) All nodes that were compressed by **CompressLightSubTrees** and **CompressPaths** can be decompressed by **DecompressLightSubTrees** and **DecompressPaths**, respectively. The algorithms run in $O(1)$ low-space MPC rounds using $O(n)$ words of global memory.

We will now show that the number of phases of (and therefore reversal phases) is bounded by $O(1)$. In particular, we want to prove that after $\ell = O(1)$ phases, graph G_ℓ consists of exactly one node. After a clever observation in Lemma 4.10, we will prove the claim in Lemma 4.11.

LEMMA 4.10. If $|G_{i+1}| \geq 2$, all nodes in G_{i+1} were heavy in G_i . Moreover, for every leaf node $w \in G_{i+1}$ it holds that $\geq n^{1/8}$ light nodes were compressed into w during phase i .

Proof. Since $|G_{i+1}| \geq 2$ (and not $|G_{i+1}| = 1$), by Lemma 4.7, there must have been heavy nodes in G_i . Since all light nodes were compressed in phase i , all nodes in G_{i+1} were heavy in G_i . Observe that even though w is a leaf in phase $i+1$, it was not a leaf node in phase i , since leaf nodes are light by definition. Let u be the unique neighbor of w in G_{i+1} . We must show that $|G_{w \rightarrow u}| > n^{1/8}$ and that $G_{w \rightarrow u} \setminus w$ was compressed into w during phase i . It must be that $|G_{w \rightarrow u}| > n^{1/8}$, since otherwise, w would have been light against u in phase i . Nodes $G_{w \rightarrow u} \setminus w$ were compressed into w during phase i by Lemma 4.7, since w was their closest heavy node (due to the graph being a tree). $\square$

LEMMA 4.11. After $\ell = O(1)$ phases, graph G_ℓ consists of exactly one node.

Proof. Consider graph G_i at the beginning of some phase i . If there are no heavy nodes in G_i , this is the last phase of the algorithm by Lemma 4.7. If there is exactly one heavy node in G_i , we are also done by Lemma 4.7. What remains to be proven is that if there are at least two heavy nodes in the graph, we reduce the size of the graph by a polynomial factor in n .

Assume that there are at least 2 heavy nodes in graph G_i , and let us analyze what happens. In Step 1(a), all light nodes are compressed into the closest heavy node by Lemma 4.7. In Step 1(b), all paths are compressed into single edges by Lemma 4.8, leaving no degree-2 nodes in the graph (compressing paths never creates new degree-2 nodes). Consider graph G_{i+1} , which by Lemma 4.7 consists of the nodes that were heavy in G_i . By Lemma 4.10 it also holds that during phase i , at least $n^{1/8}$ light nodes were compressed into every leaf node w of graph G_{i+1} . It holds that

$$n_i \geq n_{i+1} + |\{w \in G_{i+1} \mid \deg_{G_{i+1}}(w) = 1\}| \cdot n^{1/8} > n_{i+1} + n^{1/8} \cdot n_{i+1}/2 = n_{i+1}(1 + n^{1/8}/2)$$

and $n_{i+1} < n_i/(1 + n^{1/8}/2) < 2n_i/n^{1/8}$.

The first strict inequality stems from the fact that there are no degree-2 nodes left after phase i , and hence the number of leaf nodes in G_{i+1} is strictly larger than $n_{i+1}/2$. The proof is complete, as we have shown that if graph G_i contains at least 2 heavy nodes, G_{i+1} is smaller than G_i by a factor of $\Theta(n^{1/8})$. $\square$

The outline for the rest of this section is as follows. The procedure **CompressLightSubTrees** and the proof of Lemma 4.7 are presented in Section 4.3. This is the most technically involved part of our algorithm. The procedure **CompressPaths** and the proof of Lemma 4.8 are presented in Section 4.4. The procedures **DecompressPaths** and **DecompressLightSubTrees** and the proof of Lemma 4.9 are presented in Section 4.5. In Appendix A, we show technical details how **MAX-ID-Solver** can be implemented in the low-space MPC model.

4.3 MAX-ID: Single Phase (CompressLightSubTrees) In this section, we focus on a single execution of **CompressLightSubTrees**($G, \hat{D}$) on a graph G and prove Lemma 4.7. With out loss of generality, we assume there are n nodes in the graph—starting from the second phase of **MAX-ID-Solver** we will actually use this algorithm on graphs with fewer than n nodes.

At all times, every nodes v has some set of nodes S_v in its memory, which we initialize to $N(v)$. Set S_v can be thought of as the node's view or knowledge. During the execution, S_v grows, and if $|S_v| \geq 2n^{1/4}$, v becomes

full. Similarly to definitions $G_{v \rightarrow u}$ and $G_{v \nrightarrow u}$, let us define the following. For a node v and a node $u \in N(v)$, let $S_{v \rightarrow u} = S_v \cap G_{v \rightarrow u}$. Also, let $S_{v \nrightarrow u} := S_v \setminus S_{v \rightarrow u}$. Recall the definition of $r_v(w)$: for every $w \in G$ let $r_v(w)$ be $u \in N(v)$ such that $w \in G_{v \rightarrow u}$.

All nodes in the graph have the property that they are either light or heavy (see Definition 4.2). Initially, nodes themselves do not know whether they are light or heavy, since these properties depend on the topology of the graph. During the algorithm each node is in one of the four states: **active**, **happy**, **full**, or **sad**. Initially, all nodes are **active**. A node v becomes **happy**, if at some point during the execution, there exists $u \in N(v)$ such that $G_{v \nrightarrow u} \subseteq S_v$ and $|G_{v \nrightarrow u}| \leq n^{1/8}$. In that case, we say that node v is *happy against* u . If a node, that is not full, realizes that it can never become **happy** (for example by having $|S_{v \rightarrow u}| > n^{1/8}$ for two different neighbors u), it becomes **sad**. Upon becoming **happy**, **sad** or **full**, nodes do not partake in the algorithm except for answering queries from **active** nodes. We call nodes **unhappy** if they are in some other state than **happy** (including state **active**). The goal is that all light nodes eventually become **happy**, and heavy nodes always remain **unhappy**. When comparing the definitions of **happy** and light, it is evident that when a node becomes **happy**, it knows that it is light. Similarly, a node becoming **full** or **sad** knows that it is heavy.

For a node v and any $X \subseteq N(v)$, define an exponentiation operation as

$$\text{Exp}(X) : S_v \leftarrow \bigcup_{u \in X} \bigcup_{w \in S_{v \rightarrow u}} S_{w \nrightarrow r_w(v)}.$$

We say that a node v *exponentiates towards (or in the direction of)* u if $u \in N(v)$ and v performs $\text{Exp}(X)$ with $u \in X$.

High level overview of CompressLightSubTrees. The algorithm consists of $O(\log \hat{D})$ iterations, in each of which nodes perform a carefully designed graph exponentiation procedure. The aim is for light nodes v to become **happy** by learning their subtrees T_v , after which, (certain) light nodes compress T_v into their **unhappy** neighbor. If there were no memory constraints and every node could do a proper (uniform) exponentiation step in every iteration of the algorithm, i.e., execute $\text{Exp}(N(v))$, after $O(\log \hat{D})$ iterations *all* nodes would learn the whole graph—a proper exponentiation step executed on all nodes halves the diameter—and the highest ID node could compress the whole graph into itself. However, uniform exponentiation would result in all nodes exceeding their local memory $O(n)$, and also significantly breaking the global memory requirement. Even if we were to steer the exponentiation procedure such that light nodes would learn a D_{T_v} radius ball around them, where D_{T_v} is the diameter of their light subtree, this would still break global memory. In fact, we cannot even do a single exponentiation step for all nodes in the graph without breaking memory bounds! Hence, we need to steer the exponentiation with some even more stronger invariant in order to abide by the global memory constraint.

OBSERVATION 4.2. *If every light node v keeps $O(|T_{v,u}|)$ nodes in its local memory for some (possibly unique) neighbor u it is light against, this does not violate local memory $O(n)$ nor global memory $O(n \cdot \hat{D})$.*

Proof. If there is a heavy node in the graph, $|T_{v,u}|$ is unique by Lemma 4.3. The claim follows by considering a hypothetical rooting of the tree at some heavy node and applying Lemma 4.5. Otherwise, the claim holds trivially because the graph is of size $\leq 2n^{1/8}$ by Lemma 4.4. $\square$

Inspired by the observation above, we aim to steer the exponentiation such that it is performed in a balanced way, where a node learns roughly the same number of nodes in each direction (or sees only leaves in one direction). In fact, we do not want to exponentiate in a direction if that exponentiation step would provide us with $\gg |T_v|$ nodes. This step is further complicated as nodes neither know whether they are heavy or light nor do they know the size of their subtree. In our algorithm that is presented below we perform a careful probing for the number of nodes into all directions to determine in which directions we can safely exponentiate without using too much memory. In the probing procedure **ProbeDirections**, a node v computes $B_{v \rightarrow u} = \sum_{w \in S_{v \rightarrow u}} |S_{w \nrightarrow r_w(v)}|$ for every neighbor $u \in N(v)$ as an estimate for the number of nodes it may learn when exponentiating towards u . This estimate may be very inaccurate and may contain a lot of doublecounting. In Section 4.3.3, we present the full procedure and prove the following lemma.

LEMMA 4.12. (ProbeDirections) *Consider an arbitrary iteration of algorithm CompressLightSubTrees. Then algorithm ProbeDirections($\hat{D}$) returns:*

- (i) $\text{fullDirs} \subseteq N(v)$ such that if we were to exponentiate in all directions, we would obtain $|S_{v \rightarrow u'}| > n^{1/8}$ for all $u' \in \text{fullDirs}$ and $|S_{v \rightarrow u'}| \leq n^{1/8} \cdot \hat{D}$ for all $u' \in N(v) \setminus \text{fullDirs}$.
- (ii) $\text{largestDir} \in N(v)$ (returned if $\text{fullDirs} = \emptyset$) such that if we were to exponentiate in all directions, we would obtain $|S_{v \rightarrow \text{largestDir}}| \geq |S_{v \rightarrow u'}|$ for all $u' \in N(v)$ and $|S_{v \rightarrow \text{largestDir}}| \leq n^{1/8} \cdot \hat{D}$.

ProbeDirections can be implemented in $O(1)$ low-space MPC rounds, using $O(n \cdot \hat{D}^3)$ global memory. It does not alter the state of S_v for any node v in the execution of **CompressLightSubTrees**.

The main difficulty of **CompressLightSubTrees** lies in ensuring the global and local memory constraints (Lemmas 4.18 and 4.19) that prevent us from blindly exponentiating in all directions, while at the same time ensuring enough progress for light nodes such that every light node becomes **happy** by the end of the algorithm (Lemma 4.16).

CompressLightSubTrees($G_i, \hat{D}$)

All nodes are active. Initialize $S_v \leftarrow N(v)$. If $|S_v| > n^{1/8} + 1$, v becomes sad.

1. For $O(\log \hat{D})$ iterations:

(a) $\text{fullDirs}, \text{largestDir} \leftarrow \text{ProbeDirections}(\hat{D})$

// The properties of $\text{ProbeDirections}(\hat{D})$ are formally stated in Lemma 4.12. Informally, $\text{fullDirs} \subseteq N(v)$ contains directions with $> n^{1/8}$ nodes, and largestDir contains the direction with the largest number of nodes if $\text{fullDirs} = \emptyset$.

(b) If $|\text{fullDirs}| \geq 2$, v becomes sad.

(c) If $|\text{fullDirs}| = 1$:

i. Perform $\text{Exp}(N(v) \setminus \text{fullDirs})$

(d) If $|\text{fullDirs}| = 0$:

i. Perform $\text{Exp}(N(v) \setminus \text{largestDir})$

ii. If v is in S_w for some w , add w to S_v // ensure symmetric view

(e) Node v asks nodes $w \in S_v$ whether or not they are happy against $r_w(v)$, and if so, what is the size of subtree $T_{w, r_w(v)}$. Node v can locally compute if it can become happy by learning subtrees $T_{w, r_w(v)}$. If v can, it asks for them and becomes happy.

// After Step 1, all light nodes are happy, and all heavy nodes are unhappy (Lemma 4.16)

2. Happy nodes v with an unhappy neighbor u compress $S_{v \rightarrow u} = G_{v \rightarrow u}$ into v .

3. Nodes v that are happy against u such that u is happy against v update $S_v \leftarrow S_v \cup S_u$ and compress S_v into the highest ID node in S_v .

In Section 4.3.1, we discuss the measure of progress and correctness, with the final correctness proof of Lemma 4.7. In Section 4.3.2, we discuss local and global memory bounds, with the final memory proofs of Lemma 4.7. The MPC implementation is deferred to Appendix A.

4.3.1 Measure of Progress and Correctness We begin by proving the measure of progress and correctness, which will give us the means to analyze the memory requirements as if the tree was rooted.

LEMMA 4.13. *Let v be a node that is light against neighbor u . If in some iteration of **CompressLightSubTrees**, v exponentiates in the direction of u , i.e., it performs $\text{Exp}(X)$ with $u \in X$, the size of the resulting set $S_{v \rightarrow u}$ is bounded by $|T_v|$.*

Proof. Consider an arbitrary iteration of the algorithm. If $u \in \text{fullDirs}$, we do not exponentiate towards u , so there is nothing to prove. If $u \notin \text{fullDirs}$, but $\text{fullDirs} \neq \emptyset$, there is some $w \neq u$ such that, by the Probing Lemma 4.12, $|G_{v \rightarrow w}| > n^{1/8}$, which is a contradiction to v being light against u .

Hence, consider the case that $\text{fullDirs} = \emptyset$. If $\text{largestDir} = u$, we do not exponentiate towards u and there is nothing to prove. If $\text{largestDir} \neq u$, then we exponentiate towards u and by Lemma 4.12 (ii), we have $|S_{v \rightarrow u}| \leq |G_{v \rightarrow \text{largestDir}}| \leq |T_v|$. $\square$

LEMMA 4.14. *In any iteration of CompressLightSubTrees, a light node neither becomes full nor sad.*

Proof. Node v never becomes full due to initialization $S_v \leftarrow N(v)$, since for a light node it must hold that $|N(v)| \leq |T_v| + 1 \leq n^{1/8} + 1 < 2n^{1/4}$. During execution, S_v grows only in Steps 1(c)–(e). During (c), it must be that $\text{fullDirs} = u$, since otherwise it would imply that $|T_v| > n^{1/8}$. Hence, as a result of (c), v cannot become full. During (d)i, if $\text{Exp}(X)$ with $u \notin X$, it holds that $X \subset T_v$ and v cannot become full. Otherwise if $u \in X$, by Lemma 4.13, v cannot become full. Node v cannot become full even when performing Step 1(d)ii, since a hypothetical exponentiation step in the direction of u would yield a set that is bounded by $n^{1/8} \cdot \hat{D} < 2n^{1/4}$ (fullDirs is empty and u is largestDir). During (e), node v becomes happy against u and hence $|S_{v \rightarrow u}| \leq n^{1/8}$. In the worst case, $|S_{v \rightarrow u}| < n^{1/8} \cdot \hat{D} \leq n^{1/4}$. Hence, as a result of (e), v cannot become full.

A node can become sad only if its degree is too large, or in Step 1(b). A light node v never becomes sad since it must hold that $|N(v)| \leq |T_v| + 1 \leq n^{1/8} + 1$, and v cannot have two or more neighbors u with $G_{v \rightarrow u} > n^{1/8}$ (one neighbor would have to be in T_v , implying that $|T_v| > n^{1/8}$). $\square$

For the proofs of the next two lemmas, let $G = (V, E)$ be the input graph, and consider graph $G' = (V', E')$ such that $V' = V$ and $E' = E \cup \{v, w\} \mid v, w \in V \text{ and } w \in S_v \text{ or } v \in S_w\}$.

LEMMA 4.15. (MEASURE OF PROGRESS) *At the start of any iteration j , consider a light (but still active) node v , and the longest shortest path P_{vw}^j in G' between v and an a leaf node $w \in T_v$. If $|P_{vw}^j| \geq 4$ holds, then holds that $|P_{vw}^{j+1}| \leq \lceil 3/4 \cdot |P_{vw}^j| \rceil$ holds.*

Proof. Consider any subpath $P_{x_1 x_5} = \{x_1, x_2, x_3, x_4, x_5\} \subseteq P_{vw}^j$ of length 4. For $1 \leq i \leq 5$, let S_{x_i} (S'_{x_i}) be the memory of node i at the start (end) of iteration j . Note that all nodes on the path are light. By Lemma 4.14, nodes in $P_{x_1 x_5}$ never get full nor sad, and hence always exponentiate in all but one direction (either fullDirs or largestDir).

Claim. For $1 \leq i < 5$ it holds that $x_{i+1} \in S_{x_i}$.

Proof. Since edge $\{x_i, x_{i+1}\}$ exists in G' , it must be either that either $x_{i+1} \in S_{x_i}$ or $x_i \in S_{x_{i+1}}$. In the first case the claim holds, so consider the latter. Since x_i is light, $r_{x_i}(x_{i+1})$ has never been in fullDirs for x_i . Hence, whenever x_{i+1} had added x_i to $S_{x_{i+1}}$, either x_i added x_{i+1} to S_{x_i} via exponentiation, or via Step 1(d)ii. $\square$

We continue with proving that the path shortens. It is sufficient to prove that for some $i, j \in [1, 5]$, $i \neq j$, it holds that $x_i \in S'_j \setminus S_j$, as this shortens the path between x_1 and x_5 by one edge.

1. If $x_2 \notin S_{x_3}$: Since there is an edge in G' such that $x_2 \notin S_{x_3}$, it means that x_2 added x_3 to S_{x_2} during some iteration j , and since x_3 did not add x_2 in Step 1(d)ii of iteration j , direction $r_{x_3}(x_2)$ must have been in fullDirs for x_3 . Hence, x_3 will exponentiate in all directions besides $r_{x_3}(x_2)$. In particular, as $r_{x_3}(x_4) \neq r_{x_3}(x_2)$, x_3 will exponentiate towards x_4 . As $x_5 \in S_{x_4}$, we obtain $x_5 \in S'_{x_3} \setminus S_{x_3}$. This creates an edge between x_3 and x_5 in G' and shortens the path from 4 to 3, i.e., by a factor $3/4$.
2. If $x_2 \in S_{x_3}$: Assume that $x_1 \in S_{x_2}$. Since nodes in $P_{x_1 x_5}$ exponentiate in all but one direction, node x_3 will exponentiate either towards x_2 or x_4 (it must be that $x_4 \in S_{x_3}$ by the claim above). If x_3 exponentiates towards x_4 , we obtain $x_5 \in S'_{x_3} \setminus S_{x_3}$ as $x_5 \in S_{x_4}$. If x_3 exponentiates towards x_2 , we obtain $x_1 \in S'_{x_3} \setminus S_{x_3}$ as $x_1 \in S_{x_2}$. If $x_1 \notin S_{x_2}$, we can apply the analysis of 1. for node x_2 .

$\square$

LEMMA 4.16. *After $O(\log \hat{D})$ iterations, all light nodes become happy, while heavy nodes always remain unhappy.*

Proof. Let us adopt the notation of the proof of Lemma 4.15. Since Lemma 4.15 holds for any light node v , after $j = O(\log \hat{D})$ iterations it must hold that $|P_{vw}^j| \leq 3$ because $\hat{D} \in [\text{diam}(G), n^{1/8}]$. Let the resulting path be $P = \{x_1, x_2, x_3, x_4\}$, where x_4 is a leaf node. It must be the case that if x_2 learns $S_{x_3 \nrightarrow x_2}$ for all possible nodes x_3 , node x_2 becomes happy. Hence, in Step 1(e), P shortens by one. Eventually, after two iterations, P is of length one, and x_1 becomes happy.

For the second part of the claim it is sufficient to show that heavy nodes never become happy. Recall that heavy nodes are defined as nodes that are not light. Hence, for a heavy node v , there does not exist a neighbor $u \in N(v)$ such that $|G_{v \nrightarrow u}| \leq n^{1/8}$. This implies that during the algorithm, it is not possible for $|S_{v \nrightarrow u}| = |G_{v \nrightarrow u}| \leq n^{1/8}$ for any $u \in N(v)$. Hence, heavy nodes never become happy. $\square$

Proof. [Proof of Lemma 4.7 (Correctness)] By Lemma 4.16, we know that after $O(\log \hat{D})$ iterations all light nodes of G become **happy**, while all heavy nodes always remain **unhappy**. In order to prove the correctness of Lemma 4.7, we need to show that all light trees are compressed into the closest heavy node, if a heavy node exists, and that the whole tree is compressed into a single node if there are no heavy nodes. We consider both cases separately. Also consult Figure 1 for an illustration of both cases.

Case 1 (there are heavy nodes). Consider a light node v . As there are heavy nodes, Lemma 4.3 implies that there is a unique neighbor $u \in N(v)$ against which v is light. Let $T_v = G_{v \nrightarrow u}$. Now, by Lemma 4.16, v is happy at the end of the algorithm, i.e., there is a neighbor $u' \in N(v)$ for which $S_{v \nrightarrow u'} = G_{v \nrightarrow u'}$ and $|G_{v \nrightarrow u'}| \leq n^{1/8}$. The latter condition says that v is light against u' and due to the earlier discussion we deduce that $u = u'$ and $T_v = G_{v \nrightarrow u} = S_{v \nrightarrow u} \subseteq S_v$. In summary, for every light node v , the tree T_v (that does not depend on the algorithm) is contained in S_v . By Lemma 4.3, heavy nodes induce a single connected component, and hence every light node v is contained in the subtree of some light node v' that has a heavy neighbor u' . Since we are in a tree, u' is the closest heavy node for v' , and in particular, for all light nodes $v \in T_{v'}$. By Lemma 4.16, v' is happy at the end of the algorithm, and u' remains unhappy. Performing Step 2 fulfills the first claim of Lemma 4.7. Step 3 is never performed, since there are no happy nodes left in the graph.

Case 2 (all nodes of G are light). Step 2 of **CompressLightSubTrees** is never performed, since all (light) nodes are happy due to Lemma 4.16. For the sake of analysis, let each node v put one token on each incident edge $\{v, u\}$ for which $G_{v \nrightarrow u} \subseteq S_v$ holds. As all (light) nodes are happy, i.e., there is a neighbor u such that $G_{v \nrightarrow u} \subseteq S_v$ holds, the total number of tokens is at least as large as the number of nodes. Since the graph is a tree, at least one edge receives two tokens. Let $\{u, v\}$ be such an edge and observe that $G_{v \nrightarrow u} \subseteq S_v$ and $G_{u \nrightarrow v} \subseteq S_u$. Due to Observation 4.1, $G_{v \nrightarrow u} \cup G_{u \nrightarrow v} = G$ and after Step 3 of **CompressLightSubTrees** both nodes have the complete tree in their memory and both nodes trigger a compression of the whole tree into the largest ID node.

The edge $\{u, v\}$ with the above properties is not unique, but after Step 3, the endpoints of any edge having these properties yield the exact same compression. $\square$

4.3.2 Local and Global Memory Bounds The most difficult part is proving the memory bounds when there are heavy nodes. If there were no memory limitation, Lemmas 4.6 and 4.11 (building up on versions of Lemmas 4.7 to 4.9 without memory limitations) already imply that after $O(1)$ phases of **MAX-ID-Solver**, there is exactly one node left in the graph. Denote this node by r . Node r has never been compressed by definition. For the sake of analysis, we assume a rooting of G at r . We emphasize that fixing a rooting is only for analysis sake, and we do not assume that the tree is actually rooted beforehand. We define $T(v, r)$ as the subtree rooted at v (including v itself), as if tree G was rooted at r .

OBSERVATION 4.3. Consider tree G with at least one heavy node during an arbitrary iteration of **CompressLightSubTrees**. For every light node v , it holds that $T(v, r) = T_v$, and every heavy node u has a unique subtree $T(u, r)$.

Recall that the definition of T_v for a light node v was independent from any algorithmic treatment. Still, it holds that T_v equals $T(v, r)$ (that depends on our algorithm as the node r depends on it). The next lemma states that a node only exponentiates into the direction of root r if it is safe to do so in terms of memory constraints. In spirit, it is very similar to Lemma 4.13, with the slight difference that it applies to all nodes, and we prove the claim using a hypothetical rooting of the tree.

LEMMA 4.17. Let v be any node with a parent u (according to the hypothetical rooting at r). If in some iteration of **CompressLightSubTrees** when there are heavy nodes, node v exponentiates in the direction of u , i.e., it performs $\text{Exp}(X)$ with $u \in X$, the size of the resulting set $S_{v \rightarrow u}$ is bounded by $|T(v, r)| \cdot \hat{D}$.

Proof. If v performs $\text{Exp}(X)$ such that $u \in X$, there must exist either $w = \text{fullDirs}$ or $w = \text{largestDir}$ such that $w \in T(v, r)$. If $w = \text{fullDirs}$, it implies that v is heavy, and by Lemma 4.12 (i), we have that $|S_{v \rightarrow u}| \leq n^{1/8} \cdot \hat{D} < |T(v, r)| \cdot \hat{D}$. If $w = \text{largestDir}$, by Lemma 4.12 (ii), we have that $|S_{v \rightarrow u}| \leq |G_{v \rightarrow w}| \leq |T(v, r)|$. $\square$

OBSERVATION 4.4. *When node v performs an exponentiation step, multiple nodes w can send the same node to v , resulting in duplicates in set S_v . After every exponentiation step, node v has to locally remove these duplicates. As a result, when bounding the memory of a node, we have to take into account the momentary spike in global memory due to duplicates. This momentary spike can at most result in an $\hat{D}$ factor increase in the memory bounds.*

Proof. When node v exponentiates, nodes w send $S_{w \rightarrow r_w(v)}$ and not S_w . Consider node x that v has received via exponentiation, and consider the unique path P_{vx} between v and x . Since only nodes $w \in P_{vx}$ could have sent x to v , and $|P_{vx}| \leq D \leq \hat{D}$, x has at most $\hat{D}$ duplicates in S_v . $\square$

LEMMA 4.18. *In `CompressLightSubTrees`, the global memory never exceeds $O(n \cdot \hat{D}^3)$.*

Proof. The global memory $O(n \cdot \hat{D}^3)$ of `ProbeDirections` (Step 1(a)) follows from Lemma 4.12. Hence, we analyze the global memory excluding Step 1(a).

When all nodes are light, by Lemma 4.4, the size of the graph is $\leq 2n^{1/8}$. When taking duplicates into account (Observation 4.4), since $\hat{D} \leq n^{1/8}$, even if the whole graph is in the local memory of every node, this does not violate global memory constraints. For the rest of the proof assume that there is at least one heavy.

Consider an arbitrary iteration j of the algorithm when there are heavy nodes. Define set $C_v \subseteq S_v$ as the set of nodes that v has added to S_v as a result of performing Exp in all iterations up to iteration j . Let u be the parent of v (according to the hypothetical rooting at r). For that u , define $C_{v \rightarrow u} := C_v \cap G_{v \rightarrow u}$. For a node v , we have

$$|C_v| \leq |C_{v \rightarrow u}| + \sum_{w \in N(v) \setminus u} |S_{v \rightarrow w}| \leq |T(v, r)| \cdot \hat{D} + |T(v, r)| = (1 + \hat{D})|T(v, r)|$$

The bound on $|C_{v \rightarrow u}|$ is obtained by applying Lemma 4.17 for the last iteration where v has exponentiated in the direction of u , and the bound on the sum is by the definition of $T(v, r)$. Observe the crucial difference between S_v and C_v . Set S_v may contain nodes that are *not* a result of v performing Exp , but rather the result of Step 1(d)ii of the algorithm, where some other node w has added v to S_w . However, this can result in at most a factor-2 overcounting for every node. Combining this with the duplicates of Observation 4.4 results in global memory

$$\hat{D} \cdot \sum_{v \in V} |S_v| = \hat{D} \cdot \sum_{v \in V} 2|C_v| \leq \hat{D} \cdot \sum_{v \in V} 2(1 + \hat{D})|T(v, r)| = O(n \cdot \hat{D}^3),$$

where the bound on $\sum_{v \in V} |T(v, r)|$ is due to Lemma 4.5. $\square$

LEMMA 4.19. *In `CompressLightSubTrees`, the local memory of a node v never exceeds $O(n)$.*

Proof. The local memory of `ProbeDirections` (Step 1(a)) follows from Lemma 4.12. Hence, we analyze the local memory excluding Step 1(a).

When all nodes are light, by Lemma 4.4, the size of the graph is $\leq 2n^{1/8}$. When taking duplicates into account (Observation 4.4), since $\hat{D} \leq n^{1/8}$, even if the whole graph is in the local memory of every node, this does not violate global memory constraints. For the rest of the proof assume that there is at least one heavy.

Consider the start of an arbitrary phase i . If $\deg(v) > n$, we defer the discussion to Lemma A.1 on the MPC implementation details. Assuming $\deg(v) \leq n$, we prove the claim by induction. During the algorithm, the size of the local memory is at most of order $|S_v| \cdot \hat{D}$ (the extra $\hat{D}$ factor is due to Observation 4.4). The claim clearly holds in the first iteration when S_v is initialized as $N(v)$. Observe that if $\deg(v) > n^{1/8} + 1$, node v becomes *sad*. Hence, we can further assume that $\deg(v) \leq n^{1/8} + 1$. Assume the claim holds in iteration j . We perform a case distinction on the different changes of S_v , and show that for a node v , it holds that $|S_v| = O(n^{7/8})$, implying that $|S_v| \cdot \hat{D} = O(n)$ since $\hat{D} \leq n^{1/8}$.

- Step 1(c) and $|\text{fullDirs}| = 1$,

$|S_v|$ becomes at most $|S_{v \rightarrow \text{fullDirs}}| + (\deg(v) - 1) \cdot n^{1/8} \cdot \hat{D} \leq 2n^{1/4} + (n^{1/8})^3 < n^{7/8}$. The term $|S_{v \rightarrow \text{fullDirs}}|$ has a (loose) upper bound of $2n^{1/2}$, since v is not full. Observe that exponentiating in all directions except fullDirs yields $\leq n^{1/8} \cdot \hat{D}$ nodes per direction by Lemma 4.12 (i), and that $\hat{D} \leq n^{1/8}$ by assumption.

- Step 1(d) and $|\text{fullDirs}| = 0$,

$|S_v|$ becomes at most $\deg(v) \cdot n^{1/8} \cdot \hat{D} \leq (n^{1/8} + 1) \cdot (n^{1/8})^2 < n^{7/8}$. Observe that exponentiating in any direction yields $\leq n^{1/8} \cdot \hat{D}$ nodes per direction by Lemma 4.12 (ii) (fullDirs is empty), and that $\hat{D} \leq n^{1/8}$ by assumption.

- Step 1(e),

If a node v becomes happy against u , $|S_v|$ becomes $|T(v, r)| + |S_{v \rightarrow u}| \leq n^{1/8} + 2n^{1/4} < n^{7/8}$, where $n^{1/8}$ is an upper bound for $|T(v, r)|$ since it is light, and $2n^{1/4}$ is (loose) upper bound on $|S_{v \rightarrow u}|$ since v is not full.

Hence, the claim holds in iteration $j + 1$. $\square$

Proof. [Proof of Lemma 4.7 (Memory bounds)] The local memory bounds follow from Lemma 4.19, and the global memory bounds follow from Lemma 4.18. $\square$

4.3.3 Probing Our probing procedure is an integral part of `CompressLightSubTrees`, as it steers the exponentiation of every node such that, informally, a node never learns a (significantly) larger neighborhood in the direction of the root (which is imagined only for the analysis), than in the direction of its subtree.

LEMMA 4.12. (ProbeDirections) *Consider an arbitrary iteration of algorithm `CompressLightSubTrees`. Then algorithm `ProbeDirections`($\hat{D}$) returns:*

- (i) $\text{fullDirs} \subseteq N(v)$ such that if we were to exponentiate in all directions, we would obtain $|S_{v \rightarrow u'}| > n^{1/8}$ for all $u' \in \text{fullDirs}$ and $|S_{v \rightarrow u'}| \leq n^{1/8} \cdot \hat{D}$ for all $u' \in N(v) \setminus \text{fullDirs}$.
- (ii) $\text{largestDir} \in N(v)$ (returned if $\text{fullDirs} = \emptyset$) such that if we were to exponentiate in all directions, we would obtain $|S_{v \rightarrow \text{largestDir}}| \geq |S_{v \rightarrow u'}|$ for all $u' \in N(v)$ and $|S_{v \rightarrow \text{largestDir}}| \leq n^{1/8} \cdot \hat{D}$.

`ProbeDirections` can be implemented in $O(1)$ low-space MPC rounds, using $O(n \cdot \hat{D}^3)$ global memory. It does not alter the state of S_v for any node v in the execution of `CompressLightSubTrees`.

ProbeDirections($\hat{D}$)

1. For every neighbor $u \in N(v)$, compute $B_{v \rightarrow u} = \sum_{w \in S_{v \rightarrow u}} |S_{w \rightarrow r_w(v)}|$.
2. Define $\text{fullDirs} := \{u \in N(v) \mid B_{v \rightarrow u} \geq n^{1/8} \cdot \hat{D}\}$.
3. If $|\text{fullDirs}| > 0$, define $\text{largestDir} := \emptyset$. Otherwise, let $u_{\max} = \arg \max_{u \in N(v)} \{B_{v \rightarrow u}\}$ and if $B_{v \rightarrow u_{\max}} \geq \hat{D} \cdot B_{v \rightarrow u'}$ for all $u' \in N(v) \setminus u_{\max}$
 - (a) define $\text{largestDir} := u_{\max}$,
 - (b) otherwise, perform `Exp`($N(v)$) and define $\text{largestDir} := \arg \max_{u \in N(v)} \{|S_{v \rightarrow u}|\}$.
4. Return $\text{fullDirs}, \text{largestDir}$.

LEMMA 4.20. *If a node v were to perform `Exp`(u) for a neighbor $u \in N(v)$, it would hold that $B_{v \rightarrow u} / \hat{D} \leq |S_{v \rightarrow u}| \leq B_{v \rightarrow u}$.*

Proof. Recall the definition of $B_{v \rightarrow u}$. Let us compute how many times a node w in $B_{v \rightarrow u}$ can be overcounted. Consider the unique path P_{vw} from v to a node w . Observe that out of the nodes in $S_{v \rightarrow u}$, node w is in $S_{x \rightarrow v}$ only for nodes $x \in P_{vw}$. Since $|P_{vw}| \leq D + 1 \leq \hat{D} + 1$, any node w is overcounted at most $\hat{D}$ times, completing the proof. $\square$

Proof. [Proof of Lemma 4.12] Combining the condition $|B_{v \rightarrow u}| \geq n^{1/8} \cdot \hat{D}$ of Step 2 and the $\hat{D}$ -factor overcounting of Lemma 4.20 proves the properties of `fullDirs`. The properties of `largestDir` hold by definition: in the case of Step 3(a), $u_{\max}$ is the largest direction by Lemma 4.20, and in the case of Step 3(b), we exponentiate and find the absolute values. Local memory is respected in Step 1, since node v only aggregates an integer from every other node in S_v . More importantly, it is respected in Step 3: a node performing `ProbeDirections` has $\deg(v) < n^{1/8} + 1$ (otherwise it is sad), $\text{Exp}(N(v))$ is only performed if all directions yield $\leq n^{1/8} \cdot \hat{D}$ nodes (`fullDirs` is empty), and we are promised that $\hat{D} \leq n^{1/8}$.

Global memory is respected by a clever observation similar to Lemma 4.18. Similarly to Section 4.3.2, assume we have a rooting at some node r , and that node u is the parent of v . We want to bound the size of the resulting set $S_{v \rightarrow u}$ if node v performs $\text{Exp}(N(v))$. In particular, we want to show that $|S_{v \rightarrow u}| \leq |S_{v \rightarrow w}| \cdot \hat{D}^2 \leq |T(v, r)| \cdot \hat{D}^2$ for some node $w \in N(v) \setminus u$. Towards contradiction, assume that $|S_{v \rightarrow u}| > |S_{v \rightarrow w}| \cdot \hat{D}^2$ for all $w \in N(v) \setminus u$. It must then hold that

$$B_{v \rightarrow u} \geq |S_{v \rightarrow u}| > |S_{v \rightarrow w}| \cdot \hat{D}^2 \geq B_{v \rightarrow w} / \hat{D} \cdot \hat{D}^2 = B_{v \rightarrow w} \cdot \hat{D}$$

for all $w \in N(v) \setminus u$ by Lemma 4.20. However, this implies that u would have been chosen as $u_{\max}$, `largestDir` would have been defined as $u_{\max}$, and $\text{Exp}(N(v))$ would have never been performed; we have arrived at a contradiction. It holds that $|S_{v \rightarrow u}| \leq |T(v, r)| \cdot \hat{D}^2$, which bounds set S_v of every node v by $(\hat{D}^2 + 1) \cdot |T(v, r)|$, and by Lemma 4.5, the global memory is bounded by

$$\sum_{v \in V} |S_v| \leq (\hat{D}^2 + 1) \sum_{v \in V} |T(v, r)| \leq (\hat{D}^2 + 1)(D + 1) \cdot n = O(n \cdot \hat{D}^3).$$

Regarding MPC implementation, `ProbeDirections` only performs $\text{Exp}(N(v))$ (implementability proven in Lemma A.1) and computes $B_{v \rightarrow u}$, which is only a modified version of $\text{Exp}(N(v))$: instead of nodes w sending $S_{w \rightarrow r_w(v)}$ to node v , they only send $|S_{w \rightarrow r_w(v)}|$. $\square$

4.4 MAX-ID: Single Phase (CompressPaths) Let us prove the following lemma, which allows us to compress all paths in the tree into single edges. This operation does not create new paths or disconnect the graph.

LEMMA 4.8. (CompressPaths) *For any tree G and $\hat{D} \in [\text{diam}(G), n^{1/8}]$, $\text{CompressPaths}(G, \hat{D})$ returns the graph that is obtained from G by replacing all paths of G with a single edge. The algorithm runs in $O(\log \hat{D})$ low-space MPC rounds using $O(n \cdot \hat{D}^2)$ words of global memory.*

We describe a algorithm, which we denote as `CompressPaths`, and which we run on every path $P \subseteq G$. A path only includes consecutive degree-2 nodes. Similarly to `CompressLightSubTrees`, every node $v \in P$ has some set S_v in its memory, which we initialize to $N_P(v)$. Every node performs $\text{Exp}(N(v))$ until S_v no longer grows, whereupon, for every node v , it holds that $S_v = P$. The highest ID node $w \in P$ figures out the endpoints x, y of path P in G (which either have degree 1 or ≥ 3). Then, w.l.o.g., assume that $\text{ID}(x) > \text{ID}(y)$, whereupon w compresses P into x . By the definition of compression, node w also creates edge $\{x, y\}$.

Proof. [Proof of Lemma 4.8] After performing `CompressPaths`, every node $v \in P$ learns path P , i.e., it holds that $S_v = P$, after $O(\log \hat{D})$ rounds, since the path is of length at most $\text{diam}(G)$ and $\hat{D} \in [\text{diam}(G), n^{1/8}]$. Node w can learn x, y by asking for the neighbors (that are in $G \setminus P$) of the leaf.

Because $|P| \leq D \leq \hat{D} \leq n^{1/8}$, the local memory of a node is bounded by $n^{1/4}$ (when taking Observation 4.4 into account). The global memory is respected since in the worst case, all nodes have at most $\hat{D}^2$ nodes in memory (when taking Observation 4.4 into account). Compressing and creating a new edge $\{x, y\}$ comprises of sending a constant sized message to both x and y . Even in the case when x or y are endpoints to multiple paths, their total incoming message sizes are $O(\deg(x))$ and $O(\deg(y))$. The small caveat to this scheme is that if $\deg(x)$ or $\deg(y)$ are $> n^{1/8}$, we have to employ the aggregation tree structure as discussed in Lemma A.1. The implementation details of performing Exp can also be found in Lemma A.1. $\square$

4.5 MAX-ID: Single Reversal Phase A single reversal phase consist of steps `DecompressPaths` and `DecompressLightSubTrees`. In the former, we essentially reverse `CompressPaths`, and in the latter, we reverse `CompressLightSubTrees`. We prove the following.

LEMMA 4.9. (`DecompressPaths`, `DecompressLightSubTrees`) *All nodes that were compressed by `CompressLightSubTrees` and `CompressPaths` can be decompressed by `DecompressLightSubTrees` and `DecompressPaths`, respectively. The algorithms run in $O(1)$ low-space MPC rounds using $O(n)$ words of global memory.*

Let us introduce both steps formally.

- **DecompressPaths.** For every path P that was compressed in phase i into node x , node x decompresses P from itself.
- **DecompressLightSubTrees.** Every node v that had compressed T_v into a neighbor u (or itself), decompresses T_v from u (or itself).

Proof. [Proof of Lemma 4.9] As long as the nodes X that a node v wants to decompress are in its local memory, both steps are clearly correct and implementable in $O(1)$ low-space MPC steps. Observe that all nodes v that decompress a node set X , have at some point compressed set X and hence, have had X in local memory (in the form of S_v). By simply retaining set X in memory until it is time to decompress, we fulfill the requirement. $\square$

5 Connected Components (CC)

By Lemma 4.1, we can solve MAX-ID on any tree in $O(\log \hat{D})$ time using MAX-ID-Solver. The algorithm requires $O(m \cdot \hat{D}^3)$ words of global memory and value $\hat{D} \in [\text{diam}(G), n^{1/8}]$ as input. This section is mostly devoted to showing how to use MAX-ID-Solver to solve the connected components (CC) problem.

DEFINITION 5.1. (THE CC PROBLEM) *Given a graph with unique identifiers for each node, and disconnected components $C_1, \dots, C_k$, every node $v \in C_i$ outputs the maximum identifier of C_i .*

Observe that MAX-ID-Solver actually solves CC for the case when the input graph is a single tree. We show how to extend MAX-ID-Solver to solve CC for forests, effectively proving the upper bounds of the following theorem.

THEOREM 5.1. (CONNECTED COMPONENTS) *Consider the family of forests. There is a deterministic low-space MPC algorithm to detect the connected components on this family of graphs. In particular, each node learns the maximum ID of its component. The algorithm runs in $O(\log D)$ rounds, where D is the maximum diameter of any component. The algorithm requires $O(n + m)$ words of global memory, it is component-stable, and it does not need to know D . Under the 1 vs. 2 cycles conjecture, the runtime is asymptotically optimal.*

The proof is contained in Section 5.1 with references to subroutines from Sections 5.2 to 5.4. In Section 5.5, we show how to modify the algorithm of Section 5.1 to obtain a rooting.

5.1 Proof of Theorem 1.1 There are three steps to extending MAX-ID-Solver and proving Theorem 1.1: (1) reducing the global memory to $O(n + m)$; (2) removing the need to know $\text{diam}(G)$ in order to give $\hat{D}$ as input; (3) generalizing it from trees to forests while maintaining component-stability. We address all steps separately.

1. By applying Lemma 5.1 before executing MAX-ID-Solver, we reduce the number of nodes in G by a polynomial factor in $\hat{D}$. This reduces the global memory to a strict $O(n + m)$.
2. By employing the guessing scheme of Section 5.3, we perform multiple (sequential) executions of MAX-ID-Solver. Every execution is given a doubly exponentially growing guess for $\hat{D}$. The guessing scheme does not violate global memory $O(n + m)$ and results in a total runtime of $O(\log \text{diam}(G))$.
3. By the discussion in Section 5.4, we can execute MAX-ID-Solver on forests such that the runtime becomes $O(\log D)$, where D is the largest diameter of any component. Moreover, when executing MAX-ID-Solver on forests, it is component-stable.

5.2 CC: Pre- and Postprocessing The aim of our *preprocessing* is to reduce the number of nodes in the input graph G by a factor of $\text{poly}(\hat{D})$ (in fact $\hat{D}^3$ would suffice), resulting in graph G' . By executing MAX-ID-Solver on G' , we achieve a strict $O(n + m)$ global memory for one execution. When reducing the number of nodes, we must not disconnect the graph, and also keep the knowledge of the maximum ID inside the remaining graph.

After the connected components problem is solved on G' , we must extend the solution to the nodes in $G \setminus G'$ such that the solution is consistent. Extending the solution simply means informing every node in $G \setminus G'$ of ID , which is the maximum identifier of the graph. We call this stage *postprocessing*. This section is devoted to proving the following lemma.

LEMMA 5.1. *Consider a tree G with n nodes. The number of nodes can be reduced by a factor of $\text{poly}(\hat{D})$ such that the resulting graph G' remains connected, and one of the remaining nodes knows the maximum ID set $V_G \setminus V_{G'}$. If connected components is solved in G' , the solution can be extended to G . Both obtaining graph G' from G and extending the solution from G' to G takes $O(\log \hat{D})$ low-space MPC rounds using $O(n + m)$ words of global memory.*

Let us restate a known result that is going to be an essential tool in our preprocessing. We present a proof sketch to explicitly reason the memory bound.

LEMMA 5.2. ([23]) *There is an $O(1)$ -round sublinear local memory (component unstable) MPC algorithm that, given a subset $U \subseteq V$ of nodes of a graph $G = (V, E)$ with $d_G(u) = 2$ for all $u \in U$, computes a subset $S \subseteq U$ that is an independent set in G and satisfies $|S| \geq |U|/8$. The global memory used by the algorithm is $O(|U|) + O(|M| \cdot \log n) = O(n)$.*

Proof. [Proof Sketch] Consider the following random process: Each node marks itself with probability $1/2$. If a node is marked and no neighbor is marked, it joins the set S , otherwise it does not. The probability of a node to be marked and not having any of its neighbors marked is $1/8$. Thus, the expected size of S is $|U|/8$. Further, note that this analysis still holds if the randomness for the nodes is 3-independent. $|U|$ coins that are 3-independent can be created from a bitstring of length $O(3 \cdot \log |U|) = O(\log n)$ (see Definition 5.2 and Theorem 5.2).

In order to deterministically compute the set S we use the method of conditional expectation to compute a *good* bit string. For that purpose break the bitstring into $O(1)$ chunks of length at most $\delta/100 \log n$. Then we deterministically choose the bits on these segments such that the expected size of S is remains $|U|/8$ conditioned on all already determined segments of random bits. To fix one segment introduce the indicator random variable S_v that equals 1 if and only if $v \in S$. Let ϕ be the event that fixes to bitstring to what is already there and for $\alpha \in [n^{1/100}]$ let ϕ_α be the event that the to be fixed segment equals α . Knowing the IDs of its neighbors and the already fixed part of the bitstring, each machine can for each $v \in U$ that it holds compute the values $S_{v,\alpha} = \mathbb{E}[S_v \mid \psi = \alpha \wedge \phi]$. Then, nodes fix the current segment to the α_0 such that minimizes $\sum_{v \in U} S_{v,\alpha}$. By the method of conditional expectation we have $\mathbb{E}[|S| \mid \psi_{\alpha_0} \wedge \phi] \leq |U|/8$. At the end the whole bitstring is fixed and we have deterministically selected an independent set S satisfying $|S| \leq |U|/8$.

For an MPC implementation, we need to be able to globally, i.e., among all machines, to agree on the good bit string obtained from the method of conditional expectations. For this purpose, consider an aggregation tree structure, where the machines are arranged into a (roughly) $n^{1/2}$ -ary tree Definition A.1 with depth $O(1/\delta)$. In this tree, each machine can choose the (locally) good bit segments (conditioned on the previous segments) of $\delta/100 \log n$ bits. Notice that there are at most $n^{1/100}$ such bit segments. Now, we can convergecast the expected size of S , given a segment, to the root. Then, the root can decide on the *good* (prefix of a) bit string. In each round, each machine receives a $n^{1/2} \cdot n^{1/100} \ll n$ bits, which fits the local memory. Since we have $O(n^{1-})$ machines, the total memory requirement to store the bits is $O(n^{1/2+1/100+(1-)}) = O(n)$.

DEFINITION 5.2. ([37]) *For $N, M, k \in \mathbb{N}$ such that $k \leq N$, a family of functions $\mathcal{H} = \{h : [N] \rightarrow [M]\}$ is k -wise independent if for all distinct $x_1, \dots, x_k \in [N]$, the random variables $h(x_1), \dots, h(x_k)$ are independent and uniformly distributed in $[M]$ when h is chosen uniformly at random from $\mathcal{H}$.*

THEOREM 5.2. ([37]) *For every a, b, k , there is a family of k -wise independent hash functions $\mathcal{H} = \{h : \{0, 1\}^a \rightarrow \{0, 1\}^b\}$ such that choosing a random function from $\mathcal{H}$ takes $k \cdot \max\{a, b\}$ random bits.*

□

Next, we introduce elementary operations **Rake**, **Contract**, which we use during preprocessing, and **Insert**, **Expand**, which we use during postprocessing. Operation **Insert** can be thought of as the reversal of **Rake**, and operation **Expand** as the reversal of **Contract**. Recall the definition of compression and decompression from the beginning of the section.

DEFINITION 5.3. *For a degree-1 node v define the following two operations.*

- **Rake**(v): *Node v compresses into its unique neighbor u*
- + **Insert**(v): *Node v that underwent **Rake** decompresses from u .*

For a degree-2 node v define the following two operations.

- **Contract**(v): *Node v with neighbors u and w compresses into its highest ID neighbor.*
- + **Expand**(v): *Node v that underwent **Contract** decompresses from u (w.l.o.g. $ID(u) > ID(w)$).*

As long as we ensure that **Insert**(v) and **Expand**(v) are executed on nodes which have undergone **Rake**(v) and **Contract**(v) operations, respectively, we obtain the following observation.

OBSERVATION 5.1. *The operations **Rake**(v), **Insert**(v), **Contract**(v) and **Expand**(v) can be implemented in $O(1)$ low-space MPC rounds using $O(n + m)$ global memory on all nodes $v \in Z \subseteq V$ in parallel, if Z is an independent set containing only degree-1 and degree-2 nodes.*

Let us introduce preprocessing and postprocessing formally. Note that preprocessing is performed directly on the input graph G , resulting in smaller graph G' . Whereas postprocessing is performed on a solved version of G' , i.e., with all nodes v having $id_v = \mathbb{ID}$, resulting in a solved version of the input graph G . Both of the following routines use some constant c in their runtime in order to reduce the number of nodes by a factor of $\hat{D}^c$. Initialize G_0 as the input graph G .

- **Preprocessing.** For $j = 0, \dots, c \log \hat{D}$ iterations:
 1. Let H be the subgraph induced by all degree-2 nodes in G_j . Compute an independent set $Z \in H$ of size at least $|H|/8$ using Lemma 5.2.
 2. **Contract**(v) for every $v \in Z$.
 3. **Rake**(v) for every degree-1 node v . If two leaves are neighbors, perform **Rake** only on the higher ID one.
- **Postprocessing.** For $j = c \log \hat{D}, \dots, 0$ iterations:
 1. **Insert**(v) for every node v that performed **Rake**(v) in iteration j of **Preprocessing**.
 2. **Expand**(v) for every node $v \in Z$ in iteration j of **Preprocessing**.

Proof. [Proof of Lemma 5.1] Performing **Preprocessing** takes $O(\log \hat{D})$ time since Steps 1–3 can be performed in constant time. Let G (G') be the graph before (after) performing **Preprocessing**. Graph G' has a $\text{poly}(\hat{D})$ fraction less nodes than graph G because a constant fraction of nodes in a tree have degree ≤ 2 , and we compress all degree ≤ 2 nodes in the graph in each iteration.

Performing **Postprocessing** simply reverses **Preprocessing** while extending the current solution, so it has the same runtime as **Preprocessing**. Both **Preprocessing** and **Postprocessing** can be implemented in low-space MPC using $O(n + m)$ global memory, since other than Lemma 5.2, nodes only exchange constant sized messages with their neighbors, and don't store anything non-constant in local memory. $\square$

5.3 CC: Removing Knowledge of the Diameter Our MAX-ID-Solver algorithm requires value $\hat{D} \in [D, n^{1/8}]$ as input, where D denotes the diameter of the input tree. We show that we don't actually need to know D in order to give value $\hat{D}$ as input. We achieve this by sequentially executing MAX-ID-Solver with a doubly exponential guess for $\hat{D}$ as follows.

$$\hat{D} = \hat{D}_1, \hat{D}_2, \dots, \hat{D}_{\log \log n^{1/8}} = 2^{2^1}, 2^{2^2}, \dots, 2^{2^{\log \log n^{1/8}}}.$$

We proceed with the next guess only if the previous has failed to terminate after a runtime of $O(\log \hat{D}_i)$. Detecting a failure within the given runtime, and making sure that a wrong guess does not violate memory constraints is a delicate affair, and is discussed in a separate paragraph. If all of our guesses fail, it must be that $D > n^{1/8}$. In this case, we can run the deterministic $O(\log n)$ time connected components algorithm of Coy and Czumaj [22] with the requirement that all nodes output the maximum ID of the component (their algorithm is component-stable for the same reasons the algorithm in this paper is). Hence, we can safely assume that $D \leq n^{1/8}$. Assuming that failure detection can be performed within $O(\log \hat{D}_i)$ and the given memory constraints, we show that the algorithm terminates successfully for some guess $\hat{D}$, and that the runtime resulting from our guessing scheme is acceptable. Eventually for some guess l , it holds that $D \leq \hat{D}_l$ and $\hat{D}_{l'} < D$ for all $l' < l$. In particular, it holds that $D \leq \hat{D}_l \leq D^2$. Since $\hat{D}_l \in [D, \min(n^{1/8}, D^2)] \subseteq [D, n^{1/8}]$, MAX-ID-Solver will terminate successfully for $\hat{D}_l$. Our guessing scheme results in a runtime of at most

$$\sum_{i=1}^l O(\log \hat{D}_i) \leq \left(\sum_{i=0}^{\infty} \frac{1}{2^i} \right) O(\log D) + O(\log D^2) = O(\log D).$$

Detecting Failure. Let us consider the case when our guess $\hat{D}$ for the diameter is $< D$. It is possible for the algorithm to terminate even with a wrong diameter guess. However, we want to show that when the guess is wrong, we are able to detect it in $O(\log \hat{D})$ time even when the algorithm has not terminated. We also want to show that using a wrong diameter guess does not violate our memory constraints.

Our aim is to show that there exists a constant c such that if, after $c \log \hat{D}$ rounds, the algorithm is unsuccessful (failed), we can manually terminate the execution and move on to the next diameter guess. The algorithm can be seen as unsuccessful, if after a constant number of phases, the graph is larger than a singleton (Lemma 4.11). Note that failure cannot be detected during preprocessing, since there we simply free an appropriate amount of memory for the actual algorithm. The exact number of phases can be deduced from Lemma 4.11. Phases consist of algorithms `CompressLightSubTrees` and `CompressPaths` which both have a runtime of $O(\log \hat{D})$ by Lemma 4.7 and Lemma 4.8, respectively. The exact constant in Lemma 4.7 can be computed from Lemmas 4.12, 4.15 and 4.16. If the diameter guess is wrong, it will simply result in removing too small subtrees, and possibly a graph larger than a singleton being left after the phases. The exact constant in Lemma 4.8 can be computed from its proof in Section 4.4. If the diameter guess is wrong, it may still result in `CompressPaths` terminating successfully. However, it may also result in nodes not learning the whole path they are in, which we can detect and manually terminate the execution.

It is left to show that for a wrong diameter guess, the local memory $O(n)$ and global memory $O(n+m)$ are not violated. Both cases are surprisingly straightforward. The former holds since all of the local memory arguments of the section are independent of $\hat{D}$ (they only use its upper bound $n^{1/8}$). The latter holds by performing `Preprocessing` for $3 \log \hat{D}$ iterations before every execution, since all of our lemmas use at most $O((n+m) \cdot \hat{D}^3)$ global memory.

5.4 CC: Forests and Stability When executing algorithm MAX-ID-Solver and the scheme developed in Section 5.1 on forests instead of trees, we have to consider how the disjoint components can affect each other with regards to runtime, memory, and component-stability (Definition 5.4).

DEFINITION 5.4. (COMPONENT-STABILITY, [24]) *A randomized MPC algorithm A_{MPC} is component-stable if its output at any node v is entirely, deterministically, dependent on the topology and IDs (but independent of names) of v 's connected component (which we will denote $CC(v)$), v itself, the exact number of nodes n and maximum degree Δ in the entire input graph, and the input random seed S . That is, the output of A_{MPC} at v can be expressed*

as a deterministic function $A_{MPC}(CC(v), v, n, \Delta, S)$. A deterministic MPC algorithm A_{MPC} is component-stable under the same definition, but omitting dependency on the random seed S .

If we were to execute MAX-ID-Solver on a forest, nodes from disjoint components would never communicate with each other, rendering the runtime and memory arguments local. Hence, the algorithm is compatible with forests. In the scheme developed in Section 5.1 nodes from disjoint components communicate with each other only during preprocessing, when we employ the $O(1)$ time independent set algorithm of Lemma 5.2. Since the independent set is used to reduce the number of nodes *globally*, all of the runtime and memory arguments are still compatible with forests, as long as the given value D is in $[D, n^{1/8}]$, where D is the largest diameter of any component.

What is left to argue is that if the input graph is a forest, MAX-ID-Solver and the scheme developed in Section 5.1 are component-stable. This however follows directly from the stability definition (Definition 5.4) and our problem definition (Definition 5.1), because we require nodes to output the maximum identifier of their component, which is fully independent of other components.

5.5 Computing a Rooted Forest In this section, we show how to use the connected components algorithm of Theorem 1.1, with minor adjustments, to root a forest. We prove the following.

THEOREM 5.3. (ROOTING) *Consider the family of forests with component-wise maximum diameter D . There is a deterministic low-space MPC algorithm that roots the forest in $O(\log D)$ rounds using $O(n + m)$ words of global memory, and it is component-stable.*

First, we execute the algorithm of Theorem 1.1 in order for every node to learn the maximum ID of its component. Using this knowledge, we execute a modified version of the same algorithm that roots (in a component-stable way) every component towards the single node (per component) that is left after ℓ phases. The modifications are the following.

1. Redefine compression and decompression as follows

- Compress X into v : remove X (and its incident edges) from the graph. For any edge $\{x, y\}$ with $x \in X$ and $v \neq y \notin X$ we introduce a new edge $\{v, y\}$.
- Decompress X from v : Decompress X from v : add X (and its incident edges) back to the graph. If set X is a subtree, orient the revived edges towards the single node to which the subtree is attached to. If set X is a path, orient the revived edges in the same direction as edge $\{v, y\}$ that was added during the compression step of X into v .

2. In the derandomization of Lemma 5.2, the process is run independently on each connected component. For each component, we use an aggregation tree (recall Definition A.1) that consist of nodes only in the corresponding component with the maximum ID node as a root. Then, the good bitstrings can be determined through the independent aggregation trees.

Proof. [Proof of Theorem 1.2] The first modification to the algorithm ensures a rooted forest. We prove it by induction, with the base case being a rooted graph $G_\ell = \{v\}$. If graph G_i is rooted in the beginning of a reversal phase i , **DecompressPaths** extends the rooting of (some) single edges to paths, and **DecompressLightSubTrees** extends the rooting of (all) leaf nodes to subtrees, resulting in a rooted graph G_{i-1} (recall that the reversal phase indices are in decreasing order). The same exact logic also holds for **Postprocessing**, where we extend the rooting to the nodes that were removed during **Preprocessing**.

The second modification ensures component-stability. Since the maximum ID node of each component is responsible only for its own component, we can choose the bitstring independently of the other components and create the broadcast tree (like in Lemma 5.2) for each component separately and independently. Then, the orientation of the rooting (i.e., which node will become the root) only depends on the topology and the maximum ID of the component, making it independent of the other components. $\square$

6 Solving LCL Problems

In this section, we show a useful application of our forest rooting algorithm. In particular, we show that all problems contained in a wide class of problems that has been heavily studied in the distributed setting, called Locally Checkable Labelings (LCLs), can be solved in $O(\log D)$ deterministic rounds.

Informally, LCLs are a restriction of a class of problems called *locally checkable problems*. These problems satisfy that, given a solution, it is possible to check whether the solution is correct by checking the constant radius neighborhood around each node separately. Examples of these problems are classical problems such as maximal independent set, maximal matching, and $(\Delta + 1)$ -vertex coloring, but also more artificial problems, such that the problem of orienting the edges of a graph such that every node must have an odd number of outgoing edges.

The restriction that is imposed on locally checkable problems to obtain the class of LCLs is to require that the number of possible input and output labels that are required to define the problem must be constant, and moreover only graphs of bounded degree are considered. In the distributed setting, and in particular in the LOCAL model of distributed computing, LCLs have been extensively studied, see, e.g., [3, 9, 21, 12, 20, 10, 18, 16]. In particular, the imposed restriction makes it possible to prove very interesting properties on them, and to develop generic techniques to solve them. For example, we know that, if we restrict to forests, there are LCLs that can be solved in $O(1)$ rounds, there are LCLs that require $\Omega(\log^* n)$ rounds, but we also know that there is nothing in between, even if randomness is allowed (e.g., there are no LCLs with complexity $\Theta(\sqrt{\log^* n})$). Interestingly, techniques that have been developed to study LCLs have then often been extended and used to understand locally checkable problems in general (that is, problems that are not necessarily LCLs).

Since the LOCAL model is very powerful, and allows to send arbitrarily large messages, any solvable problem can be solved in $O(D)$ rounds. In this section, we provide an MPC algorithm for solving any solvable LCL problem on forests. The algorithm is deterministic, component-stable, and runs in $O(\log D)$ time in the low-space MPC model using $O(n + m)$ words of global memory (formal statement in Theorem 1.3). Moreover, our algorithm can be used *even for unsolvable LCLs*, that is, problems for which there exists some instance in which they are unsolvable. Hence, given any LCL, our algorithm produces a correct output on any instance that admits a solution, and it outputs “not solvable” on those instances where the LCL is not solvable.

THEOREM 1.3. (LCLs ON TREES) *All LCL problems on forests with maximum component diameter D can be solved in $O(\log D)$ time in the low-space MPC model using $O(n + m)$ words of global memory. The algorithm is deterministic and does not require prior knowledge of D .*

Notice that it is enough to prove Theorem 1.3 for rooted forests, since we can first root the forest by spending the same runtime and memory (Theorem 1.1), and then solve the LCL.

The remaining of the section is structured as follows: we start by giving a formal definition of LCLs (Section 6.1); we proceed by providing a high-level overview of our algorithm (Section 6.2); then, we provide the definition of the concept of “compatibility tree”, that will be useful later (Section 6.3); in Section 6.4 we give the explicit algorithm, called LCLSolver; in Sections 6.6 to 6.10 we show some properties of the subroutines used in LCLSolver, and we bound the time complexity of each of them; finally, we put things together and prove the main theorem of this section in Section 6.11.

We note that, for the sake of simplicity, algorithm LCLSolver is described for trees, but we will show in Section 6.11 that it can also be executed on forests.

6.1 Locally Checkable Labelings Locally Checkable Labeling (LCL) problems have been introduced in a seminal work of Naor and Stockmeyer [35]. The definition they provide restricts attention to problems where the goal is to label nodes (such as vertex coloring problems), but they remark that a similar definition can be given for problems where the goal is to label edges (such as edge coloring problems). A modern way to define LCL problems that captures both of the above types of problems (and combinations thereof) consists of labeling of *half-edges*, i.e., pairs (v, e) where e is an edge incident to vertex v . Let us first formally define half-edge labelings, and then provide this modern LCL problem definition.

DEFINITION 6.1. (HALF-EDGE LABELING) *A half-edge in a graph $G = (V, E)$ is a pair (v, e) , where $v \in V$, and $e = \{u, v\} \in E$. We say that a half-edge (v, e) is incident to some vertex w if $v = w$. We denote the set of half-edges of G by $H = H(G)$. A half-edge labeling of G with labels from a set Σ is a function $g: H(G) \rightarrow \Sigma$.*

We distinguish between two kinds of half-edge labelings: *input labelings*, that are labels that are part of the

input, and *output labelings*, that are provided by an algorithm executed on input-labeled instances. Throughout the paper, we will assume that any considered input graph G comes with an input labeling $g_{\text{in}}: H(G) \rightarrow \Sigma_{\text{in}}$ and will refer to Σ_{in} as the *set of input labels*; if the considered LCL problem does not have input labels, we can simply assume that $\Sigma_{\text{in}} = \{\perp\}$ and that each half-edge is labeled with $\perp$.

Informally, LCLs are defined on bounded-degree graphs, where each node may have in input a label from a constant-size set Σ_{in} of labels, and must produce in output a label from a constant-size set Σ_{out} of labels. Then, an LCL is defined through a set of locally checkable constraints that must be satisfied by all nodes.

DEFINITION 6.2. (LCL) *An LCL problem $\Pi = (\Sigma_{\text{in}}, \Sigma_{\text{out}}, C, r)$ is defined as follows:*

- Σ_{in} and Σ_{out} are sets of constant size that represent, respectively, the possible input and output labels.
- The parameter r is a constant called *checkability radius* of Π .
- C is a set of constant size, containing allowed neighborhoods. Each element $c_i = (G_i, v_i)$ of C , where $G_i = (V_i, E_i)$, is such that:
 - G_i is a graph satisfying that $v_i \in V_i$ and that the eccentricity of v_i in G_i is at most r ;
 - Every half-edge of G_i is labeled with a label in Σ_{in} and a label in Σ_{out} .

DEFINITION 6.3. (SOLVING AN LCL) *In order to solve an LCL on a given graph $G = (V, E)$ where to each element $(v, e) \in V \times E$ is assigned an input label from Σ_{in} , we must assign to each element $(v, e) \in V \times E$ an output label from Σ_{out} such that, for every $v \in V$, it holds that $(G_r(v), v) \in C$, where $G_r(v)$ is the subgraph of G induced by nodes at distance at most r from v and edges incident to at least one node at distance at most $r - 1$ from v .*

EXAMPLE 6.1. (MAXIMAL INDEPENDENT SET) *In the maximal independent set problem, the goal is to select an independent set of nodes that cannot be extended. That is, selected nodes must not be neighbors, and non-selected nodes must have at least one neighbor in the set.*

For this problem, $\Sigma_{\text{in}} = \{\perp\}$. Then, we can use two possible output labels, 1 to indicate nodes that are in the set, and 0 to indicate nodes that are not in the set. Hence, $\Sigma_{\text{out}} = \{0, 1\}$. Finally, we need to define r and C . For this problem, it is sufficient to pick $r = 1$. In C , we put all possible pairs (G, v) satisfying the following:

- G is a star centered at v ;
- G has at most Δ leaves (and there can be 0 leaves);
- For each node in G , either all incident half-edges are output labeled 0, or all incident half-edges are output labeled 1;
- If v is labeled 1, then all leaves are labeled 0;
- If v is labeled 0, then at least one leaf is labeled 1.

Hence, the idea is that MIS can be checked by just inspecting the radius-1 neighborhood of each node, which is a star, and we just list all stars that are valid.

While Definition 6.2 gives an easy way to define problems, such a definition is not the most convenient for proving statements about LCLs. In order to make our proofs more accessible, we consider an alternative definition of LCLs, called *node-edge formalism*. It is known that, on trees and forests, any LCL defined as in Definition 6.2 can be converted, in a mechanical way, into an LCL described by using this formalism, such that the obtained LCL has the same asymptotic complexity of the original one [10].

DEFINITION 6.4. (NODE-EDGE-CHECKABLE LCL) *In this formalism, a problem Π is a tuple $(\Sigma_{\text{in}}, \Sigma_{\text{out}}, C_V, C_E)$ satisfying the following:*

- As before, Σ_{in} and Σ_{out} are sets of constant size that represent, respectively, the possible input and output labels;

- C_V and C_E are both sets of multisets of pairs of labels, where each pair is in $\Sigma_{\text{in}} \times \Sigma_{\text{out}}$, and multisets in C_E have size 2.

DEFINITION 6.5. (SOLVING A NODE-EDGE CHECKABLE LCL) *Solving an LCL given in this formalism means that we are given a graph $G = (V, E)$ where to each element $(v, e) \in V \times E$ is assigned a label $i_{v,e}$ from Σ_{in} , and to each element $(v, e) \in V \times E$ we must assign a label $o_{v,e}$ from Σ_{out} such that:*

- *For every node $v \in V$ it holds that the multiset $M_v = \{(i_{v,e}, o_{v,e}) \mid e \text{ is incident to } v\}$ satisfies $M_v \in C_V$;*
- *For every edge $e \in E$ it holds that the multiset $M_e = \{(i_{v,e}, o_{v,e}) \mid e \text{ is incident to } v\}$ satisfies $M_e \in C_E$.*

Hence, in the node-edge checkable formalism, we are given a graph where each half-edge (that is, an element from $V \times E$) is labeled with a label from Σ_{in} , the task is to label each half-edge from a label from Σ_{out} , and the LCL constraints are expressed by listing tuples of size at most Δ representing allowed configurations for the nodes, and tuples of size 2 representing allowed configurations for the edges. In [10] it has been shown that any LCL Π defined on trees or forests can be converted into a node-edge checkable LCL Π' satisfying that the complexity of Π and Π' differ only by an additive constant. Hence, for the purposes of this work, we can safely restrict our attention to node-edge-checkable LCLs.

EXAMPLE 6.2. (MAXIMAL INDEPENDENT SET) *Sometimes, defining an LCL in the node-edge checkable formalism is non-trivial. MIS is an example of problems in which the conversion requires a bit of work (it can be done mechanically, though, as shown in [10]). We hence use MIS as an example for this formalism.*

As before, $\Sigma_{\text{in}} = \{\perp\}$, and hence when listing the elements in C_V and C_E we will not specify the input labels. This time, it is not actually possible to use just 2 labels as output. In fact, we define $\Sigma_{\text{out}} = \{0, 1, P\}$. Then, C_V contains all multisets of size at most Δ satisfying that:

- *All elements are 1, or*
- *one element is P and all the others are 0.*

Then, $C_E = \{\{1, 0\}, \{1, P\}, \{0, 0\}\}$. In other words, nodes in the MIS output 1 on all their incident half-edges, nodes not in the MIS output P on one incident half-edge and 0 on all the others. The label P is used to prove maximality. That is, nodes not in the set must point to one neighbor in the set by using the label P. In fact, on the edge constraint, P is only compatible with 1. Observe that, given a solution for the standard MIS problem, a solution for this variant can be produced with just one round of communication.

6.2 Overview On a high level, our algorithm works as follows. We describe it from the point of view of a single node, and for a single tree of the forest. Firstly, we root the tree, obtaining that each node knows the edge connecting it to its parent. Then, the algorithm proceeds in phases, and in total the number of phases is going to be a constant that depends on the amount of memory available to the machines. In each phase, we *compress* the tree into a smaller tree, as follows:

- all subtrees containing less than a fixed amount of nodes are compressed to their root;
- all paths are compressed into a single edge.

Each phase is going to require $O(\log D)$ time. In other words, this part of our algorithm works similar to the standard rake-and-compress algorithm. Moreover, while compressing the tree, we maintain some information about the LCL that we are trying to solve. This information is called *compatibility tree*.

The compatibility tree, for each node and for each edge, keeps track of the possible configurations that they can use. At the beginning, for each node, these configurations correspond to the configurations in C_V that are compatible with the given input, and for each edge, these configurations correspond to the configurations in C_E that are compatible with the given input.

When compressing a subtree into a single node, we update the list of the configurations usable on that node, in such a way that each configuration satisfies the following: if the node uses it, then it is possible to assign a labeling on the subtree compressed into that node, in such a way that, for each node and edge in the compressed subtree we use only configurations allowed by the compatibility tree before the compression.

Similarly, when compressing a path, for the new edge that we add, we store a list of configurations satisfying that, if we label the first and last half-edge of the removed path with the labels of the configuration, then we can complete the compressed path by only using configurations allowed by the compatibility tree before the compression.

At the end, we obtain that the whole tree is recursively compressed on a single node v . If the compatibility tree does not allow any configuration for v , then we know that the LCL is unsolvable. Otherwise, we can pick an arbitrary configuration allowed by the compatibility tree and assign it to v . By performing this operation, we know that we can safely put back the paths and subtrees that were compressed on v and have the guarantee that we can label them using only allowed configurations. Hence, we again proceed in phases, where we put back compressed paths and subtrees in the opposite order in which they have been compressed, and each time we assign labels allowed by the compatibility tree. At the end, we obtain that the whole tree is labeled correctly, and hence the LCL is solved.

6.3 Compatibility Tree A compatibility tree is an assignment of sets of allowed configurations to nodes and edges, where this time configurations are not just multisets, but they are tuples. In other words, we may allow a node to use a configuration, but only if the labels of that configuration are used in a very specific order.

DEFINITION 6.6. (COMPATIBILITY TREE) *A compatibility tree of a tree $G = (V, E)$ is a pair of functions ϕ and ψ , where ϕ maps each node $v \in V$ into a set of tuples of size at most Δ , and ψ maps each edge $e \in E$ into a set of tuples of size 2.*

In order to specify how the compatibility tree is initialized, it is useful to first assign an order to the edges incident to each node, and to the nodes incident to each edge. This ordering is called port numbering assignment. Observe that an arbitrary port numbering assignment can be trivially computed in 1 round of communication.

DEFINITION 6.7. (PORT NUMBERING) *A node port numbering is a labeling of every half-edge satisfying that, for each node v , half-edges incident to v have pairwise distinct values in $\{1, \dots, \deg(v)\}$. An edge port numbering is a labeling of every half-edge satisfying that, for each edge e , half-edges incident to e have pairwise distinct values in $\{1, 2\}$. A port numbering is the union of a node port numbering and an edge port numbering.*

Assume that the tree G is already provided with a port numbering. The compatibility tree of G is initialized as follows. For each node v , $\phi(v) = \{(\ell_1, \dots, \ell_{\deg(v)}) \mid \{(i_1, \ell_1), \dots, (i_{\deg(v)}, \ell_{\deg(v)})\} \in C_V\}$, where i_j is the input assigned to the half-edge incident to v with node port number j . For each edge e , $\psi(e) = \{(\ell_1, \ell_2) \mid \{(i_1, \ell_1), (i_2, \ell_2)\} \in C_E\}$, where i_j is the input assigned to the half-edge incident to e with edge port number j . In other words, we initialize ϕ and ψ with everything that is allowed by the constraints of the problem, in all possible orders that are compatible with the given input.

We can observe that, by construction, ϕ and ψ still encode the original problem. In other words, we can now forget about C_V and C_E , and try to find a labeling assignment that is valid according to ϕ and ψ . We make this observation more formal in the following statement.

OBSERVATION 6.1. *The LCL problem Π is solvable if and only if there is a labeling $g_{\text{out}} : H \rightarrow \Sigma_{\text{out}}$ that solves Π that satisfies that:*

- *For each node v , let ℓ_j be the label assigned by g_{out} to the half-edge incident to v with port number j . It must hold that $(\ell_1, \dots, \ell_{\deg(v)}) \in \phi(v)$.*
- *For each edge e , let ℓ_j be the label assigned by g_{out} to the half-edge incident to e with port number j . It must hold that $(\ell_1, \ell_2) \in \psi(e)$.*

On a high level, when compressing a subtree into a node v , we will redefine $\phi(v)$ and discard some tuples. The discarded tuples are the ones satisfying that, if node v uses such a configuration, there is no way to complete the labeling of the subtree in a valid way. Similarly, when compressing a path, we will define $\psi(e)$, where e is the new (virtual) edge that we use to replace the path, in such a way that, if $\psi(e)$ contains the tuple (ℓ_1, ℓ_2) and we label the first half-edge of the compressed path with ℓ_1 and the last half-edge with ℓ_2 , then we can correctly complete the labeling inside the path. Here it should become clear why we use tuples and not just multisets: it may be that v can use a label on the half-edge connecting it to one child (because that subtree can be competed

by starting with that label), but the same label cannot be used on the half-edge connecting v to a different child. A similar situation could happen on a compressed path: it could be that it is possible to label ℓ_1 the half-edge connecting the first endpoint to the path and ℓ_2 the half-edge of the second endpoint to the path, but not vice versa.

In the algorithm, we will solve the problem Π in the tree obtained by compressing some subtrees into single nodes, and some paths into single edges. We now formally define what it means to partially solve an LCL Π w.r.t. a compatibility tree (ϕ, ψ) . Observe that, in a tree G obtained after performing some compression steps, a node v may have a degree that is smaller than the size of the tuples given by $\phi(v)$, that always have size equal to the original degree of v , denoted by $\text{origdeg}(v)$, and hence the ports incident to v may be just a subset of $\{1, \dots, \text{origdeg}(v)\}$.

DEFINITION 6.8. (PARTIALLY SOLVING AN LCL W.R.T. THE COMPATIBILITY TREE) *Let G be a tree, and let (ϕ, ψ) be a compatibility tree for G . A solution for Π that is correct according to ϕ and ψ is a labeling g_{out} satisfying that:*

- *For each node v , let $\text{origdeg}(v)$ be the size of the tuples given by $\phi(v)$, and let $P(v) \subseteq \{1, \dots, \text{origdeg}(v)\}$ be the subset of ports of v that are present in G . For each $j \in P(v)$, let ℓ_j be the label assigned by g_{out} to the half-edge incident to v with port number j . There must exist labels ℓ_k , for all $k \in \{1, \dots, \text{origdeg}(v)\} \setminus P(v)$, such that it holds that $(\ell_1, \dots, \ell_{\text{origdeg}(v)}) \in \phi(v)$.*
- *For each edge e , let ℓ_j be the label assigned by g_{out} to the half-edge incident to e with port number j . It must hold that $(\ell_1, \ell_2) \in \psi(e)$.*

In other words, solving the LCL in the tree obtained by performing some compression steps, means to pick, for each node, a configuration allowed by ϕ , in such a way that all edges that are still present have a configuration allowed by ψ .

6.4 The Algorithm Let $\Pi = (\Sigma_{\text{in}}, \Sigma_{\text{out}}, C_V, C_E)$ be the considered LCL problem, and let $G_0 = G$ denote a rooted input tree with root r . The high-level idea of our approach is to first initialize ϕ_0 and ψ_0 as the functions ϕ and ψ shown in Section 6.3. Then, we perform the following distinct parts.

Steps 1–2 of LCLSolver. From G_0 , we iteratively derive a sequence $G_1, G_2, \dots, G_t$ of smaller trees until eventually, for some $t = O(1)$, it holds that G_t consists of a single node (the root r). In the meanwhile, we also update the compatibility tree, and compute ϕ_j and ψ_j for all $0 < j \leq t$. The sequence is derived such that G_j ($0 < j \leq t$) is obtained from G_{j-1} by first compressing all subtrees of size $\leq n^{1/2}$ into their respective roots (we refer to the roots of the subtrees and not (necessarily) the actual root node r), and then compressing all paths into single edges. Throughout the sequence of compatibility trees, we maintain the following property, which we prove in Lemmas 6.3 and 6.6.

CLAIM 6.1. *Let $1 \leq j \leq t$. If there exists a correct solution for G_{j-1} according to ϕ_{j-1} and ψ_{j-1} (w.r.t. Definition 6.8), then there exists also a correct solution for G_j according to ϕ_j and ψ_j (w.r.t. Definition 6.8). Moreover, given any correct solution for G_j , we can transform it into a correct solution for G_{j-1} .*

Steps 3–4 of LCLSolver. For all $0 \leq i \leq t$, we define $\hat{G}_i$ to be G_i where to each node v is assigned a configuration $c(v) \in \phi_i(v)$ in such a way that the assignment c induces a labeling g_{out} that is correct according to ϕ_i and ψ_i (w.r.t. Definition 6.8). The sequence is derived such that $\hat{G}_i$ is obtained from $\hat{G}_{i+1}$ by decompressing the subtrees and paths that were compressed when G_{i+1} was obtained from G_i , and simultaneously solving the problem, which is possible by Claim 6.1. Finally, the solution on $\hat{G}_0$ is a solution for Π on G by Observation 6.1.

The Algorithm. Let us now formally present the algorithm LCLSolver (along with its subroutines) that solves any LCL on rooted trees in $O(\log D)$ time. Note that the subroutines are state-changing functions, i.e., they modify their input graphs.

LCLSolver($\Pi, G(V, E)$)

1. Initialize ϕ_0 and ψ_0 according to Section 6.3.

2. Initialize phase counter $j \leftarrow 0$. Repeat the following until the graph is a singleton.

- (a) **CountSubtreeSizes**(G)
- (b) **GatherSubtrees**(G)
- (c) **CompressSubtrees**(G)
// compress all subtrees of size $\leq n^{1/2}$ into single nodes, also compute ϕ_{j+1}
- (d) **AdvancedCompressPaths**(G)
// compress all paths into single edges, also compute ψ_{j+1}
- (e) Update $j \leftarrow j + 1$

3. Set $c(v)$ to be an arbitrary element of $\phi_j(v)$, where v is the obtained singleton.
// the graph $\dot{G}_j = \dot{G}$ is obtained

4. Initialize repetition counter $k \leftarrow j$. Repeat the following while $k \geq 0$.

- (a) **DecompressPaths**($\dot{G}$)
// decompress the paths from phase k
- (b) **DecompressSubtrees**($\dot{G}$)
// decompress the subtrees from phase k , the graph $\dot{G}_{k-1}$ is obtained
- (c) Update $k \leftarrow k - 1$

Observe that the phase counter is incremented, while the repetition counter is decremented, which is inline with the indexing used in the previous high-level overview. Next, we give a brief introduction to the subroutines, before defining them formally in the following subsections. Recall that $T(v)$ denotes the subtree that is rooted at a node v such that v belongs to $T(v)$ and $G \setminus T(v)$ is connected.

- **CountSubtreeSizes**: Every node v learns either the exact size of $T(v)$ or that $|T(v)| > n^{1/2}$. In the former case, v marks itself as *light*, and in the latter case, as *heavy*. If a heavy node has a light child, it remarks itself as a *local root*.
- **GatherSubtrees**: Every local root v learns $T(u)$ for every light child u .
- **CompressSubtrees**: Every local root v checks, for each half-edge h connecting it to a light child u , what are the possible labelings of h that allow to complete the labeling of $T(u)$ in such a way that it is valid according to ϕ and ψ . Then, v compresses all of these trees into itself, and updates ϕ in such a way that any LCL solution on the remaining graph can be extended to a solution on $T(u)$ for every light child u . This is done according to the computed possible labelings of the half-edges.
- **AdvancedCompressPaths**: For every path P with some endpoints u and w (both have either degree 1 or ≥ 3), compress P into a new edge $\{u, w\}$, and assign an arbitrary edge port numbering to this edge. The value of $\psi(\{u, w\})$ is then defined in a way that any LCL solution on the edge $\{u, w\}$ can be extended to a solution on P . After performing **CompressPaths**, there are no degree-2 nodes left, which will be crucial for the analysis.
- **DecompressPaths**: The LCL problem on the input graph is solved. In repetition k , we decompress all paths that were compressed during phase k . While decompressing, we extend the solution to the paths.
- **DecompressSubtrees**: The LCL problem on the input graph is solved. In repetition k , every local root of phase k decompresses all subtrees that it compressed during phase k . While decompressing, we extend the solution to the subtrees.

The following sections are rather self-contained and correspond to a specific subroutine that is called by **LCLSolver** (in the order they are called). They are written from a node's point of view, with the proofs intertwining correctness, runtime, and MPC details.

6.5 Solving LCLs: CountSubtreeSizes During the execution of the subroutine, every node v maintains the following variables:

- i : iteration counter
- $s(v)$: size of $T(v)$ until depth 2^i (v is at depth 0)
- $C(v)$: set of all descendant nodes (if any) of v at depth 2^i (v is at depth 0).

The aim of the subroutine is to detect all heavy nodes, i.e., nodes which have a subtree of size $> n^{1/2}$ rooted at them. This can be thought of as a preprocessing step for **GatherSubtrees**. All nodes are initially marked as active.

CountSubtreeSizes(G)

Each node v initializes: $C(v) \leftarrow$ set of children of v in G , $s(v) \leftarrow |C(v)|$, and $i \leftarrow 0$.

1. Repeat the following steps until $C(v) = \emptyset$ for every node v .

(a) If v is active and all $u \in C(v)$ are also active, v updates

- $C(v) \leftarrow \bigcup_{u \in C(v)} C(u)$
- $s(v) \leftarrow s(v) + \sum_{u \in C(v)} s(u)$.

Otherwise, v marks itself as heavy, and it becomes inactive.

(b) If $s(v) > n^{1/2}$, v marks itself as heavy, and it becomes inactive. Heavy nodes update $C(v) \leftarrow \emptyset$.

(c) All active nodes update $i \leftarrow i + 1$.

Non-heavy nodes marks themselves as light.

Upon termination, for every heavy node v it holds that $|T(v)| > n^{1/2}$. Note that the ancestors of heavy nodes are also heavy; heavy nodes induce a single connected component in G .

LEMMA 6.1. (CountSubtreeSizes) *Every node v learns either the exact size of $T(v)$ or that $|T(v)| > n^{1/2}$. The algorithm terminates in $O(\log D)$ low-space MPC deterministic rounds using $O(n + m)$ words of global memory.*

Proof. We first show that every active node v maintains the correct values for $s(v)$ and $C(v)$ throughout the algorithm. In iteration $i = 0$, values $s(v)$ and $C(v)$ are correct by initialization. During iteration i in Step 1(a), v updates its values only if v together with all of its descendants in $C(v)$ are active, resulting in the correct values for iteration $i + 1$ by construction (see Figure 2). A node v marks itself as heavy only when $s(v) > n^{1/2}$ or when one of its descendants is heavy. Both conditions imply that $|T(v)| > n^{1/2}$. If neither conditions are met and $C(v) = \emptyset$ at some point, then the value $s(v)$ for node v is the exact size of $T(v)$ and v marks itself light.

The algorithm terminates in $O(\log D)$ iterations (with each iteration taking $O(1)$ MPC rounds), since an active node knows $T(v)$ until depth 2^i in iteration i , and the depth of a tree is D . Observe that for every light node v , it holds that $|C(v)| \leq s(v) \leq n^{1/2}$. Hence, local memory is never violated, because when a node updates $C(v)$ in Step 1(a), the resulting set is of size at most $n^{1/2} \cdot n^{1/2}$ nodes. Also, storing value $s(v)$ takes only $O(\log n)$ bits. Global memory is never violated, since, by design, a node u is only kept in the set $C(\cdot)$ of exactly one node.

Algorithm **CountSubtreeSizes** can be thought of as a modified version of graph exponentiation where nodes only keep track of the furthest away descendants. For the communication in Step 1(a) to be feasible, the set $C(v)$ is simply a set of IDs corresponding to the desired nodes. Observe that the communication in Step 1(a) is always initialized by v , and not by the descendants in $C(v)$ (nodes in $C(v)$ don't even know the ID of v). This is feasible, because, by design, every node has at most one ancestor that initializes communication. $\square$

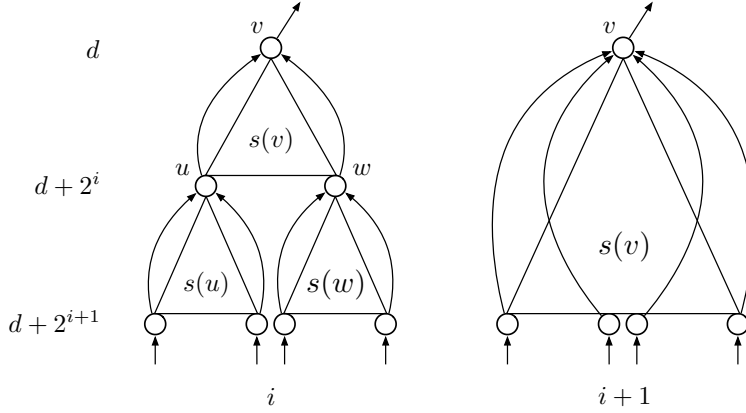


Figure 2: An illustration of an update in Step 1(a) of `CountSubtreeSizes`. The intuition is that node v learns the size of the partial subtrees hanging from every child in $C(v)$. The values d and $d + 2^i$ on the left refer to the depth of nodes v and u, w , respectively, with regards to the whole tree. The oriented edges are not the actual edges of G , but rather a representation of sets $C(\cdot)$. The incoming edges of node v are incident to the nodes in set $C(v)$ in both iterations i and $i + 1$. The value $s(v)$ in iteration $i + 1$ is simply the sum of $s(v)$, $s(u)$ and $s(w)$ from iteration i .

6.6 Solving LCLs: GatherSubtrees After executing `CountSubtreeSizes`, by Lemma 6.1, every node knows if it is heavy or light. Moreover, every heavy node v knows if it has a light child or not. If so, node v remarks itself from heavy to *local root*. If there are no local roots in G , mark the actual root of the tree as a local root. During algorithm `GatherSubtrees`, heavy nodes do nothing, and all other nodes (including local roots) maintain the following variables:

- i : iteration counter
- $C(v)$: a subset of descendant nodes.

The procedure is as follows (see Figure 3 for an example).

GatherSubtrees(G)

Each node v initializes: $C(v) \leftarrow$ set of light children of v in G , and $i \leftarrow 0$.

1. Repeat the following steps until $C(v)$ for every local root v consist of the union of subtrees $T(u)$ for every light child u .
 - (a) Every local root v updates $C(v) \leftarrow C(v) \cup \bigcup_{u \in C(v)} C(u)$.
 - (b) Every light node w updates $C(w) \leftarrow \bigcup_{u \in C(w)} C(u)$.
 - (c) Update $i \leftarrow i + 1$.

We phrase the algorithm in terms of the subtrees of the light children of a local root v , instead of the subtree of v directly, and we do this for a simple reason: a local root v may have children that are also local roots, in which case, v does not want to learn anything in their direction.

LEMMA 6.2. (`GatherSubtrees`) *Every local root has gathered the subtree $T(u)$ for every light child u . The algorithm terminates in $O(\log D)$ low-space MPC deterministic rounds using $O(n + m)$ words of global memory.*

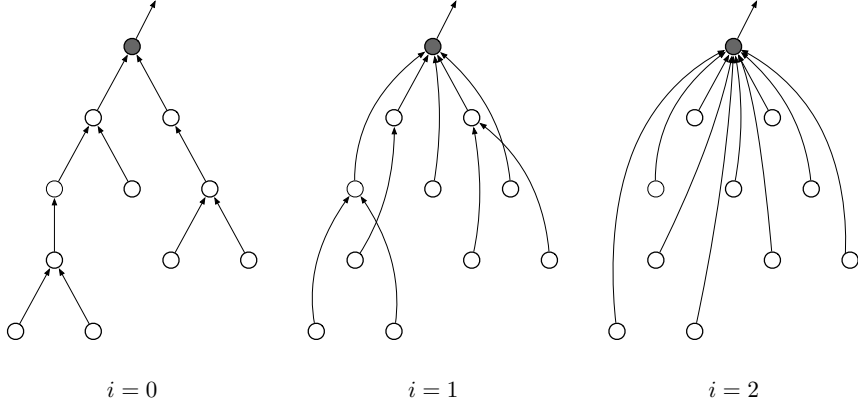


Figure 3: Three iterations of **GatherSubtrees**, with a local root marked black. The directed edges are not necessarily the actual edges of G , but rather a representation of sets $C(\cdot)$. The incoming edges of a node v are incident to the nodes in set $C(v)$. The figure illustrates how local roots behave differently than other nodes: local roots aggregate all descendants, while other nodes replace current ones with new ones.

Proof. By design, the set $C(v)$ of a local root v contains $T(u)$ until depth $2^i - 1$ for every light child u in iteration i . Since the depth of a tree is D , after at most $O(\log D)$ iterations, for every local root v , $C(v)$ contains $T(u)$ for every light child u .

Observe that $|T(u)| \leq n^{1/2}$ for every light node u . This implies that $|C(v)| \leq n$ for every local root v , since it has a constant number of children (the maximum degree of the graph is constant). Hence, local memory is respected. Global memory is never violated, since by design, a node u is only kept in set $C(\cdot)$ of exactly one node.

Similarly to **CountSubtreeSizes**, algorithm **GatherSubtrees** can be thought of as a modified version of graph exponentiation. However, as opposed to **CountSubtreeSizes**, algorithm **GatherSubtrees** actually gathers the whole subtrees into the memory of preselected nodes (local roots). Similarly to **CountSubtreeSizes**, we store IDs in the sets $C(\cdot)$ in order for the communication in Step 1 to be feasible. Also, the communication is made possible due to every node having at most one ancestor that initializes the communication. $\square$

6.7 Solving LCLs: **CompressSubtrees** After executing **GatherSubtrees**, by Lemma 6.2, every local root has gathered the IDs of the nodes in the subtree $T(u)$ for every light child u .

CompressSubtrees(G)

1. Perform the following step for every light child u of every local root v . Denote $e^* = (u, v)$. Every local root v gathers the topology of $T(u)$, along with $\phi(w)$ for every node w in $T(u)$ and $\psi(e)$ for every edge e in $T(u)$. Every local root v computes the set of labels $L(e^*)$ satisfying that, by labeling the half-edge (v, e^*) with a label in $L(e^*)$, it is possible to complete the labeling in $T(u)$ by only using configurations allowed by ϕ and ψ .
2. Every local root v updates $\phi(v)$ by possibly discarding some tuples. Let $P(v)$ be the set of ports of v connecting it to light nodes, and let e_i be the edge reached from v by following port i . A tuple $(\ell_1, \dots, \ell_d)$ is kept in $\phi(v)$ if and only if $\ell_i \in L(e_i)$ for all $i \in P(v)$.

LEMMA 6.3. Let ϕ and ψ be the compatibility tree before performing **CompressSubtrees**, and let ϕ' be the updated compatibility tree after performing **CompressSubtrees**. Let G' be the resulting graph after performing

CompressSubtrees. *The following holds.*

- If there exists a correct solution for the input graph G according to ϕ and ψ , then there exists also a correct solution for G' according to ϕ' and ψ .
- Given any correct solution for G' according to ϕ' and ψ , it can be transformed into a correct solution for G according to ϕ and ψ .

Proof. In order to show the first statement, suppose that there is a correct solution for G . Notice that, since G' is a subgraph of G , if, for every $v \in G'$, we use the tuple in $\phi(v)$ of the correct solution in G , then we get a correct solution for G' . Hence, we need to ensure that the tuple used by v is in $\phi'(v)$. But this is exactly what we do: every tuple excluded from $\phi(v)$ in Step 2 is not part of any correct solution for G , and hence the first statement holds. For the second statement, observe that from the definition of G' , it follows that any correct solution for G' provides a partial solution for G (all labels are fixed except the ones in the compressed subtrees), and this partial solution is part of a correct solution for G . Hence, a correct solution for G can be obtained by extending the provided solution to the compressed subtrees. An extension is guaranteed to exist, since all non-extendable tuples of $\phi(v)$ were removed previously in Step 2. Note that this extension can be performed by all local roots simultaneously, since there are no dependencies between subtrees. $\square$

LEMMA 6.4. (CompressSubtrees) *The algorithm terminates in $O(1)$ time in the low-space MPC model and uses $O(n + m)$ words of global memory.*

Proof. Gathering the topology of $T(u)$, along with $\phi(w)$ for every node w in $T(u)$ and $\psi(e)$ for every edge e in $T(u)$ is possible: by Lemma 6.2, the local root v knows the IDs of all nodes in $T(u)$, and hence node v can simply gather all incident edges from all nodes in $T(u)$ in constant time, and reconstruct $T(u)$ locally. This does not break any memory constraints, since v receives every edge from at most two nodes, the number of edges is bounded by the number of nodes, and the sets $\phi(w)$ and $\psi(e)$ are of constant size. Computing the sets $L(\cdot)$ does not require communication, and can be done locally in constant time. Finally, the removal (contraction) of nodes and updating the set $\phi(v)$ also takes constant time, concluding the runtime proof. Moreover, the global memory is not violated, since similarly to **GatherSubtrees**, a node u is gathered by only one local root v . $\square$

6.8 Solving LCLs: AdvancedCompressPaths The aim of this algorithm is to compress all paths into single edges while retaining the compatibility information of the paths, i.e., if the problem is solved, the solution can be extended to the paths that were compressed. As opposed to the previous subroutines, **AdvancedCompressPaths** does not capitalize on anything that is done by the previous routines.

We begin with a slight detour and first show how all degree-2 nodes can compute their distance to the highest ID endpoint with the following algorithm. To keep things simple, we present an algorithm for a single path H with two endpoints of degree 1.

CountDistances(G)

1. Define degree-2 nodes as *internal* nodes, degree-1 nodes as *endpoints*, and the higher ID endpoint as the *head*. The head is denoted by h . Assign weight $w(e) \leftarrow 1$ for every edge e . Repeat the following steps until all internal nodes share a weighted edge with both endpoints; the weight of the edge equals the distance.

- (a) Every internal node with incident edges $e = \{u, v\}$ and $e' = \{v, w\}$, removes* e and e' from H and replaces them with a new edge $e'' = \{u, w\}$ and sets $w(e'') \leftarrow w(e) + w(e')$.

*If u (or w) is an endpoint, node v does not remove e (or e').

LEMMA 6.5. (CountDistances) *Every degree-2 node knows its distance to both endpoints. The algorithm does not require prior knowledge of D , terminates in $O(\log D)$ low-space MPC rounds using $O(n + m)$ words of global memory.*

Proof. The weight of each edge $\{v, u\}$ in the graph equals the number of edges between v and u in the original graph. The base case being evident from the initialization of the weight of each edge, and the induction step from the update step $w(e'') \leftarrow w(e) + w(e')$. Since the shortest path from either endpoint to any other node in the path decreases by a factor of at least $3/2$, **CountDistances** terminates in $O(\log D)$ time.

Creating edges and communicating through them can be done in constant time in MPC, since storing an edge equals storing the ID of the neighbor. Observe that every internal node keeps exactly two edges in memory. In order to not break their local memory, endpoints do not keep track of any edges. Since all nodes keep at most two edges in memory, global memory is respected. $\square$

Next, we show that, by using the distances computed with **CountDistances**, we can compress paths of all lengths in $O(\log D)$ time, while respecting both local and global memory. Recall that for a path, h denotes its highest ID endpoint.

AdvancedCompressPaths(G)

1. Set $i \leftarrow 0$ and execute the following steps for every path H until it consists of one edge.
 - (a) Define an MIS set $Z_i := \{v \in H \mid \deg(v) = 2 \text{ and } d_G(v, h) \text{ is not divisible by } 2^{i+1}\}$.
 - (b) Every node $v \in Z_i$ with incident edges $e = \{u, v\}$ and $e' = \{v, w\}$ removes e and e' from G' and replaces them with a new edge $e'' = \{u, w\}$ (with port 1 connected to u and port 2 connected to w). Furthermore, for the new edge, v sets $\psi(e'')$ to the set of all tuples (ℓ_1, ℓ_2) satisfying that there exist two labels x, y satisfying the following. Let p_1 be the port connecting e to node u , let p_2 be the port connecting e to v , let p_3 be the port connecting v to e , let p_4 be the port connecting v to e' , let p_5 be the port connecting e' to v , and let p_6 be the port connecting e' to w :
 - There is a tuple in $\psi(e)$ with label ℓ_1 in position p_1 and label x in position p_2 ;
 - There is a tuple in $\phi(v)$ with label x in position p_3 and label y in position p_4 ;
 - There is a tuple in $\psi(e')$ with label y in position p_5 and label ℓ_2 in position p_6 .
 - (c) Update $i \leftarrow i + 1$.

From the perspective of the nodes u and w , the new edge e'' replaces the old edges e and e' , respectively. In other words, if u was connected through port j to edge e , now it is connected through port j to edge e'' . Notice that, the reason for which, at each step, we compute an MIS, is that, if MIS nodes replace their two incident edges of the path with a single edge, we still obtain a (shorter) path as a result.

Consecutive MIS. Executing **CountDistances** gives us the means to compute consecutive maximal independent sets in **AdvancedCompressPaths**, which is not exactly obvious nor easily attainable using other means. If we were to compute an MIS directly with, e.g., Linial's [32] algorithm in every iteration of Step 2, we would end up with a total runtime of $O(\log D \cdot \log^* n)$. An alternative approach would be to employ the component-unstable $O(1)$ -time algorithm that computes an independent set of size $\Omega(n/\Delta)$ by [24]. This approach also fails for multiple paths, since the algorithm in [24] does not give the guarantee that a constant fraction of nodes in *all* paths join the independent set, leading to a total runtime of $O(\log n)$. To summarize, **CountDistances** is a novel approach to a very non-trivial problem, yielding component stability and a sharp $O(\log D)$ runtime.

LEMMA 6.6. *Let ϕ and ψ be the compatibility tree before performing **AdvancedCompressPaths**, and let ψ' be the updated compatibility tree after performing **AdvancedCompressPaths**. Let G' be the resulting graph after performing **AdvancedCompressPaths**. The following holds.*

- *If there exists a correct solution for the input graph G according to ϕ and ψ , then there also exists a correct solution for G' according to ϕ and ψ' .*
- *Given any correct solution for G' according to ϕ and ψ' , it can be transformed into a correct solution for G according to ϕ and ψ .*

Proof. Assuming that Z_i is indeed an MIS, the first statement follows from the fact that acting nodes (i.e., MIS nodes) are never neighbors and every set $\psi(\{u, w\})$ that an acting node v creates only discards configurations that do not correspond to valid solutions for the subpath (u, v, w) . The second statement holds by the definition of labels ℓ_1, ℓ_2, x, y , since we can perform the process in reverse.

Let us show that Z_i constitutes an MIS. For any i , observe that the distances of the remaining nodes constitute all multiples of 2^i up until some number (the length of the path). Hence, every second node is not divisible by 2^{i+1} and joins Z_i , proving the statement. $\square$

LEMMA 6.7. (AdvancedCompressPaths) *There are no degree-2 nodes left in the graph. The algorithm terminates in $O(\log D)$ low-space MPC rounds using $O(n + m)$ words of global memory.*

Proof. Since Z_i constitutes an MIS, every path shortens by a constant factor. After $O(\log D)$ iterations, every path is compressed into a single edge. Every iteration consists of a constant number of communication rounds, every node uses a constant amount of memory, and compressing paths into edges never creates new degree-2 nodes. $\square$

6.9 Solving LCLs: DecompressPaths Assuming that the problem of interest is solved in the current graph, we essentially reverse **AdvancedCompressPaths** and iteratively extend the solution from certain edges to the paths that were previously compressed into those edges. By “the problem is solved in the current graph” we simply mean that the output labels of the half-edges in the current graph satisfies Definition 6.8.

DecompressPaths($\dot{G}$)

1. All nodes that performed **AdvancedCompressPaths** in phase k , know the last iteration i and can perform the following until $i = 0$.
 - (a) Every node $v \in Z_i$ learns the fixed half-edge labels (ℓ_1, ℓ_2) assigned to $e'' = (u, w)$ (e'' is the edge v had created). Node v removes e'' from the graph and replaces it with $e = (u, v)$ and $e' = (v, w)$ (edges e and e' are the edges v had removed). Furthermore, v assigns half-edge labels ℓ_1, x to edge e and labels y, ℓ_2 to edge e' such that the labeling satisfies $\psi(e)$, $\phi(v)$, and $\psi(e')$.
 - (b) Update $i \leftarrow i - 1$

LEMMA 6.8. (DecompressPaths) *The LCL problem on the graph is solved according to Definition 6.8, and the graph has the same node and edge sets as G in phase k before executing **AdvancedCompressPaths**. The algorithm terminates in $O(\log D)$ low-space MPC rounds using $O(n + m)$ words of global memory.*

Proof. All we do is reversing the steps of **AdvancedCompressPaths** and extending the solution for $\dot{G}$ to the decompressed paths, resulting in a correct solution on the graph that has the same node and edge sets that we had before the compression. Since the computation of the solution is done locally, and extending the solution requires a constant amount of memory and communication, the lemma follows from Lemmas 6.6 and 6.7. $\square$

6.10 Solving LCLs: DecompressSubtrees The assumption for this algorithm, similarly to **DecompressPaths**, is that the LCL problem on the graph is solved correctly according to Definition 6.8. We also assume, and keep the invariant, that we do not only know the partial output assignment given to $\dot{G}$, but we also know, for each node v of G , the tuple $c(v) \in \phi(v)$ assigned to it. This is especially useful at the beginning, when we have the root that is a singleton, and hence has no incident edges in the current graph, but we still want to know how to label its incident half-edges after decompressing the subtrees rooted at its children.

DecompressSubtrees($\dot{G}$)

1. Every local root v of phase k decompresses every subtree $T(u)$ compressed into it during phase k , while simultaneously solving the LCL problem on $T(u)$.

LEMMA 6.9. (DecompressSubtrees) *The LCL problem on the graph is solved according to Definition 6.8, and it has the same node and edge sets as G in phase k before executing CompressSubtrees. The algorithm terminates in $O(1)$ low-space MPC rounds using $O(n + m)$ words of global memory.*

Proof. The first statement follows from Lemma 6.3, since the solution for $\dot{G}$ can be extended to the decompressed trees, resulting in a correct solution on the graph that has the same node and edge sets that we had before the compression. For each node u in the decompressed trees, we store in $c(u)$ the tuple used to label its incident half-edges. The runtime follows from Lemma 6.4, and the memory is respected trivially. $\square$

6.11 Proof of Theorem 1.3 By the lemmas in Sections 6.6 to 6.10, the problem is solved in the original forest, and all subroutines of LCLSolver have time complexity $O(\log D)$ in the low-space MPC model and use $O(n + m)$ words of global memory. All of the subroutines are clearly deterministic. What is left to prove is that

- (i) after a constant number of phases in Step 2, the graph is reduced to a single node;
- (ii) after a constant number of repetitions in Step 4, the graph is expanded to its original form;
- (iii) if the input graph is a forest, the algorithm is component-stable, and the runtime becomes $O(\log D_{\max})$, where $D_{\max}$ denotes the maximum diameter of any component.

Proof. [Proof of (i)] The proof is very similar to the proof of Lemma 4.11, but we restate the claims for completeness. Let us recall what effectively happens during a phase. There are only two subroutines that alter the graph: in CompressSubtrees, all subtrees of size $\leq n^{1/2}$ are compressed into the first ancestor v with a subtree of size $> n^{1/2}$; then, in AdvancedCompressPaths, all paths are compressed into single edges, leaving no degree-2 nodes in the graph. Let G_j and $n_j = |G_j|$ denote the graph and the size of the graph at the beginning of phase j , respectively. We claim that after one phase, the number of nodes in the graph drops by a factor of $\Theta(n^{1/2})$. Observe, that after CompressSubtrees every leaf w in the graph corresponds to a subtree of size $\geq n^{1/2}$ that was removed. Moreover, the same holds also after AdvancedCompressPaths. Hence,

$$\begin{aligned} n_j &\geq n_{j+1} + |\{w \in G_{j+1} \mid \deg_{G_{j+1}}(w) = 1\}| \cdot n^{1/2} \\ &> n_{j+1} + n^{1/2} \cdot n_{j+1}/2. \\ &= n_{j+1}(1 + n^{1/2}/2), \end{aligned}$$

implying that

$$n_{j+1} \leq \frac{n_j}{1 + n^{1/2}/2}.$$

The first strict inequality stems from the fact that there are no degree-2 nodes left after phase j , and hence the number of leaf nodes in G_{j+1} is strictly larger than $n_{j+1}/2$. It is clear that after $O(1/\delta)$ phases, the graph is reduced to one node. $\square$

Proof. [Proof of (ii)] Let us recall what effectively happens during a repetition. Both subroutines DecompressPaths and DecompressSubtrees alter the graph by decompressing the paths and subtrees that were compressed previously in some phase. Hence, the number of repetitions is equal to number of phases, which is constant. $\square$

Proof. [Proof of (iii)] During the algorithm, the only communication between the components happens in order to start the subroutines in synchrony, which does not affect the LCL solution. It does however affect the runtime, since smaller components may be stalled behind larger components. Hence, in all runtime arguments, D can be substituted with $D_{\max}$. $\square$

Extension to unsolvable LCL problems. If the LCL problem is unsolvable, we can detect it in the following way. If, during any phase of Step 2, a local root v ends up with an empty set of tuples in $\phi(v)$, the original LCL problem must be unsolvable. Node v can then broadcast to all nodes in the graph to output label $\perp$ on their incident half-edges, indicating that there is no solution to the LCL problem.

7 Conditional Hardness Results

In this section, we show that our algorithm for solving all LCLs is optimal, assuming a widely believed conjecture about MPC. By earlier work, we consider the following more convenient problem that is also hard under the conjecture. We note that, due to technical reasons, our problem definition is slightly different to the one in [26]. Following in the footsteps of previous work, we will show that our version of the problem is also hard under the 1 vs. 2 cycles conjecture.

DEFINITION 7.1. (THE D -DIAMETER s - t PATH-CONNECTIVITY PROBLEM) *Consider a graph that consists of a collection of paths of diameter $O(D)$, for some parameter D satisfying $D \in \Omega(\log n)$ and $D = n^{o(1)}$. Given two special nodes s and t of degree 1 in the graph, the algorithm should provide the following guarantee: If s and t are in the same connected component, then the algorithm should output YES. If s and t are in different connected components, the algorithm should output NO.*

LEMMA 7.1. *Assuming that the 1 vs. 2 cycles conjecture holds, there is no deterministic low-space MPC algorithm with $\text{poly}(n)$ global memory to solve the D -diameter s - t path-connectivity problem in $o(\log D)$ rounds.*

Proof. On a high level, we show that we can use an algorithm for the D -diameter s - t path-connectivity problem to reduce the size of the given cycles by a multiplicative factor D , unless the given cycles are already too small. We then show that, by recursively applying this algorithm, we obtain a solution for the 1 vs. 2 cycles conjecture.

In more detail, we are given a graph G that is either one or two cycles, where each cycle is of length at least $n/2$. Let D be in $\Omega(\log n)$ and in $n^{o(1)}$. We proceed in phases, starting in phase $i = 0$. We assume that at the beginning of phase i the graph G contains at least $(n/D^i)/2^i$ nodes and at most $(n/D^i) \cdot 2^i$ nodes, and we guarantee that at the end of phase i the graph contains at least $(n/D^{i+1})/2^{i+1}$ nodes and at most $(n/D^{i+1}) \cdot 2^{i+1}$ nodes. If the graph contains two cycles, this factor- D reduction will actually independently hold for the size of each cycle. This is performed by running the algorithm A that solves the D -diameter s - t path-connectivity problem. We stop when the number of nodes is $n^{o(1)}$, which requires $O(\log_D n)$ phases. Then, we can spend $o(\log n)$ rounds to solve the problem with known techniques (e.g., [14]). If the D -diameter s - t path-connectivity problem could be solved in $o(\log D)$ rounds, we would obtain a total running time of $O(\log_D n) \cdot o(\log D) + o(\log n) = o(\log n)$, which violates the 1 vs. 2 cycles conjecture. We now explain a single phase of the algorithm.

In each phase i , we maintain the invariant that, if there are two cycles, the larger one contains at most 4^i times the nodes of the smaller one. Assume that at the beginning of phase i there are at least $(4^i + 1)cD \log n$ nodes, for a sufficiently large constant c . If it is not the case, then we are done, because the number of nodes is in $n^{o(1)}$.

Sample the nodes in G with probability $1/D$ and turn each sampled node *inactive*. At the end of the phase, only inactive nodes will remain, and by a standard Chernoff bound, with high probability, the number of inactive nodes is at least a factor $D/2$ and at most a factor $2D$ smaller than the original amount of nodes. Moreover, this holds independently on each cycle, and hence the ratio of the sizes of the obtained cycles can increase by at most a factor 4, hence maintaining the invariant.

We now show an upper bound on the length of the obtained paths, induced by active nodes. Consider a sequence of $c'D \log n$ nodes, for some sufficiently large constant c' . The probability that none of them is sampled is $(1 - 1/D)^{c'D \log n}$, and hence, with high probability, each path has length $O(D \log n)$. Moreover, since the shortest cycle has at least $cD \log n$ nodes, then, by fixing c sufficiently larger than c' , we obtain that each cycle contains at least one sampled node with high probability, and hence G' is a collection of paths, as required.

We create many instances of the D -diameter s - t path-connectivity problem from these paths as follows. Fix a node u with degree 1 in G' . We set $u := s$ and create an instance of D -diameter s - t path-connectivity for each possible choice of $t \neq s$, where t is also a degree 1 node. Notice that there can be at most n of such choices. Furthermore, we do the same construction for all possible choices of s , which results in $O(n^2)$ instances of the D -diameter s - t path-connectivity problem.

Suppose now that we have a deterministic $o(\log D)$ time algorithm A to solve the D -diameter s - t path-connectivity problem. The paths have length $O(D \log n)$, and hence running this algorithm requires $o(\log(D \log n))$

rounds, which, by the assumption on D , is still in $o(\log D)$. Run A independently on each of the $O(n^2)$ instances of the D -diameter s - t path-connectivity problem. On an instance where s and t are on the same path, the algorithm returns YES and otherwise NO. Hence, we can derive which endpoints in G' are on the same path in $o(\log D)$ time.

Then, we create a new instance of the 1 vs. 2 cycles cycle problem as follows. For each pair s and t on the same path, we create a virtual edge between the inactive neighbors of s and t and remove the active nodes. The number of nodes decreases at least by a factor $D/2$ and at most by a factor $2D$, as required. $\square$

Since a connected component algorithm clearly solves the D -diameter s - t path-connectivity problem, we obtain the following corollary.

COROLLARY 7.1. *Assuming the 1 vs. 2 cycles conjecture, there is no low-space memory MPC algorithm to solve connected components in $o(\log D)$ rounds on forests.*

We now show that we can define an LCL problem Π for which we can convert any solution into a solution for the problem of Definition 7.1 in constant time. This implies a conditional lower bound of $\Omega(\log D)$ for Π , implying also that our generic solver, that runs in $O(\log D)$ rounds, is optimal. Instead of defining Π by defining C_V and C_E formally, which makes it difficult to parse the definition, we provide a human understandable description of the constraints.

- The possible inputs of the nodes are 0 or 1. In the instances that we create, all nodes will be labeled 0, except for s , which will be labeled 1.
- The possible outputs are on edges, and every edge needs to be either oriented or unoriented.
- All nodes of degree 2 must have either both incident edges unoriented, or both incident edges oriented. If they are oriented, one must be incoming and the other outgoing.
- Any node of degree 1 with input 1 must have its incident edge oriented.
- Any node of degree 1 with input 0 must have its incident edge either unoriented, or oriented incoming.

We can observe some properties on the possible solutions for this problem:

- The edges of a path are either all oriented or all unoriented.
- The edges of a path containing only nodes with input 0 must all be unoriented, because a path needs to be oriented consistently, and endpoints with input 0 must have their edge oriented incoming.
- All the edges of a path containing an endpoint with input 1 must be oriented.

We can use an algorithm for Π to solve the problem of Definition 7.1 as follows. By giving 0 as input to all nodes except s , and 1 to s , and solving Π , we obtain a solution in which only the other endpoint of the path containing s has an oriented incident edge, and we can hence check if this node is t . Since Π is an LCL, we obtain the following.

THEOREM 7.1. *Assuming the 1 vs. 2 cycles conjecture, there is no low-space memory MPC algorithm to solve any solvable LCL in $o(\log D)$ rounds on forests.*

A MPC Implementation Details

Initially, before executing any algorithm, the input graph of n nodes and m edges is distributed among the machines arbitrarily. By applying Definition A.1, we can organize the input such that every node and its edges are hosted on a single machine, or, in the case of high degree, on multiple consecutive machines.

DEFINITION A.1. (AGGREGATION TREE STRUCTURE, [13]) *Assume that an MPC algorithm receives a collection of sets $A_1, \dots, A_k$ with elements from a totally ordered domain as input. In an aggregation tree structure for $A_1, \dots, A_k$, the elements of $A_1, \dots, A_k$ are stored in lexicographically sorted order (they are primarily sorted by the number $i \in \{1, \dots, k\}$ and within each set A_i they are sorted increasingly). For each $i \in \{1, \dots, k\}$ such that the elements of A_i appear on at least 2 different machines, there is a tree of constant depth containing the*

machines that store elements of A_i as leafs and where each inner node of the tree has at most $n^{1/2}$ children. The tree is structured such that it can be used as a search tree for the elements in A_i (i.e., such that an in-order traversal of the tree visits the leaves in sorted order). Each inner node of these trees is handled by a separate additional machine. In addition, there is a constant-depth aggregation tree of degree at most $n^{1/2}$ connecting all the machines that store elements of $A_1, \dots, A_k$.

This section is dedicated to showing how MAX-ID-Solver can be implemented in the low-space MPC model. We only cover routine `CompressLightSubTrees`, since the implementation details for `CompressPaths`, `DecompressLightSubTrees`, and `DecompressPaths` are simple, and included in the corresponding proofs.

In the proof of `CompressLightSubTrees`, we have reasoned that the local memory of a node never exceeds $O(n)$, and that the total memory never exceeds $O(n \cdot \tilde{D}^3)$. However, we have to also ensure that the low-space MPC's communication bandwidth of $O(n)$ is respected throughout the routines (Lemma A.1). Also, we have to address the possibility of a node having degree $> n$, since we work with arbitrary degree trees.

If, during some iteration of `CompressLightSubTrees`, the degree of a node is $> n$, it is clearly heavy, and does not partake in the ongoing iteration. In fact, if the degree is $> n^{1/8} + 1$, it is also heavy and does not partake. Hence, in the following lemma, we can assume that every node v and its edges are hosted on a single machine, and that $\deg(v) \leq n^{1/8} + 1$.

LEMMA A.1. *The following routines can be performed in $O(1)$ low-space MPC rounds:*

1. *A node can detect whether it is happy or full,*
2. $\text{Exp}(X), X \subseteq N(v)$,
3. *If node v is added in S_w for some w , v is able to add w to S_v .*

Proof. We prove the three statements separately. All three statements have the a common technical difficulty: it is possible for node v to be included in S_w for some w , such that $w \notin S_v$, which causes communication bandwidth congestion. We address this common issue shortly, after reasoning about the separate challenges of each routine.

1. Since the property of being full depends solely on the size of S_v , it can be computed locally. In order for a node v to detect if it is happy, v only has to ask all nodes $w \in S_v$ for their degrees.
2. In order for a node v to perform $\text{Exp}(X), X \subseteq N(v)$, v must ask a subset of nodes $w \in S_v$ for their $S_{w \rightarrow r_w(v)}$, which is straightforward to implement.
3. When needed, a node w can inform nodes $v \in S_v$ that they have been added to S_w . After which it is straightforward for v to add w to S_v .

In all of the routines above, it is possible for $v \in S_w$ for some w , such that $w \notin S_v$. This can happen when v does not maintain a symmetric view towards a direction in `fullDirs` in Step 1(c). This can cause $> n$ nodes querying node v , breaking the communication constraint of the low-space MPC model. The following scheme resolves the issue. Let us first restate a tree structure that is useful to carry out computations on a set or on a collection of sets, in $O(1)$ low-space MPC rounds and with $O(n + m)$ global memory.

Denote the collection of machines we are using for the algorithm as $M = \{M_1, M_2, \dots, M_l\}$. Let us allocate a new collection of empty machines $M' = \{M'_1, M'_2, \dots, M'_l\}$. For every node $w \in S_v$ of a node v hosted by M_j , send a directed edge (v, w) to M'_j . Let us call all edges of form (x, y) as the *outgoing* edges of x and *incoming* edges of y . Along with the edge, send the address of machine M_j .

Define sets $A_1, \dots, A_k$ such that set A_i contains all incoming edges of node i . Apply Definition A.1 such that sets $A_1, \dots, A_k$ are stored in M' in lexicographically sorted order (by the ID of $i \in \{1, \dots, k\}$ and within each set A_i , the elements are sorted increasingly). By Definition A.1, for each $i \in \{1, \dots, k\}$ such that the elements of A_i appear on at least 2 different machines, there is a tree of constant depth containing the machines that store elements of A_i as leafs and where each inner node of the tree has at most $n^{1/2}$ children. Let us denote this kind of tree as $\mathcal{A}_i$.

Observe that every $\mathcal{A}_i$ corresponds to a node i that has $> n$ incoming edges, which is exactly the problematic case we have set out to deal with. The root of $\mathcal{A}_i$ can ask for S_i from the machine in M hosting node i , and

distributes S_i to all leaf nodes hosting the incoming edges (this requires a communication bandwidth of $O(n^{3/2})$). We also establish a mapping from the machines in M to machines in M' such that the machine in M hosting u and S_u knows the machines in M' hosting edges (u, v) for every $v \in S_u$. This is straightforward to implement, since when we distributed the edges to M' , we also distributed the corresponding addresses of machines in M .

Let us describe what effectively happens when a node u asks for $S_{v \rightarrow r_v(u)}$ of node $v \in S_u$ if v has $> n$ incoming edges. The machine $M_u \in M$ hosting node u queries the machine $M'_{(u,v)} \in M'$ for $S_{v \rightarrow r_v(u)}$, where $M'_{(u,v)}$ is the machine hosting edge (u, v) . Due to the design of the aggregation tree, machine $M'_{(u,v)}$ is a leaf of tree $\mathcal{A}_v$ and has at most n elements (edges) stored on it. The queries to machine $M'_{(u,v)}$ comprise an incoming message size of $O(n)$. Answering the queries would require a communication bandwidth of $O(n^2)$.

We can reduce the communication bandwidth of $O(n^{3/2})$ (the distribution of S_i) and $O(n^2)$ (the leaves of $\mathcal{A}_v$ answering queries) to $O(n)$ by using $\delta/2$ instead of δ for the whole algorithm. $\square$

References

- [1] Noga Alon, László Babai, and Alon Itai. A Fast and Simple Randomized Parallel Algorithm for the Maximal Independent Set Problem. *Journal of Algorithms*, 7(4):567–583, 1986.
- [2] Alexandr Andoni, Zhao Song, Clifford Stein, Zhengyu Wang, and Peilin Zhong. Parallel Graph Connectivity in Log Diameter Rounds. In *FOCS*, pages 674–685, 2018.
- [3] Alkida Balliu, Sebastian Brandt, Yuval Efron, Juho Hirvonen, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Classification of Distributed Binary Labeling Problems. In *DISC*, 2020.
- [4] Alkida Balliu, Sebastian Brandt, Manuela Fischer, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Exponential Speedup over Locality in MPC with Optimal Memory. In Christian Scheideler, editor, *36th International Symposium on Distributed Computing (DISC 2022)*, volume 246 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 9:1–9:21, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [5] Alkida Balliu, Sebastian Brandt, Juho Hirvonen, Dennis Olivetti, Mikael Rabie, and Jukka Suomela. Lower Bounds for Maximal Matchings and Maximal Independent Sets. *Journal of the ACM*, 68(5):39:1–39:30, 2021.
- [6] Alkida Balliu, Sebastian Brandt, Fabian Kuhn, and Dennis Olivetti. Improved Distributed Lower Bounds for MIS and Bounded (Out-)Degree Dominating Sets in Trees. In *Proc. 40th ACM Symposium on Principles of Distributed Computing (PODC)*, 2021.
- [7] Alkida Balliu, Sebastian Brandt, Fabian Kuhn, and Dennis Olivetti. Distributed Δ -coloring plays hide-and-seek. In *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing*, pages 464–477. ACM, 2022.
- [8] Alkida Balliu, Sebastian Brandt, and Dennis Olivetti. Distributed Lower Bounds for Ruling Sets. *SIAM Journal on Computing*, 51(1):70–115, 2022.
- [9] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. Almost Global Problems in the LOCAL Model. In *DISC*, pages 9:1–9:16, 2018.
- [10] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally Checkable Labelings with Small Messages. In *the Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 8:1–8:18, 2021.
- [11] Alkida Balliu, Mohsen Ghaffari, Fabian Kuhn, and Dennis Olivetti. Node and Edge Averaged Complexities of Local Graph Problems. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC)*, 2022.
- [12] Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. New Classes of Distributed Time Complexity. In *STOC*, pages 1307–1318, 2018.
- [13] Philipp Bamberger, Fabian Kuhn, and Yannic Maus. Efficient Deterministic Distributed Coloring with Small Bandwidth. *CoRR*, abs/1912.02814, 2019.
- [14] Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Lacki, and Vahab Mirrokni. Near-Optimal Massively Parallel Graph Connectivity. In *FOCS*, 2019.
- [15] Sebastian Brandt, Manuela Fischer, and Jara Uitto. Breaking the Linear-Memory Barrier in MPC: Fast MIS on Trees with Strongly Sublinear Memory. In *SIROCCO*, 2019.
- [16] Sebastian Brandt, Christoph Grunau, and Vaclav Rozhon. The landscape of distributed complexities on trees and beyond. In *the Proceedings of the 41st Symposium on Principles of Distributed Computing (PODC)*, 2022.
- [17] Sebastian Brandt, Rustam Latypov, and Jara Uitto. Brief Announcement: Memory Efficient Massively Parallel Algorithms for LCL Problems on Trees. In *35th International Symposium on Distributed Computing (DISC 2021)*, pages 50:1–50:4, 2021.

- [18] Yi-Jun Chang. The Complexity Landscape of Distributed Locally Checkable Problems on Trees. In *DISC*, pages 18:1–18:17, 2020.
- [19] Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. The Complexity of $(\Delta + 1)$ -Coloring in Congested Clique, Massively Parallel Computation, and Centralized Local Computation. In *PODC*, 2019.
- [20] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. An Exponential Separation between Randomized and Deterministic Complexity in the LOCAL Model. *SIAM J. Comput.*, 48(1):122–143, 2019.
- [21] Yi-Jun Chang and Seth Pettie. A Time Hierarchy Theorem for the LOCAL Model. *SIAM J. Comput.*, 48(1):33–69, 2019.
- [22] Sam Coy and Artur Czumaj. Deterministic Massively Parallel Connectivity. In *Proceedings of the ACM Symposium on Theory of Computing (STOC)*, 2022.
- [23] Artur Czumaj, Peter Davies, and Merav Parter. Graph Sparsification for Derandomizing Massively Parallel Computation with Low Space. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 175–185, 2020.
- [24] Artur Czumaj, Peter Davies, and Merav Parter. Component Stability in Low-Space Massively Parallel Computation. In *PODC*, 2021.
- [25] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, pages 107–113, 2008.
- [26] Mohsen Ghaffari, Fabian Kuhn, and Jara Uitto. Conditional Hardness Results for Massively Parallel Computation from Distributed Lower Bounds. In *FOCS*, pages 1650–1663, 2019.
- [27] Mohsen Ghaffari and Jara Uitto. Sparsifying Distributed Algorithms with Ramifications in Massively Parallel Computation and Centralized Local Computation. In *SODA*, 2019.
- [28] Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. *ACM SIGOPS Operating Systems Review*, pages 59–72, 2007.
- [29] Howard J. Karloff, Siddharth Suri, and Sergei Vassilvitskii. A Model of Computation for MapReduce. In *SODA*, 2010.
- [30] Fabian Kuhn, Thomas Moscibroda, and Roger Wattenhofer. Local Computation: Lower and Upper Bounds. *Journal of ACM*, 63:17:1–17:44, 2016.
- [31] Christoph Lenzen and Roger Wattenhofer. Brief Announcement: Exponential Speed-Up of Local Algorithms Using Non-Local Communication. In *PODC*, 2010.
- [32] Nathan Linial. Distributive Graph Algorithms – Global Solutions from Local Data. In *FOCS*, 1987.
- [33] Nathan Linial. Locality in Distributed Graph Algorithms. *SIAM J. Comput.*, 21(1):193–201, 1992.
- [34] Michael Luby. A Simple Parallel Algorithm for the Maximal Independent Set Problem. *SIAM Journal on Computing*, 15:1036–1053, 1986.
- [35] Moni Naor and Larry Stockmeyer. What Can Be Computed Locally? *SIAM Journal on Computing*, 24(6):1259–1277, 1995.
- [36] Tim Roughgarden, Sergei Vassilvitskii, and Joshua R. Wang. Shuffles and Circuits (On Lower Bounds for Modern Parallel Computation). *Journal of the ACM*, 2018.
- [37] Salil P. Vadhan. Pseudorandomness. *Foundations and Trends in Theoretical Computer Science*, 7(1-3):1–336, 2012.
- [38] Tom White. *Hadoop: The Definitive Guide*. O’Reilly Media, Inc., 2009.
- [39] Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *the Proceedings of the SENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2010.

Publication III

Christoph Grunau, Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. Conditionally Optimal Parallel Coloring of Forests. In *International Symposium on Distributed Computing (DISC) 2023*, Volume 281, 23:1-23:20, doi.org/10.4230/LIPIcs.DISC.2023.23, L'Aquila, Italy, October 2023.

© 2023 Schloss Dagstuhl – Leibniz-Zentrum für Informatik
Reprinted with permission.

Conditionally Optimal Parallel Coloring of Forests

Christoph Grunau ✉ 

ETH Zürich, Switzerland

Rustam Latypov ✉ 

Aalto University, Finland

Yannic Maus ✉ 

TU Graz, Austria

Shreyas Pai ✉ 

Aalto University, Finland

Jara Uitto ✉ 

Aalto University, Finland

Abstract

We show the first conditionally optimal deterministic algorithm for 3-coloring forests in the low-space massively parallel computation (MPC) model. Our algorithm runs in $O(\log \log n)$ rounds and uses optimal global space. The best previous algorithm requires 4 colors [Ghaffari, Grunau, Jin, DISC'20] and is randomized, while our algorithm are inherently deterministic.

Our main technical contribution is an $O(\log \log n)$ -round algorithm to compute a partition of the forest into $O(\log n)$ ordered layers such that every node has at most two neighbors in the same or higher layers. Similar decompositions are often used in the area and we believe that this result is of independent interest. Our results also immediately yield conditionally optimal deterministic algorithms for maximal independent set and maximal matching for forests, matching the state of the art [Giliberti, Fischer, Grunau, SPAA'23]. In contrast to their solution, our algorithms are not based on derandomization, and are arguably simpler.

2012 ACM Subject Classification Theory of computation → Massively parallel algorithms

Keywords and phrases massively parallel computation, coloring, forests, optimal

Digital Object Identifier 10.4230/LIPIcs.DISC.2023.23

Related Version *Full Version:* <https://arxiv.org/abs/2308.00355>

Funding *Rustam Latypov:* Academy of Finland, Grant 334238.

Yannic Maus: Austrian Science Fund (FWF), Grant P36280-N.

Shreyas Pai: Academy of Finland, Grant 334238.

1 Introduction

A recent sequence of papers investigates fundamental symmetry-breaking problems such as coloring, maximal independent set and maximal matching on trees [9, 7, 32, 22, 26]. We conclude, simplify and unify this line of work by giving a conceptually simple algorithm for 3-coloring, maximal independent set and maximal matching. We solve the three problems in a unified way by computing a so-called H -decomposition (we discuss these in more detail in Section 1.2). Even though such decompositions are the natural tool for solving the aforementioned problems on trees, computing them efficiently in the MPC model remained outside the reach of previous techniques.

► **Theorem 1.** *There are deterministic $O(\log \log n)$ -round low-space MPC algorithms for 3-coloring, maximal matching and maximal independent set (MIS) on forests. These algorithms use $O(n)$ global space.*



© Christoph Grunau, Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto;
licensed under Creative Commons License CC-BY 4.0

37th International Symposium on Distributed Computing (DISC 2023).

Editor: Rotem Oshman; Article No. 23; pp. 23:1–23:20



Leibniz International Proceedings in Informatics

LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

The runtimes of our algorithms are conditionally optimal, conditioned on the 1 vs 2 cycle conjecture, at least if one restricts to so-called component stable algorithms [23, 15, 35, 41] (see Section 1.4 for a brief discussion about component-stability).

We note that algorithms for maximal matching and maximal independent set matching our guarantees are known from a very recent work [26]. However, their algorithms are quite complicated and technical, and use sophisticated derandomization techniques. Moreover, their techniques inherently cannot be used to color a tree with a small number of colors. Indeed, the 3-coloring problem is considered to be the hardest of the three problems, e.g., once such a coloring is known one can compute an MIS in $O(1)$ rounds. Additionally, a crucial property used in previous MPC algorithms for MIS and maximal matching is that any partial solution can be extended to a solution of the whole graph; a property that does not hold for 3-coloring. The best previous algorithm for coloring trees uses 4 colors and is randomized [22]. If one allows for randomization the single additional color makes the problem significantly easier by the following divide and conquer approach: if one partitions the tree into two parts by letting each node join one of the parts uniformly at random, the connected components induced by each part have logarithmic diameter. Once the diameter is small, one can use $O(\log \log n)$ MPC rounds to color each component independently with two colors in a brute force manner. In the next section we zoom out and present the bigger picture of our work.

1.1 MPC Model and Exponential Speed-Up Over LOCAL Algorithms

The Massively Parallel Computation (MPC) model [30] is a mathematical abstraction of modern frameworks of parallel computing such as Hadoop [43], Spark [44], MapReduce [18], and Dryad [29]. In the MPC model, we have M machines that communicate in all-to-all fashion, in synchronous rounds. In each round, every machine receives the messages sent in the previous round, performs (arbitrary) local computations, and is allowed to send messages to any other machine. Initially, an input graph of n nodes and m edges is arbitrarily distributed among the machines. At the end of the computation, each machine needs to know the output of each node it holds, e.g., their color in the vertex-coloring problem.

The MPC model is typically divided into 3 regimes according to the local space S . The *superlinear* and the *linear* regimes allow for $S = n^{1+\Omega(1)}$ and $S = \tilde{O}(n)$ words¹ of space (memory) per machine. A word is $O(\log n)$ bits and is enough to store a node or a machine identifier from a polynomial (in n) domain. The local space restricts the amount of data a machine initially holds and is allowed to send and receive per round. Both linear and superlinear regimes allow for very efficient algorithms because machines can get a “global view” of the graph in the sense that it can store information for each node of the graph [31, 20, 24]. However, the growing size of most real-world graphs makes it impossible to get such a global view on a single machine and hence research in recent years has focused on the most challenging *low-space* (or *sublinear*) regime with $S = n^\delta$, for some constant $\delta < 1$, where we cannot even store the whole neighborhood of a single node in a single machine. As each machine can only get a local view there are close connections to the LOCAL model of distributed computing that we further elaborate on below.

Furthermore, we focus on the most restricted case of *linear global space*, i.e., $S \cdot M = \Theta(n + m)$. Notice that $\Omega(n + m)$ words are *required* to store the input graph.

¹ The $\tilde{O}$ notation hides polylogarithmic factors.

The LOCAL Model and Graph Exponentiation. The LOCAL model is a classic model of distributed message passing. Each node of an input graph hosts a processor and the nodes communicate along the edges of the graph in synchronous rounds. The local computation, local space, and message sizes are unbounded in this model. Most research in the LOCAL model has focused on symmetry breaking problems like graph colorings, MIS, and maximal matchings. For most of these classic problems $O(\log n)$ -round randomized algorithms are known [36, 1, 38] which can directly be translated to the MPC model. A major focus on recent and current research is to develop *sublogarithmic* MPC algorithms that beat the logarithmic baseline.

In fact, the strong connection between the models also shows up in faster algorithms, as almost all recent MPC algorithms for such problems are MPC-optimized implementations of algorithms that were originally developed for the LOCAL model. The main technique to obtain this speedup is the graph *exponentiation technique* [33]. It allows to gather the T -hop radius neighborhood of a node in $O(\log T)$ MPC rounds. So, as long as these neighborhoods fit the space constraints, after gathering them one can simulate T -round LOCAL algorithms locally to compute the output for each node. Furthermore, for component-stable algorithms, the connection also goes the other way around, that is, an $\Omega(T)$ lower bound on the round complexity in the LOCAL model implies an exponentially lower $\Omega(\log T)$ conditional lower bound in the MPC model. Thus, the holy grail is to obtain this exponential speedup over the LOCAL model. A central open problem in the area is to find an $O(\log \log n)$ round MPC algorithm for the classic MIS problem on general graphs, which enjoys a matching conditional $\Omega(\log \log n)$ -round lower bound.

Unfortunately, we are very far from answering this question. The current state of the art is an $\tilde{O}(\sqrt{\log \Delta} + \log \log \log n)$ -round randomized MPC algorithm [25]. We note that we take into account the new results on network decomposition, which reduce the dependency on n [42, 21]. This result is obtained by combining the graph exponentiation technique with *sparsification* methods [33, 25]. The exponentiation technique is used to simulate Ghaffari's $O(\log \Delta + \text{poly} \log \log n)$ -round MIS algorithm for the LOCAL model [19].

From a high-level perspective they break the LOCAL algorithm into $O(\sqrt{\log \Delta})$ phases each of length $T = O(\sqrt{\log \Delta})$. In the beginning of each phase, the graph is subsampled so that the maximum degree of any node is at most $2^{\sqrt{\log \Delta}}$. Then, we can gather the T -hop neighborhood of each node in $O(\log \log \Delta)$ MPC rounds and simulate T rounds of the LOCAL algorithm in a single MPC round. The main benefit of simulating (shorter) phases and subsampling to smaller degree graphs is to reduce the memory resources needed during the exponentiation technique. Unfortunately, this phase-based approach seems to hit a fundamental barrier at $\sqrt{\log \Delta}$ rounds, and it is unclear how to reduce memory usage without it. Due to little progress in improving on this result, recent research has focused on special graph classes such as trees and bounded arboricity graphs.

Symmetry Breaking on Trees and Bounded Arboricity Graphs. Studying low-space MPC algorithms for MIS on trees and forests has been fruitful. This line of work started with a randomized $O(\log^3 \log n)$ round low-space MPC algorithm for MIS and maximal matching on trees [9]. Later, the round complexity was first improved to $O(\log^2 \log n)$ [7] and finally to $O(\log \log n)$ [22], where both algorithms extend to low-arboricity graphs. The $O(\log \log n)$ algorithm is conditionally optimal, at least if one restricts oneself to component-stable algorithms. Finally, a recent work derandomized the $O(\log \log n)$ round algorithm using MPC specific derandomization techniques and thus obtained a deterministic $O(\log \log n)$ round MIS and Maximal Matching algorithm for trees and more generally low-arboricity graphs [26].

While especially the $O(\log \log n)$ round algorithms are quite technical and involved, all of the aforementioned previous algorithms rely on the same fundamental idea. Namely, to interleave graph exponentiation with the computation of partial solutions to rapidly decrease the maximum degree of the remaining graph. Unfortunately, it seems unlikely that such a rapid degree reduction is possible in general graphs; thus it seems that new approaches are necessary in order to get an $O(\log \log n)$ round algorithm for general graphs.

Also, their approach does not work for coloring a forest with a constant number of colors. The main reason is that they critically rely on the fact that any partial solution can be extended to a full solution, which is not the case for coloring a forest with a fixed number of colors.

1.2 Our Technical Contribution

We present a unified solution for 3-coloring, MIS, maximal matching that takes $O(\log \log n)$ rounds. The core technical contribution that unifies these is an efficient algorithm to compute H -decompositions.

► **Theorem 2.** *There is a deterministic $O(\log \log n)$ -rounds low-space MPC algorithm that computes a strict H -decomposition with $O(\log n)$ layers on forests in $O(n)$ global space.*

H -decompositions were introduced to the area of distributed computing by Barenboim and Elkin [5]. An H -decomposition (of a forest) partitions the vertices of the graph into layers such that every node has at most two neighbors² in higher or equal layers. For forests an H -partition with $O(\log n)$ layers always exist. In the LOCAL model, such an H -decomposition immediately implies an algorithm for 3-coloring in $O(\log n)$ rounds. Essentially, one can iterate through the layers in a reverse order and color all nodes in a layer while avoiding conflicts with the already colored neighbors in higher layers.

The novelty of our approach is not that we use such decompositions to compute a 3-coloring of a forest, in fact, this straightforward approach has made it into the classrooms of many graduate programs of universities, but in the way how we compute it. We detail on our solution in more detail in the nutshell, but the main take-away is as follows. We steer every machine to learn some parts of the graph (to large extent in an uncoordinated fashion) such that every machine can compute a partial H -decomposition locally, which we can later unify to a global decomposition. We are not aware of any other MPC algorithm for H -decompositions with a similar approach.

Balanced Exponentiation. In order to achieve exponential speedup, our algorithms rely on graph exponentiation. However, there are no known sparsification techniques that can cope with the memory resources that are needed for the classic graph exponentiation technique. Instead, we provide a self-contained exponentiation procedure whose memory overhead is very mild on forests. To explain our procedure, we first need to define a *subtree*. A subtree is a subgraph of a tree, such that if it is removed, the rest of the tree stays connected. A node is *important* if it is contained in a subtree of size $n^{\delta/8}$. We present the following result (the formal statement appears in the full version).

² There are generalizations to higher number of neighbors that are important when dealing with bounded arboricity graphs [37, 6].

Let $0 < k \leq n^{\delta/8}$ be a parameter. There is a deterministic low-space MPC algorithm that, given an n -node forest F , uses $O(\log k)$ rounds in which every important node $v \in F$ discovers its k -hop neighborhood in every direction of the graph, except for at most one.

Given a node $v \in F$, we refer to each of its neighbors $x \in N(v)$ as a *direction* with regard to v . Informally, what node v can discover in direction x is simply the subgraph of F that is connected to v via x , which is uniquely defined, since F is a forest.

This result above may be of independent interest and may be useful to design algorithms for other graph problems. We obtain it by extending the exponentiation technique of a recent work by [3]. Their work designs an exponentiation technique which (almost) equals ours in the special case when k equals the maximum diameter of a component of the forest. In their work it is used in an $O(\log \text{diam})$ -round algorithm to compute the connected components of a forest. Later it has also been used to solve certain dynamic programming tasks on tree-structured data [28], also in time that is logarithmic in the diameter. In the full version we present a more detailed discussion on the similarities and the difference between the exponentiation result in this work and the one in their work, and why our result requires a different analysis. The main benefit of our result is that a flexible choice of k allows the runtime and space to be small, if the required “view” for the nodes is small, which we heavily utilize in our algorithm to compute H -decompositions.

1.3 Our Method in a Nutshell

As mentioned in the previous section, our key technical contribution is to compute a so-called H -decomposition of the input forest F . In particular, the goal is to compute a partition $V(F) = V_1 \sqcup V_2 \sqcup \dots \sqcup V_L$ of the vertices into $L = O(\log n)$ layers such that each node in V_i has at most two neighbors in $\bigcup_{j \geq i} V_j$. There exists a simple peeling algorithm which computes such a partition; iteratively peel off all nodes of degree at most 2 and define V_i as the set of nodes that got peeled off in the i -th iteration. A simple calculation shows that at least half of all the remaining nodes get peeled off in each iteration, and hence we get a decomposition into $O(\log n)$ layers. Moreover, one can determine the iteration in which a node gets peeled off by only looking at its $O(\log n)$ -hop neighborhood. Thus, if we could compute for a given node its entire $O(\log n)$ -neighborhood and store it in a single machine, then we could locally determine the layer of that node with no further communication.

One way to compute the $O(\log n)$ -neighborhood of each node in the MPC model is the well-known graph exponentiation technique. Generally speaking, graph exponentiation allows to learn the 2^i -hop neighborhood of each node in $O(i)$ MPC rounds. Thus, we could in principle hope to learn the $O(\log n)$ -hop neighborhood of each node in just $O(\log \log n)$ rounds. However, one obviously necessary precondition of the graph exponentiation technique is that the $O(\log n)$ -hop neighborhood of each node has size n^δ , as otherwise we cannot possibly store the neighborhood in one machine. This is quite a limiting condition. If the input is for example a star, even the two-hop neighborhood of each node contains $\Omega(n)$ vertices. Moreover, even if each local neighborhood would fit into one machine, the global space required to store all the neighborhoods might still be prohibitively large, especially if one aims for near-linear global space.

Thus, we cannot use the vanilla graph exponentiation technique. Instead, we use the balanced graph exponentiation technique for forests mentioned in the previous section. The output guarantee of the balanced exponentiation algorithm, running in $O(\log \log n)$ rounds, weakens the guarantee that each node sees its $O(\log n)$ -hop in two ways. First, it only gives a guarantee for nodes that are contained in a sufficiently small subtree, namely of size at most n^δ . Second, for each node v in a small subtree, it computes all nodes of distance $O(\log n)$, except for nodes in one direction.

We start by briefly discussing how one can deal with the first shortcoming. If one iteratively removes all nodes that are contained in a subtree of size at most x from F and all nodes of degree at most 2, then all nodes are removed within $O(\log_x(n))$ iterations. This fact was used in similar forms in previous results and for completeness we give a standalone proof (see Lemma 7). Thus, if we repeatedly assign nodes in subtrees of size at most n^δ and nodes of degree at most 2 to one of $O(\log n)$ layers, then after $O(1/\delta)$ iterations, we assigned each node to one of $O((1/\delta) \log n)$ layers. Thus, it intuitively suffices to focus on nodes in subtrees of size at most n^δ . Section 4 gives a formal treatment of this argument.

The more severe difficulty stems from the fact that there might not be a single node in the forest for which we have stored its entire $O(\log n)$ -neighborhood in one machine. This makes it impossible to locally determine the layer of each node, or even a single one, assigned by the simple peeling process described in the beginning. Instead, each node v locally simulates a conservative variant of the peeling algorithm described above; in each iteration not all the nodes of degree at most 2 are removed, but only those that v has stored in its machine. Note that if v has strictly more than 2 neighbors not stored in its machine, then the conservative peeling algorithm would never peel off v . Moreover, even if v would eventually be peeled off, then there is no guarantee that it happens within the first $O(\log n)$ iterations. However, the fact that v has stored all the nodes in its $O(\log n)$ -hop neighborhood except for nodes in one direction in its machine suffices to show that v gets peeled off within the first $O(\log n)$ iterations (see Lemma 21). Thus, each node v locally computes a layering $V^v = V_1^v \sqcup \dots \sqcup V_L^v$ for some $L = O(\log n)$ such that $v \in V^v$ and each node in V_i^v has at most two neighbors contained in $(\bigcup_{j \geq i} V_j^v) \cup (V(F) \setminus V^v)$. As some nodes might not be assigned to any layer, we refer to such a decomposition as a partial H -decomposition. Note that a node might get assigned to different layers from different nodes. Fortunately, this is not a problem because of the following nice structural property about (partial) H -decompositions: if we are given multiple (partial) H -decompositions, then we can get another (partial) H -decomposition by assigning each node to the smallest layer assigned by any of the H -decompositions. This structural observation allows us to combine the different locally computed (partial) H -decompositions into a single partial H -decomposition where each node in a small subtree is assigned to one of the $O(\log n)$ layers.

Rooted vs. Unrooted Forests. Our results are for unrooted forests, which are indeed more difficult than rooted forests. In fact, the fastest known MPC algorithm to root a forest takes $O(\log \text{diam})$ rounds (and at least on general forests this runtime is conditionally tight) [3]; so rooting the forest does not fit our time budget of $O(\log \log n)$ rounds. Many steps of our algorithm would simplify (or maybe even allow for alternative solutions) if the forest was rooted. For example, in a directed forests, we would not need our balanced exponentiation procedure. One can show that nodes can just exponentiate towards their children until the local memory is full without breaking any global memory bounds. However we would still need our combinatorial algorithm for creating a (global) H -partition. Observe that [3] also contains a $O(\log \text{diam})$ -round 2-coloring algorithm for rooted constant-degree forests, which can be generalized to rooted unbounded-degree forests.

1.4 Further Related Work

Component Stability. Roughly speaking, an MPC algorithm is component-stable, if the outputs of nodes in different components are independent of each other. Low-space component-stable MPC algorithms are closely connected to algorithms in the LOCAL model and this connection was used to lift (unconditional) lower bounds from the LOCAL model into

conditional lower bounds in the MPC model [23]. Under the 1 vs 2 cycle conjecture, this technique turns an $\Omega(T)$ -round lower bound in LOCAL into an $\Omega(\log T)$ lower bound in low-space MPC. This approach was used to establish, among others, $\Omega(\log \log n)$ randomized lower bounds for MIS and maximal matching. Later, the technique was extended to deterministic component-stable algorithms as well [15]. While the assumption of component-stability might seem very natural to MPC algorithms, it is known that component-instability can help. For example, any component-stable algorithm for finding an independent set of size $\Omega(n/\Delta)$ requires $\Omega(\log \log^* n)$ rounds, while there is an $O(1)$ -round algorithm that is not component-stable [15].

log(diam) Algorithms on Forests. There are surprisingly few works with a strict $\log(\text{diam})$ runtime for any graph families in any MPC regimes, where diam refers to the diameter. To our knowledge, the only existing ones are low-space algorithms for forests [3, 28]. The authors of [3] show that connectivity, rooting, and all LCL (locally checkable labeling) problems can be solved on forests in $O(\log \text{diam})$ using optimal global space $O(n)$. The authors of [28] build on top of the works of [3] by introducing a framework to solve dynamic programming tasks and optimization problems, all in time $\log(\text{diam})$ and global space $O(n)$. We note that given a double-logarithmic dependency on n , the connectivity problem can be solved on general graphs (even deterministically) in $O(\log \text{diam} + \log \log n)$ time using linear total space [8, 13]. Also, the 1 vs 2 cycle conjecture directly rules out an $o(\log \text{diam})$ for connectivity.

Symmetry-Breaking on General Graphs. In general graphs, $(\Delta + 1)$ -vertex coloring is an intensively studied symmetry breaking problem, where Δ is the maximum degree of the graph. A series of works [39, 4, 40, 11] for Congested Clique model which is similar to the MPC model with linear local memory has culminated in a deterministic $O(1)$ -round algorithm [17].

In the low-space MPC model, the first algorithm for the problem was randomized and used $O(\log \log \log n)$ rounds with almost linear $\tilde{O}(m)$ global space [11]³. By derandomizing the classic logarithmic-time algorithms, one can obtain an $O(\log \Delta + \log \log n)$ -round algorithm for $(\Delta + 1)$ -coloring, MIS, and maximal matching [14, 17]. For coloring, this was improved to $O(\log \log \log n)$ through derandomizing a tailor-made algorithm [16]. The deterministic algorithms require $n^{1+\Omega(1)}$ global space.

1.5 Outline

We define strict H -decompositions in Section 3, and then we show how to compute them in $O(\log \log n)$ low-space MPC rounds using $O(n \cdot \text{poly}(\log n))$ global space in Section 4. The key subroutine for the algorithm in Section 4 is discussed in Appendix A. In Section 5 we show how to use these decompositions to compute a coloring, MIS, and matching. In Appendix B we show how to reduce the global memory usage of our algorithms from $O(n \cdot \text{poly}(\log n))$ to $O(n)$. Due to space constraints, the balanced exponentiation procedure and the missing proofs of Section 4 appear in the full version. Many of the proofs are omitted due to the page limit and deferred to the full version.

³ The $O(\sqrt{\log \log n})$ runtime stated in the paper is automatically improved to $O(\log \log \log n)$ through developments in network decomposition [42].

2 Preliminaries and Notation

The input graph is an undirected, finite, simple forest $F = (V, E)$ with $n = |V|$ nodes and $m = |E|$ edges such that $E \subseteq [V]^2$ and $V \cap E = \emptyset$. For a subset $S \subseteq V$, we use $G[S]$ to denote the subgraph of G induced by nodes in S .

Let $\deg_F(v)$ denote the degree of a node v in F and let Δ denote the maximum degree of F . For node set $S \subseteq V(F)$ and a node $v \in S$ we write $\deg_S(v)$ for the degree of v in $F[S]$. The distance $d_F(v, u)$ between two vertices v, u in F is the number of edges in the shortest $v - u$ path in F ; if no such path exists, we set $d_F(v, u) := \infty$. Sometimes we simply write $\deg(v)$ and $d(v, u)$ if it is clear from context that we refer to the degree and distance in graph F . The greatest distance between any two vertices in F is the diameter of F , denoted by $\text{diam}(F)$.

For each node v and for every $k \in \mathbb{N}$, we denote the k -hop (or k -radius) neighborhood of v as $N^k(v) = \{u \in V : d(v, u) \leq k\}$. Set $N^1(v)$ is simply the set of neighbors of v , to which often refer to as $N(v)$. We often consider sets of nodes S from which we need to remove a single node u . Hence, we use the notation $S \setminus u$ as a shorthand for $S \setminus \{u\}$.

3 Strict H -decompositions

We begin with the formal definition of an H -decomposition, that is, a partition of the graph into layers such that every node has at most two neighbors in higher or equal layers. We also extend the definition to the setting where some nodes remain without a layer.

► **Definition 3** ((Partial) H -Decomposition). *Let F be a forest and $\text{layer} : V(F) \mapsto \mathbb{N} \cup \{\infty\}$. For $i \in \mathbb{N} \cup \{\infty\}$ define $V_i = \{v \in V(F) \mid \text{layer}(v) = i\}$, $V_{\geq i} = \bigcup_{j \geq i} V_j$.*

We say that layer is a partial H -decomposition if $\deg_{V_{\geq i}}(v) \leq 2$ holds for every v with $\text{layer}(v) = i$. We speak of an H -decomposition if $V_\infty = \emptyset$, and $L = \max\{\text{layer}(v) \mid v \in V(F), \text{layer}(v) \neq \infty\}$ is the length of the decomposition.

We also refer to the V_i 's as the *layers* of the (partial) H -decomposition.

Why 3-coloring and strict H -decompositions? H -decompositions were introduced to the area of distributed computing by Barenboim and Elkin [5]. Nowadays, they are a frequent tool in the area and by increasing the degree bound 2 to $(2 + \varepsilon)a$ the concept also extends to graphs with arboricity at most a (this is the original setting considered in [5]). More generally, it can be shown that H -decompositions with $O(\log n)$ layers exist. In the LOCAL model, an H -decomposition of a tree with $O(\log n)$ layers can be computed by iteratively removing nodes of degree 1 (rake) and nodes of degree 2 (compress).

In the LOCAL model, one can 3-color any graph with a given H -decomposition as in Definition 3 with L layers in $O(L + \log^* n)$ rounds. Each layer induces a graph with maximum degree 2. First, use Linial's algorithm to color each layer in parallel with $C = O(1)$ colors. This coloring may contain lots of monochromatic edges between different layers and is only used as a schedule to compute the final 3-coloring. In order to compute that final coloring, iterate through the layers in a decreasing order, and in each layer iterate through the C colors. When processing one of the C color classes, every node picks one color in $\{1, 2, 3\}$ not used by any of its already colored neighbors (at most two).

Optimally, we would like to use the above LOCAL model algorithm as the base for our exponentially faster MPC algorithm. However, even if we were given an H -decomposition for free it is non-trivial to actually use it for 3-coloring a graph if the runtime is restricted

to $O(\log \log n)$ rounds and you only allow for a polylogarithmic memory overhead. Going through the layers in some sequential manner would be way too slow as it would require logarithmically rounds. Still, as the LOCAL algorithm has locality $T = \Theta(\log n)$ the output of a node may depend on the topology in logarithmic distance. In order to achieve a fast MPC algorithm, we want the nodes to use the graph exponentiation technique to learn the part G_v of their T -hop neighborhood that is relevant to determine their output in $O(\log T) = O(\log \log n)$ rounds. We refer to G_v as the predecessor graph of node v . The challenge with the standard H -decomposition as given by Definition 3 is that, even though $G_v \subseteq V_{\geq \text{layer}(v)}$, it may be of size $\Theta(n)$. Hence, even if every node could learn its predecessor graph and store it in its local memory S_v (formally, the memory of every node is stored on some machine), the global space bound can only be upper bounded by $\sum_{v \in V} |G_v| = O(n^2)$, drastically, violating the desired near-linear bound. In order to circumvent this issue we introduce the concept of a strict H -decomposition which is optimized for its usage in the MPC model. The bottom line of this decomposition is that besides the properties of a classic H -decomposition, we also have a set V^{pivot} . The set V^{pivot} induces a graph with maximum degree 2 and hence can be colored with 3 colors in $O(\log^* n)$ rounds with Linial's algorithm [35]. The main gain compared to the classic H -decomposition is, that once we have colored the nodes in V^{pivot} , we can show that the predecessor graph of every node $v \in V \setminus V^{\text{pivot}}$ is of logarithmic size (when considering the same LOCAL model algorithm that colors these nodes layer by layer). Hence, each node can learn its predecessor graph in $O(\log \log n)$ rounds without violating global space constraints. We next present the definition of a strict H -decomposition.

► **Definition 4** ((Partial) Strict H -Decomposition). *Let F be a forest and $\text{layer}: V(F) \mapsto \mathbb{N} \cup \{\infty\}$ be a function. We define $V_{<\infty} = \{v \in V(F) \mid \text{layer}(v) < \infty\}$ and*

$$V^{\text{pivot}} := \{v \in V_{<\infty} \mid \text{layer}(v) \geq \text{layer}(w) \text{ for every } w \in N_F(v)\}.$$

We refer to a (partial) H -decomposition layer as strict if for every $v \in V_{<\infty} \setminus V^{\text{pivot}}$, it holds that

$$|\{w \in N_F(v) \setminus V^{\text{pivot}} \mid \text{layer}(w) = \text{layer}(v)\} \cup \{w \in N_F(v) \mid \text{layer}(w) > \text{layer}(v)\}| \leq 1. \quad (1)$$

That is, the total number of non-pivot neighbors with the same layer and of neighbors with a strictly higher layer is at most 1.

There is some similarity between the definition of a strict H -decomposition and the H -decompositions used in the theory of so called locally checkable labelings [12, 10, 2]. These decompositions iteratively layer degree 1 nodes and paths of length at least ℓ . Our strict H -decomposition is similar to the case when we remove paths of length at least $\ell = 3$ (see Lemma 10). The following lemma is one of the most crucial structural properties of partial H -decompositions that we exploit in the core of our algorithm (see Appendix A).

► **Lemma 5** (Partial Strict H -Decomposition, Closure under taking minimums). *Let F be a forest and $\text{layer}_1, \text{layer}_2: V(F) \rightarrow \mathbb{N} \cup \{\infty\}$ be two partial strict H -decompositions. Let $\text{layer}: V(F) \rightarrow \mathbb{N} \cup \{\infty\}$ with*

$$\text{layer}(v) = \min(\text{layer}_1(v), \text{layer}_2(v))$$

for every $v \in V(F)$. Then, layer is also a partial strict H -decomposition.

From a high level point of view, Lemma 5 says that we can independently compute two partial strict H -decompositions, and even though they might contain conflicting layer assignments for certain nodes, we can obtain a unified decomposition, by assigning each node to the smaller layer of the two choices. In fact, this insight also generalizes to more than two (possibly conflicting) decompositions. At the core of our procedure in Appendix A, many nodes (independently) learn large parts of the graph. Then, every node computes a partial decomposition on the parts that it has learned, and in a second step all these partial decompositions are combined, where each node takes the minimum layer that it got assigned in any of the decompositions. Taking the minimum is a very efficient procedure in the MPC model and only requires constant time. The remaining difficulty in Appendix A is to show that nodes learn large enough parts in the graph in order to make very fast global progress, that is, we show that the unified decomposition assigns a layer to a large fraction of the nodes.

4 Strict H -decomposition in MPC

In this section, we present our $O(\log \log n)$ -round MPC algorithm for computing a strict H -decomposition. However, the hardest part of that algorithm, that is, assigning each node that is contained in a small subtree (see definition below) to a layer is deferred to Appendix A. The algorithm in this section uses $n \cdot \text{poly} \log n$ global space. In Appendix B we explain how to extend the algorithm to optimal space.

High Level Overview. For the sake of this high level overview let us first assume that we compute an H -decomposition with $O(\log n)$ layers that may not be strict. Similar to the classic rake & compress algorithm, our algorithm iteratively assigns nodes to layers. After assigning a node to some layer we *remove* it from the graph and continue on the remaining graph, which may actually become disconnected and turn into a forest. In order to present the details of the high level intuition we require the definition of a subtree which is central to our whole approach.

► **Definition 6 (Subtree).** *Let T be a tree. A subtree $T' \subseteq T$ is a connected induced subgraph of T such that $T \setminus T'$ contains at most one component.*

A subtree $T' \subseteq F$ of a forest F is a connected induced subgraph of F such that the number of components of $F \setminus T'$ is not larger than the number of connected components of F .

The definition of a subtree is best understood in a rooted tree, where the subtree rooted at a node v is formed by all its descendants.

In order to assign a layer to all nodes of the graph, we iterate the following two steps until all nodes have received a layer:

1. Assigns a layer to each node contained in a *small subtree* of size $\leq n^{\delta/10} \cdot \text{SUBTREERC}(F)$,
2. Assign a layer to each node of degree ≤ 2 in the remaining graph.

This process can be seen as a generalization of the classic rake and compress procedure, in which one iteratively removes leaves, i.e., subtrees of size 1, and nodes of degree 2. The rake and compress procedure requires $O(\log n)$ iterations to *remove* all nodes of the graph. Our generalized process requires $O(1/\delta) = O(1)$ in order to assign a layer to every node of the graph (see Lemma 7 for $x = n^{\delta/10}$). Note that the lemma statement considers a slightly different process than the one presented in this overview; the difference lies in the fact that we actually want to compute a strict H -decomposition. However, a similar lemma holds for the process of this overview. The main contribution and the main difficulty of

our work lies in the procedure $\text{SUBTREERC}(F)$ as nodes do not know whether they are contained in a small subtree, but still these subtrees can have diameter up to $n^{\delta/10}$, so conditioned on the 1 vs 2 cycle conjecture it is impossible that a single node can learn the whole subtree in $O(\log \log n)$ rounds (we don't prove this formally, but it's very unlikely that such a result holds without breaking the conjecture). We explain the details of the procedure $\text{SUBTREERC}(F)$ in Appendix A.

We continue with our generalized rake and compress statement that shows that a constant number of iterations of the aforementioned process suffice. As we want to compute a strict H -decomposition (see Definition 4), we need to slightly modify Step 2 of the above outline, for which we require further definitions; the details of why $\text{SUBTREERC}(F)$ returns layers that induce a strict H -decomposition are presented in Appendix A.

A path in a graph is a *degree-2 path* if all of its nodes, including its endpoints have degree 2. The *length of a path* is the number of nodes in the path, e.g., a single node is a path of length 1.

The following lemma is easiest to be understood when setting $x = \ell = 1$ where the process (almost) equals the classic rake & compress process – in fact it consists of a rake step, a compress step, and another rake step – and the theorem shows that it removes $1/3$ of the nodes ($2/3$ of the nodes remain in the graph).

► **Lemma 7** (Generalized rake and compress). *Let $x, \ell \in \mathbb{Z}$. Consider a process on a tree T that consists of the following steps:*

1. *Remove (at least) all subtrees of size $\leq x$ from T , resulting in T_1 ,*
2. *Remove (at least) all nodes contained in a degree-2 path of length at least ℓ from T_1 , resulting in T_2 ,*
3. *Remove (at least) all nodes with degree ≤ 1 from T_2 .*

The number of nodes remaining is at most a $1/(1 + (x + 1)/2\ell) = O(\ell/x)$ fraction of the nodes from T . The degrees of nodes in Step 2) and 3) of the process are with respect to the graph induced by remaining nodes at the respective step.

We now state a lemma for a key subroutine that we will use as black box in this section and dedicate Appendix A to designing an algorithm that proves the lemma.

► **Lemma 8** (SUBTREERC). *Let F be a forest on n vertices. There exists a deterministic MPC algorithm SUBTREERC with $O(n^\delta)$ local space, $0 < \delta < 1$, and $\tilde{O}(n)$ global space which takes F as input and computes in $O(\log \log n)$ rounds a partial strict H -decomposition layer: $V(F) \mapsto [\lceil \log(|V(F)| + 1) \rceil] \cup \{\infty\}$ such that $\text{layer}(v) < \infty$ for every node $v \in V(F)$ contained in a subtree of size $n^{\delta/10}$.*

Our MPC algorithm for computing strict H -decomposition appears in Algorithm 1. We will now prove the correctness and progress guarantees of our algorithm.

► **Lemma 9.** *At the end of each iteration i , we have that layer is a partial strict H -decomposition with at most $(i + 1) \cdot \text{offset}$ layers.*

► **Lemma 10.** *In iteration i , Algorithm 1 correspond to a generalized rake and compress step with $x = n^{\delta/10}$ and $\ell = 3$.*

► **Corollary 11.** *In iteration i , Algorithm 1 correspond to a generalized rake and compress step with $x = 1$ and $\ell = 3$.*

■ **Algorithm 1** Strict H -decomposition.

```

1: Throughout  $V_\infty = \{v \in F \mid \text{layer}(v) = \infty\}$  denotes the set of nodes whose layer equals  $\infty$ .
2: function STRICTHDECOMP(Forest  $F$ )
3:   Initialize:  $\text{layer}(v) = \infty$  for all  $v \in V(F)$ ;  $\text{offset} \leftarrow \lceil \log(|V(F)| + 1) \rceil + 1$ 
4:   for  $i = 1, 2, \dots, \lceil 10/\delta \rceil$  do
5:      $F_i \leftarrow F[V_\infty]$ 
6:      $\text{layer} \leftarrow i \cdot \text{offset} + \text{SUBTREERC}(F_i, x = n^{\delta/10})$ 
7:     Let  $V_i^{\text{pivot}} \leftarrow \{v \in V_\infty \mid d_{V_\infty}(v) \leq 2, d_{V_\infty}(w) \leq 2 \text{ for all } w \in N(v)\}$ 
8:      $\text{layer}(v) \leftarrow (i + 1) \cdot \text{offset}$  for every node  $v \in V_i^{\text{pivot}}$ 
9:      $\text{layer}(v) \leftarrow (i + 1) \cdot \text{offset}$  for every node  $v \in V_\infty$  with  $\leq 1$  in  $V_\infty$ 
10:  return layer
    
```

► **Theorem 12.** *Algorithm STRICTHDECOMP(F) (Algorithm 1) applied to some forest F computes a strict H -decomposition of F with $O(\log n)$ layers, uses $O(\log \log n)$ low-space MPC rounds and $\tilde{O}(n)$ global space.*

Proof. By Lemmas 7 and 10, in each iteration, the number of nodes in the forest shrinks by a factor of $O(n^{\delta/10})$. Therefore, after $O(1/\delta)$ iterations of the for loop, the number of nodes with layer ∞ will be zero.

By Lemma 9, in an iteration i , we produce a partial strict H -decomposition with at most $(i + 1) \cdot \text{offset}$ layers and in the next iterations $j > i$, we compute a partial strict H -decomposition of the nodes that received layer ∞ (V_∞) in iteration i . The offset value ensures that the nodes in V_∞ get a higher layer than the nodes in $V \setminus V_\infty$. After $i = O(1/\delta)$ iterations, each node has a layer at most $O(\log n)$ since $(i + 1) \cdot \text{offset} = O(\log n)$, and hence we produce a valid strict H -decomposition.

Lemma 8 ensures that implementing each iteration takes $O(\log \log n)$ low-space MPC rounds and $\tilde{O}(n)$ global space. The theorem follows because there are just $O(1/\delta)$ iterations. ◀

5 Coloring, MIS, and Matching

The following theorem is proven at the end of the section.

► **Theorem 13.** *There is a deterministic $O(\log \log n)$ round algorithm for 3-coloring trees in the low-space MPC model using $\tilde{O}(n)$ words of global space.*

For an input tree F , consider having a strict H -decomposition $\text{layer} : V(F) \rightarrow \mathbb{N}$ described in Definition 4, which we get from Algorithm 1 in $O(\log \log n)$ rounds and $\tilde{O}(n)$ words of global space. We first color the subgraph induced by the nodes in V^{pivot} . Recall that V^{pivot} is the set of nodes that have no neighbor with a higher layer.

Coloring the Pivot Nodes. The subgraph $F[V^{\text{pivot}}]$ has maximum degree 2 each node $v \in V^{\text{pivot}}$ has at most two neighbors in V^{pivot} with the same layer, and no neighbors with higher layer. In order to color $F[V^{\text{pivot}}]$, we first run Linial's $O(\Delta^2)$ -coloring algorithm [34], which requires $O(\log^* n)$ rounds. Since $\Delta(F[V^{\text{pivot}}]) \leq 2$, Linial's algorithm results in an $O(1)$ -coloring which we can convert to a 3-coloring by performing the following: In each round, all nodes with the highest color among their neighbors in V^{pivot} recolor themselves with

the smallest color such that a proper coloring is preserved. Clearly, one color is eliminated in each round and since each node v has at most 2 neighbors in $F[V^{pivot}]$, we achieve a 3-coloring of $F[V^{pivot}]$ in a constant number of rounds.

Coloring the Remaining Nodes. We will now compute a 3-coloring of the nodes in $V \setminus V^{pivot}$. We first orient all edges $e = \{u, v\}$ with $u, v \in V \setminus V^{pivot}$ from u to v if $\text{layer}(u) < \text{layer}(v)$ and arbitrarily if $\text{layer}(u) = \text{layer}(v)$. The following lemma will help us to ensure that we do not create conflicts with the 3-coloring computed on V^{pivot} .

► **Lemma 14.** *Each node in $v \in V \setminus V^{pivot}$ has at most two forbidden colors. If v has an outgoing edge, then it can have at most one forbidden color.*

Proof. Each node in $v \in V \setminus V^{pivot}$ has at most two neighbors in V^{pivot} . This is because nodes in V^{pivot} do not have neighbors in higher layer, so v can only have neighbors in V^{pivot} at the same or higher layer. By Definition 4, v can have at most two such neighbors.

Nodes v with one outgoing edge can have at most one neighbor in V^{pivot} , as otherwise v has three neighbors with same or higher layer, and Definition 4 is violated. So if v has an outgoing edge, it can have at most one forbidden color. ◀

In what follows, each node $v \in V \setminus V^{pivot}$ will remember its at most two forbidden colors due to neighbors in V^{pivot} . Definition 4 also guarantees that all nodes in $V \setminus V^{pivot}$ will have at most one outgoing edge. So the nodes $w \in V \setminus V^{pivot}$ with no outgoing edge pick an arbitrary color that is not forbidden as their final color.

In order to properly color the nodes with exactly one outgoing edge, consider the following centralized procedure: Color the nodes one by one in a greedy manner starting from the highest layer and with an arbitrary order within one layer. Here, greedy means, that a node picks the smallest color that is not forbidden and not used by any of its already colored neighbors. This process computes a proper 3-coloring as each node will have one color used by the neighbor along its outgoing edge, and at most one forbidden color. The output of a node $v \in V \setminus V^{pivot}$ in this centralized procedure only depends on the *directed path* of v obtained by following outgoing edges starting at v . In the following lemma we show that this directed path cannot be too long.

► **Lemma 15.** *The directed path of a node $v \in V \setminus V^{pivot}$ obtained by following outgoing edges starting at v has length at most $O(\log n)$.*

Proof. Consider a directed edge (u, w) in the directed path of v . If $\text{layer}(u) = \text{layer}(w)$, then w cannot have an outgoing edge as it will have two neighbors in $V \setminus V^{pivot}$ with same or higher layer, violating Definition 4. In other words, if $\text{layer}(u) = \text{layer}(w)$, then the directed path of v ends at w .

Therefore, if we go along the directed path, the layer of the nodes either strictly increases or the path does not continue. Since there are $O(\log n)$ layers in the H -decomposition, the length of a directed path is at most $O(\log n)$. ◀

In our MPC algorithm, the idea is for each node to learn its $O(\log n)$ length directed path by performing graph exponentiation only along the directed edges. Since all nodes with no outgoing edges are already colored with their final color, consider performing the following MPC algorithm only for nodes with one outgoing edge: Each node computes its final color after gathering its directed path by performing $O(\log \log n)$ graph exponentiation steps along directed edges.

Proof of Theorem 13. Correctness follows from the fact that each node can recolor itself with its final color when seeing its whole directed path. The runtime follows from the fact that we only perform $O(\log \log n)$ graph exponentiation steps and color the directed paths.

Let us analyze the space usage of our algorithm. Since the length of a directed path stored by each node during the algorithm is at most $O(\log n)$, we do not violate global space. Note that the sequential coloring of frozen layers does not require additional space. Notice that even though each node v is the source of at most one request, multiple nodes may send a request to v . Hence, during graph exponentiation, node v may have to communicate with a large number of nodes in lower layers. To mitigate this issue, we perform a load balancing process by sorting all the at most n requests by the ID of their destination. This can be done deterministically in $O(1)$ rounds. Now, all the requests with destination v lie in consecutive machines, and therefore, we can broadcast the response of v to all these machines in $O(1)$ rounds by creating a constant depth broadcast tree on these machines. Therefore, each step of graph exponentiation can be done in $O(1)$ rounds, which leads to an overall running time of $O(\log \log n)$ rounds. ◀

MIS and Maximal Matching

The maximal independent set and maximal matching algorithms follow from Theorem 13.

► **Theorem 16.** *There is a deterministic $O(\log \log n)$ round MIS algorithm for trees in the low-space MPC model using $\tilde{O}(n)$ words of global space.*

Proof. By Theorem 13, we can color the tree with 3 colors. For all colors i , perform the following. Nodes colored i add themselves to the independent set, and all nodes adjacent to nodes colored i remove themselves from the graph. Clearly this results in a maximal independent set in $O(1)$ rounds and the space requirements are satisfied. ◀

► **Theorem 17.** *There is a deterministic $O(\log \log n)$ round maximal matching algorithm for trees in the low-space MPC model using $\tilde{O}(n)$ words of global space.*

Proof. By Theorem 13, we can color the tree with 3 colors using a H -decomposition. Recall that in the decomposition, each node v with $\text{layer}(v) = i$ has at most two neighbors with layer at least i . Let us define the parent nodes of v . We orient an edge $\{u, v\}$ from v to u if (i) u belongs to a strictly higher layer than v or (ii) u belongs to the same layer and has a higher ID. For all colors i , perform the following. Node v colored i proposes to its highest ID outgoing neighbor u , and u accepts the proposal of the highest ID proposer. If u accepts v 's proposal in which case the edge $\{u, v\}$ joins the matching. If u rejects v 's proposal, it means that u is matched with some other node and then we repeat the same procedure with v 's other possible out-neighbor. Note that when a node joins the matching, it prevents all other incident edges from joining the matching. As a result, all nodes colored i have either joined the matching or they have no out-going edges. After iterating through all color classes, all nodes have either joined the matching or they have no incident edges, implying that all their original neighbors belong to the matching. This results in a maximal matching in $O(1)$ rounds and the space requirements are satisfied. ◀

References

- 1 Noga Alon, László Babai, and Alon Itai. A Fast and Simple Randomized Parallel Algorithm for the Maximal Independent Set Problem. *Journal of Algorithms*, 7(4):567–583, 1986. doi:10.1016/0196-6774(86)90019-2.
- 2 Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally Checkable Labelings with Small Messages. In *the Proceedings of the International Symposium on Distributed Computing (DISC)*, pages 8:1–8:18, 2021. doi:10.4230/LIPIcs.DISC.2021.8.
- 3 Alkida Balliu, Rustam Latypov, Yannic Maus, Dennis Olivetti, and Jara Uitto. Optimal Deterministic Massively Parallel Connectivity on Forests. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2589–2631, 2023. doi:10.1137/1.9781611977554.ch99.
- 4 Philipp Bamberger, Fabian Kuhn, and Yannic Maus. Efficient Deterministic Distributed Coloring with Small Bandwidth. In *PODC '20: ACM Symposium on Principles of Distributed Computing (PODC)*, pages 243–252, 2020. doi:10.1145/3382734.3404504.
- 5 Leonid Barenboim and Michael Elkin. Sublogarithmic distributed MIS algorithm for sparse graphs using nash-williams decomposition. *Distributed Comput.*, 22(5-6):363–379, 2010. doi:10.1007/s00446-009-0088-2.
- 6 Leonid Barenboim, Michael Elkin, Seth Pettie, and Johannes Schneider. The Locality of Distributed Symmetry Breaking. *Journal of the ACM*, 63(3):20:1–20:45, 2016.
- 7 Soheil Behnezhad, Sebastian Brandt, Mahsa Derakhshan, Manuela Fischer, MohammadTaghi Hajiaghayi, Richard M. Karp, and Jara Uitto. Massively parallel computation of matching and mis in sparse graphs. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing*, PODC '19, pages 481–490, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3293611.3331609.
- 8 Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Łącki, and Vahab Mirrokni. Near-Optimal Massively Parallel Graph Connectivity. In *FOCS*, 2019. doi:10.1109/FOCS.2019.00095.
- 9 Sebastian Brandt, Manuela Fischer, and Jara Uitto. Breaking the Linear-memory Barrier in MPC: Fast MIS on Trees with Strongly Sublinear Memory. *Theoretical Computer Science*, 849:22–34, 2021. doi:10.1016/j.tcs.2020.10.007.
- 10 Yi-Jun Chang. The Complexity Landscape of Distributed Locally Checkable Problems on Trees. In *DISC*, pages 18:1–18:17, 2020. doi:10.4230/LIPIcs.DISC.2020.18.
- 11 Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. The Complexity of $(\Delta + 1)$ -Coloring in Congested Clique, Massively Parallel Computation, and Centralized Local Computation. In *PODC*, 2019. doi:10.1145/3293611.3331607.
- 12 Yi-Jun Chang and Seth Pettie. A Time Hierarchy Theorem for the LOCAL Model. *SIAM J. Comput.*, 48(1):33–69, 2019. doi:10.1137/17M1157957.
- 13 Sam Coy and Artur Czumaj. Deterministic Massively Parallel Connectivity. In *Proceedings of the ACM Symposium on Theory of Computing (STOC)*, 2022. doi:10.1145/3519935.3520055.
- 14 Artur Czumaj, Peter Davies, and Merav Parter. Graph Sparsification for Derandomizing Massively Parallel Computation with Low Space. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 175–185, 2020. doi:10.1145/3350755.3400282.
- 15 Artur Czumaj, Peter Davies, and Merav Parter. Component Stability in Low-Space Massively Parallel Computation. In *PODC*, 2021. doi:10.1145/3465084.3467903.
- 16 Artur Czumaj, Peter Davies, and Merav Parter. Improved Deterministic $(\Delta + 1)$ -Coloring in Low-Space MPC. In *PODC*, pages 469–479, 2021. doi:10.1145/3465084.3467937.
- 17 Artur Czumaj, Peter Davies, and Merav Parter. Simple, deterministic, constant-round coloring in congested clique and MPC. *SIAM Journal on Computing*, 50(5):1603–1626, 2021. doi:10.1137/20M1366502.
- 18 Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, pages 107–113, 2008. doi:10.1145/1327452.1327492.

- 19 Mohsen Ghaffari. An Improved Distributed Algorithm for Maximal Independent Set. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 270–277, 2016.
- 20 Mohsen Ghaffari, Themis Gouleakis, Christian Konrad, Slobodan Mitrovic, and Ronitt Rubinfeld. Improved Massively Parallel Computation Algorithms for MIS, Matching, and Vertex Cover. In *PODC*, pages 129–138, 2018. doi:10.1145/3212734.3212743.
- 21 Mohsen Ghaffari, Christoph Grunau, Bernhard Haeupler, Saeed Ilchi, and Václav Rozhoň. Improved Distributed Network Decomposition, Hitting Sets, and Spanners, via Derandomization. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2532–2566, 2023. doi:10.1137/1.9781611977554.ch97.
- 22 Mohsen Ghaffari, Christoph Grunau, and Ce Jin. Improved MPC Algorithms for MIS, Matching, and Coloring on Trees and Beyond. In *DISC*, 2020. doi:10.4230/LIPIcs.DISC.2020.34.
- 23 Mohsen Ghaffari, Fabian Kuhn, and Jara Uitto. Conditional Hardness Results for Massively Parallel Computation from Distributed Lower Bounds. In *FOCS*, pages 1650–1663, 2019. doi:10.1109/FOCS.2019.00097.
- 24 Mohsen Ghaffari and Ali Sayyadi. Distributed Arboricity-Dependent Graph Coloring via All-to-All Communication. In *ICALP*, pages 142:1–142:14, 2019. doi:10.4230/LIPIcs.ICALP.2019.142.
- 25 Mohsen Ghaffari and Jara Uitto. Sparsifying Distributed Algorithms with Ramifications in Massively Parallel Computation and Centralized Local Computation. In *SODA*, 2019. doi:10.1137/1.9781611975482.99.
- 26 Jeff Giliberti, Manuela Fischer, and Christoph Grunau. Deterministic massively parallel symmetry breaking for sparse graphs. *CoRR*, abs/2301.11205, 2023. doi:10.48550/arXiv.2301.11205.
- 27 Michael T. Goodrich, Nodari Sitchinava, and Qin Zhang. Sorting, searching, and simulation in the mapreduce framework. In Takao Asano, Shin-ichi Nakano, Yoshio Okamoto, and Osamu Watanabe, editors, *Algorithms and Computation*, pages 374–383, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- 28 Chetan Gupta, Rustam Latypov, Yannic Maus, Shreyas Pai, Simo Särkkä, Jan Studený, Jukka Suomela, Jara Uitto, and Hossein Vahidi. Fast dynamic programming in trees in the mpc model, 2023. arXiv:2305.03693.
- 29 Michael Isard, Mihai Budiu, Yuan Yu, Andrew Birrell, and Dennis Fetterly. Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks. *ACM SIGOPS Operating Systems Review*, pages 59–72, 2007. doi:10.1145/1272996.1273005.
- 30 Howard J. Karloff, Siddharth Suri, and Sergei Vassilvitskii. A Model of Computation for MapReduce. In *SODA*, 2010. doi:10.1137/1.9781611973075.76.
- 31 Silvio Lattanzi, Benjamin Moseley, Siddharth Suri, and Sergei Vassilvitskii. Filtering: A Method for Solving Graph Problems in MapReduce. In *SPAA*, pages 85–94, 2011. doi:10.1145/1989493.1989505.
- 32 Rustam Latypov and Jara Uitto. Deterministic 3-coloring of trees in the sublinear MPC model. *CoRR*, abs/2105.13980, 2021. arXiv:2105.13980.
- 33 Christoph Lenzen and Roger Wattenhofer. Brief Announcement: Exponential Speed-Up of Local Algorithms Using Non-Local Communication. In *PODC*, 2010. doi:10.1145/1835698.1835772.
- 34 Nathan Linial. Distributive Graph Algorithms – Global Solutions from Local Data. In *FOCS*, 1987. doi:10.1109/SFCS.1987.20.
- 35 Nathan Linial. Locality in Distributed Graph Algorithms. *SIAM J. Comput.*, 21(1):193–201, 1992. doi:10.1137/0221015.
- 36 Michael Luby. A Simple Parallel Algorithm for the Maximal Independent Set Problem. *SIAM Journal on Computing*, 15:1036–1053, 1986. doi:10.1137/0215074.
- 37 Crispin Nash-Williams. Decomposition of Finite Graphs Into Forests. *Journal of the London Mathematical Society*, s1-39:12, 1964. doi:10.1112/jlms/s1-39.1.12.

- 38 Öjvind Johansson. Simple Distributed $\Delta + 1$ -coloring of Graphs. *Information Processing Letters*, pages 229–232, 1999. doi:10.1016/S0020-0190(99)00064-2.
- 39 Merav Parter. $(\delta + 1)$ coloring in the congested clique model. In Ioannis Chatzigiannakis, Christos Kaklamanis, Daniel Marx, and Donald Sannella, editors, *International Colloquium on Automata, Languages, and Programming, (ICALP)*, volume 107, pages 160:1–160:14, 2018. doi:10.4230/LIPIcs.ICALP.2018.160.
- 40 Merav Parter and Hsin-Hao Su. Randomized $(\delta + 1)$ -coloring in $o(\log^* \delta)$ congested clique rounds. In *32nd International Symposium on Distributed Computing (DISC)*, volume 121, pages 39:1–39:18, 2018. doi:10.4230/LIPIcs.DISC.2018.39.
- 41 Tim Roughgarden, Sergei Vassilvitskii, and Joshua R. Wang. Shuffles and Circuits (On Lower Bounds for Modern Parallel Computation). *Journal of the ACM*, 2018. doi:10.1145/3232536.
- 42 Václav Rozhoň and Mohsen Ghaffari. Polylogarithmic-time deterministic network decomposition and distributed derandomization. In *STOC*, pages 350–363, 2020. doi:10.1145/3357713.3384298.
- 43 Tom White. *Hadoop: The Definitive Guide*. O'Reilly Media, Inc., 2009.
- 44 Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster Computing with Working Sets. In *the Proceedings of the SENIX Conference on Hot Topics in Cloud Computing (HotCloud)*, 2010. doi:10.5555/1863103.1863113.

A Massively Parallel Subtree Rake and Compress

This section is dedicated to designing an algorithm that proves Lemma 8, which states that we can compute in $O(\log \log n)$ rounds a partial strict H -decomposition that assigns each node contained in a subtree of size $O(n^{\delta/10})$ to one of $O(\log n)$ layers. We restate the lemma.

► **Lemma 8 (SUBTREERC).** *Let F be a forest on n vertices. There exists a deterministic MPC algorithm SUBTREERC with $O(n^\delta)$ local space, $0 < \delta < 1$, and $\tilde{O}(n)$ global space which takes F as input and computes in $O(\log \log n)$ rounds a partial strict H -decomposition layer: $V(F) \mapsto [\lceil \log(|V(F)| + 1) \rceil] \cup \{\infty\}$ such that $\text{layer}(v) < \infty$ for every node $v \in V(F)$ contained in a subtree of size $n^{\delta/10}$.*

Our algorithm critically relies on the balanced graph exponentiation technique mentioned in Section 1.2 and explained in detail in the full version. In the following definition, you should think about U as being the set of nodes that v has stored in its local memory after the balanced graph exponentiation. We refer to U as good if it contains all nodes within distance $O(\log n)$ of v , except for potentially one direction, for which no node is contained in U .

► **Definition 18** (U is a good subset for v). *Let F be a forest, $U \subseteq V(F)$ and $v \in V(F)$. We say that U is a good subset for v if*

1. $v \in U$,
2. $|N_F(v) \setminus U| \leq 1$, i.e., v has at most one neighbor in F not in U ,
3. $N_F(w) \subseteq U$ for every $w \in U \setminus \{v\}$ with $d_F(v, w) \leq 3L$ where $L := \lceil \log(|U| + 1) \rceil$.

In Appendix A.1, we give a peeling algorithm that takes as input a set U and computes a partial strict H -decomposition with $O(\log n)$ layers by repeatedly peeling off low-degree vertices contained in U . Moreover, if U is good for v , then v gets assigned to one of the $O(\log n)$ layers. This peeling algorithm will later be simulated without any further communication on the machine that has stored the set U in its memory.

Using the balanced graph exponentiation technique, we can compute a collection of sets $U_1, U_2, \dots, U_k$ such that for each node v contained in a subtree of size at most $n^{\delta/10}$ there exists some subset U_j that is good for v . In particular, in the full version, we prove the following statement.

► **Lemma 19** (Lemma from Balanced Exponentiation). *Let F be a forest on n vertices. There exists a deterministic low-space MPC algorithm with $O(n^\delta)$ local space, $0 < \delta < 1$, and $O(n \cdot \text{poly}(\log n))$ global space which takes F as input and computes in $O(\log \log n)$ rounds a collection of non-empty sets $U_1, U_2, \dots, U_k \subseteq V(F)$ such that*

1. (Local Space) $|U_j| = O(n^\delta)$ for every $j \in [k]$,
2. (Global Space) $\sum_{j=1}^k |U_j| = O(n \cdot \text{poly}(\log n))$ and
3. for every $v \in V(F)$ which is contained in a subtree of size at most $n^{\delta/10}$, there exists a $j \in [k]$ such that U_j is a good subset for v (see Definition 18).

Moreover, the algorithm also computes for each U_j the forest $F[U_j]$ induced by vertices in U_j and stores it on a single machine.

Our final MPC algorithm for proving Lemma 8 first computes a collection of sets $U_1, U_2, \dots, U_k$ using Lemma 19. Then, the machine storing U_j locally simulates the peeling algorithm of Appendix A.1 with input U_j . As a result, we obtain one partial strict H -decomposition for each set U_j . These partial strict H -decompositions are then combined into one partial strict H -decomposition by assigning each node to the smallest layer assigned by any of the partial strict H -decompositions. More details can be found in Appendix A.2.

A.1 The Conservative Peeling Algorithm

Algorithm 2 computes a partial strict H -decomposition by repeatedly removing low-degree vertices contained in U .

■ **Algorithm 2** Conservative Peeling Algorithm.

```

1: function CONSERVATIVEPEELING(Forest  $F$ , Subset  $U \subseteq V(F)$ )
2:    $V_{\geq 1} \leftarrow V(F)$ , layer:  $V(F) \mapsto \mathbb{N} \cup \{\infty\}$ ,  $L \leftarrow \lceil \log(|U| + 1) \rceil$ 
3:   for  $i = 1, 2, \dots, L$  do
4:     We define  $N_{\geq i}(v) := N_{F[V_{\geq i}]}(v)$  for every  $v \in V_{\geq i}$ .
5:      $V_i^{\text{pivot}} \leftarrow \{v \in V_{\geq i} \cap U \mid N_{\geq i}(v) \subseteq U \text{ and } \forall w \in N_{\geq i}(v) \cup \{v\}: |N_{\geq i}(w)| \leq 2\}$ .
6:      $V_i \leftarrow V_i^{\text{pivot}} \cup \{v \in V_{\geq i} \cap U \mid |N_{\geq i}(v) \setminus V_i^{\text{pivot}}| \leq 1\}$ 
7:      $V_{\geq i+1} \leftarrow V_{\geq i} \setminus V_i$ 
8:     layer( $v$ )  $\leftarrow i$  for every  $v \in V_i$ 
9:     layer( $v$ )  $\leftarrow \infty$  for every  $v \in V_{\geq L+1}$ 
10:  return layer
    
```

If we would just be interested in computing a partial H -decomposition instead of a strict one, then we could replace Algorithm 2 with the single line $V_i \leftarrow \{v \in V_{\geq i} \cap U \mid |N_{\geq i}(v)| \leq 2\}$.

We first show that Algorithm 2 indeed computes a partial strict H -decomposition.

► **Lemma 20.** *Let F be a forest and $U \subseteq V(F)$. Let layer: $V(F) \mapsto \mathbb{N} \cup \{\infty\}$ be the mapping computed by Algorithm 2 when given F and U as input. Then, layer is a partial strict H -decomposition as defined in Definition 4.*

Next, we show that if U is a good subset for v , as defined in Definition 18, then v gets assigned to one of the $O(\log n)$ layers.

► **Lemma 21.** *Let F be a forest, $U \subseteq V(F)$ and $v \in V(F)$. Let $\text{layer}: V(F) \mapsto \mathbb{N} \cup \{\infty\}$ be the mapping computed by Algorithm 2 when given F and U as input. If U is a good subset for v (see Definition 18), then $\text{layer}(v) \leq L := \lceil \log(|U| + 1) \rceil$.*

Finally, we show that we can locally simulate Algorithm 2 by only knowing the forest induced by vertices in U and the degree of each node U in the original forest.

► **Lemma 22** (Local Sequential Simulation). *Let F be an arbitrary forest and $U \subseteq V(F)$ be a non-empty subset. Let $\text{layer}: V(F) \mapsto \mathbb{N} \cup \{\infty\}$ be the mapping computed by Algorithm 2 when given F and U as input. There exists a sequential algorithm running in $O(|U|)$ space with the following guarantee: The input of the algorithm is the forest $F[U]$ and the degree $\deg_F(u)$ of each node $u \in U$ in the forest F . The algorithm outputs for each node $v \in U$ its layer $\text{layer}(v)$.*

A.2 Subtree Rake and Compress

Algorithm 3 computes a partial strict H -decomposition with $O(\log n)$ layers where each node in a subtree of size at most x is assigned to one of the layers. We later set $x = n^{\delta/10}$. The correctness follows from the key structural property that partial strict H -decompositions are closed under taking minimums (Lemma 5).

Algorithm 3 SUBTREERC Algorithm.

```

1: function SUBTREERC(forest  $F$ ,  $x \in \mathbb{N}$ )
2:   Let  $U_1, U_2, U_3, \dots, U_k \subseteq V(F)$  such that for every node  $v \in V(F)$  contained in a
   subtree of size at most  $x$  in  $F$ , there exists some  $j \in [k]$  such that  $U_j$  is a good subset for
    $v$  (see Definition 18)
3:    $\text{layer}_j \leftarrow \text{ConservativePeeling}(F, U_j)$  for every  $j \in [k]$   ▷  $\text{layer}_j: V(F) \mapsto \mathbb{N} \cup \{\infty\}$ 
4:    $\text{layer}(v) = \min_{j \in [k]} \text{layer}_j(v)$   ▷  $\text{layer}: V(F) \mapsto \mathbb{N} \cup \{\infty\}$ 
5:   return  $\text{layer}$ 

```

► **Lemma 23.** *The algorithm above computes a partial H decomposition $\text{layer}: V(F) \mapsto [\lceil \log(|V(F)| + 1) \rceil] \cup \{\infty\}$ such that $\text{layer}(v) < \infty$ for every node $v \in V(F)$ contained in a subtree of size at most x .*

Proof. Lemma 20 states that layer_j is a strict partial H -decomposition for every $j \in [k]$. Hence, Lemma 5 implies that layer is also a strict H -decomposition. Moreover, for every node $v \in V(F)$ contained in a subtree of size at most x in F , there exists some $j \in [k]$ such that U_j is a good subset for v . Thus, Lemma 21 gives that $\text{layer}_j(v) < \infty$ and therefore $\text{layer}(v) < \infty$. ◀

We are now ready to prove Lemma 8.

Proof of Lemma 8. We first run the balanced exponentiation algorithm of Lemma 19 which runs in $O(\log \log n)$ rounds and needs $\tilde{O}(n)$ global space. As a result, we obtain a collection of non-empty subsets $U_1, U_2, \dots, U_k \subseteq V(F)$ satisfying the three properties stated in Lemma 19. In particular, for each $j \in [k]$, there exists one machine which has stored $F[U_j]$. As $|U_j| = O(n^\delta)$ and F is a forest, $F[U_j]$ indeed fits into one machine. Moreover, one can compute in $O(1)$ rounds for each node $v \in V(F)$ its degree $\deg_F(v)$ and store $\deg_F(v)$ for every node $v \in U_j$ in the same machine as we store $F[U_j]$ using standard MPC primitives [27]. Let $\text{layer}_j \leftarrow \text{ConservativePeeling}(F, U_j)$. Lemma 22 implies that we can compute

$\text{layer}_j(u)$ for every node $u \in U_j$ locally on the machine that stores $F[U_j]$ without any further communication. Then, in $O(1)$ rounds we can compute $\text{layer}(v) = \min_{j \in [k]} \text{layer}_j(v)$ for every $v \in V$ using the fact that we can sort N items in $O(1)$ rounds in the low-space MPC model with $\tilde{O}(N)$ global space [27]. In more detail, we create one tuple $(v, \text{layer}_j(v))$ for every $j \in [k]$ and $u \in U_j$ and one tuple (v, ∞) for every node $v \in V(F)$. Then, we sort the tuples according to the lexicographic order. Given the sorted tuples, it is straightforward to determine $\text{layer}(v)$ for every $v \in V$. As $\sum_{j=1}^k |U_j| = \tilde{O}(n)$, it follows that the algorithm needs $\tilde{O}(n)$ global space. It thus remains to argue about the correctness, which directly follows from the third property of Lemma 19 and Lemma 23. $\blacktriangleleft$

B **Coloring, MIS, Matching, and H -decomposition with Optimal Space**

In this section we show how to obtain optimal global space by equipping the algorithm from Theorems 13, 16, and 17 with suitable pre- and processing steps that free additional space.

► **Theorem 1.** *There are deterministic $O(\log \log n)$ -round low-space MPC algorithms for 3-coloring, maximal matching and maximal independent set (MIS) on forests. These algorithms use $O(n)$ global space.*

Proof. We perform the standard procedure of iteratively putting in layer i nodes of degree at most 2 for $i = 1$ to $O(\log \log n)$. This removes $O(\text{poly log } n)$ fraction of the nodes since each iteration layers a constant fraction of the nodes. Therefore, the new number of nodes is $n' = n / \text{poly log } n$, and an MPC algorithm using $\tilde{O}(n')$ global space uses $O(n)$ words of global space.

So we freeze these initial $O(\log \log n)$ layers obtain G' remaining graph with $n' = n / \text{poly log } n$ nodes. Then we apply Theorem 13 to compute a 3-coloring in G' in $O(\log \log n)$ rounds and $O(n)$ global space. Finally we complete the solution on the nodes in the frozen layers one layer at a time taking an additional $O(\log \log n)$ rounds.

The claim for MIS and maximal matching follows by the proofs of Theorem 16 and Theorem 17 respectively after computing the H -decomposition and the 3-coloring. $\blacktriangleleft$

Using a similar preprocessing step, we can also show that a strict H -decomposition of Theorem 12 can be computed with optimal global space.

► **Theorem 2.** *There is a deterministic $O(\log \log n)$ -rounds low-space MPC algorithm that computes a strict H -decomposition with $O(\log n)$ layers on forests in $O(n)$ global space.*

Proof. Same as above, iteratively putting in layer i the pivot nodes and nodes of degree 1 as in Algorithm 1 of Algorithm 1 for $i = 1$ to $O(\log \log n)$. By Corollary 11 and using Lemma 7 with $x = 1$ and $\ell = 3$, we get that each iteration layers a constant fraction of nodes, which implies that $O(\text{poly log } n)$ fraction of the nodes are removed after $O(\log \log n)$ iterations.

Now we have a partial strict H -decomposition if we assign layer ∞ to the remaining nodes. These nodes form a graph G' with $n' = n / \text{poly log } n$ nodes, and so we can compute a strict H -decomposition on G' using Algorithm 1 in $O(\log \log n)$ rounds and $\tilde{O}(n') = O(n)$ global space. Therefore, we have computed a strict H -decomposition of G in $O(\log \log n)$ rounds and in $O(n)$ global space. $\blacktriangleleft$

Publication IV

Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. Adaptive Massively Parallel Coloring in Sparse Graphs. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC) 2024*, pages 508-518, doi.org/10.1145/3662158.3662821, Nantes, France, June 2024.

© 2024 ACM

Reprinted with permission.

Adaptive Massively Parallel Coloring in Sparse Graphs

Rustam Latypov
rustam.latypov@aalto.fi
Aalto University
Helsinki, Finland

Shreyas Pai
shreyas@cse.iitm.ac.in
Indian Institute of Technology Madras
Chennai, India

Yannic Maus
yannic.maus@ist.tugraz.at
Graz University of Technology
Graz, Austria

Jara Uitto
jara.uitto@aalto.fi
Aalto University
Helsinki, Finland

ABSTRACT

Classic symmetry-breaking problems on graphs have gained a lot of attention in models of modern parallel computation. The Adaptive Massively Parallel Computation (AMPC) is a model that captures the central challenges in data center computations. Chang et al. [PODC'2019] gave an extremely fast, constant time, algorithm for the $(\Delta + 1)$ -coloring problem, where Δ is the maximum degree of an input graph of n nodes. The algorithm works in the most restrictive *low-space* setting, where each machine has n^δ local space for a constant $0 < \delta < 1$.

The standard approaches for $(\Delta + 1)$ -coloring are ignorant about the graph topology in the following sense: They exploit the property that *any* partial coloring can be extended to a feasible $(\Delta + 1)$ -coloring of the whole graph. For most graphs, the chromatic number is much smaller than $\Delta + 1$ and we would like to find colorings with fewer colors. However, as soon as we have fewer than $\Delta + 1$ colors, it might not be possible to complete partial colorings.

In this work, we study the vertex-coloring problem in sparse graphs parameterized by their *arboricity* α , a standard measure for sparsity. We give deterministic algorithms that in constant, or almost constant, time give $\text{poly } \alpha$ and $O(\alpha)$ -colorings, where α can be arbitrarily smaller than Δ . A strong and standard approach to compute arboricity-dependent colorings is through the Nash-Williams forest decomposition, which gives rise to an (acyclic) orientation of the edges such that each node has a small out-degree.

Our main technical contribution is giving efficient deterministic algorithms to compute these orientations and showing how to leverage them to find colorings in low-space AMPC. A key technical challenge is that the color of a node may depend on *almost all* of the other nodes in the graph and these dependencies cannot be stored on a single machine. Nevertheless, our novel and careful exploration technique yields the orientation, and the arboricity-dependent coloring, with a sublinear number of adaptive queries per node.

CCS CONCEPTS

• Theory of computation → Massively parallel algorithms; Graph algorithms analysis.

KEYWORDS

graph coloring, adaptive massively parallel computation

ACM Reference Format:

Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. 2024. Adaptive Massively Parallel Coloring in Sparse Graphs. In *ACM Symposium on Principles of Distributed Computing (PODC '24)*, June 17–21, 2024, Nantes, France. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3662158.3662821>

1 INTRODUCTION

In this paper, we study the graph coloring problem in sparse graphs in a modern model of parallel computation, the AMPC-Model [5]. The Adaptive Massively Parallel Computation (AMPC) model of computation captures the challenges in modern platforms of processing massive data. We have P machines with $S = O(n^\delta)$ local space each, where $0 < \delta < 1$ and n is the number of nodes in the input graph. In synchronous rounds, the machines can communicate through a distributed key-value storage. The machines have read access to the storage and, at the end of each round, the machines can write to the storage. A crucial limitation is that the local space limits the number of reads and writes per round. The AMPC model is a practically motivated extension of the standard MPC model, where communication is restricted to all-to-all synchronous message-passing [23].

Before delving into our results, let us explain the necessary background. The goal of the k -vertex-coloring problem is to assign a color out of k colors to every node of a graph such that no neighboring nodes have the same color. This is a classic and central problem that has been intensively studied for decades. Vertex-coloring also has close connections to other classic problems, such as the maximal independent set and scheduling.

In the distributed and parallel setting, the typical goal is to color the graph with $\Delta + 1$ colors, where Δ corresponds to the maximum degree of the input graph. Indeed, any graph can be colored with $\Delta + 1$ colors, and for cliques and odd cycles, this is tight. However, many graphs can be colored with significantly fewer than $\Delta + 1$ colors. For example, sparse graphs can have a high maximum degree but can often be colored with much fewer colors. Sparse graphs often occur in practice, which is causing a demand for better coloring algorithms for sparse graphs.



This work is licensed under a Creative Commons Attribution International 4.0 License. *PODC '24*, June 17–21, 2024, Nantes, France
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0668-4/24/06
<https://doi.org/10.1145/3662158.3662821>

Arboricity. In this work, we study the vertex-coloring problem in graphs parameterized by *arboricity*, which is a standard measure of “everywhere” sparsity. The arboricity α corresponds to the minimum number of forests that the edges of a graph can be partitioned into [28]. It is well known that any graph can be colored with 2α colors, and in some cases this is tight. As a straightforward example, a tree clearly has arboricity 1 and can be colored with 2 colors, while the maximum degree Δ can be up to $n-1$. Hence, arboricity captures the number of colors needed to color sparse graphs.

Finding *arboricity-dependent* colorings with $O(\alpha)$, $\text{poly}(\alpha)$, or $\exp(\alpha)$ colors has gained attention in various models of computation like LOCAL [1–3, 15, 26, 27], MPC [6, 14, 18], dynamic algorithms [7, 10, 11, 22], and streaming [6]. A common approach is to find an acyclic orientation of the edges such that each node has an out-degree of $O(\alpha)$ or $\text{poly } \alpha$. Starting from sinks, the nodes can then be iteratively colored with out-degree + 1 colors. Typically, the out-degree gives a trade-off between the runtime and the number of required colors. This approach to coloring highlights a crucial difference between $(\Delta + 1)$ -coloring and arboricity-dependent coloring and pinpoints a key challenge. In the former case, any partial solution can be extended to a complete solution, but this is not the case for the latter. In this light, the iterative coloring approach can be seen as carefully choosing partial solutions that can be extended to a complete solution.

In recent work, the problem has been studied in the context of dynamic algorithms [10]. They showed that *given such an orientation*, there is an efficient Local Computation Algorithm (LCA) to find an arboricity-dependent coloring. They also showed how to maintain such an orientation efficiently. In the LCA model, each node can be queried for the i th entry in their adjacency list and their degree [30]. The goal is to implement an oracle that returns the output at any node v , say, the color of v in the coloring problem. The crux is that the outputs (per node) must yield a consistent and feasible solution throughout the graph.

In our setting, the orientation is not given for free and one of our contributions is an LCA to find an acyclic orientation with a small out-degree. As pointed out by the authors in [10], there is a caveat to designing such an LCA. Although there is no formal proof, it seems likely that the problem has a high volume, that is, any LCA needs to do linear (in n) queries in the worst case. To circumvent this obstacle, we design an LCA that orients *almost* all of the edges. We show that this LCA is sufficient for designing efficient AMPC algorithms to compute acyclic low out-degree orientations orienting *all* edges. Lastly, we obtain various AMPC coloring algorithms that employ different trade-offs between their runtime and the number of colors used. We next detail these three contributions in order, (1) the LCA for computing “weak orientations”, (2) AMPC algorithms for computing acyclic low out-degree orientations, and (3) AMPC coloring algorithms. We emphasize that all our algorithms are *deterministic*.

Contribution 1: An LCA for computing a weak orientation.

A β -partition of a graph is a partition of its vertex set into layers such that every node has at most β neighbors in higher (or the same) layers. The size of the partition is the number of layers. This partition naturally gives rise to the desired orientation required for efficient graph coloring algorithms by orienting edges from smaller

to larger layers while orienting them arbitrarily within the layers. In graphs with arboricity at most α , it is known that such partitions with $O(\log n)$ layers exist when $\beta \geq (2 + \epsilon)\alpha$ for any constant $\epsilon > 0$ [3]. Our core building block and also the largest challenge of our approach is the design of an LCA algorithm that computes a weak version of the desired partition. In Section 2 we underline the technical challenges associated with obtaining such a result. The following simplified version of the main lemma assumes that β and α are constant and that $\beta \geq (2 + \epsilon)\alpha$ holds for any $\epsilon > 0$. We denote by $G[S]$ the subgraph induced by the node set S in graph G .

LEMMA 1.1 (SIMPLIFIED VERSION OF LEMMA 4.7). *For any constant $\delta > 0$ there is deterministic LCA algorithm that uses at most $O(n^\delta)$ queries per node on a graph G with arboricity α and assigns each node a layer from $\mathbb{N} \cup \{\infty\}$ such that the following holds:*

► *There exists a subset $S \subseteq V$ containing at least $1 - 1/n^{O(\delta)}$ fraction of vertices such that the layering of S forms a β -partition of $G[S]$ with $O(\log_\beta n)$ layers. ◀*

Contribution 2: AMPC algorithms for finding acyclic low out-degree orientations. Our second contribution is designing low-space AMPC algorithms to find acyclic orientations in graphs with any arboricity α . Ultimately, our AMPC algorithm iteratively uses the above LCA to find a complete partition and the number of nodes that are not layered (after one application of the LCA) dictates the round complexity of the AMPC algorithm.

Essentially, Lemma 1.1 can be applied recursively¹ on the subgraph induced by the nodes that are assigned an ∞ layer, yielding $O(1/\delta) = O(1)$ recursion levels and implying the following theorem.

THEOREM 1.2 (β -PARTITIONING). *For any constant $\epsilon > 0$ and any $\beta \geq (2 + \epsilon)\alpha$ there is a deterministic AMPC algorithm that computes a β -partition of size $O(\log_{\beta/(2\alpha)} n)$ in $O(\log_{\beta/(2\alpha)} \beta)$ rounds on any n -node graph with arboricity α . The algorithm requires $O(n^\delta)$ local space and $O(n^{1+\delta})$ total space for any constant $\delta > 0$. In particular,*

- *when $\beta = O(\alpha)$, we get size $O(\log n)$ in $O(\log \alpha)$ rounds and*
- *when $\beta = O(\alpha^{1+\epsilon})$, we get size $O(\log_\alpha n)$ in $O(1)$ rounds.*

Contribution 3: Coloring sparse graphs in AMPC. We design AMPC algorithms for coloring with different trade-offs between their runtime and the number of colors used.

THEOREM 1.3 (COLORING RESULTS). *For any constant $\epsilon > 0$ on graphs with possibly non-constant arboricity α , there are deterministic low-space AMPC algorithms to compute:*

- (1) *An $O(\alpha^{2+\epsilon})$ -coloring in $O(1/\epsilon)$ rounds.*
- (2) *An $O(\alpha^2)$ -coloring in $O(\log \alpha)$ rounds.*
- (3) *A $((2 + \epsilon)\alpha + 1)$ -coloring in $\tilde{O}(\alpha/\epsilon)$ rounds.²*

All of our coloring algorithms heavily exploit that we first compute a small out-degree orientation using Theorem 1.2 with different parameters. Although Theorem 1.3 has various trade-offs between the runtime and the number of colors, its main strength lies in the following corollary.

¹This recursion does not work as stated, because we have omitted some details to simplify the exposition. We explain these subtleties in Section 2.3.

²The $\tilde{O}(\cdot)$ notation hides $O(\log \alpha)$ factors.

COROLLARY 1.4. *For any constant $\varepsilon > 0$ there is a constant time AMPC algorithm to compute a $((2 + \varepsilon)\alpha + 1)$ -coloring on any graph with bounded arboricity $\alpha = O(1)$.*

The Quadratic Barrier. One of our easiest results of Theorem 1.3, once we're given the small out-degree orientation, is the one coloring with quadratic number of colors. Here, we essentially simulate the LOCAL model algorithm by Linial that, very efficiently, computes a β^2 -coloring on a graph with an out-degree β and only uses local information. The fact that the algorithm works in graphs with a small out-degree (instead of small maximum degree as usually cited) was first observed in [3]. Understanding when one can go below $o(\alpha^2)$ colors with low volume and/or round complexity seems fundamentally hard and is an intriguing open question (see, e.g., [4, 16]).

Deterministic algorithms. We emphasize that, in contrast to most prior algorithms in the AMPC model and also the most efficient LCA algorithms for large degree graphs, all our algorithms are deterministic. We note that deterministic algorithms seem fundamentally more constrained than randomized ones. As an example, in the deterministic setting, even on a tree, it seems out of reach to design a sublinear LCA that finds the nearest leaf of each node. In a closely related VOLUME model³, it is known that $\Omega(n)$ deterministic queries are required to solve closely related problems on trees [29]. Furthermore, there is an *exponential* gap between the randomized and the deterministic complexities.

Very High Degrees. A particular challenge in massively parallel computation models is to efficiently deal with nodes whose degree is significantly larger than the local space of each machine, especially for deterministic algorithms. As we desire to have efficient algorithms for all ranges of parameters, we design an algorithm using orthogonal techniques. We note that the following theorem holds in the strictly weaker [5] low-space massively parallel computing model (MPC).

THEOREM 1.5 (DETERMINISTIC COLORING). *For any $x > 1$, there is a deterministic low-space MPC algorithm to compute an $2x\Delta$ -coloring in any n -node graph with maximum degree Δ in $O(\log_x n)$ rounds using $O(n + m)$ poly $\log n$ total space.*

In the case of $\alpha > n^\delta$, we can still compute the low out-degree partitions as claimed in Theorem 1.2. Actually, the partition comes with the property that each layer also has a small maximum degree, and hence, we can apply Theorem 1.5 (with $x = \alpha^{O(\varepsilon)} = n^{O(1)}$) to color each layer independently with $O(\alpha^{1+\varepsilon})$ colors in $O(\log_x n) = O(1)$ time, using a separate color space for each layer. This is one of the ways, how we use Theorem 1.5 in the proof of Theorem 1.3 in the case of huge arboricity.

1.1 Related Work

In the LCA model, there is an algorithm for the classic $(\Delta + 1)$ -coloring problem with $\text{poly } \Delta \cdot \log n$ queries [8]. Given that the local space is larger than a small polynomial of Δ , we can turn the LCA into a randomized AMPC algorithm that runs in constant time. The current *deterministic* state-of-the-art originates from the

³A crucial difference between LCA and VOLUME is that in VOLUME the queries must form a single connected component.

LOCAL model of distributed message-passing [25], where the edges of an input graph correspond to communication links along which the nodes communicate in synchronous rounds. Using the graph-exponentiation technique, we can simulate a known $\text{poly } \log \log n$ -round algorithm [9] in $O(\log \log \log n)$ rounds. This can be further derandomized to match the same runtime deterministically and similar techniques apply to $(\text{degree} + 1)$ -coloring as well [12, 13]. In AMPC, we can do even better. The aforementioned algorithms consist of two parts: Solving the problem on low-degree instances in $O(\log \log \log n)$ time and high-degree instances in $O(\log^* n)$ time. The former part can easily be simulated in $O(1)$ rounds in the low-space AMPC even though not explicitly stated in the papers. The runtime then reduces to $O(\log^* n)$.

The coloring problem parameterized by the arboricity α has enjoyed attention in other related models of distributed and parallel computation as well. In the LOCAL model, Barenboim and Elkin gave deterministic $O(\log n)$ and $O(\alpha \log n)$ round algorithms for computing $O(\alpha^2)$ and $(2 + \varepsilon)\alpha$ -colorings, respectively. It is also known that $\Omega(\log n)$ rounds are required [25] for arboricity-dependent colorings. On the randomized side, it is known how to obtain an $O(\alpha \log \alpha)$ coloring in $O(\log n)$ rounds and $O(\alpha)$ -coloring in $O(\log n \cdot \min\{\log \alpha, \log \log n\})$ rounds [17]. We note that there are also $\text{poly } \log n$ time algorithms for finding $(1 + \varepsilon)\alpha$ -out-degree orientations, but these orientations need not be acyclic [21].

In the CONGESTED CLIQUE model, we have n machines that communicate in an all-to-all fashion. Each machine corresponds to a node in the input graph and initially knows its incident edges. The CONGESTED CLIQUE model is very close to the non-adaptive MPC model with linear memory per machine, i.e., $\Omega(n)$ words per machine. There are deterministic algorithms for $O(\alpha)$ -colorings in both of the models that run in $O(1)$ rounds [14]. Finally, through the rake-and-compress decomposition on trees, one can also obtain an orientation with an out-degree at most 2. In a recent work, it was shown how to obtain this decomposition in $O(1)$ rounds of AMPC [20]. For the special case of forests ($\alpha = 1$), [19] give a 3-coloring algorithm that runs in $O(\log \log n)$ rounds and uses optimal global space. This algorithm is conditionally optimal in the MPC model.

1.2 Outline

We start with giving a technical overview of our main contributions and the challenges we need to overcome in Section 2. In Section 3, we define the AMPC model and also give several key definitions for structures like partial, induced, and natural β -partitions. In Section 4, we give details of the LCA for computing partial β -partition and define a coin dropping game which is useful in describing the query structure of the LCA. Due to space constraints, we defer the proofs of Theorems 1.2, 1.3 and 1.5 to the full version in [24].

2 SUBLINEAR LCA: HIGH LEVEL CHALLENGES AND TECHNIQUES

We design a deterministic LCA with a sublinear number of queries to find an acyclic orientation with a small out-degree $\beta = \Omega(\alpha)$ for a very large fraction of the nodes in a graph with arboricity α . The informal statement is Lemma 1.1 and the formal one is Lemma 4.7.

The ultimate goal for our LCA would be to compute a so called β -partition, which is a partition of its vertex set into layers such that every node has at most β neighbors in higher (or the same) layers, with as few layers as possible. This partition naturally gives rise to the desired orientation required for efficient graph coloring algorithms by orienting edges from smaller to larger layers while orienting them arbitrarily within the layers. As discussed in the introduction, it seems unlikely that an LCA with sublinear query complexity can compute such a partition. Instead, we design a sublinear LCA that computes the layers of *most* of the nodes in the graph, see Figure 1.

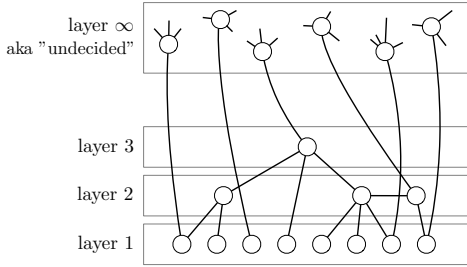


Figure 1: An illustration of a β -partition where most of nodes are assigned a layer.

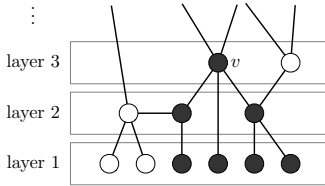


Figure 2: The first three layers of a β -partition. Black nodes form the dependency graph $D(v)$ of node v .

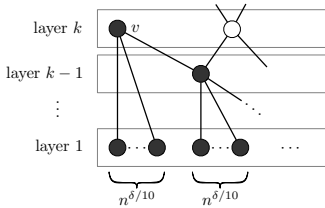


Figure 3: A skewed dependency graph, which is a counterexample to naively volume-based querying.

2.1 Where Should One Query?

A natural question to ask is what part of the graph is sufficient for a node to learn (or query), in order to be able to compute its layer? The question is easy to answer if we already knew the β -partition ℓ that we will compute. This β -partition implies a subgraph $D(v) := G[D(v)]$ for each node v that testifies the layer of node v in ℓ . In other words, the layer of node v depends on $D(v)$, that's why we call it the *dependency graph* of v . See Figure 2 for an example. **Once node v has learned $D(v)$ with its queries, it can compute its layer.** More formally, the dependency graph $D(v)$ of a node v contains all nodes u that are reachable from v via a path of nodes with strictly decreasing layers. Again, see Figure 2. Observe that every node in $D(v)$ has at most β neighbors outside of $D(v)$. Of course, the imagined β -partition ℓ is something that we can utilize in the analysis and to build intuition, but we cannot use it to guide the nodes to explore the graph, simply because it is not known beforehand.

Observe that if the dependency graph of a node v is too large, there is no hope for v to learn it in a sublinear number of queries. Hence, our only hope is that at least nodes with small dependency graphs ($\leq n^\delta$) manage to learn their respective dependency graphs and compute their layers. In our analysis, we show that most of the nodes have small dependency graphs.

Everything boils down to solving an impossible sounding task: **guide the graph exploration of a node v such that most of what v queries is within its dependency graph, with no prior information on where to search.** Deterministic graph exploration without prior knowledge is hard, especially since the graph may look identical in every direction at a node v , but only a small fraction of the directions contain the dependency graph and there is no way to identify these directions that we want to explore. Let us briefly explain why the classic search paradigms fail in this setting.

- *Depth-first-search:* Recall that node v can have up to β neighbors outside the dependency graph. If the algorithm happens to initiate the DFS from a neighbor outside of $D(v)$, all of the queries might be used on learning something that is not the dependency graph.
- *Breadth-first-search:* In order to avoid the pitfalls of DFS, one could employ a BFS with the hope of balancing the exploration of the graph. Indeed, after learning its 1-hop surroundings, at least a $1/\beta$ fraction of what is queried is the dependency graph, which sounds promising. Unfortunately, this approach is also flawed. Consider a node v and its neighbor u that is outside v 's dependency graph. The degree of u is unbounded, so if for example $\deg(u) = n^\delta$, most of the queries in the second level of the BFS will be spent on learning the neighbors of u , which are outside $D(v)$, and hence useless.

Clearly, a more sophisticated and fine-tune exploration procedure is required. Our actual approach explores the graph in a "volume-balanced" way, instead of "distance-balanced" like BFS and DFS.

Volume-based querying. In order to steer the direction of the queries, consider the following coin dropping procedure. Give n^δ coins to node v , who naively (this approach will still not work)

forwards the coins to all of its neighbors, giving each neighbor an equal share of the total number of v 's coins. Consider repeating this process recursively for every node holding coins, until coins cannot be further divided, which is in some sense a "volume-based" BFS. The forwarding of coins represents querying for nodes, and every node that has received a coin has been queried by v and hence *known* to v . Optimally, all nodes of $D(v)$ receive a coin. Unfortunately, this approach fails for a large family of counter-examples: skewed dependency graphs (Figure 3), comprising of a long path where every node has, e.g., $n^{\delta/10}$, neighbors in the lowest layer. In this case, n^δ coins will be already spent after only 10 steps of distributing coins, effectively learning only roughly $10 \cdot n^{\delta/10}$ dependency graph nodes.

2.2 Our Solution

Our solution uses the same coin dropping procedure, but we devise more sophisticated *forwarding rules* that determine how to distribute the coins. These rules utilize the topology of the graph induced by v 's knowledge aka the nodes v has discovered so far. The forwarding rules are also updated frequently during the course of the algorithm to steer the future exploration of the graph and making very strong usage of the adaptivity of the queries.

Recall that every node in $D(v)$ has at most β neighbors outside of $D(v)$. Hence, when coins are at a node $u \in D(v)$ and u forwards coins to $\beta + 1$ neighbors, we ensure that at least one neighbor from $D(v)$ receives coins, or more precisely at least a $1/(\beta + 1)$ -fraction of the coins goes to neighbors in $D(v)$. If we keep recursively applying this approach to all nodes that still have coins available, this roughly tells us, that after k steps of forwarding, at least $x/(\beta + 1)^k$ coins are at nodes that are contained in the dependency graph. Here x is the number of coins that v started with. However, it is not sufficient to choose $\beta + 1$ neighbors arbitrarily, since it may be that we keep forwarding coins to nodes that we already know, i.e., nodes that have seen a coin before. In the end, it may be that all of these coins are just accumulated in a small part of $D(v)$ that we already know, or in other words, the process does not lead v to querying new, undiscovered, parts of the dependency graph.

Our coin forwarding rules. Hence, we need to choose the *correct neighbors* to whom we forward the coins such that enough coins "flow" into the part of the dependency graph we have not yet learned, and not too many "leak out" outside of the dependency graph or "get stuck" in a part of the dependency graph that we already know. The rules for choosing the correct neighbors are chosen adaptively and frequently updated. Node v keeps simulating an estimation of the β -partition (this estimation is called the *induced β -partition*, formalized in Definition 3.6) for the whole subgraph it has queried so far, which we call S_v (which is only a small part of the whole graph). In the simulation, every node in S_v is assigned a layer by v , or if no layer is assigned, we assume it is ∞ . During an iteration, nodes forward coins to the $\beta + 1$ neighbors with the largest simulated layer. If more than $\beta + 1$ neighbors have an ∞ layer, a node can forward the coins to any such $\beta + 1$ neighbors. We show that with this forwarding rule, enough coins traverse to parts of $D(v)$ that have not received any coins before, and we keep learning more and more nodes of $D(v)$.

Surprisingly, we cannot halt the coin dropping game at the first point where node v is able to compute a layer $\neq \infty$ for itself. It is possible, that after querying most (but not all) of its dependency graph, node v computes its estimated layer to be, e.g., $n^{\delta/10}$, while its actual layer, if v would have found the whole dependency graph, would be significantly smaller. This is unacceptable, since we ultimately want the number of layers to be logarithmic in n . The final success of our method depends on the frequency of updating the forwarding rules and the duration for which we run the algorithm.

2.3 Using Our LCA Recursively

Due to the sublinear query complexity, our LCA can only compute the β -partition layers for a large fraction of nodes, but not all. One would hope to simply recurse on the subgraph remaining after removing all nodes with a $\neq \infty$ layer. However, this step turns out to require more care. Taking a closer look at Lemma 1.1, we only get the guarantee that there exists a subset $S \subseteq V$ containing at least a $1 - 1/n^{O(\delta)}$ fraction of vertices such that the layering of S forms a β -partition of $G[S]$. The difficulty of recursion stems from the fact that we cannot identify the nodes belonging to this magical set S and that there may exist nodes outside of set S that have a $\neq \infty$ layer, which are problematic, since including their layering in the partition may result in a *globally inconsistent* partition. Consider nodes $v, u \notin S$; for example, a node v may compute layer 10 for itself based on the fact that its neighbor u should be in layer 9. However, node u , for some reason, has computed its layer to be 11. If we place nodes v and u in their preferred layers, node v might end up with more than β neighbors in higher layers.

We resolve the issue with a slightly stronger LCA than stated in the informal Lemma 1.1. In addition to outputting a layer $\ell_u(u)$, each node $u \in V$ outputs a *proof* of its own layer in the form of a β -partition ℓ_u on a small subgraph of G . The proof is basically a set of instructions for layering a part of the graph such that v can obtain layer $\ell_u(u)$.

Having access to this proof actually relieves us from needing to know which nodes having layer $\neq \infty$ belong to S and which do not. We can merge all β -partitions ℓ_u that are given as proofs by nodes $u \in V$ with $\neq \infty$ via a global minimum function

$$\ell(v) = \min_{u \in V} \ell_u(v)$$

and get a globally consistent β -partition ℓ on the subgraph induced by all nodes with layer $\neq \infty$. This is sufficient for using our querying routines for recursion in AMPC.

3 MODEL AND DEFINITIONS

3.1 The AMPC Model

In the AMPC model, we have an input of size N , which is processed by P machines, each with space S . The unit of measure is a word of size $O(\log n)$ bits. The total space of all machines is $T = S \cdot P$ and we assume that $S = \Theta(N^{1-\Omega(1)})$. In addition, there is a collection of distributed data stores (DDS) that we denote by $D_0, D_1, \dots$. We assume that all DDS provide a key-value store semantics: they store a collection of key-value pairs, such that given a key, a DDS returns the corresponding value. We require that both key and value consist of a constant number of words. The input data is stored in D_0 and

uses a set of keys known to all machines (e.g. consecutive integers). The solution to a problem is stored in the last data store.

If there are $k > 1$ key-value pairs stored in the DDS with the same key x , the individual values can be accessed by querying for keys $(x, 1), \dots, (x, k)$. Note that the indices from 1 to k are assigned arbitrarily. A query for a key that does not occur in the DDS, results in an empty response. The computation proceeds in rounds. In the i th round, each machine can read data from D_{i-1} and write to D_i . Within a round, each machine can perform $O(S)$ reads (henceforth called queries) and $O(S)$ writes, and carry out arbitrary computation. Each query refers to querying for a single key-value pair from D_{i-1} and each write refers to writing a single key-value pair to D_i . The amount of communication that a machine performs per round is equal to the total number of queries and writes. The crucial property of the model is that the queries that a machine makes within a round may depend on each other. In particular, say g is a function from X to X and for each $x \in X$, D_{i-1} stores a key-value pair $(x, g(x))$. Then, in round i a machine can compute $g^k(y)$ if $k = O(S)$.

The realism of the AMPC model is discussed in more depth in [5]. The algorithms presented in this paper take a graph with n nodes and m edges as an input. In this case, the input size is $N = n + m$. We also assume that $S = n^\delta$, where $\delta \in (0, 1)$ is some constant.

3.2 Arboricity

Definition 3.1 (Arboricity). The arboricity $\alpha(G)$ of a graph $G = (V, E)$ is defined as

$$\alpha(G) = \max_{\substack{G' \subseteq G, \\ |V(G')| \geq 2}} \left\lceil \frac{|E(G')|}{|V(G')| - 1} \right\rceil.$$

An equivalent definition for $\alpha(G)$ is the minimum number of edge-disjoint forests into which the edge set of G can be partitioned [28]. In this work, we will consider graphs with arboricity bounded by parameter α that is $\alpha(G) \leq \alpha$ for some parameter α . To simplify notation we will assume that the arboricity of the graph is α . We now state some simple facts about the arboricity that immediately follow from Definition 3.1 and were also observed in [3].

Fact 3.2 ([3]). For any subgraph G' of G , the arboricity of G' is at most the arboricity of G .

Fact 3.3 ([3]). For any $\varepsilon > 0$, a graph $G = (V, E)$ with arboricity α has at least $\varepsilon|V|/(2 + \varepsilon)$ vertices with degree at most $(2 + \varepsilon) \cdot \alpha$.

An equivalent formulation of Fact 3.3 is that, for any $\varepsilon > 0$, a graph with arboricity α has less than $2|V|/(2 + \varepsilon)$ vertices with degree greater than $(2 + \varepsilon) \cdot \alpha$. We generalize this statement by parameterizing the degree bound as follows.

Lemma 3.4 (Generalization of Fact 3.3). For any $\beta \geq 1$, a graph $G = (V, E)$ with arboricity α has less than $2\alpha|V|/\beta$ vertices with degree greater than β .

Proof. Suppose for a contradiction that there are at least $2\alpha|V|/\beta$ vertices with degree greater than β . It follows that,

$$\begin{aligned} 2|E| &= \sum_{v \in V} \deg(v) > \beta \cdot \frac{2\alpha \cdot |V|}{\beta} = 2\alpha \cdot |V| \\ &\stackrel{*}{\geq} 2 \cdot \frac{|E|}{|V| - 1} \cdot |V| \\ &> 2|E|, \end{aligned}$$

which is a contradiction. The inequality $*$ follows from Definition 3.1, as $|E|/(|V| - 1)$ must be a lower bound for arboricity. $\square$

3.3 β -Partition

We begin with generalizing the graph-theoretic structure known as H -partition, which were introduced in the context of distributed message passing models by Barenboim and Elkin [3]. Informally, they define an H -partition as a partition of a graph with arboricity α into $O(\log n)$ layers such that every node has at most $(2 + \varepsilon) \cdot \alpha$ neighbors in higher or equal layers, where n is the number of nodes in the graph.

For our purposes, we define a β -partition, which is a partition of a graph into layers such that every node has at most β neighbors in higher or equal layers, for any positive value β . Our definition also permits an infinity layer: nodes in the infinity layer can have an unbounded number of neighbors in the infinity layer. If the infinity layer is non-empty, we call the partition a *partial β -partition*. Otherwise, we simply call it a β -partition.

Definition 3.5 ((partial) β -partition). Consider a graph $G = (V, E)$ and a function $\lambda: V(G) \rightarrow \mathbb{N} \cup \{\infty\}$. For $i \in \mathbb{N} \cup \{\infty\}$ define $V_i = \{v \in V(G) \mid \lambda(v) = i\}$ and $V_{\geq i} = \bigcup_{j \geq i} V_j$.

We say that λ is a β -partition if $\deg_{V_{\geq i}}(v) \leq \beta$ for every node $v \in G$ with $\lambda(v) \neq \infty$ and $|V_\infty| = 0$. If $|V_\infty| > 0$, we call λ a *partial β -partition*. We refer to V_i as *layer i* of the β -partition and $\lambda(v)$ as the *layer* of a node v . The *size* of a (partial) β -partition is the number of unique non-infinity layers.

Observe that if $\beta = n$ on a n -node graph, the β -partition would just contain one layer – the input graph. Hence, β -partitions are useful only for certain ranges of β . For example, when $\beta = (2 + \varepsilon)\alpha$ where α is the arboricity, we get the well known H -partition by Barenboim and Elkin [3].

We extend the definition of β -partitions further to node-induced subgraphs, where it is a priori unclear how to deal with neighbors not in the subgraph.

3.4 Induced β -Partition

Barenboim and Elkin [3] define an algorithm where we iteratively put all nodes with degree at most $(2 + \varepsilon)\alpha$ in the top-most layer and delete them from the graph. This algorithm has the property that each node will have at most $(2 + \varepsilon)\alpha$ neighbors in the same layer or above, and therefore, the output of this algorithm is a $(2 + \varepsilon)\alpha$ -partition. Using Facts 3.2 and 3.3, it is easy to see that size of the partition produced by this algorithm is at most $O(\log_{(2+\varepsilon)/2} n)$. In Definition 3.6, we generalize the algorithm of [3] in two ways: first, by making the degree bound β and second, by running it only on a subset S of the nodes.

In the following, we emphasize that all notions refer to the original graph G if not specified otherwise, even when we consider an (induced) subgraph of G . In particular, $N(u)$ and $\deg(u)$ always refer to $N_G(u)$ and $\deg_G(u)$, respectively. The following definitions do not assume any particular model of computation.

Definition 3.6 (S -induced β -partition). Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. Consider the partial β -partition $\sigma_{S,\beta}: V(G) \rightarrow \mathbb{N} \cup \{\infty\}$ constructed as follows.

- (1) Set $\sigma_{S,\beta}(v) \leftarrow \infty$ for every $v \in V$.
- (2) For $i = 0, \dots, |S|$:
Set $\sigma_{S,\beta}(v) \leftarrow i$ for all nodes $v \in S$ with at most β nodes $u \in N(v)$ for which $\sigma_{S,\beta}(u) = \infty$.

We refer to $\sigma_{S,\beta}(v)$ as the *layer* of a node v .

We often write σ or σ_S instead of $\sigma_{S,\beta}$ when subset S and/or parameter β are clear from context. The following lemma proves many useful properties about function $\sigma_{S,\beta}$ produced by Definition 3.6. In particular, part ii. proves that $\sigma_{S,\beta}$ is indeed a partial β -partition.

LEMMA 3.7. Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. The partial β -partition $\sigma_{S,\beta}$ from Definition 3.6 satisfies the following properties for a node $v \in S$.

- i. If $\sigma_{S,\beta}(v) = \infty$, then $\sigma_{S,\beta}(w) = \infty$ for at least $\beta + 1$ nodes $w \in N(v)$.
- ii. If $\sigma_{S,\beta}(v) \neq \infty$, then $\sigma_{S,\beta}(w) \geq \sigma_{S,\beta}(v)$ for at most β nodes $w \in N(v)$.
- iii. If $\sigma_{S,\beta}(v) \neq \infty$ and $\deg(v) \geq \beta + 1$, then $\sigma_{S,\beta}(w) \geq \sigma_{S,\beta}(v) - 1$ for at least $\beta + 1$ nodes $w \in N(v)$.

PROOF. In the following, an iteration refers to one round of Step 2 in Definition 3.6. Let $\sigma_S^k(v)$ denote value $\sigma_S(v)$ at the beginning of iteration k . For readability, we write σ instead of $\sigma_{S,\beta}$.

i. Observe that if there is an iteration where no node is assigned a σ value, none of the following iterations assign σ values. Hence, if value $|S|$ is assigned, $\sigma(v) \neq \infty$ holds for all nodes $v \in S$. Towards a contradiction, consider an iteration $i < |S|$ after which $\sigma(v) = \infty$ and there are at most β nodes $w \in N(v)$ with $\sigma(w) = \infty$. In iteration $i + 1$, it must be that $\sigma(v) \leftarrow i + 1 \neq \infty$, a contradiction.

ii. Consider $\sigma(v) = \sigma^i(v) = i \neq \infty$. By definition, it holds that $\sigma^i(w) = \infty$ for at most β neighbors $w \in N(v)$. Hence, for said neighbors it will hold that $\sigma(w) = \infty$ or $\sigma(w) \in [i, |S|]$. In either case, $\sigma(w) \geq \sigma(v)$.

iii. Consider $\sigma(v) = \sigma^i(v) = i \neq \infty$. Since $\deg(v) \geq \beta + 1$, it must hold that $i > 0$. Define I_i to be the set of nodes $w \in N(v)$ for which $\sigma^i(w) = \infty$. Define I_{i-1} similarly and observe that $I_i \subseteq I_{i-1}$: since $\sigma(v) = i$, it must be that $|I_{i-1}| \geq \beta + 1$ and $|I_i| \leq \beta$. Hence, for nodes $w \in I_{i-1} \setminus I_i$ it holds that $\sigma(w) = i - 1$. Since $\sigma^i(w) = \infty$ for nodes $w \in I_i$, it must be that $\sigma(w) \geq i$. Hence, for $|I_{i-1}| \geq \beta + 1$ nodes $w \in N(v)$ it holds that $\sigma(w) \geq i - 1 = \sigma(v) - 1$. $\square$

The following lemma reveals the so called non-increasing nature of the S -induced β -partition. Very informally, the intuition for the lemma is that the larger the subgraph used to construct the S -induced β -partition, the smaller the layer assignment for every node in the graph. In other words, more information gives a better partition.

LEMMA 3.8. Consider a graph $G = (V, E)$, subsets $S \subseteq T \subseteq V$ and any $\beta \geq 1$. Then $\sigma_{S,\beta}(v) \geq \sigma_{T,\beta}(v)$ for every node $v \in V$.

PROOF. For readability, we write σ_S and σ_T instead of $\sigma_{S,\beta}$ and $\sigma_{T,\beta}$, respectively. If $S = T$, then σ_S equals σ_T by definition. Hence, consider only $S \subset T$. The claim holds for all nodes v for which $\sigma_S(v) = \infty$. Since $\sigma_S(v) = \infty$ holds for all nodes $v \in V \setminus S$ by definition, we only need to prove the claim for nodes $v \in S$ for which $\sigma_S(v) \neq \infty$.

Consider the construction of the induced partitions σ_S and σ_T from Definition 3.6 side-by-side. Towards a contradiction, let i be the first iteration (side-by-side) after which $\sigma_S(v) < \sigma_T(v)$ holds for a node $v \in S$. It must be that $\sigma_S(v) = i \neq \infty$ and $\sigma_T(v) = \infty$, which implies that $\sigma_S(w) = \infty$ for at most β nodes $w \in N(v)$ and that $\sigma_T(w) = \infty$ for at least $\beta + 1$ nodes $w \in N(v)$. Hence there must have existed one node x for which $\sigma_S(x) < \sigma_T(x)$ after iteration $i - 1$, a contradiction. $\square$

The analysis of our algorithm will crucially rely on the properties of a so called *dependency graph* of a node, which is defined through a (partial) β -partition.

Definition 3.9 (Dependency graph). Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. The dependency graph $G[D(\sigma_{S,\beta}, v)]$ of a node $v \in V$ is defined as the output of the following recursive procedure.

- If $\sigma_{S,\beta}(v) = \infty$, then $D(\sigma_{S,\beta}, v) = \emptyset$
- If $\sigma_{S,\beta}(v) = 0$, then $D(\sigma_{S,\beta}, v) = \{v\}$
- otherwise $D(\sigma_{S,\beta}, v) = \{\bigcup_{w \in N^{<}(v)} D(\sigma_{S,\beta}, w)\} \cup \{v\}$ where $N^{<}(v)$ is the set of nodes $w \in N(v)$ such that $\sigma_{S,\beta}(w) < \sigma_{S,\beta}(v)$.

OBSERVATION 3.10 (NESTED PROPERTY OF DEPENDENCY GRAPHS). Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. For all $w \in D(\sigma_{S,\beta}, v)$ for a node $v \in S$ it holds that $D(\sigma_{S,\beta}, w) \subseteq D(\sigma_{S,\beta}, v)$.

PROOF. By Definition 3.9, a node w can be added to $D(\sigma_{S,\beta}, v)$ only by adding $D(\sigma_{S,\beta}, w)$ to $D(\sigma_{S,\beta}, v)$, and $D(\sigma_{S,\beta}, w)$ contains w by definition. $\square$

LEMMA 3.11. Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. If $\sigma_{S,\beta}(v) \neq \infty$ for a node $v \in S$, then $|N(v) \setminus D(\sigma_{S,\beta}, v)| \leq \beta$.

PROOF. We write σ instead of $\sigma_{S,\beta}$. By Definition 3.9, set $N(v) \cap D(\sigma, v)$ contains all nodes $w \in N(v)$ for which $\sigma(w) < \sigma(v)$. Hence, $N(v) \setminus D(\sigma, v)$ contains all nodes for which $\sigma(w) \geq \sigma(v)$. Because $\sigma(v) \neq \infty$, by Lemma 3.7 it holds that $\sigma(w) \geq \sigma(v)$ for at most β nodes $w \in N(v)$. $\square$

3.5 Natural β -Partition

We want to accentuate the case when the whole node set of a graph is used to construct an induced β -partition. For a graph $G = (V, E)$, we call the V -induced β -partition the *natural β -partition*.

Definition 3.12 (Natural β -partition). Consider a graph $G = (V, E)$, and any $\beta \geq 1$. The natural β -partition ℓ_β is defined as the V -induced β -partition $\sigma_{V,\beta}$.

For $\beta = (2 + \varepsilon)\alpha$ the natural β -partition corresponds to the H -partition from [3]. Our motivation behind the natural β -partition

definition is that ℓ_β assigns the lowest possible layer to all nodes in the graph among all induced β -partitions. More formally, we have the following lemma.

LEMMA 3.13. *Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. It holds that $\sigma_{S,\beta}(v) \geq \ell_\beta(v)$ for every $v \in V$.*

PROOF. By Lemma 3.8, it holds that $\sigma_{S,\beta}(v) \geq \sigma_{T,\beta}(v)$ for any subgraphs $S \subseteq T \subseteq V$ and every node $v \in V$. When $T = V$, it holds that $\sigma_{S,\beta}(v) \geq \sigma_{V,\beta}(v) = \ell_\beta(v)$. $\square$

Later, in our algorithmic usage of induced β -partitions, we will compute many of these partitions in parallel on various induced subgraphs $G[S]$, $S \subseteq V$ of the input graph G . The next lemma states that the layer assignment of a node v in $\sigma_{S,\beta}$ corresponds to its layer assignment in the natural β -partition if $G[S]$ contains the dependency graph of v .

LEMMA 3.14. *Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. If $D(\ell_\beta, v) \subseteq S$ for a node $v \in V$, then $\sigma_{S,\beta}(w) = \ell_\beta(w)$ for every $w \in D(\ell_\beta, v)$.*

PROOF. For readability, we write σ_S and ℓ instead of $\sigma_{S,\beta}$ and ℓ_β , respectively.

Let us first prove the claim for $S = D(\ell, v)$ by induction on the values of $\ell(v)$, the base case being $\ell(w) = 0$. The base case implies that $\deg(w) \leq \beta$, so by Definition 3.6 it holds that $\sigma_S(w) = 0$. Assume that $\sigma_S(w) = \ell(w)$ for all nodes $w \in S$ with $\ell(w) < i$. Consider a node $u \in S$ with layer $\ell(u) = i$. Observe that all nodes $w \in N(u)$ with $\ell(w) < \ell(u) = i$ are in S and $\sigma_S(w) = \ell(w)$ by induction. Since $\ell(u) = i$, by Lemma 3.7, there are at most β nodes $w \in N(u)$ with $\ell(w) \geq \ell(u)$, implying that $\sigma_S(u)$ will be i .

We have established that $\sigma_{D(\ell,v)}(w) = \ell(w)$ for every node $w \in D(\ell, v)$ if $S = D(\ell, v)$. If $S \supset D(\ell, v)$, the previous implies $\sigma_S(w) \leq \ell(w)$, because by Lemma 3.8 it holds that $\sigma_S(w) \leq \sigma_{D(\ell,v)}(w)$. On the other hand, by Lemma 3.13, for any subgraph $S \subseteq G$ it holds that $\sigma_S(w) \geq \ell(w)$ for every node $w \in S$. Hence, if $D(\ell, v) \subseteq S$ it holds that $\sigma_S(w) = \ell(w)$ for every node $w \in D(\ell, v)$. $\square$

4 AN LCA FOR PARTIAL β -PARTITION

In this section, we present an LCA for constructing a partial β -partition (Definition 3.5) of the input graph using a coin dropping game. The game is formulated from the perspective of a single node making local queries to the graph and can be easily implemented as an LCA (see Lemma 4.7). Later, we will simulate this game for many nodes in parallel in the AMPC model.

4.1 The Coin Dropping Game

The high-level idea of the coin dropping game for a node v in graph $G = (V, E)$ is as follows. During the game, we keep track of a certain subset of nodes $S_v \subseteq V$ that initially contains only node v , but grows in every super-iteration. In the beginning of every super-iteration, we compute the S_v -induced β -partition $\sigma_{S_v,\beta}$ from Definition 3.6. One can think of $\sigma_{S_v,\beta}(v)$ as an approximation of the natural β -partition $\ell_\beta = \sigma_{V,\beta}$ (see Definition 3.12). During a super-iteration, we perform a coin distribution process (consisting of iterations) that uses $\sigma_{S_v,\beta}$ as a guiding mechanism to add certain nodes to S_v such that $\sigma_{S_v,\beta}(v)$ of the next super-iteration is a “better” approximation of $\ell_\beta(v)$.

The coin distribution process consist of iterations, in each of which, nodes holding coins forward them along the edges of the graph according to certain *local forwarding rules*. In the very first iteration, only node v is given coins. The whole process can be viewed as a flow of coins that starts at node v and spreads through the graph. The flow stops when it reaches nodes outside of S_v , after which all nodes outside of S_v that have received a coin are added to S_v of the next super-iteration. The local forwarding rules manifest as *forwarding sets* for every node in S_v , which require computing the S_v -induced β -partition. Informally, the forwarding set of a node u is a small subset of u 's neighbors, to whom u will be forwarding its coins during the super-iteration. More formally, see Definition 4.1. While keeping the size of set S_v roughly below x^3 for some parameter x , we ensure that eventually $\sigma_{S_v,\beta}(v) = \ell_\beta(v)$ for all nodes that have small enough dependency graphs, i.e., $|D(\ell_\beta, v)| < x^2$ and a small enough layer in the natural β -partition, i.e., $\ell_\beta(v) \leq \log_{\beta+1} x$. We emphasize that the game does not guarantee that $D(\ell_\beta, v) \subseteq S_v$, which complicates the analysis.

Definition 4.1 (Forwarding set). Consider a graph $G = (V, E)$, a subset $S \subseteq V$ and any $\beta \geq 1$. Let $\sigma_{S,\beta}$ be the S -induced β -partition. The forwarding set $F(\sigma_{S,\beta}, u)$ for a node $u \in S$ is the set of any $\min\{\deg(u), \beta + 1\}$ nodes $w \in N(u)$ with the highest $\sigma_S(w)$ values.

(x, β, F) -coin dropping game. Consider executing the (x, β, F) -coin dropping game for a node v in a graph $G = (V, E)$. Throughout the game, we keep track of a subset $S_v \subseteq V$. The game consists of x^2 super-iterations, where one super-iteration proceeds as described in Algorithm 1. In the first super-iteration S_v is initialized to $S_v \leftarrow \{v\}$.

Algorithm 1 A super-iteration of the (x, β, F) -coin dropping game for a node $v \in V$.

- 1: Compute the S_v -induced β -partition $\sigma_{S_v,\beta}$ and the forwarding set $F(\sigma_{S_v,\beta}, u)$ for all $u \in S_v$.
 - 2: Node v is given x coins.
 - 3: For $|V|$ iter: every node $u \in S_v$ with $x' \geq |F(\sigma_{S_v,\beta}, u)|$ coins forwards $x'/|F(\sigma_{S_v,\beta}, u)|$ coins to every node in $F(\sigma_{S_v,\beta}, u)$.
 - 4: Every node $u \notin S_v$ with coins is added to S_v .
-

4.2 Game Analysis

Let us analyze the coin dropping game. Recall that Algorithm 1 refers to one super-iteration of the game. Recall that nodes cannot compute the natural β -partition ℓ_β themselves. However, we utilize ℓ_β for analysis. The following lemma ensures progress, and informally it states that if the S_v -induced β -partition assigns a layer to node v which does not equal $\ell_\beta(v)$ (it can be anything between $\ell_\beta(v)$ and ∞), a node from the dependency graph $D(\ell_\beta, v)$ is added to S_v which was not in S_v previously.

LEMMA 4.2 (MEASURE OF PROGRESS). *Consider a graph $G = (V, E)$, any $\beta, x \geq 1$ and the natural β -partition ℓ_β . Consider the beginning of an arbitrary super-iteration of the (x, β, F) -coin dropping game for a fixed node $v \in V$ and the corresponding set $S_v \subseteq V$. If $|D(\ell_\beta, v)| \leq x^2$, $\ell_\beta(v) \leq \log_{\beta+1} x$ and $\sigma_{S_v,\beta}(v) > \ell_\beta(v)$, a node $w \in D(\ell_\beta, v) \setminus S_v$ is added to S_v at the end of the super-iteration.*

PROOF. For readability, we write σ and ℓ instead of $\sigma_{S_v, \beta}$ and ℓ_β , respectively. Consider a node $v \in V$ as in the lemma statement. Recall that $v \in S_v$ by definition. Since $\sigma(v) = \infty$ by the lemma statement, we can assume that $\deg(v) \geq \beta + 1$ (otherwise $\sigma(v) = 0 = \sigma_V(v)$ by definition). Let $P = (w_0, w_1, \dots, w_k)$ be the longest path path where

$$w_0 = v \text{ and } w_i \in F(\sigma, w_{i-1}) \cap D(\ell, w_{i-1})$$

By the guarantees of Lemma 4.3, for every w_i it holds that $\sigma(w_i) > \sigma(w_{i-1})$. Observe that by Observation 3.10, it holds that $P \subseteq D(\ell, v)$. Because P cannot be any longer, Lemma 4.3 cannot be applied to w_k , so either $w_k \notin S_v$ or $\deg(w_k) \leq \beta$. There are two possible cases.

- $w_k \in S_v$ and $\deg(w_k) \leq \beta$: This implies $\sigma(w_k) = 0$, contradicting $\sigma(w_k) > \ell(w_k)$, since ℓ cannot be negative.
- $w_k \notin S_v$: Because $w_i \in D(\ell, w_{i-1})$, it holds that $\ell(w_{i-1}) > \ell(w_i)$ by definition for every $i \in [1, k]$. Since $\ell(v) \leq \log x$ by the lemma statement, it must hold be $k \leq \log x$. In Algorithm 1, $v(= w_0)$ forwards $x/(\beta + 1)$ coins to w_1 , who forwards $x/(\beta + 1)^2$ coins to w_2 , and so on. Since $k \leq \log x$, node w_k will receive at least $x/(\beta + 1)^{\log_{\beta+1} x} = 1$ coin(s) and will be added to S_v at the end of the super-iteration. Because $w_k \notin S$ and $w_k \in P \subseteq D(\ell, v)$, it holds that $w_k \in D(\ell, v) \setminus S_v$, completing the proof. $\square$

The following lemma was used in Lemma 4.2. Informally, it ensures that more than half of coins forwarded by a node $u \in S_v$ are forwarded to nodes in $D(\ell_\beta, v)$.

LEMMA 4.3. Consider a graph $G = (V, E)$, any $\beta, x \geq 1$ and the natural β -partition ℓ_β . Consider the (x, β, F) -coin dropping game for a fixed node $v \in G$ and the corresponding set $S_v \subseteq V$ during any super-iteration. Let $u \in S_v$ be a node with $\deg(u) \geq \beta + 1$. If $\sigma_{S_v, \beta}(u) > \ell_\beta(u)$, then there exists at least one node $w \in F(\sigma_{S_v, \beta}, u) \cap D(\ell_\beta, v)$ for which $\sigma_{S_v, \beta}(w) > \ell_\beta(w)$.

PROOF. For readability, we write σ and ℓ instead of $\sigma_{S_v, \beta}$ and ℓ_β , respectively. Consider $u \in S_v$ as in the lemma statement. Since $\deg(u) \geq \beta + 1$, it holds that $|F(\sigma, u)| = \beta + 1$ by Definition 4.1. Since $\ell(u) \neq \infty$, it holds that $|N(u) \setminus D(\ell, u)| \leq \beta$ by Lemma 3.11. Recall that $\ell(w) < \ell(u)$ holds for every node $w \in D(\ell, u) \setminus \{u\}$ by Definition 3.9. Hence, it holds that $\ell(w) < \infty$ for every node $w \in D(\ell, u) \setminus \{u\}$.

Let us consider the case where $\sigma(u) = \infty$. By Lemma 3.7 there are at least $\beta + 1$ nodes $w' \in N(u)$ with $\sigma(w') = \infty$. Hence, it holds that $\sigma(w) = \infty$ for every node $w \in F(\sigma, u)$. Since $|N(u) \setminus D(\ell, u)| \leq \beta$, $|F(\sigma, u)| = \beta + 1$, and $F(\sigma, u) \subseteq N(u)$, there must be at least one node $w \in F(\sigma, u) \cap D(\ell, u)$. Because $w \in F(\sigma, u)$, it holds that $\sigma(w) = \infty$. Because $w \in D(\ell, u) \setminus \{u\}$, it holds that $\ell(w) < \infty$, proving the claim.

Let us consider the case where $\sigma(u) \neq \infty$. By Lemma 3.7, it holds that $\sigma(w) \geq \sigma(u) - 1$ for at least $\beta + 1$ nodes $w \in N(u)$. Hence, it holds that $\sigma(w) \geq \sigma(u) - 1$ for every node $w \in F(\sigma, u)$. Since $|N(u) \setminus D(\ell, u)| \leq \beta$, $|F(\sigma, u)| = \beta + 1$, and $F(\sigma, u) \subseteq N(u)$, there must be at least one node $w \in F(\sigma, u) \cap D(\ell, u)$. Because $w \in F(\sigma, u)$, it holds that $\sigma(w) \geq \sigma(u) - 1 \geq \ell(u)$. Because $w \in D(\ell, u)$, it holds that $\ell(u) > \ell(w)$. We conclude that $\sigma(w) > \ell(w)$. $\square$

The following two lemmas bring everything together to prove that after x^2 super-iterations of the coin dropping game it holds that $\sigma_{S_v, \beta}(v) = \ell_\beta(v)$ and $|S_v| \leq x^3 + 1$.

LEMMA 4.4 (CORRECTNESS). Consider a graph $G = (V, E)$, any $\beta, x \geq 1$ and the natural β -partition ℓ_β . Consider the (x, β, F) -coin dropping game for a fixed node $v \in G$ and the corresponding set $S_v \subseteq V$. If $|D(\ell_\beta, v)| \leq x^2$ and $\ell_\beta(v) \leq \log_{\beta+1} x$, then upon termination $\sigma_{S_v, \beta}(v) = \ell_\beta(v)$.

PROOF. Recall that the (x, β, F) -coin dropping game comprises of x^2 super-iterations. Consider $v \in V$ as in the lemma statement. Towards a contradiction, assume that $\sigma_{S_v, \beta}(v) \neq \ell_\beta(v)$ after x^2 super-iterations. By Lemma 3.13, this implies that $\sigma_{S_v, \beta}(v) > \ell_\beta(v)$ holds for x^2 super-iterations. Observe that S_v can only increase in size during Algorithm 1. Since $\sigma_{S_v, \beta}(v) > \ell_\beta(v)$ in every super-iteration, a node $w \in D(\ell_\beta, v) \setminus S_v$ is added to S_v by Lemma 4.2 in every super-iteration. Since $|D(\ell_\beta, v)| \leq x^2$, after x^2 super-iterations it will hold that $D(\ell_\beta, v) \subseteq S_v$, implying $\sigma_{S_v, \beta}(v) = \ell_\beta(v)$ by Lemma 3.14, a contradiction. $\square$

REMARK 4.5 (NODES MAY NOT FIND THEIR DEPENDENCY GRAPHS). In Lemma 4.4 we do not guarantee that eventually $D(\ell_\beta, v) \subseteq S_v$, which would directly yield that $\sigma_{S_v, \beta}(v) = \ell_\beta(v)$ by Lemma 3.14. Rather we prove only the sufficient statement $\sigma_{S_v, \beta}(v) = \ell_\beta(v)$ that can also happen even when $D(\ell_\beta, v) \not\subseteq S_v$.

LEMMA 4.6 (QUERY COMPLEXITY). Consider a graph $G = (V, E)$, any $\beta, x \geq 1$ and the (x, β, F) -coin dropping game for a fixed node $v \in V$ and the corresponding set $S_v \subseteq V$. Upon termination, it holds that $G[S_v]$ is connected and contains at most x^6 edges.

PROOF. During the game, a node can be added to S_v only if it is adjacent to a node already in S_v . Hence, $G[S_v]$ is always connected.

Consider one super-iteration of the (x, β, F) -coin dropping game. Observe that the forwarding set of any node $u \in S_v$ is of size $\leq \beta + 1$, and only nodes with $\geq \beta + 1$ coins forward coins. Hence, if a node $w \notin S_v$ receives coins, it receives ≥ 1 coins. Since the total number of usable coins in a super-iteration is x , the number of nodes that can be added to S_v in a super-iteration is at most x . The claim follows since there are x^2 super-iterations and $G[S_v]$ is connected. $\square$

4.3 The LCA

In this section, we show the LCA for computing a partial β -partition with a large fraction of nodes being assigned a layer $\neq \infty$. In particular, we prove the following lemma.

LEMMA 4.7 (SUBLINEAR LCA FOR PARTIAL β -PARTITIONS). For any constant $\varepsilon > 0$ and $\beta \geq (2 + \varepsilon)\alpha$ there is a deterministic LCA algorithm that uses at most x^6 queries per node on a graph G with arboricity α and assigns each node a layer from $\mathbb{N} \cup \{\infty\}$ such that the following holds: There exists a subset $S \subseteq V$ containing at least a $1 - 2^{1 - \log x / \log_{\beta/(2\alpha)}(\beta+1)}$ fraction of vertices such that the layering of S forms a β -partition of $G[S]$ with $\log_{\beta+1} x$ layers.

PROOF. Consider ε and β as in the lemma statement. When questioned about vertex v , the LCA performs the (x, β, F) -coin dropping game and outputs the S_v -induced β -partition layer of v . In order

to perform the coin dropping game, the LCA needs to be able to compute the S_v -induced β -partition (Definition 3.6) locally. This is possible since it only requires knowing $G[S_v]$ and the degrees of nodes in S_v . By Lemma 4.6, subgraph $G[S_v]$ contains at most x^6 edges at any point during the game, bounding the query complexity of the LCA.

By Lemma 4.4, if $|D(\ell_\beta, v)| \leq x^2$ and $\ell_\beta(v) \leq \log_{\beta+1} x$, then upon termination the LCA can output $\ell_\beta(v)$, which is the layer of v in the natural β -partition of graph G . By Lemma 4.9, there are at least $|V| \cdot (1 - 2^{1-\log x / \log_{\beta/(2\alpha)}(\beta+1)})$ such nodes, completing the proof. $\square$

REMARK 4.8. *Instead of Lemma 4.7, we actually obtain a stronger statement. Our LCA, if queried about a node u , outputs a partial β -partition $\ell_u : V \rightarrow \mathbb{N} \cup \{\infty\}$. There exists a subset $S \subseteq V$ containing at least a $1 - 2^{1-\log x / \log_{\beta/(2\alpha)}(\beta+1)}$ fraction of vertices, and $\lambda : V \rightarrow \mathbb{N} \cup \{\infty\} : v \mapsto \min_{u \in S} \ell_u(v)$ is a partial β -partition of G with $\log_{\beta+1} x$ layers.*

PROOF. For a node $u \in V$, instead of only outputting the S_v -induced β -partition layer $\ell_u(u)$ in the proof of Lemma 4.7, the LCA can simply output the whole partition ℓ_u . By Lemma 4.10, function λ is a partial β -partition of G with the desired properties. $\square$

The following lemmas are purely combinatorial and do not depend on the coin dropping game nor any model of computation.

LEMMA 4.9. *Consider a graph $G = (V, E)$ with arboricity α and the natural β -partition ℓ_β for any constant $\varepsilon > 0$ and $\beta \geq (2 + \varepsilon)\alpha$. For any $x > 1$, let $X \subseteq V$ such that $|D(\ell_\beta, v)| \leq x^2$ and $\ell_\beta(v) \leq \log_{\beta+1} x$ for $v \in X$. It holds that*

$$|V \setminus X| \leq |V| \cdot 2^{1-\log x / \log_{\beta/(2\alpha)}(\beta+1)}.$$

PROOF. Divide the nodes of V into sets $V_>$ and $V_\leq$, such that $\ell_\beta(w) > \log_{\beta+1} x$ holds for nodes $w \in V_>$ and $\ell_\beta(w) \leq \log_{\beta+1} x$ holds for nodes $w \in V_\leq$. Furthermore, let us define $V_\leq^X = V_\leq \cap X$. Observe that $V \setminus X = V_> \cup V_\leq^X$. Let us bound the terms separately.

$V_\leq^X$: By definition, it holds that $|D(\ell_\beta, w)| > x^2$ for nodes $w \in V_\leq^X$.

Towards a contradiction, assume that $|V_\leq^X| > |V| \cdot x^{-1}$. Let every node $w \in V_\leq^X$ give every node in its dependency graph one coin. Note that all nodes in the dependency graph of w are in $V_\leq$. The total number of coins is hence at least $|V| \cdot x^{-1} \cdot x^2 = |V| \cdot x$. On the other hand, any node u with $\ell_\beta(u) \leq \log_{\beta+1} x$ has at most β neighbors in higher layers, and hence can be given a coin by at most $\beta^{\log_{\beta+1} x} < x$ nodes in V . Since the total number of coins is at least $|V_\leq| \cdot x$, and every node can only be charged at most x coins, it must be that $|V| > (|V| \cdot x) / x = |V|$, which is a contradiction and hence it holds that $|V_\leq^X| \leq |V| \cdot x^{-1}$.

$V_>$: By Lemma 3.4, we get that for each layer i the number of nodes that will receive a layer larger than i shrinks by a factor of $2\alpha/\beta$. Therefore, the number of nodes with $\ell_\beta(w) >$

$\log_{\beta+1} x$ is

$$\begin{aligned} |V_>| &\leq |V| \cdot (2\alpha/\beta)^{\log_{\beta+1} x} \\ &= |V| \cdot (\beta/(2\alpha))^{-\log_{\beta+1} x} \\ &= |V| \cdot x^{-1/\log_{\beta/(2\alpha)}(\beta+1)}. \end{aligned}$$

Hence, it holds that

$$\begin{aligned} |V \setminus X| &\leq |V| \cdot x^{-1} + |V| \cdot x^{-1/\log_{\beta/(2\alpha)}(\beta+1)} \\ &\leq |V| \cdot 2^{1-\log x / \log_{\beta/(2\alpha)}(\beta+1)}. \end{aligned} \quad \square$$

LEMMA 4.10 (CLOSURE UNDER MINIMUM, PARTIAL β -PARTITIONS). *Let $\ell_1, \dots, \ell_k$ be partial β -partitions of a graph $G = (V, E)$ for any $\beta \geq 1$. The following function $\lambda : V \rightarrow \mathbb{N} \cup \{\infty\}$ assigning a layer to every node $v \in V$ is also a partial β -partition:*

$$\lambda(v) = \min_{1 \leq i \leq k} \ell_i(v).$$

Moreover, if $\ell_i(v) \neq \infty$ for any $i \in [1, k]$ then $\lambda(v) \neq \infty$.

PROOF. In order to prove that λ is a partial β -partition, we need to show that $\lambda(w) \geq \lambda(v) \neq \infty$ for at most β nodes $w \in N(v)$. Let $N^\geq(v)$ be the set of neighbors $w \in N(v)$ with $\lambda(w) \geq \lambda(v)$.

Without loss of generality, we can assume that $\lambda(v) = \ell_1(v)$. Let $N_1^<(v)$ be the set of neighbors $w \in N(v)$ with $\ell_1(w) < \ell_1(v)$, and let $N_1^\geq(v)$ be the set of neighbors $w \in N(v)$ with $\ell_1(w) \geq \ell_1(v)$. Let us count how many nodes from sets $N_1^<(v)$ and $N_1^\geq(v)$ could be in $N^\geq(v)$.

- Because $\lambda(w) = \min\{\ell_1(w), \dots, \ell_k(w)\}$ for every node $w \in N_1^<(v)$, it holds that $\lambda(w) \leq \ell_1(w)$. Hence, no nodes from $N_1^<(v)$ can be in $N^\geq(v)$.
- Because ℓ_1 is a partial H -partition, it holds that $|N_1^\geq(v)| \leq \beta$ so at most β nodes from $N_1^\geq(v)$ can be in $N^\geq(v)$.

We conclude that $|N^\geq(v)| \leq \beta$, proving that λ is a partial β -partition. If $\ell_i(v) \neq \infty$ for any $i \in [1, k]$ then $\lambda(v) \neq \infty$ because function λ takes the minimum and $\ell_i(v) < \infty$. $\square$

ACKNOWLEDGMENTS

Rustam Latypov is supported by the Research Council of Finland, Grant 334238. Yannic Maus is supported by the Austrian Science Fund (FWF), Grant P36280-N. This work was done while Shreyas Pai was a postdoctoral researcher at Aalto University, supported in part by Research Council of Finland, Grant 334238, and Helsinki Institute for Information Technology HIIT.

REFERENCES

- [1] L. Barenboim. 2015. Deterministic $(\Delta + 1)$ -Coloring in Sublinear (in Δ) Time in Static, Dynamic and Faulty Networks. In *Proc. 34th ACM Symposium on Principles of Distributed Computing (PODC)*. 345–354.
- [2] L. Barenboim and M. Elkin. 2010. Deterministic Distributed Vertex Coloring in Polylogarithmic Time. In *Proc. 29th Symp. on Principles of Distributed Computing (PODC)*.
- [3] Leonid Barenboim and Michael Elkin. 2010. Sublogarithmic distributed MIS algorithm for sparse graphs using Nash-Williams decomposition. *Distributed Comput.* 22, 5–6 (2010), 363–379. <https://doi.org/10.1007/s00446-009-0088-2>
- [4] Leonid Barenboim and Michael Elkin. 2013. *Distributed Graph Coloring: Fundamentals and Recent Developments*. Morgan & Claypool Publishers.
- [5] Soheil Behnezhad, Laxman Dhulipala, Hossein Esfandiari, Jakub Łącki, Vahab Mirrokni, and Warren Schudy. 2019. Massively Parallel Computation via Remote Memory Access. In *The Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 59–68. <https://doi.org/10.1145/3323165.3323208>

- [6] Suman K. Bera, Amit Chakrabarti, and Prantar Ghosh. 2020. Graph Coloring via Degeneracy in Streaming and Other Space-Conscious Models. In *47th International Colloquium on Automata, Languages, and Programming (ICALP 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 168)*, Artur Czumaj, Anuj Dawar, and Emanuela Merelli (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 11:1–11:21. <https://doi.org/10.4230/LIPIcs.ICALP.2020.11>
- [7] Sayan Bhattacharya, Martin Costa, Nadav Panski, and Shay Solomon. 2023. Arboricity-Dependent Algorithms for Edge Coloring. *arXiv preprint arXiv:2311.08367* (2023).
- [8] Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. 2019. The Complexity of $(\Delta + 1)$ -Coloring in Congested Clique, Massively Parallel Computation, and Centralized Local Computation. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*. <https://doi.org/10.1145/3293611.3331607>
- [9] Yi-Jun Chang, Wenzheng Li, and Seth Pettie. 2018. An Optimal Distributed $(\Delta + 1)$ -Coloring Algorithm?. In *the Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*, 445–456.
- [10] Aleksander Björn Grodt Christiansen, Krzysztof Nowicki, and Eva Rotenberg. 2023. Improved Dynamic Colouring of Sparse Graphs. In *the Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*, 1201–1214. <https://doi.org/10.1145/3564246.3585111>
- [11] Aleksander B. G. Christiansen and Eva Rotenberg. 2022. Fully-Dynamic $\alpha + 2$ Arboricity Decompositions and Implicit Colouring. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 229)*, Miłkołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 42:1–42:20. <https://doi.org/10.4230/LIPIcs.ICALP.2022.42>
- [12] Sam Coy, Artur Czumaj, Peter Davies-Peck, and Gopinath Mishra. 2024. Parallel Derandomization for Coloring.
- [13] Artur Czumaj, Peter Davies, and Merav Parter. 2021. Improved Deterministic $(\Delta + 1)$ -Coloring in Low-Space MPC. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 469–479.
- [14] Manuela Fischer, Jeff Giliher, and Christoph Grunau. 2023. Deterministic Massively Parallel Symmetry Breaking for Sparse Graphs. In *the Proceedings of the Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 89–100. <https://doi.org/10.1145/3558481.3591081>
- [15] Pierre Fraigniaud, Marc Heinrich, and Adrian Kosowski. 2016. Local Conflict Coloring. In *the Proceedings of the Symposium on Foundations of Computer Science (FOCS)*, 625–634.
- [16] Pierre Fraigniaud, Marc Heinrich, and Adrian Kosowski. 2016. Local Conflict Coloring. In *FOCS*, 625–634.
- [17] Mohsen Ghaffari and Christiana Lymouri. 2017. Simple and Near-Optimal Distributed Coloring for Sparse Graphs. In *the International Symposium on Distributed Computing (DISC)*, 20:1–20:14. <https://doi.org/10.4230/LIPIcs.DISC.2017.20>
- [18] Mohsen Ghaffari and Ali Sayyadi. 2019. Distributed Arboricity-Dependent Graph Coloring via All-to-All Communication. In *the Proceedings of the International Colloquium on Automata, Languages, and Programming (ICALP)*, 142:1–142:14. <https://doi.org/10.4230/LIPIcs.ICALP.2019.142>
- [19] Christoph Grunau, Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. 2023. Conditionally Optimal Parallel Coloring of Forests. In *37th International Symposium on Distributed Computing (DISC 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 281)*, Rotem Oshman (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 23:1–23:20. <https://doi.org/10.4230/LIPIcs.DISC.2023.23>
- [20] MohammadTaghi Hajiaghayi, Marina Knittel, Hamed Saleh, and Hsin-Hao Su. 2022. Adaptive Massively Parallel Constant-Round Tree Contraction. 83:1–83:23. <https://doi.org/10.4230/LIPIcs.ITCS.2022.83>
- [21] David G. Harris, Hsin-Hao Su, and Hoa T. Vu. 2021. On the Locality of Nash-Williams Forest Decomposition and Star-Forest Decomposition. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (Virtual Event, Italy) (PODC'21)*. Association for Computing Machinery, New York, NY, USA, 295–305. <https://doi.org/10.1145/3465084.3467908>
- [22] Monika Henzinger, Stefan Neumann, and Andreas Wiese. 2020. Explicit and Implicit Dynamic Coloring of Graphs with Bounded Arboricity. *CoRR* abs/2002.10142 (2020). [arXiv:2002.10142](https://arxiv.org/abs/2002.10142) <https://arxiv.org/abs/2002.10142>
- [23] Howard J. Karloff, Siddharth Suri, and Sergei Vassilvitskii. 2010. A Model of Computation for MapReduce. In *SODA*. <https://doi.org/10.1137/1.9781611973075.76>
- [24] Rustam Latypov, Yannic Maus, Shreyas Pai, and Jara Uitto. 2024. Adaptive Massively Parallel Coloring in Sparse Graphs. [arXiv:2402.13755 \[cs.DC\]](https://arxiv.org/abs/2402.13755)
- [25] Nati Linial. 1992. Locality in Distributed Graph Algorithms. *SIAM J. Comput.* 21, 1 (1992), 193–201.
- [26] Yannic Maus. 2021. Distributed Graph Coloring Made Easy. In *SPAA '21: 33rd ACM Symposium on Parallelism in Algorithms and Architectures, Virtual Event, USA, 6–8 July, 2021*, Kunal Agrawal and Yossi Azar (Eds.). ACM, 362–372. <https://doi.org/10.1145/3409964.3461804>
- [27] Yannic Maus and Tigran Tonoyan. 2022. Linial for lists. *Distributed Comput.* 35, 6 (2022), 533–546. <https://doi.org/10.1007/S00446-022-00424-Y>
- [28] Crispin Nash-Williams. 1964. Decomposition of Finite Graphs Into Forests. *Journal of the London Mathematical Society* s1-39 (1964), 12. <https://doi.org/10.1112/jlms/s1-39.1.12>
- [29] Will Rosenbaum and Jukka Suomela. 2020. Seeing Far vs. Seeing Wide: Volume Complexity of Local Graph Problems. In *the Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 89–98.
- [30] R. Rubinfeld, G. Tamir, S. Vardi, and N. Xie. 2011. Fast Local Computation Algorithms. In *the Proceedings of the Symposium on Innovations in Computer Science (ICS)*, 223–238.

Business, Economy
Art, Design, Architecture
Science, Technology
Crossover

Doctoral Theses

Aalto DT 82/2026

ISBN 978-952-64-3082-9
ISBN 978-952-64-3081-2 (pdf)

Aalto University
School of Science
Department of Computer Science
aalto.fi